

```
=== FILE: .\config.py ===
```

```
import os
```

```
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
```

```
class Config:
```

```
    SECRET_KEY = os.environ.get('SECRET_KEY', 'typesphere-production-secret-key-2026')
```

```
    # Handle Render/Heroku postgresql:// URL syntax
```

```
    raw_db_url = os.environ.get('DATABASE_URL', f"sqlite:///({os.path.join(BASE_DIR, 'typesphere.db')})")
```

```
    if raw_db_url.startswith("postgres://"):
```

```
        raw_db_url = raw_db_url.replace("postgres://", "postgresql://", 1)
```

```
    SQLALCHEMY_DATABASE_URI = raw_db_url
```

```
    SQLALCHEMY_TRACK_MODIFICATIONS = False
```

```
    SESSION_COOKIE_HTTPONLY = True
```

```
    SESSION_COOKIE_SAMESITE = 'Lax'
```

```
    WTF_CSRF_TIME_LIMIT = None
```

```
class TestConfig(Config):
```

```
    TESTING = True
```

```
    SQLALCHEMY_DATABASE_URI = 'sqlite:///memory:'
```

```
    WTF_CSRF_ENABLED = False
```

```
=== FILE: .\requirements.txt ===
```

```
Flask==3.0.2
```

```
Flask-SQLAlchemy==3.1.1
```

```
Flask-Login==0.6.3
```

```
Flask-WTF==1.2.1
```

```
Flask-Migrate==4.0.7
```

```
Flask-SocketIO==5.3.6
```

```
python-socketio==5.11.1
```

```
python-engineio==4.9.0
```

```
email-validator==2.1.1
```

```
Werkzeug==3.0.1
```

```
SQLAlchemy>=2.0.36
```

```
WTForms==3.1.2
```

```
gunicorn==21.2.0
```

```
=== FILE: .\run.py ===
```

```
import os
```

```
import socket
```

```
from app import create_app, socketio
```

```
app = create_app()
```

```
def get_local_ip():
```

```
    try:
```

```
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
        s.connect(("8.8.8.8", 80))
```

```
        ip = s.getsockname()[0]
```

```
        s.close()
```

```
        return ip
```

```
    except Exception:
```

```
        return "127.0.0.1"
```

```
if __name__ == '__main__':
```

```
    port = int(os.environ.get('PORT', 5000))
```

```
    local_ip = get_local_ip()
```

```
    mail_user = app.config.get('MAIL_USERNAME', '')
```

```
    is_mail_ready = bool(mail_user and 'your_' not in mail_user)
```

```
    print("\n" + "=" * 65)
```

```
    print("\n TypeSphere Server is Running!")
```

```
    print(f" On your PC:      http://127.0.0.1:{port}")
```

```
    print(f" On your Phone:    http://{local_ip}:{port}")
```

```
print(f"📧 Mail Sender Active: {mail_user if is_mail_ready else '❌ NOT CONFIGURED (Edit config.py)'}")
```

```
print(f"=" * 65 + "\n")
```

```
socketio.run(
    app,
    host='0.0.0.0',
    port=port,
    debug=True,
    allow_unsafe_werkzeug=True
)
```

```
=== FILE: .\seed_data.py ===
```

```
import sys

from datetime import date, datetime

from sqlalchemy import text

from app import create_app, db

from app.models.user import User

from app.models.typing import TypingText, TypingTest

from app.models.challenge import Achievement, DailyChallenge

from app.models.curriculum import LessonStage

from app.models.arcade_content import ArcadeGameConfig, ArcadeContentItem

from app.models.admin import PlatformConfig

from app.models.feedback import RatingReview
```

```
app = create_app()
```

```
def seed_database():
```

```
    with app.app_context():

        print("🗄️ Initializing Database Schema...")

        db.create_all()
```

```
    # 1. Resilient Auto-Migration Patch for SQLite
```

```
    try:
```

```
with db.engine.connect() as conn:
```

```
    if 'sqlite' in str(db.engine.url):
```

```
        # Patch users table
```

```
        res = conn.execute(text("PRAGMA table_info(users)"))
```

```
        cols = {row[1] for row in res.fetchall()}
```

```
        if cols:
```

```
            if 'is_verified' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN is_verified BOOLEAN DEFAULT 1"))
```

```
            if 'elo_rating' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN elo_rating INTEGER DEFAULT 1000"))
```

```
            if 'ranked_wins' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN ranked_wins INTEGER DEFAULT 0"))
```

```
            if 'ranked_losses' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN ranked_losses INTEGER DEFAULT 0"))
```

```
            if 'is_suspended' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN is_suspended BOOLEAN DEFAULT 0"))
```

```
            if 'is_banned' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN is_banned BOOLEAN DEFAULT 0"))
```

```
            if 'status_reason' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN status_reason VARCHAR(255)"))
```

```
            if 'last_active' not in cols:
```

```
                conn.execute(text("ALTER TABLE users ADD COLUMN last_active DATETIME"))
```

```
        # Patch user_settings table
```

```
        res = conn.execute(text("PRAGMA table_info(user_settings)"))
```

```
        s_cols = {row[1] for row in res.fetchall()}
```

```
        if s_cols:
```

```
            if 'blind_mode' not in s_cols:
```

```
                conn.execute(text("ALTER TABLE user_settings ADD COLUMN blind_mode BOOLEAN DEFAULT 1"))
```

```
            if 'ghost_mode' not in s_cols:
```

```
                conn.execute(text("ALTER TABLE user_settings ADD COLUMN ghost_mode BOOLEAN DEFAULT 0"))
```

```
            if 'default_duration' not in s_cols:
```

```
                conn.execute(text("ALTER TABLE user_settings ADD COLUMN default_duration INTEGER DEFAULT 60"))
```

```

if 'default_content' not in s_cols:

    conn.execute(text("ALTER TABLE user_settings ADD COLUMN default_content VARCHAR(32) DEFAULT 'words'"))

if 'default_level' not in s_cols:

    conn.execute(text("ALTER TABLE user_settings ADD COLUMN default_level VARCHAR(32) DEFAULT 'moderate'"))

if 'sound_volume' not in s_cols:

    conn.execute(text("ALTER TABLE user_settings ADD COLUMN sound_volume FLOAT DEFAULT 0.7"))

```

Patch typing_tests table

```
res = conn.execute(text("PRAGMA table_info(typing_tests)"))
```

```
t_cols = {row[1] for row in res.fetchall()}
```

if t_cols:

if 'title' not in t_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN title VARCHAR(128) DEFAULT 'Passage'"))
```

if 'word_count' not in t_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN word_count INTEGER DEFAULT 0"))
```

if 'character_count' not in t_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN character_count INTEGER DEFAULT 0"))
```

if 'complexity_score' not in t_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN complexity_score FLOAT DEFAULT 1.0"))
```

if 'is_active' not in t_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN is_active BOOLEAN DEFAULT 1"))
```

if 'created_by' not in t_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN created_by VARCHAR(64) DEFAULT 'System'"))
```

if 'updated_by' not in t_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN updated_by VARCHAR(64)"))
```

Patch typing_tests table

```
res = conn.execute(text("PRAGMA table_info(typing_tests)"))
```

```
test_cols = {row[1] for row in res.fetchall()}
```

if test_cols:

if 'is_ranked' not in test_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN is_ranked BOOLEAN DEFAULT 0"))
```

if 'content_category' not in test_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN content_category VARCHAR(64) DEFAULT 'General'"))
```

if 'difficulty' not in test_cols:

```
    conn.execute(text("ALTER TABLE typing_tests ADD COLUMN difficulty VARCHAR(32) DEFAULT 'moderate'"))
```

Patch lesson_stages table

```
res = conn.execute(text("PRAGMA table_info(lesson_stages)"))
```

```
l_cols = {row[1] for row in res.fetchall()}
```

if l_cols:

if 'track' not in l_cols:

```
    conn.execute(text("ALTER TABLE lesson_stages ADD COLUMN track VARCHAR(32) DEFAULT 'beginner'"))
```

if 'hand_position_hint' not in l_cols:

```
    conn.execute(text("ALTER TABLE lesson_stages ADD COLUMN hand_position_hint VARCHAR(128)"))
```

if 'keyboard_row' not in l_cols:

```
    conn.execute(text("ALTER TABLE lesson_stages ADD COLUMN keyboard_row VARCHAR(32) DEFAULT 'home'"))
```

if 'required_attempts' not in l_cols:

```
    conn.execute(text("ALTER TABLE lesson_stages ADD COLUMN required_attempts INTEGER DEFAULT 2"))
```

```
conn.commit()
```

except Exception as e:

```
    print("Auto-migration note:", e)
```

2. Seed Super Admin & Test Typist

```
print("🌱 Seeding Accounts...")
```

```
admin = User.query.filter_by(username='admin').first()
```

if not admin:

```
    admin = User(
```

```
        username='admin',
```

```
        email='admin@typesphere.io',
```

```
        role='super_admin',
```

```
        is_verified=True,
```

```
        elo_rating=1500,
```

```
        ranked_wins=24,
```

```
        ranked_losses=2
```

```
)

admin.set_password('AdminMaster2026!')

db.session.add(admin)

else:

    admin.role = 'super_admin'

    admin.is_verified = True


typist = User.query.filter_by(username='speeddemon').first()

if not typist:

    typist = User(

        username='speeddemon',

        email='speed@typesphere.io',

        role='user',

        is_verified=True,

        elo_rating=1350,

        ranked_wins=15,

        ranked_losses=4

    )

    typist.set_password('SpeedPass123!')

    db.session.add(typist)

else:

    typist.is_verified = True


# 3. Seed Platform Achievements

print("🌱 Seeding Achievements...")

badges = [

    ('first_test', 'First Contact', 'Complete your very first typing test on TypeSphere.', 'zap'),

    ('wpm_50', 'Cruising Velocity', 'Reach a verified speed of 50 WPM.', 'award'),

    ('wpm_75', 'Sonic Cadence', 'Reach a verified speed of 75 WPM.', 'trending-up'),

    ('wpm_100', 'Century Club', 'Surpass the triple-digit barrier of 100 WPM.', 'cpu'),

    ('perfect_acc', 'Flawless Flow', 'Complete a full test with exactly 100% accuracy.', 'shield'),

    ('night_owl', 'Night Owl', 'Complete a verified typing run between 12:00 AM and 5:00 AM.', 'moon'),

    ('early_bird', 'Early Bird', 'Complete a verified typing run between 5:00 AM and 8:00 AM.', 'sun'),
```

```
('survival_master', 'Survival Master', 'Complete a full test in Survival Mode without losing all 3 hearts.', 'heart'),

('certified_tyrist', 'Certified Scholar', 'Generate an official proficiency certificate.', 'file-text'),

('century_veteran', 'Century Veteran', 'Complete 100 verified tests on TypeSphere.', 'activity')

]

for code, title, desc, icon in badges:

    if not Achievement.query.filter_by(code=code).first():

        db.session.add(Achievement(code=code, title=title, description=desc, icon=icon))


# 4. Seed Multi-Track Academy Curriculum (12 Stages across 4 Tracks)

print("🌱 Seeding Academy Curriculum (4 Tracks, 12 Stages)...")

curriculum_data = [

    ('beginner', 1, "Keyboard Orientation & Posture", "intro", "Learn home row resting anchors, hand ergonomics, and finger coordination.", "f j f j f j dk sl a; a; f j dk sl a;", 15.0, 90.0, 2, "F J D K"),

    ('beginner', 2, "Home Row Mastery", "home-row", "Anchor muscle memory exclusively on the baseline resting keys.", "asdf jkl; a fad flask fall glad half dash salad flash salsa lad fall", 20.0, 92.0, 2, "A S D F J K L ;"),

    ('beginner', 3, "Top Row Extensions", "top-row", "Train upward vertical reaches without shifting wrists off the desk.", "quit write power tower quote trip your wipe wire pure quiet root weep", 25.0, 92.0, 2, "Q W E R T Y U I O P"),

    ('beginner', 4, "Bottom Row Transitions", "bottom-row", "Practice downward finger tucks while maintaining palm stability.", "cabin van mix zoom bomb zinc exam calm civic move bank comb mimic", 28.0, 92.0, 2, "Z X C V B N M"),

    ('intermediate', 1, "Contralateral Shift Synchronization", "shifts", "Synchronize shift-key holds for effortless capitalization.", "The Quick Brown Fox Jumps Over The Lazy Dog London Paris Tokyo", 38.0, 94.0, 2, "Shift Keys"),

    ('intermediate', 2, "High-Frequency Core Vocabulary", "core-words", "Fluid execution on the top 100 most frequent English building blocks.", "their there would about which could people other first water after", 45.0, 94.0, 2, "General Vocabulary"),

    ('intermediate', 3, "Complete Sentence Cadence", "sentences", "Punctuation rhythms, sentence capitalization, and natural phrase timing.", "Consistent practice builds velocity. Never compromise accuracy for speed.", 52.0, 95.0, 2, "Punctuation & Syntax"),

    ('intermediate', 4, "Top Number Row Reaches", "numbers", "Reach the top number row with correct finger stretches without looking.", "Invoice 8402 total 195 dollars on 2026-09-15 tracking number 7391054", 42.0, 92.0, 2, "1 2 3 4 5 6 7 8 9 0"),

    ('advanced', 1, "Complex Symbols & Brackets", "symbols", "Master hyphens, parentheses, semicolons, quotes, and code brackets.", "function(arg) { return [x, y]; } value = 'true'; (check == 100)", 60.0, 95.0, 2, "{} [] () <> /"),

    ('advanced', 2, "High-Speed N-Gram Sprints", "ngrams", "Rapid bursts of frequent bigrams and trigrams (the, ing, tion, ment).", "the and for that with this from have they which would there their about", 72.0, 95.0, 2, "the and ing tion"),

    ('advanced', 3, "Zero-Error Precision Gauntlet", "precision", "Eliminate stutter pauses with strict 98% accuracy thresholds.", "Deliberate movement precedes true velocity. Calm hands strike cleanly.", 78.0, 98.0, 3, "All Standard Keys"),

    ('expert', 1, "Professional Workplace Execution", "professional", "Simulated office memos, executive briefings, and technical reports.", "Please review the quarterly engineering deployment roadmap attached.", 95.0, 96.0, 3, "Office Vocabulary"),
```

```
    ('expert', 2, "Grandmaster Endurance Sprint", "grandmaster", "Sustained high-velocity paragraphs demanding flawless cognitive stamina.",
"Synchronized kinetic movement compounds into unstoppable tactile velocity under intense pressure.", 105.0, 97.0, 3, "Mastery Corpus")
```

```
]
```

```
for track, num, title, slug, desc, practice, min_w, min_a, req_att, f_keys in curriculum_data:
```

```
    stage = LessonStage.query.filter_by(slug=slug).first()
```

```
    if not stage:
```

```
        stage = LessonStage(

            track=track,

            stage_number=num,

            title=title,

            slug=slug,

            description=desc,

            practice_material=practice,

            min_wpm_to_pass=min_w,

            min_accuracy_to_pass=min_a,

            required_attempts=req_att,

            focus_keys=f_keys

        )

        db.session.add(stage)
```

5. Seed Arcade Game Configs

```
print("🎮 Seeding 12 Arcade Game Configs & Content Items...")
```

```
game_configs = [

    ('speed_racer', 'Speed Racer', 'Control a live vehicle on an open race track. Correct strokes accelerate your car forward; typos apply
backward recoil without halting momentum.', True, True, True, 4, "60,120,300", "moderate"),

    ('falling_words', 'Falling Words', 'Target descending words before they cross the defense laser.', True, True, False, 2, "60,120", "moderate"),

    ('bubble_pop', 'Bubble Pop', 'Floating bubbles contain single isolated target characters.', True, True, False, 2, "30,60,120", "moderate"),

    ('whack_a_word', 'Whack-A-Word', '3x3 grid where targets pop up for brief exposure windows.', True, True, False, 2, "30,60,90",
"moderate"),

    ('zombie_defense', 'Zombie Defense', 'Incoming hordes march towards your base barrier. Type overhead words to eliminate them.', True,
False, False, 1, "60,120", "moderate"),

    ('zombie_duel', 'Zombie Duel', '1v1 combat duel. Fast keystrokes launch attack spells; typos break defensive shields.', True, True, False, 2,
"60,120", "moderate"),

    ('cipher_hacker', 'Cipher Hacker', 'Progressive security layers. Decrypt hexadecimal, alphanumeric tokens, and code syntax.', True, True,
False, 2, "60,120", "hard"),
```

```
    ('space_defender', 'Space Defender', 'Defend your starship from incoming asteroids in all orbital quadrants.', True, True, False, 2,
"45,60,120", "moderate"),
```

```
    ('bomb_defuse', 'Bomb Defuse', 'Defuse time bombs with sequential multi-character cryptograms.', True, True, False, 2, "40,60", "hard"),
```

```
    ('typing_ninja', 'Typing Ninja', 'Slice across floating words as they launch into the air.', True, True, False, 2, "45,60", "moderate"),
```

```
    ('memory_type', 'Memory Type', 'Sequences flash for 1.5 seconds and disappear. Type from mental recall.', True, True, False, 2, "60",
"hard"),
```

```
    ('keyboard_quest', 'Keyboard Quest', 'Ergonomic stage map training home-row, upper reaches, bottom tucks, and numbers.', True, False,
False, 1, "60", "easy")
```

```
]
```

```
for slug, title, desc, s_ai, a_1v1, a_mp, max_p, durs, def_ai in game_configs:
```

```
    cfg = ArcadeGameConfig.query.filter_by(game_slug=slug).first()
```

```
    if not cfg:
```

```
        cfg = ArcadeGameConfig(

            game_slug=slug,

            display_title=title,

            description=desc,

            allows_solo_ai=s_ai,

            allows_1v1=a_1v1,

            allows_multiplayer=a_mp,

            max_multiplayer_players=max_p,

            durations=durs,

            default_ai_level=def_ai,

            is_enabled=True

        )

        db.session.add(cfg)
```

6. Seed Complete Typing Content Library

```
print("📖 Seeding Complete Content Library (Literature, Tech, Science, Business, Code)...")
```

```
passages_catalog = [
```

```
    ("The Morning Walk", "Daily Life", "Easy", False, None,
```

"The sun was up early and the sky was clear and bright. A small bird sat on the high green tree and sang a sweet song. The cool morning air made the walk pleasant and relaxing. People walked their dogs along the quiet path near the pond while children laughed and played with a red ball."),

```
    ("The Small Town Baker", "Daily Life", "Easy", False, None,
```

"Every morning before dawn the baker would turn on the lights in his warm shop. He made fresh bread with flour yeast water and salt. The sweet smell of warm loaves filled the whole street. Neighbors stopped by on their way to work to buy fresh rolls and say hello with a smile."),

("Learning to Read", "Education", "Easy", False, None,

"A good book can take you to places you have never seen before. When you open a page you can travel to high mountains deep blue oceans or distant green forests. Reading every day helps your mind grow strong and makes your imagination fly."),

("Pride and Prejudice Excerpt", "Literature", "Medium", False, None,

"It is a truth universally acknowledged, that a single man in possession of a good fortune, must be in want of a wife. However little known the feelings or views of such a man may be on his first entering a neighbourhood, this truth is so well fixed in the minds of the surrounding families, that he is considered as the rightful property of some one or other of their daughters."),

("A Tale of Two Cities", "Literature", "Medium", False, None,

"It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of light, it was the season of darkness, it was the spring of hope, it was the winter of despair."),

("Cloud Computing Fundamentals", "Technology", "Medium", False, None,

"Distributed computing architecture enables modern web applications to dynamically scale compute resources based on incoming request volume. Load balancers distribute user traffic evenly across independent server instances, ensuring high availability and fault tolerance even during hardware failures."),

("The Structure of DNA", "Science", "Medium", False, None,

"Deoxyribonucleic acid is the hereditary material in humans and almost all other living organisms. The information stored in DNA is encoded in a sequence of four chemical bases: adenine, guanine, cytosine, and thymine. These bases pair up with each other to form the rungs of a twisted ladder."),

("The Printing Press Revolution", "History", "Medium", False, None,

"Johannes Gutenberg introduced movable metal type printing to Europe around fourteen forty, transforming the dissemination of knowledge forever. Books that previously required months of manual copying by scribes could now be reproduced rapidly, accelerating the Renaissance and scientific discovery."),

("Quarterly Financial Review", "Business", "Hard", False, None,

"During the fourth fiscal quarter of 2025, operating revenue increased by 14.8% year-over-year to \$42.6 million. However, administrative expenditures rose sharply due to localized inflation, regulatory compliance auditing, and accelerated software capitalization. The board recommended a quarterly dividend of \$0.45 per share."),

("Biomechanical Ergonomics", "Science", "Hard", False, None,

"Repetitive strain injuries often manifest when typists maintain static isometric contraction of the forearm extensor muscles. Ergonomic keyboards designed with split key clusters, tenting angles between 10 and 15 degrees, and low actuation-force mechanical switches substantially reduce carpal tunnel hydrostatic pressure."),

("Decoupled Message Queues", "Technology", "Hard", False, None,

"Asynchronous event-driven architectures decouple producers from consumers using partitioned commit logs. By enforcing strict message ordering per partition key, streaming platforms guarantee that high-throughput downstream workers process state updates with at-least-once delivery semantics."),

("Legal Jurisprudence Memo", "Professional", "Hard", False, None,

"Pursuant to Section 104(b) of the revised commercial code, the plaintiff must establish prima facie evidence that the contractual indemnification clause was both procedurally and substantively unconscionable at the time of execution. Laches and equitable estoppel arguments remain unavailing."),

("Phenomenological Inquiry", "Education", "Expert", False, None,

"Phenomenology seeks an epochal suspension of the natural attitude, interrogating the intentionality that constitutes consciousness toward transcendental objects. Such transcendental bracketing reveals that subjectivity is inextricably bound to the spatio-temporal horizon of lived experience, precluding naive empirical reductionism."),

("Quantum Computing Paradigms", "Science", "Expert", False, None,

"Superconducting qubits utilize Josephson junctions to create non-linear dissipationless inductance, establishing discrete macroscopic quantum eigenstates. Mitigating phase decoherence and environmental noise requires topological braiding or fault-tolerant surface codes operating at sub-millikelvin temperatures."),

("Python Quicksort Algorithm", "Technology", "Medium", True, "python",

```
"def quicksort(arr):\n    if len(arr) <= 1:\n        return arr\n    pivot = arr[len(arr) // 2]\n    left = [x for x in arr if x < pivot]\n    middle = [x for x\nin arr if x == pivot]\n    right = [x for x in arr if x > pivot]\n    return quicksort(left) + middle + quicksort(right)",
```

("JavaScript Async Debounce", "Technology", "Medium", True, "javascript",

```
"const debounce = (func, delay = 300) => {\n    let timerId;\n    return (...args) => {\n        clearTimeout(timerId);\n        timerId = setTimeout(() =>\nfunc.apply(this, args), delay);\n    }; \n};",
```

("SQL Analytical Window Function", "Business", "Hard", True, "sql",

```
"SELECT department_id, employee_name, salary,\n    AVG(salary) OVER(PARTITION BY department_id) as dept_avg_salary,\n    RANK()\nOVER(PARTITION BY department_id ORDER BY salary DESC) as salary_rank\nFROM corporate_payroll\nWHERE employment_status = 'ACTIVE';")\n]
```

for title, cat, diff, is_code, code_lang, content in passages_catalog:

existing = TypingText.query.filter_by(content=content).first()

if not existing:

```
t_item = TypingText(\n\n    title=title,\n\n    category=cat,\n\n    difficulty=diff,\n\n    content=content,\n\n    is_code=is_code,\n\n    code_lang=code_lang,\n\n    is_active=True,\n\n    created_by="System"
```

```

    )

    t_item.calculate_stats()

    db.session.add(t_item)

# 7. Seed Daily Challenge

print("🌱 Seeding Daily Challenge...")

today = date.today()

daily = DailyChallenge.query.filter_by(target_date=today).first()

if not daily:

    daily = DailyChallenge(

        target_date=today,

        title="The Kinetic Discipline",

        content="True velocity is not rushed chaos; it is calm, deliberate movement free of hesitation and unnecessary recoil. Keep your hands balanced and let cadence carry your speed."

    )

    db.session.add(daily)

# 8. Seed Platform Parameters

print("📦 Seeding Platform Parameters...")

configs = [

    ('maintenance_mode', 'false', 'general', 'Master toggle for system maintenance'),

    ('allow_registrations', 'true', 'auth', 'Enables new user registration'),

    ('default_test_duration', '60', 'typing', 'Default timed test duration in seconds'),

    ('min_wpm_cutoff', '250', 'security', 'Anti-cheat upper bound for human velocity')

]

for key, val, cat, desc in configs:

    if not PlatformConfig.query.filter_by(key=key).first():

        db.session.add(PlatformConfig(key=key, value=val, category=cat, description=desc, updated_by="System"))

# 9. Commit All Seed Data

db.session.commit()

print("\n" + "=" * 65)

print("✅ SUCCESS: TypeSphere Master Database Populated Cleanly!")

```

```

print("  • Accounts: admin (super_admin) & speeddemon (verified)")

print("  • Achievements: 10 Core Badges")

print("  • Academy: 12 Stages across 4 Skill Tracks")

print("  • Arcade: 12 Playable Games Configured")

print("  • Content: 18 Multi-Tier Passages (Literature, Tech, Science, Business, Code)")

print("  • Daily Challenge & System Parameters: Initialized")

print("=" * 65 + "\n")

```

```

if __name__ == '__main__':

```

```

    seed_database()

```

```

    sys.exit(0)

```

```

=== FILE: \app\__init__.py ===

```

```

from flask import Flask

```

```

from flask_sqlalchemy import SQLAlchemy

```

```

from flask_login import LoginManager

```

```

from flask_migrate import Migrate

```

```

from flask_socketio import SocketIO

```

```

from sqlalchemy import text

```

```

from config import Config

```

```

# Instantiate database and extensions FIRST

```

```

db = SQLAlchemy()

```

```

login_manager = LoginManager()

```

```

migrate = Migrate()

```

```

socketio = SocketIO(cors_allowed_origins="*", async_mode='threading')

```

```

def create_app(config_class=Config):

```

```

    app = Flask(__name__)

```

```

    app.config.from_object(config_class)

```

```

    db.init_app(app)

```

```

    login_manager.init_app(app)

```

```

migrate.init_app(app, db)

socketio.init_app(app)


login_manager.login_view = 'auth.login'

login_manager.login_message_category = 'info'


# Register all database models before create_all

from app.models.user import User

from app.models.typing import TypingTest, TypingText, TypingDNA

from app.models.challenge import DailyChallenge, Achievement, UserAchievement

from app.models.settings import UserSettings

from app.models.feedback import ContactMessage, FeedbackItem, RatingReview, SiteSetting, Announcement

from app.models.curriculum import LessonStage, UserLessonProgress

from app.models.content import ContentItem, ExamTemplate

from app.models.game import GameRecord, ArcadeLeaderboard

from app.models.arcade_content import ArcadeContentItem, ArcadeGameConfig

from app.models.plan import UserSubscription

from app.models.admin import RolePermission, AdminAuditLog, UserActivity, VisitorTraffic, SecurityEvent, PlatformConfig


@login_manager.user_loader

def load_user(user_id):

    return User.query.get(int(user_id))


# Auto-patch missing tables & columns for SQLite databases

with app.app_context():

    db.create_all()

    try:

        with db.engine.connect() as conn:

            if 'sqlite' in str(db.engine.url):

                res = conn.execute(text("PRAGMA table_info(user_settings)"))

                cols = [row[1] for row in res.fetchall()]

                if cols and 'blind_mode' not in cols:

                    conn.execute(text("ALTER TABLE user_settings ADD COLUMN blind_mode BOOLEAN DEFAULT 1"))

```

```

if cols and 'ghost_mode' not in cols:

    conn.execute(text("ALTER TABLE user_settings ADD COLUMN ghost_mode BOOLEAN DEFAULT 0"))

    conn.commit()

except Exception:

    pass


# Register Blueprints

from app.routes.auth import auth_bp

from app.routes.typing import typing_bp

from app.routes.dashboard import dashboard_bp

from app.routes.leaderboard import leaderboard_bp

from app.routes.challenges import challenges_bp

from app.routes.multiplayer import multiplayer_bp

from app.routes.admin import admin_bp

from app.routes.settings import settings_bp

from app.routes.feedback import feedback_bp

from app.routes.learn import learn_bp

from app.routes.games import games_bp


app.register_blueprint(auth_bp, url_prefix='/auth')

app.register_blueprint(typing_bp, url_prefix='/typing')

app.register_blueprint(dashboard_bp, url_prefix='/dashboard')

app.register_blueprint(leaderboard_bp, url_prefix='/leaderboard')

app.register_blueprint(challenges_bp, url_prefix='/challenges')

app.register_blueprint(multiplayer_bp, url_prefix='/multiplayer')

app.register_blueprint(admin_bp, url_prefix='/admin')

app.register_blueprint(settings_bp, url_prefix='/settings')

app.register_blueprint(feedback_bp, url_prefix='/feedback')

app.register_blueprint(learn_bp, url_prefix='/learn')

app.register_blueprint(games_bp, url_prefix='/games')


# Register traffic & telemetry analytics hooks

from app.services.admin_security import capture_traffic

```



```
capture_traffic(app)

# Register Arcade real-time multiplayer sockets

from app.services.game_socket_engine import register_arcade_socket_events

register_arcade_socket_events()

# Root route and /typing/ share the same home test page

@app.route('/')

def root_home():

    from app.routes.typing import test_page

    return test_page()

return app

=== FILE: .\app\models\achievement.py ===

=== FILE: .\app\models\admin.py ===

from datetime import datetime

from app import db

class RolePermission:

    """Explicit permission mappings for RBAC."""

    SUPER_ADMIN = 'super_admin'

    ADMIN = 'admin'

    MODERATOR = 'moderator'

    CONTENT_MANAGER = 'content_manager'

    ANALYST = 'analyst'

    ALL_ROLES = [SUPER_ADMIN, ADMIN, MODERATOR, CONTENT_MANAGER, ANALYST]

    HIERARCHY = {

        SUPER_ADMIN: ['*'],
```

```
ADMIN: [

    'view_dashboard', 'manage_users', 'manage_content', 'manage_games',

    'manage_leaderboard', 'manage_challenges', 'manage_feedback',

    'manage_learning', 'view_analytics', 'view_security', 'view_audit_logs',

    'export_data', 'manage_settings'

],

MODERATOR: [

    'view_dashboard', 'manage_users_basic', 'manage_leaderboard',

    'manage_feedback', 'view_audit_logs'

],

CONTENT_MANAGER: [

    'view_dashboard', 'manage_content', 'manage_challenges', 'manage_learning'

],

ANALYST: [

    'view_dashboard', 'view_analytics', 'view_traffic', 'export_data'

]

}

@classmethod

def has_permission(cls, role, permission):

    if role == cls.SUPER_ADMIN:

        return True

    allowed = cls.HIERARCHY.get(role, [])

    return permission in allowed or '*' in allowed

class AdminAuditLog(db.Model):

    __tablename__ = 'admin_audit_logs'

    id = db.Column(db.Integer, primary_key=True)

    admin_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True, index=True)

    admin_username = db.Column(db.String(64), nullable=False)

    action = db.Column(db.String(128), nullable=False, index=True) # e.g. "USER_SUSPEND", "PASSAGE_DELETE"

    target_type = db.Column(db.String(64), nullable=False, index=True) # "user", "passage", "setting", etc.
```

```
target_id = db.Column(db.String(64), nullable=True)

details = db.Column(db.Text, nullable=True) # JSON or descriptive string of changes

ip_address = db.Column(db.String(45), nullable=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow, index=True)
```

```
admin = db.relationship('User', foreign_keys=[admin_id])
```

```
class UserActivity(db.Model):
```

```
    __tablename__ = 'user_activities'
```

```
id = db.Column(db.Integer, primary_key=True)

user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=True, index=True)

action = db.Column(db.String(64), nullable=False, index=True) # 'LOGIN', 'TEST_COMPLETE', etc.

feature = db.Column(db.String(64), default='Platform')

details = db.Column(db.String(255), nullable=True)

ip_address = db.Column(db.String(45), nullable=True)

user_agent = db.Column(db.String(255), nullable=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow, index=True)
```

```
user = db.relationship('User', backref=db.backref('activities', lazy='dynamic', cascade='all, delete-orphan'))
```

```
class VisitorTraffic(db.Model):
```

```
    __tablename__ = 'visitor_traffic'
```

```
id = db.Column(db.Integer, primary_key=True)

session_id = db.Column(db.String(64), nullable=False, index=True)

user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True, index=True)

ip_address = db.Column(db.String(45), nullable=True)

path = db.Column(db.String(255), nullable=False, index=True)

method = db.Column(db.String(10), default='GET')

referrer = db.Column(db.String(255), nullable=True)

browser = db.Column(db.String(64), nullable=True)

os = db.Column(db.String(64), nullable=True)
```

```
device_type = db.Column(db.String(32), default='desktop') # 'desktop', 'mobile', 'tablet'

status_code = db.Column(db.Integer, default=200)

created_at = db.Column(db.DateTime, default=datetime.utcnow, index=True)
```

```
class SecurityEvent(db.Model):
```

```
    __tablename__ = 'security_events'

id = db.Column(db.Integer, primary_key=True)

event_type = db.Column(db.String(64), nullable=False, index=True) # 'FAILED_LOGIN', 'ABUSE_FLAG', 'IP_BLOCK'

severity = db.Column(db.String(20), default='LOW', index=True) # 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'

identifier = db.Column(db.String(128), nullable=True) # email or username or token

ip_address = db.Column(db.String(45), nullable=True)

user_agent = db.Column(db.String(255), nullable=True)

description = db.Column(db.Text, nullable=False)

created_at = db.Column(db.DateTime, default=datetime.utcnow, index=True)
```

```
class PlatformConfig(db.Model):
```

```
    __tablename__ = 'platform_configs'

id = db.Column(db.Integer, primary_key=True)

key = db.Column(db.String(64), unique=True, nullable=False, index=True)

value = db.Column(db.Text, nullable=False)

category = db.Column(db.String(32), default='general', index=True)

description = db.Column(db.String(255), nullable=True)

updated_by = db.Column(db.String(64), nullable=True)

updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

```
=== FILE: .\app\models\arcade_content.py ===
```

```
from datetime import datetime
```

```
from app import db
```

```
class ArcadeContentItem(db.Model):
```

```
    __tablename__ = 'arcade_content_items'
```

```
id = db.Column(db.Integer, primary_key=True)

game_mode = db.Column(db.String(50), nullable=False, index=True)

difficulty = db.Column(db.String(20), default='moderate', index=True) # easy, moderate, hard, expert

target_text = db.Column(db.String(255), nullable=False)

category = db.Column(db.String(50), default='general', index=True)

associated_keys = db.Column(db.String(64), nullable=True)

word_length = db.Column(db.Integer, default=5)

is_active = db.Column(db.Boolean, default=True, index=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
class ArcadeGameConfig(db.Model):

    __tablename__ = 'arcade_game_configs'

    id = db.Column(db.Integer, primary_key=True)

    game_slug = db.Column(db.String(50), unique=True, nullable=False)

    display_title = db.Column(db.String(100), nullable=False)

    description = db.Column(db.Text, nullable=False)

    allows_solo_ai = db.Column(db.Boolean, default=True)

    allows_1v1 = db.Column(db.Boolean, default=True)

    allows_multiplayer = db.Column(db.Boolean, default=False)

    max_multiplayer_players = db.Column(db.Integer, default=2)

    durations = db.Column(db.String(50), default="60")

    default_ai_level = db.Column(db.String(20), default="moderate")

    is_enabled = db.Column(db.Boolean, default=True)
```

```
=== FILE: .\app\models\challenge.py ===

from datetime import date, datetime

from app import db
```

```
class DailyChallenge(db.Model):

    __tablename__ = 'daily_challenges'
```

```
id = db.Column(db.Integer, primary_key=True)

target_date = db.Column(db.Date, default=date.today, unique=True, index=True)

title = db.Column(db.String(128), default='Daily Typing Mastery')

content = db.Column(db.Text, nullable=False)

created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
class Achievement(db.Model):

    __tablename__ = 'achievements'

    id = db.Column(db.Integer, primary_key=True)

    code = db.Column(db.String(64), unique=True, nullable=False)

    title = db.Column(db.String(128), nullable=False)

    description = db.Column(db.String(256), nullable=False)

    icon = db.Column(db.String(64), default='trophy')
```

```
class UserAchievement(db.Model):

    __tablename__ = 'user_achievements'

    id = db.Column(db.Integer, primary_key=True)

    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    achievement_id = db.Column(db.Integer, db.ForeignKey('achievements.id'), nullable=False)

    unlocked_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
achievement = db.relationship('Achievement')
```

```
=== FILE: .\app\models\content.py ===

from datetime import datetime

from app import db
```

```
class ContentItem(db.Model):

    __tablename__ = 'content_items'

    id = db.Column(db.Integer, primary_key=True)
```

```
title = db.Column(db.String(150), nullable=True)

category = db.Column(db.String(64), nullable=False, index=True)

# 'words', 'sentences', 'paragraphs', 'quotes', 'stories', 'realworld', 'data_entry', 'numbers', 'punctuation', 'coding', 'exam'

subcategory = db.Column(db.String(64), nullable=True, index=True)

# e.g., 'emails', 'invoices', 'python', 'ssc_cgl', 'court_steno'

difficulty = db.Column(db.String(32), default='moderate', index=True) # 'easy', 'moderate', 'hard', 'expert'
```

```
body = db.Column(db.Text, nullable=False)

source_author = db.Column(db.String(128), nullable=True)
```

```
# Quantitative Lexical Metadata

word_count = db.Column(db.Integer, default=0)

character_count = db.Column(db.Integer, default=0)

punctuation_density = db.Column(db.Float, default=0.0) # percentage of punct chars

numeric_density = db.Column(db.Float, default=0.0) # percentage of digits

avg_word_length = db.Column(db.Float, default=0.0)

complexity_score = db.Column(db.Float, default=1.0) # 1.0 to 5.0 difficulty rating
```

```
# Premium-Ready Feature Flags (All initially completely accessible)

is_premium_exclusive = db.Column(db.Boolean, default=False, index=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
def calculate_metrics(self):

    chars = len(self.body)

    words = self.body.split()

    num_words = len(words) if words else 1

    punct_count = sum(1 for c in self.body if c in "!\"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~")

    num_count = sum(1 for c in self.body if c.isdigit())

    total_letters = sum(len(w) for w in words)

    self.character_count = chars

    self.word_count = num_words
```

```
self.punctuation_density = round((punct_count / chars) * 100.0, 2) if chars else 0.0

self.numeric_density = round((num_count / chars) * 100.0, 2) if chars else 0.0

self.avg_word_length = round(total_letters / num_words, 2) if num_words else 0.0
```

```
class ExamTemplate(db.Model):

    __tablename__ = 'exam_templates'

    id = db.Column(db.Integer, primary_key=True)

    exam_code = db.Column(db.String(50), unique=True, nullable=False) # 'ssc_chsl', 'rrb_ntpc', 'court_clerk'

    exam_title = db.Column(db.String(150), nullable=False)

    description = db.Column(db.Text, nullable=True)

    duration_seconds = db.Column(db.Integer, default=600) # e.g. 10 or 15 mins

    required_wpm = db.Column(db.Float, default=35.0)

    max_error_percentage = db.Column(db.Float, default=5.0) # Strict government threshold

    allow_backspace = db.Column(db.Boolean, default=True)

    strict_case_match = db.Column(db.Boolean, default=True)

    content = db.Column(db.Text, nullable=False)
```

```
=== FILE: .\app\models\curriculum.py ===

from datetime import datetime

from app import db
```

```
class LessonStage(db.Model):

    __tablename__ = 'lesson_stages'

    __table_args__ = (

        db.UniqueConstraint('track', 'stage_number', name='uix_track_stage'),

    )

    id = db.Column(db.Integer, primary_key=True)

    track = db.Column(db.String(32), default='beginner', index=True) # 'beginner', 'intermediate', 'advanced', 'expert'

    stage_number = db.Column(db.Integer, nullable=False) # Sequence within track
```

```
title = db.Column(db.String(120), nullable=False)

slug = db.Column(db.String(64), unique=True, nullable=False)

description = db.Column(db.Text, nullable=False)


# Ergonomic Guidance

focus_keys = db.Column(db.String(64), nullable=True) # e.g., 'F J D K'

hand_position_hint = db.Column(db.String(128), nullable=True)

keyboard_row = db.Column(db.String(32), default='home') # 'home', 'top', 'bottom', 'numbers', 'symbols'


practice_material = db.Column(db.Text, nullable=False)

min_wpm_to_pass = db.Column(db.Float, default=20.0)

min_accuracy_to_pass = db.Column(db.Float, default=95.0)

required_attempts = db.Column(db.Integer, default=2) # Multi-attempt verification
```

```
class UserLessonProgress(db.Model):

    __tablename__ = 'user_lesson_progress'


    id = db.Column(db.Integer, primary_key=True)

    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    stage_id = db.Column(db.Integer, db.ForeignKey('lesson_stages.id'), nullable=False)


    completed = db.Column(db.Boolean, default=False)

    successful_attempts = db.Column(db.Integer, default=0)

    best_wpm = db.Column(db.Float, default=0.0)

    best_accuracy = db.Column(db.Float, default=0.0)

    total_attempts = db.Column(db.Integer, default=0)

    unlocked_at = db.Column(db.DateTime, default=datetime.utcnow)

    completed_at = db.Column(db.DateTime, nullable=True)


    stage = db.relationship('LessonStage')

    user = db.relationship('User', backref=db.backref('curriculum_progress', lazy='dynamic'))
```

```
=== FILE: .\app\models\feedback.py ===

from datetime import datetime

from app import db


class ContactMessage(db.Model):

    __tablename__ = 'contact_messages'


    id = db.Column(db.Integer, primary_key=True)

    name = db.Column(db.String(100), nullable=False)

    email = db.Column(db.String(120), nullable=False)

    subject = db.Column(db.String(200), nullable=False)

    message = db.Column(db.Text, nullable=False)

    is_read = db.Column(db.Boolean, default=False, index=True)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
class FeedbackItem(db.Model):

    __tablename__ = 'feedback_items'


    id = db.Column(db.Integer, primary_key=True)

    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    category = db.Column(db.String(50), default='General') # 'Bug', 'Feature', 'UI/UX', 'Typing Test', 'General'

    title = db.Column(db.String(150), nullable=False)

    message = db.Column(db.Text, nullable=False)

    status = db.Column(db.String(30), default='Pending') # 'Pending', 'Under Review', 'Resolved', 'Closed'

    admin_response = db.Column(db.Text, nullable=True)

    created_at = db.Column(db.DateTime, default=datetime.utcnow)


    user = db.relationship('User', backref=db.backref('feedback_submissions', lazy='dynamic'))
```

```
class RatingReview(db.Model):

    __tablename__ = 'rating_reviews'


    id = db.Column(db.Integer, primary_key=True)
```

```
user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False, unique=True)

rating = db.Column(db.Integer, nullable=False) # 1 to 5 Stars

review_title = db.Column(db.String(150), nullable=False)

review_text = db.Column(db.Text, nullable=False)

is_approved = db.Column(db.Boolean, default=True, index=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow)

updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

```
user = db.relationship('User', backref=db.backref('rating_review', uselist=False))
```

```
class SiteSetting(db.Model):
```

```
    __tablename__ = 'site_settings'
```

```
id = db.Column(db.Integer, primary_key=True)

setting_key = db.Column(db.String(64), unique=True, nullable=False, index=True)

setting_value = db.Column(db.Text, nullable=False)

description = db.Column(db.String(255), nullable=True)
```

```
class Announcement(db.Model):
```

```
    __tablename__ = 'announcements'
```

```
id = db.Column(db.Integer, primary_key=True)

title = db.Column(db.String(200), nullable=False)

content = db.Column(db.Text, nullable=False)

is_active = db.Column(db.Boolean, default=True, index=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
=== FILE: .\app\models\game.py ===
```

```
from datetime import datetime
```

```
from app import db
```

```
class GameRecord(db.Model):
```

```
    __tablename__ = 'game_records'
```

```
id = db.Column(db.Integer, primary_key=True)
```

```
user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=True, index=True)
```

```
game_mode = db.Column(db.String(50), nullable=False, index=True)
```

```
play_type = db.Column(db.String(30), default='solo_ai') # solo_practice, solo_ai, 1v1_private, multiplayer
```

```
result_outcome = db.Column(db.String(20), default='FINISHED') # WIN, LOSS, DRAW, FINISHED
```

```
score = db.Column(db.Integer, default=0, index=True)
```

```
net_wpm = db.Column(db.Float, default=0.0)
```

```
accuracy = db.Column(db.Float, default=100.0)
```

```
errors = db.Column(db.Integer, default=0)
```

```
highest_combo = db.Column(db.Integer, default=0)
```

```
average_reaction_ms = db.Column(db.Float, default=0.0)
```

```
meta_stats_json = db.Column(db.Text, default='{}')
```

```
duration_seconds = db.Column(db.Float, default=0.0)
```

```
created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
user = db.relationship('User', backref=db.backref('game_records', lazy='dynamic'))
```

```
class ArcadeLeaderboard(db.Model):
```

```
    __tablename__ = 'arcade_leaderboards'
```

```
id = db.Column(db.Integer, primary_key=True)
```

```
user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
```

```
game_mode = db.Column(db.String(50), nullable=False, index=True)
```

```
high_score = db.Column(db.Integer, default=0, index=True)
```

```
best_wpm = db.Column(db.Float, default=0.0)
```

```
best_accuracy = db.Column(db.Float, default=0.0)
```

```
updated_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
user = db.relationship('User')
```

=== FILE: .\app\models\plan.py ===

from datetime import datetime

from app import db

class UserSubscription(db.Model):

"""

Extensible access control layer. All users initially default to 'free' with full capability.

"""

__tablename__ = 'user_subscriptions'

id = db.Column(db.Integer, primary_key=True)

user_id = db.Column(db.Integer, db.ForeignKey('users.id'), unique=True, nullable=False)

tier = db.Column(db.String(32), default='free', nullable=False) # 'free', 'pro', 'enterprise'

is_active = db.Column(db.Boolean, default=True)

Feature Toggles (All unlocked initially)

ai_coach_unlimited = db.Column(db.Boolean, default=True)

unlimited_multiplayer = db.Column(db.Boolean, default=True)

official_certification = db.Column(db.Boolean, default=True)

custom_themes_access = db.Column(db.Boolean, default=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow)

expires_at = db.Column(db.DateTime, nullable=True)

user = db.relationship('User', backref=db.backref('subscription', uselist=False))

=== FILE: .\app\models\settings.py ===

from app import db

class UserSettings(db.Model):

__tablename__ = 'user_settings'

id = db.Column(db.Integer, primary_key=True)

user_id = db.Column(db.Integer, db.ForeignKey('users.id'), unique=True, nullable=False)

Appearance & Ergonomics

theme = db.Column(db.String(32), default='dark') # dark, light, cyber, sepia

font_family = db.Column(db.String(64), default='JetBrains Mono')

font_size = db.Column(db.Integer, default=22)

caret_style = db.Column(db.String(32), default='smooth') # smooth, block, underline

sound_enabled = db.Column(db.Boolean, default=True)

sound_theme = db.Column(db.String(32), default='mechanical')

sound_volume = db.Column(db.Float, default=0.7)

show_keyboard = db.Column(db.Boolean, default=True)

Initial Test Defaults (Blind Mode: ON, Ghost Mode: OFF)

default_duration = db.Column(db.Integer, default=60) # 60s, 120s, 300s, 600s, 1200s, 0 (No Limit)

default_content = db.Column(db.String(32), default='words') # words, quotes, numbers, punctuation, mixed, code, custom

default_level = db.Column(db.String(32), default='moderate') # easy, moderate, hard, expert

blind_mode = db.Column(db.Boolean, default=True) # Blind Mode ON by default

ghost_mode = db.Column(db.Boolean, default=False) # Ghost Mode OFF by default

=== FILE: .\app\models\typing.py ===

from datetime import datetime

import json

from app import db

class TypingTest(db.Model):

__tablename__ = 'typing_tests'

id = db.Column(db.Integer, primary_key=True)

user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=True, index=True)

Mode & Ranked Classification

mode = db.Column(db.String(32), default='timed') # 'timed', 'words', 'quote', 'custom', 'code', etc.

is_ranked = db.Column(db.Boolean, default=False, index=True) # True only for verified standard runs

```
content_category = db.Column(db.String(64), default='General')

difficulty = db.Column(db.String(32), default='moderate') # 'easy', 'moderate', 'hard', 'expert'
```

```
# Core Telemetry

duration = db.Column(db.Float, nullable=False) # In seconds

wpm = db.Column(db.Float, nullable=False, index=True) # Net WPM

raw_wpm = db.Column(db.Float, nullable=False) # Gross WPM

accuracy = db.Column(db.Float, nullable=False, index=True)

consistency = db.Column(db.Float, nullable=False)

rhythm_score = db.Column(db.Float, default=100.0)
```

```
# Keystroke Breakdown

errors = db.Column(db.Integer, default=0)

correct_chars = db.Column(db.Integer, default=0)

incorrect_chars = db.Column(db.Integer, default=0)

extra_chars = db.Column(db.Integer, default=0)

missed_chars = db.Column(db.Integer, default=0)
```

```
# Integrity & Anti-Cheat

suspicious = db.Column(db.Boolean, default=False, index=True)

suspicion_reason = db.Column(db.String(255), nullable=True)

completed_at = db.Column(db.DateTime, default=datetime.utcnow, index=True)
```

```
# Serialized Keystroke Log & WPM Timeline for Replays

timeline_data = db.Column(db.Text, nullable=True)

events_data = db.Column(db.Text, nullable=True)
```

```
def get_timeline(self):

    return json.loads(self.timeline_data) if self.timeline_data else []
```

```
def get_events(self):

    return json.loads(self.events_data) if self.events_data else []
```

```
class TypingText(db.Model):

    __tablename__ = 'typing_texts'

    id = db.Column(db.Integer, primary_key=True)

    title = db.Column(db.String(128), default="Untitled Passage")

    category = db.Column(db.String(64), default='General', index=True)

    difficulty = db.Column(db.String(32), default='Medium', index=True) # Easy, Medium, Hard, Expert

    language = db.Column(db.String(32), default='english')

    content = db.Column(db.Text, nullable=False)

    source = db.Column(db.String(128), default='System')

    is_code = db.Column(db.Boolean, default=False)

    code_lang = db.Column(db.String(32), nullable=True)
```

```
# Content Metadata

word_count = db.Column(db.Integer, default=0)

character_count = db.Column(db.Integer, default=0)

complexity_score = db.Column(db.Float, default=1.0)

is_active = db.Column(db.Boolean, default=True, index=True)

created_at = db.Column(db.DateTime, default=datetime.utcnow)

updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

created_by = db.Column(db.String(64), default="System")

updated_by = db.Column(db.String(64), nullable=True)
```

```
def calculate_stats(self):

    words = self.content.strip().split()

    self.word_count = len(words)

    self.character_count = len(self.content.strip())
```

```
class TypingDNA(db.Model):

    __tablename__ = 'typing_dna'

    id = db.Column(db.Integer, primary_key=True)

    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), unique=True, nullable=False)
```



```

left_hand_accuracy = db.Column(db.Float, default=100.0)

right_hand_accuracy = db.Column(db.Float, default=100.0)

punctuation_accuracy = db.Column(db.Float, default=100.0)

numbers_accuracy = db.Column(db.Float, default=100.0)

capitals_accuracy = db.Column(db.Float, default=100.0)

rhythm_consistency_avg = db.Column(db.Float, default=100.0)


key_stats_json = db.Column(db.Text, default='{}')

confusion_matrix_json = db.Column(db.Text, default='{}')

updated_at = db.Column(db.DateTime, default=datetime.utcnow)


def get_key_stats(self):

    return json.loads(self.key_stats_json) if self.key_stats_json else {}


def get_confusion_matrix(self):

    return json.loads(self.confusion_matrix_json) if self.confusion_matrix_json else {}


=== FILE: .\app\models\user.py ===

from datetime import datetime

from werkzeug.security import generate_password_hash, check_password_hash

from flask_login import UserMixin

from app import db


class User(UserMixin, db.Model):

    __tablename__ = 'users'


    id = db.Column(db.Integer, primary_key=True)

    username = db.Column(db.String(64), unique=True, nullable=False, index=True)

    email = db.Column(db.String(120), unique=True, nullable=False, index=True)

    password_hash = db.Column(db.String(256), nullable=False)


    # Granular Roles: 'super_admin', 'admin', 'moderator', 'content_manager', 'analyst', 'user'

```

```

role = db.Column(db.String(32), default='user', nullable=False, index=True)

is_verified = db.Column(db.Boolean, default=True, nullable=True)


# Account Moderation States

is_suspended = db.Column(db.Boolean, default=False, index=True)

is_banned = db.Column(db.Boolean, default=False, index=True)

status_reason = db.Column(db.String(255), nullable=True)

last_active = db.Column(db.DateTime, default=datetime.utcnow)


# Version 2.0 Ranked Fields

elo_rating = db.Column(db.Integer, default=1000, nullable=False, index=True)

ranked_wins = db.Column(db.Integer, default=0, nullable=False)

ranked_losses = db.Column(db.Integer, default=0, nullable=False)


created_at = db.Column(db.DateTime, default=datetime.utcnow)


# Relationships

tests = db.relationship("TypingTest", backref='user', lazy='dynamic', cascade="all, delete-orphan")

dna_profile = db.relationship("TypingDNA", uselist=False, backref='user', cascade="all, delete-orphan")

user_settings = db.relationship("UserSettings", uselist=False, backref='user', cascade="all, delete-orphan")

achievements = db.relationship("UserAchievement", backref='user', lazy='dynamic', cascade="all, delete-orphan")


def set_password(self, password):

    self.password_hash = generate_password_hash(password)


def check_password(self, password):

    return check_password_hash(self.password_hash, password)


@property

def is_admin(self):

    return self.role in ['super_admin', 'admin', 'moderator', 'content_manager', 'analyst']


@property

```

```
def is_super_admin(self):

    return self.role == 'super_admin'

@property

def is_active_account(self):

    return not self.is_banned and not self.is_suspended

@property

def rank_division(self):

    elo = self.elo_rating or 1000

    if elo >= 1900: return "Grandmaster"

    if elo >= 1700: return "Diamond"

    if elo >= 1500: return "Platinum"

    if elo >= 1300: return "Gold"

    if elo >= 1100: return "Silver"

    return "Bronze"

@property

def rank_badge(self):

    division = self.rank_division

    badges = {

        "Grandmaster": {"icon": "♔", "color": "#ff00ff"},

        "Diamond": {"icon": "♕", "color": "#58a6ff"},

        "Platinum": {"icon": "♖", "color": "#3fb950"},

        "Gold": {"icon": "♗", "color": "#f59e0b"},

        "Silver": {"icon": "♘", "color": "#94a3b8"},

        "Bronze": {"icon": "♙", "color": "#b45309"}

    }

    return badges.get(division, {"icon": "♚", "color": "#b45309"})

=== FILE: .\app\models\__init__.py ===

from app.models.user import User

from app.models.typing import TypingTest, TypingText, TypingDNA
```

```
from app.models.challenge import DailyChallenge, Achievement, UserAchievement

from app.models.settings import UserSettings

from app.models.feedback import ContactMessage, FeedbackItem, RatingReview, SiteSetting, Announcement

__all__ = [

    'User',

    'TypingTest',

    'TypingText',

    'TypingDNA',

    'DailyChallenge',

    'Achievement',

    'UserAchievement',

    'UserSettings',

    'ContactMessage',

    'FeedbackItem',

    'RatingReview',

    'SiteSetting',

    'Announcement'

]

=== FILE: .\app\routes\admin.py ===

import io

import csv

import json

from datetime import datetime, timedelta

from flask import Blueprint, render_template, request, redirect, url_for, flash, jsonify, Response, session

from flask_login import login_required, current_user, login_user

from sqlalchemy import func, desc

from app import db

from app.models.user import User

from app.models.typing import TypingTest, TypingText

from app.models.challenge import DailyChallenge, Achievement

from app.models.feedback import ContactMessage, FeedbackItem, RatingReview, Announcement
```

```
from app.models.arcade_content import ArcadeContentItem, ArcadeGameConfig

from app.models.game import GameRecord

from app.models.admin import (

    RolePermission, AdminAuditLog, VisitorTraffic,

    SecurityEvent, UserActivity, PlatformConfig

)

from app.services.admin_security import admin_permission_required, log_admin_action


admin_bp = Blueprint('admin', __name__)


@admin_bp.before_request
@login_required
def enforce_admin_global():

    if not current_user.is_admin:

        flash("Unauthorized area. Access denied.", "danger")

        return redirect(url_for('typing.test_page'))


# =====

# 1. Admin Dashboard Overview

# =====

@admin_bp.route("")
@admin_bp.route('/')

@admin_permission_required('view_dashboard')

def dashboard():

    now = datetime.utcnow()

    today_start = datetime(now.year, now.month, now.day)

    week_start = now - timedelta(days=7)

    month_start = now - timedelta(days=30)


# User Metrics

total_users = User.query.count()

new_today = User.query.filter(User.created_at >= today_start).count()

new_week = User.query.filter(User.created_at >= week_start).count()
```

```
new_month = User.query.filter(User.created_at >= month_start).count()

suspended_users = User.query.filter((User.is_suspended == True) | (User.is_banned == True)).count()

active_now = User.query.filter(User.last_active >= (now - timedelta(minutes=15))).count()


# Typing Test Metrics

total_tests = TypingTest.query.count()

tests_today = TypingTest.query.filter(TypingTest.completed_at >= today_start).count()

avg_wpm = db.session.query(func.avg(TypingTest.wpm)).scalar() or 0

avg_acc = db.session.query(func.avg(TypingTest.accuracy)).scalar() or 0

max_wpm = db.session.query(func.max(TypingTest.wpm)).scalar() or 0

suspicious_tests = TypingTest.query.filter_by(suspicious=True).count()


# Engagement Metrics

games_played = GameRecord.query.count()

daily_challenges = DailyChallenge.query.count()

reviews_count = RatingReview.query.count()

unread_contacts = ContactMessage.query.filter_by(is_read=False).count()


# Traffic Metrics

total_pageviews = VisitorTraffic.query.count()

unique_visitors = db.session.query(func.count(func.distinct(VisitorTraffic.session_id))).scalar() or 0


# Recent Audit Log for Super Admin preview

recent_audits = AdminAuditLog.query.order_by(AdminAuditLog.created_at.desc()).limit(8).all()

recent_security = SecurityEvent.query.order_by(SecurityEvent.created_at.desc()).limit(5).all()


return render_template(

    'admin/dashboard.html',

    stats={

        'total_users': total_users,

        'new_today': new_today,

        'new_week': new_week,

        'new_month': new_month,
```

```

'suspended_users': suspended_users,

'active_now': active_now,

'total_tests': total_tests,

'tests_today': tests_today,

'avg_wpm': round(avg_wpm, 1),

'avg_acc': round(avg_acc, 1),

'max_wpm': round(max_wpm, 1),

'suspicious_tests': suspicious_tests,

'games_played': games_played,

'reviews_count': reviews_count,

'unread_contacts': unread_contacts,

'total_pageviews': total_pageviews,

'unique_visitors': unique_visitors
},

recent_audits=recent_audits,

recent_security=recent_security
)

```

```
# =====
```

2. User Management & Impersonation

```
# =====
```

```
@admin_bp.route('/users')
```

```
@admin_permission_required('manage_users')
```

```
def users_list():
```

```
    page = request.args.get('page', 1, type=int)
```

```
    search = request.args.get('q', "").strip()
```

```
    role_filter = request.args.get('role', "")
```

```
    status_filter = request.args.get('status', "")
```

```
    query = User.query
```

```
    if search:
```

```
        query = query.filter((User.username.ilike(f"%{search}%")) | (User.email.ilike(f"%{search}%")))
```

```
    if role_filter:
```

```
        query = query.filter_by(role=role_filter)
```

```
    if status_filter == 'suspended':
```

```
        query = query.filter_by(is_suspended=True)
```

```
    elif status_filter == 'banned':
```

```
        query = query.filter_by(is_banned=True)
```

```
    elif status_filter == 'active':
```

```
        query = query.filter_by(is_suspended=False, is_banned=False)
```

```
    users_page = query.order_by(User.id.desc()).paginate(page=page, per_page=20, error_out=False)
```

```
    return render_template(
```

```
        'admin/users.html',
```

```
        users=users_page,
```

```
        search=search,
```

```
        role_filter=role_filter,
```

```
        status_filter=status_filter,
```

```
        roles=RolePermission.ALL_ROLES
```

```
    )
```

```
@admin_bp.route('/users/<int:user_id>/profile')
```

```
@admin_permission_required('manage_users')
```

```
def user_detail(user_id):
```

```
    user = User.query.get_or_404(user_id)
```

```
    tests = TypingTest.query.filter_by(user_id=user.id).order_by(TypingTest.completed_at.desc()).limit(15).all()
```

```
    games = GameRecord.query.filter_by(user_id=user.id).order_by(GameRecord.created_at.desc()).limit(10).all()
```

```
    activities = UserActivity.query.filter_by(user_id=user.id).order_by(UserActivity.created_at.desc()).limit(20).all()
```

```
    return render_template(
```

```
        'admin/user_detail.html',
```

```
        user=user,
```

```
        tests=tests,
```

```
        games=games,
```

```

activities=activities,

roles=RolePermission.ALL_ROLES

)

@admin_bp.route('/users/<int:user_id>/update-role', methods=['POST'])

@admin_permission_required('manage_users')

def update_user_role(user_id):

    user = User.query.get_or_404(user_id)

    new_role = request.form.get('role')

    if user.id == current_user.id:

        flash("Modifying your own role is restricted.", "danger")

        return redirect(url_for('admin.user_detail', user_id=user.id))

    if current_user.role != RolePermission.SUPER_ADMIN and new_role == RolePermission.SUPER_ADMIN:

        flash("Only Super Administrators can promote to Super Admin.", "danger")

        return redirect(url_for('admin.user_detail', user_id=user.id))

    old_role = user.role

    user.role = new_role

    db.session.commit()

    log_admin_action('ROLE_CHANGE', 'user', user.id, f"Role changed from {old_role} to {new_role}")

    flash(f"User @{user.username} updated to role: {new_role.upper()}", "success")

    return redirect(url_for('admin.user_detail', user_id=user.id))

@admin_bp.route('/users/<int:user_id>/moderate', methods=['POST'])

@admin_permission_required('manage_users')

def moderate_user(user_id):

    user = User.query.get_or_404(user_id)

    action = request.form.get('action') # 'suspend', 'unsuspend', 'ban', 'unban', 'delete'

    reason = request.form.get('reason', 'Administrative moderation decision')

```

```

if user.id == current_user.id:

    flash("You cannot perform moderation actions on your own profile.", "danger")

    return redirect(url_for('admin.user_detail', user_id=user.id))

if action == 'suspend':

    user.is_suspended = True

    user.status_reason = reason

    log_admin_action('USER_SUSPEND', 'user', user.id, f"Suspended. Reason: {reason}")

    flash(f"User @{user.username} suspended.", "warning")

elif action == 'unsuspend':

    user.is_suspended = False

    user.status_reason = None

    log_admin_action('USER_UNSPEND', 'user', user.id, "Suspension lifted")

    flash(f"User @{user.username} suspension lifted.", "success")

elif action == 'ban':

    user.is_banned = True

    user.status_reason = reason

    log_admin_action('USER_BAN', 'user', user.id, f"Banned. Reason: {reason}")

    flash(f"User @{user.username} banned permanently.", "danger")

elif action == 'unban':

    user.is_banned = False

    user.status_reason = None

    log_admin_action('USER_UNBAN', 'user', user.id, "Ban lifted")

    flash(f"User @{user.username} unbanned.", "success")

elif action == 'delete':

    if current_user.role != RolePermission.SUPER_ADMIN:

        flash("Only Super Administrators can permanently delete user accounts.", "danger")

        return redirect(url_for('admin.user_detail', user_id=user.id))

    username = user.username

    db.session.delete(user)

    log_admin_action('USER_DELETE', 'user', user_id, f"Permanently deleted @ {username}")

    db.session.commit()

    flash(f"User @{username} permanently deleted.", "info")

```

```

        return redirect(url_for('admin.users_list'))

db.session.commit()

return redirect(url_for('admin.user_detail', user_id=user.id))

@admin_bp.route('/users/<int:user_id>/impersonate', methods=['POST'])
@admin_permission_required('super_admin')
def impersonate_user(user_id):
    """Super Admin secure user simulation."""

    target_user = User.query.get_or_404(user_id)

    session['admin_impersonator_id'] = current_user.id

    log_admin_action('IMPERSONATION_START', 'user', target_user.id, f'SuperAdmin impersonated {target_user.username}')

    login_user(target_user)

    flash(f'Impersonating @{target_user.username}. Use top bar to exit.', "warning")

    return redirect(url_for('typing.test_page'))

@admin_bp.route('/stop-impersonation')
@login_required
def stop_impersonation():
    admin_id = session.pop('admin_impersonator_id', None)

    if not admin_id:
        return redirect(url_for('typing.test_page'))

    admin_user = User.query.get(admin_id)

    if admin_user:
        log_admin_action('IMPERSONATION_END', 'user', current_user.id, "Impersonation session concluded")

        login_user(admin_user)

        flash("Returned to Super Administrator console.", "info")

        return redirect(url_for('admin.dashboard'))

    return redirect(url_for('auth.logout'))

# =====

# 3. Content & Passage Library Management

# =====

```

```

@admin_bp.route('/passages')
@admin_permission_required('manage_content')
def passages_list():
    page = request.args.get('page', 1, type=int)

    search = request.args.get('q', '').strip()

    category = request.args.get('category', '')

    difficulty = request.args.get('difficulty', '')

    query = TypingText.query

    if search:
        query = query.filter((TypingText.title.ilike(f"%{search}%")) | (TypingText.content.ilike(f"%{search}%")))

    if category:
        query = query.filter_by(category=category)

    if difficulty:
        query = query.filter_by(difficulty=difficulty)

    passages_page = query.order_by(TypingText.id.desc()).paginate(page=page, per_page=15, error_out=False)

    categories = [c[0] for c in db.session.query(TypingText.category).distinct().all()]

    return render_template(
        'admin/passages.html',
        passages=passages_page,
        search=search,
        category=category,
        difficulty=difficulty,
        categories=categories
    )

@admin_bp.route('/passages/create', methods=['GET', 'POST'])
@admin_permission_required('manage_content')
def passage_create():
    if request.method == 'POST':

```

```

title = request.form.get('title', 'Untitled Passage').strip()

category = request.form.get('category', 'General').strip()

difficulty = request.form.get('difficulty', 'Medium').strip()

content = request.form.get('content', '').strip()

source = request.form.get('source', 'Administrator').strip()

is_code = request.form.get('is_code') == 'on'

code_lang = request.form.get('code_lang') if is_code else None


if not content:

    flash("Passage text content cannot be blank.", "danger")

    return render_template('admin/passage_form.html', action="Create")


passage = TypingText(

    title=title,

    category=category,

    difficulty=difficulty,

    content=content,

    source=source,

    is_code=is_code,

    code_lang=code_lang,

    created_by=current_user.username

)

passage.calculate_stats()

db.session.add(passage)

db.session.commit()


log_admin_action('PASSAGE_CREATE', 'passage', passage.id, f"Added passage '{title}'")

flash(f"Passage #{passage.id} ({title}) added successfully.", "success")

return redirect(url_for('admin.passages_list'))


return render_template('admin/passage_form.html', action="Create", passage=None)


@admin_bp.route('/passages/<int:passage_id>/edit', methods=['GET', 'POST'])

```

```

@admin_permission_required('manage_content')

def passage_edit(passage_id):

    passage = TypingText.query.get_or_404(passage_id)

    if request.method == 'POST':

        passage.title = request.form.get('title', passage.title).strip()

        passage.category = request.form.get('category', passage.category).strip()

        passage.difficulty = request.form.get('difficulty', passage.difficulty).strip()

        passage.content = request.form.get('content', passage.content).strip()

        passage.source = request.form.get('source', passage.source).strip()

        passage.is_code = request.form.get('is_code') == 'on'

        passage.code_lang = request.form.get('code_lang') if passage.is_code else None

        passage.is_active = request.form.get('is_active') == 'on'

        passage.updated_by = current_user.username

        passage.calculate_stats()


        db.session.commit()

        log_admin_action('PASSAGE_UPDATE', 'passage', passage.id, f"Modified passage '{passage.title}'")

        flash("Passage updated successfully.", "success")

        return redirect(url_for('admin.passages_list'))


    return render_template('admin/passage_form.html', action="Edit", passage=passage)


@admin_bp.route('/passages/<int:passage_id>/delete', methods=['POST'])

@admin_permission_required('manage_content')

def passage_delete(passage_id):

    passage = TypingText.query.get_or_404(passage_id)

    title = passage.title

    db.session.delete(passage)

    db.session.commit()

    log_admin_action('PASSAGE_DELETE', 'passage', passage_id, f"Deleted '{title}'")

    flash("Passage deleted.", "info")

    return redirect(url_for('admin.passages_list'))

```

```

@admin_bp.route('/passages/<int:passage_id>/toggle-status', methods=['POST'])

@admin_permission_required('manage_content')

def passage_toggle(passage_id):

    passage = TypingText.query.get_or_404(passage_id)

    passage.is_active = not passage.is_active

    db.session.commit()

    log_admin_action('PASSAGE_TOGGLE', 'passage', passage.id, f"Active state: {passage.is_active}")

    return redirect(url_for('admin.passages_list'))

# =====

# 4. Traffic & Visitor Analytics

# =====

@admin_bp.route('/analytics')

@admin_permission_required('view_analytics')

def traffic_analytics():

    days = request.args.get('days', 30, type=int)

    cutoff = datetime.utcnow() - timedelta(days=days)

    total_views = VisitorTraffic.query.filter(VisitorTraffic.created_at >= cutoff).count()

    unique_sessions = db.session.query(func.count(func.distinct(VisitorTraffic.session_id))).filter(VisitorTraffic.created_at >= cutoff).scalar() or 0

    registered_views = VisitorTraffic.query.filter(VisitorTraffic.created_at >= cutoff, VisitorTraffic.user_id.isnot(None)).count()

    guest_views = total_views - registered_views

    # Popular Pages

    popular_pages = db.session.query(

        VisitorTraffic.path, func.count(VisitorTraffic.id).label('views')

    ).filter(VisitorTraffic.created_at >= cutoff).group_by(VisitorTraffic.path).order_by(desc('views')).limit(10).all()

    # Browser Breakdown

    browsers = db.session.query(

        VisitorTraffic.browser, func.count(VisitorTraffic.id).label('count')

    ).filter(VisitorTraffic.created_at >= cutoff).group_by(VisitorTraffic.browser).order_by(desc('count')).limit(6).all()

```

```

# Device Breakdown

devices = db.session.query(

    VisitorTraffic.device_type, func.count(VisitorTraffic.id).label('count')

).filter(VisitorTraffic.created_at >= cutoff).group_by(VisitorTraffic.device_type).all()

return render_template(

    'admin/analytics.html',

    days=days,

    total_views=total_views,

    unique_sessions=unique_sessions,

    guest_views=guest_views,

    registered_views=registered_views,

    popular_pages=popular_pages,

    browsers=browsers,

    devices=devices

)

# =====

# 5. Leaderboard Moderation

# =====

@admin_bp.route('/leaderboard')

@admin_permission_required('manage_leaderboard')

def leaderboard_manage():

    page = request.args.get('page', 1, type=int)

    filter_suspicious = request.args.get('suspicious', 'all')

    query = TypingTest.query.filter(TypingTest.user_id.isnot(None))

    if filter_suspicious == 'flagged':

        query = query.filter_by(suspicious=True)

    elif filter_suspicious == 'verified':

        query = query.filter_by(suspicious=False)

    tests_page = query.order_by(TypingTest.wpm.desc()).paginate(page=page, per_page=20, error_out=False)

```



```

return render_template('admin/leaderboard.html', tests=tests_page, filter_suspicious=filter_suspicious)

@admin_bp.route('/leaderboard/test/<int:test_id>/toggle-flag', methods=['POST'])
@admin_permission_required('manage_leaderboard')
def test_toggle_flag(test_id):
    test = TypingTest.query.get_or_404(test_id)

    test.suspicious = not test.suspicious

    test.suspicion_reason = "Flagged manually by Administrator" if test.suspicious else None

    db.session.commit()

    log_admin_action('TEST_FLAG_TOGGLE', 'typing_test', test.id, f"Suspicious set to {test.suspicious}")

    flash(f"Test #{test.id} verification state toggled.", "success")

    return redirect(url_for('admin.leaderboard_manage'))

@admin_bp.route('/leaderboard/test/<int:test_id>/delete', methods=['POST'])
@admin_permission_required('manage_leaderboard')
def test_delete(test_id):
    test = TypingTest.query.get_or_404(test_id)

    db.session.delete(test)

    db.session.commit()

    log_admin_action('TEST_RECORD_DELETE', 'typing_test', test_id, "Record deleted from leaderboards")

    flash("Typing score record removed.", "info")

    return redirect(url_for('admin.leaderboard_manage'))

# =====

# 6. Security Center & Uneditable Audit Logs

# =====

@admin_bp.route('/security')
@admin_permission_required('view_security')
def security_center():
    page = request.args.get('page', 1, type=int)

    events = SecurityEvent.query.order_by(SecurityEvent.created_at.desc()).paginate(page=page, per_page=25, error_out=False)

    failed_logins = SecurityEvent.query.filter_by(event_type='FAILED_LOGIN').count()

```

```
critical_events = SecurityEvent.query.filter_by(severity='CRITICAL').count()
```

```

return render_template(
    'admin/security.html',
    events=events,
    failed_logins=failed_logins,
    critical_events=critical_events
)

```

```
@admin_bp.route('/audit-logs')
```

```
@admin_permission_required('view_audit_logs')
```

```
def audit_logs():
```

```
    page = request.args.get('page', 1, type=int)
```

```
    action_filter = request.args.get('action', '').strip()
```

```
    query = AdminAuditLog.query
```

```
    if action_filter:
```

```
        query = query.filter(AdminAuditLog.action.ilike(f"%{action_filter}%"))
```

```
    logs_page = query.order_by(AdminAuditLog.created_at.desc()).paginate(page=page, per_page=30, error_out=False)
```

```
    return render_template('admin/audit_logs.html', logs=logs_page, action_filter=action_filter)
```

```
# =====
```

```
# 7. Platform Settings & Data Export
```

```
# =====
```

```
@admin_bp.route('/settings', methods=['GET', 'POST'])
```

```
@admin_permission_required('manage_settings')
```

```
def system_settings():
```

```
    if request.method == 'POST':
```

```
        # Maintenance mode, test ranges, registration toggle
```

```
        for key in ['maintenance_mode', 'allow_registrations', 'default_test_duration', 'min_wpm_cutoff']:
```

```
            val = request.form.get(key, '').strip()
```

```
cfg = PlatformConfig.query.filter_by(key=key).first()

if not cfg:

    cfg = PlatformConfig(key=key, value=val, updated_by=current_user.username)

    db.session.add(cfg)

else:

    cfg.value = val

    cfg.updated_by = current_user.username

db.session.commit()

log_admin_action('PLATFORM_CONFIG_UPDATE', 'settings', details="Updated platform runtime configs")

flash("Platform parameters updated.", "success")

return redirect(url_for('admin.system_settings'))


configs = {c.key: c.value for c in PlatformConfig.query.all()}

return render_template('admin/settings.html', configs=configs)


@admin_bp.route('/export/<data_type>')
@admin_permission_required('export_data')
def export_csv(data_type):

    """Clean CSV data export for Super Admins and Analysts."""

    log_admin_action('EXPORT_DATA', 'system', details=f"Exported {data_type}.csv")


    output = io.StringIO()

    writer = csv.writer(output)


    if data_type == 'users':

        writer.writerow(['ID', 'Username', 'Email', 'Role', 'Suspended', 'Banned', 'Elo', 'Created_At'])

        for u in User.query.all():

            writer.writerow([u.id, u.username, u.email, u.role, u.is_suspended, u.is_banned, u.elo_rating, u.created_at])

    elif data_type == 'tests':

        writer.writerow(['ID', 'User_ID', 'Mode', 'WPM', 'Accuracy', 'Consistency', 'Duration', 'Suspicious', 'Date'])

        for t in TypingTest.query.limit(2000).all():

            writer.writerow([t.id, t.user_id, t.mode, t.wpm, t.accuracy, t.consistency, t.duration, t.suspicious, t.completed_at])

    elif data_type == 'audit':
```

```
writer.writerow(['ID', 'Admin', 'Action', 'Target_Type', 'Target_ID', 'IP', 'Date'])

for a in AdminAuditLog.query.order_by(AdminAuditLog.id.desc()).limit(2000).all():

    writer.writerow([a.id, a.admin_username, a.action, a.target_type, a.target_id, a.ip_address, a.created_at])

else:

    return "Unknown dataset", 400


output.seek(0)

return Response(

    output.getvalue(),

    mimetype="text/csv",

    headers={"Content-Disposition": f"attachment;filename=typesphere_{data_type}_{datetime.utcnow().strftime('%Y%m%d')}.csv"}

)


=== FILE: .\app\routes\auth.py ===

# (In app/routes/auth.py)

from flask import Blueprint, render_template, redirect, url_for, flash, request, session

from flask_login import login_user, logout_user, login_required, current_user

from app import db

from app.models.user import User

from app.models.settings import UserSettings

from app.services.admin_security import record_user_activity, log_security_incident

from flask_wtf import FlaskForm

from wtforms import StringField, PasswordField, SubmitField

from wtforms.validators import DataRequired, Email, Length, EqualTo, ValidationError


auth_bp = Blueprint('auth', __name__)


class LoginForm(FlaskForm):

    email = StringField('Username or Email', validators=[DataRequired()])

    password = PasswordField('Password', validators=[DataRequired()])

    submit = SubmitField('Sign In')


class RegisterForm(FlaskForm):
```

```
username = StringField('Username', validators=[DataRequired(), Length(3, 30)])

email = StringField('Email Address', validators=[DataRequired(), Email()])

password = PasswordField('Password', validators=[DataRequired(), Length(4, 64)])

confirm_password = PasswordField('Confirm Password', validators=[DataRequired(), EqualTo('password')])

submit = SubmitField('Create Account')
```

```
def validate_username(self, field):

    if User.query.filter_by(username=field.data.strip()).first():

        raise ValidationError('Username is already taken.')
```

```
def validate_email(self, field):

    if User.query.filter_by(email=field.data.strip().lower()).first():

        raise ValidationError('Email address is already registered.')
```

```
@auth_bp.route('/register', methods=['GET', 'POST'])
```

```
def register():

    if current_user.is_authenticated:

        return redirect(url_for('admin.dashboard')) if current_user.is_admin else url_for('typing.test_page')
```

```
form = RegisterForm()
```

```
if form.validate_on_submit():
```

```
    user = User(

        username=form.username.data.strip(),

        email=form.email.data.strip().lower(),

        role='user'
```

```
    )
```

```
    user.set_password(form.password.data)
```

```
    db.session.add(user)
```

```
    db.session.flush()
```

```
    settings = UserSettings(user_id=user.id)
```

```
    db.session.add(settings)
```

```
    db.session.commit()
```

```
record_user_activity(user.id, 'REGISTER', 'Auth', 'User registered new account')
```

```
login_user(user)
```

```
flash(f'Account created successfully. Welcome, {user.username}!', 'success')
```

```
return redirect(url_for('typing.test_page'))
```

```
return render_template('auth/register.html', form=form)
```

```
@auth_bp.route('/login', methods=['GET', 'POST'])
```

```
def login():
```

```
    if current_user.is_authenticated:
```

```
        return redirect(url_for('admin.dashboard')) if current_user.is_admin else url_for('typing.test_page')
```

```
form = LoginForm()
```

```
if form.validate_on_submit():
```

```
    login_input = form.email.data.strip()
```

```
    user = User.query.filter((User.email == login_input.lower()) | (User.username == login_input)).first()
```

```
    if user and user.check_password(form.password.data):
```

```
        # Check Account Moderation Status
```

```
        if user.is_banned:
```

```
            log_security_incident('BANNED_USER_LOGIN_ATTEMPT', 'MEDIUM', f'Banned user {user.username} tried logging in',
            identifier=user.username)
```

```
            flash(f"This account has been permanently banned: {user.status_reason or 'Terms violation'}", "danger")
```

```
            return render_template('auth/login.html', form=form)
```

```
        if user.is_suspended:
```

```
            log_security_incident('SUSPENDED_USER_LOGIN_ATTEMPT', 'LOW', f'Suspended user {user.username} tried logging in",
            identifier=user.username)
```

```
            flash(f"This account is temporarily suspended: {user.status_reason or 'Investigation pending'}", "warning")
```

```
            return render_template('auth/login.html', form=form)
```

```
login_user(user)
```

```

record_user_activity(user.id, 'LOGIN', 'Auth', f"User signed in via {request.user_agent.platform}")

# Routing separation: Admins go to Admin Center; Typists go to public typing

if user.is_admin:

    flash(f"Admin console authorized. Welcome, {user.username}.", "info")

    return redirect(url_for('admin.dashboard'))

next_page = request.args.get('next')

return redirect(next_page) if next_page else redirect(url_for('typing.test_page'))

log_security_incident('FAILED_LOGIN', 'MEDIUM', f"Invalid credentials supplied for identifier '{login_input}'", identifier=login_input)

flash('Invalid username/email or password.', 'danger')

return render_template('auth/login.html', form=form)

@auth_bp.route('/logout')

@login_required

def logout():

    record_user_activity(current_user.id, 'LOGOUT', 'Auth', "User logged out")

    logout_user()

    flash('Signed out successfully.', 'info')

    return redirect(url_for('typing.test_page'))

=== FILE: .\app\routes\challenges.py ===

from flask import Blueprint, render_template, redirect, url_for

from datetime import date

from app import db

from app.models.challenge import DailyChallenge

from app.models.typing import TypingTest

challenges_bp = Blueprint('challenges', __name__)

@challenges_bp.route('/')

```

```

def index():

    return render_template('challenges/index.html')

@challenges_bp.route('/daily')

def daily():

    today = date.today()

    challenge = DailyChallenge.query.filter_by(target_date=today).first()

    if not challenge:

        challenge = DailyChallenge(

            target_date=today,

            title="Sovereignty of Focus",

            content="Consistency is the hallmark of the master craftsman. Every accurate strike compounds into pure flow and unmatched velocity."

        )

        db.session.add(challenge)

        db.session.commit()

# Today's Leaderboard

daily_tests = TypingTest.query.filter(

    TypingTest.mode == 'daily',

    TypingTest.suspicious == False

).order_by(TypingTest.wpm.desc()).limit(20).all()

# Previous Daily Challenges Archive

previous_challenges = DailyChallenge.query.filter(

    DailyChallenge.target_date < today

).order_by(DailyChallenge.target_date.desc()).limit(7).all()

return render_template(

    'challenges/daily.html',

    challenge=challenge,

    leaderboard=daily_tests,

    history=previous_challenges

)

```

```
=== FILE: .\app\routes\dashboard.py ===

from flask import Blueprint, render_template, abort

from flask_login import login_required, current_user

from app import db

from app.models.user import User

from app.models.typing import TypingTest, TypingDNA

from app.models.challenge import Achievement, UserAchievement

from app.services.achievement_service import AchievementService


dashboard_bp = Blueprint('dashboard', __name__)


@dashboard_bp.route('/')

@login_required

def index():

    AchievementService.check_and_award(current_user)


    tests = TypingTest.query.filter_by(user_id=current_user.id).order_by(TypingTest.completed_at.desc()).all()

    valid_tests = [t for t in tests if not t.suspicious]


    solo_tests = [t for t in valid_tests if t.mode != 'multiplayer']

    multiplayer_tests = [t for t in valid_tests if t.mode == 'multiplayer']


    total_tests = len(valid_tests)

    avg_wpm = round(sum(t.wpm for t in valid_tests) / total_tests, 1) if total_tests else 0

    best_wpm = max((t.wpm for t in valid_tests), default=0)

    avg_accuracy = round(sum(t.accuracy for t in valid_tests) / total_tests, 1) if total_tests else 0

    total_time_minutes = round(sum(t.duration for t in valid_tests) / 60.0, 1) if total_tests else 0


    achievements = UserAchievement.query.filter_by(user_id=current_user.id).all()


    return render_template(

        'dashboard/index.html',
```

```
total_tests=total_tests,

avg_wpm=avg_wpm,

best_wpm=best_wpm,

avg_accuracy=avg_accuracy,

total_time_minutes=total_time_minutes,

recent_tests=solo_tests[:8],

multiplayer_tests=multiplayer_tests[:8],

achievements=achievements

)


@dashboard_bp.route('/achievements')

@login_required

def achievements_showcase():

    AchievementService.check_and_award(current_user)


    all_achievements = Achievement.query.order_by(Achievement.id.asc()).all()

    unlocked_map = {

        ua.achievement_id: ua.unlocked_at

        for ua in UserAchievement.query.filter_by(user_id=current_user.id).all()

    }


    badge_data = []

    for ach in all_achievements:

        is_unlocked = ach.id in unlocked_map

        badge_data.append({

            'code': ach.code,

            'title': ach.title,

            'description': ach.description,

            'icon': ach.icon,

            'unlocked': is_unlocked,

            'unlocked_at': unlocked_map.get(ach.id)

        })
```

```

unlocked_count = len(unlocked_map)

total_count = len(all_achievements)

mastery_pct = round((unlocked_count / total_count * 100.0), 1) if total_count else 0.0


return render_template(

    'dashboard/achievements.html',

    badges=badge_data,

    unlocked_count=unlocked_count,

    total_count=total_count,

    mastery_pct=mastery_pct

)


@dashboard_bp.route('/dna')

@login_required

def dna_profile():

    dna = TypingDNA.query.filter_by(user_id=current_user.id).first()

    stats = dna.get_key_stats() if dna else {}

    confusions = dna.get_confusion_matrix() if dna else {}

    return render_template('dashboard/dna.html', dna=dna, stats=stats, confusions=confusions)


@dashboard_bp.route('/analytics')

@login_required

def analytics():

    tests = TypingTest.query.filter_by(user_id=current_user.id, suspicious=False).order_by(TypingTest.completed_at.asc()).all()


    total_tests = len(tests)

    total_words = sum(int((t.correct_chars or 0) / 5) for t in tests)

    total_seconds = sum(t.duration for t in tests)

    total_hours = round(total_seconds / 3600.0, 2)

    best_wpm = max((t.wpm for t in tests), default=0)

    avg_wpm = round(sum(t.wpm for t in tests) / total_tests, 1) if total_tests else 0

    avg_accuracy = round(sum(t.accuracy for t in tests) / total_tests, 1) if total_tests else 0

    avg_consistency = round(sum(t.consistency for t in tests) / total_tests, 1) if total_tests else 0

```

```

improvement_rate = 0.0

if total_tests >= 6:

    early_avg = sum(t.wpm for t in tests[:5]) / 5.0

    recent_avg = sum(t.wpm for t in tests[-5:]) / 5.0

    if early_avg > 0:

        improvement_rate = round(((recent_avg - early_avg) / early_avg) * 100.0, 1)

```

```

data_points = [{

    'date': t.completed_at.strftime('%m-%d %H:%M'),

    'wpm': round(t.wpm, 1),

    'raw_wpm': round(t.raw_wpm, 1),

    'accuracy': round(t.accuracy, 1),

    'consistency': round(t.consistency, 1)

} for t in tests]

```

```

return render_template(

    'dashboard/analytics.html',

    data_points=data_points,

    total_tests=total_tests,

    total_words=total_words,

    total_hours=total_hours,

    best_wpm=best_wpm,

    avg_wpm=avg_wpm,

    avg_accuracy=avg_accuracy,

    avg_consistency=avg_consistency,

    improvement_rate=improvement_rate

)

```

```

@dashboard_bp.route('/profile/<username>')

def public_profile(username):

    user = User.query.filter_by(username=username).first_or_404()

    tests = TypingTest.query.filter_by(user_id=user.id, suspicious=False).order_by(TypingTest.wpm.desc()).all()

```

```

total_tests = len(tests)

best_test = tests[0] if tests else None

best_wpm = best_test.wpm if best_test else 0

avg_wpm = round(sum(t.wpm for t in tests) / total_tests, 1) if total_tests else 0

avg_accuracy = round(sum(t.accuracy for t in tests) / total_tests, 1) if total_tests else 0

```

```

dna = TypingDNA.query.filter_by(user_id=user.id).first()

achievements = UserAchievement.query.filter_by(user_id=user.id).all()

```

```

return render_template(

    'dashboard/profile.html',

    profile_user=user,

    total_tests=total_tests,

    best_test=best_test,

    best_wpm=best_wpm,

    avg_wpm=avg_wpm,

    avg_accuracy=avg_accuracy,

    dna=dna,

    achievements=achievements

)

```

```

=== FILE: .\app\routes\feedback.py ===

```

```

from flask import Blueprint, render_template, request, redirect, url_for, flash, jsonify

from flask_login import login_required, current_user

from datetime import datetime, timedelta

from app import db

from app.models.feedback import ContactMessage, FeedbackItem, RatingReview

```

```

feedback_bp = Blueprint('feedback', __name__)

```

```

@feedback_bp.route('/contact', methods=['GET', 'POST'])

```

```

def contact():

```

```

    if request.method == 'POST':

```

```

        name = request.form.get('name', '').strip()

        email = request.form.get('email', '').strip().lower()

        subject = request.form.get('subject', '').strip()

        message = request.form.get('message', '').strip()

```

```

        if not name or not email or not message:

```

```

            flash('Please complete all required fields.', 'danger')

            return redirect(url_for('feedback.contact'))

```

```

        # Anti-spam check: prevent duplicate submissions within 2 minutes from same email

```

```

        recent_msg = ContactMessage.query.filter_by(email=email).order_by(ContactMessage.id.desc()).first()

```

```

        if recent_msg and (datetime.utcnow() - recent_msg.created_at) < timedelta(minutes=2):

```

```

            flash('Message already received. Please wait before submitting another inquiry.', 'warning')

            return redirect(url_for('feedback.contact'))

```

```

        msg_record = ContactMessage(name=name, email=email, subject=subject, message=message)

```

```

        db.session.add(msg_record)

```

```

        db.session.commit()

```

```

        flash("Your message has been received! Our team will respond shortly.", 'success')

```

```

        return redirect(url_for('feedback.contact'))

```

```

    return render_template('feedback/contact.html')

```

```

@feedback_bp.route('/submit', methods=['GET', 'POST'])

```

```

def submit_feedback():

```

```

    if request.method == 'POST':

```

```

        if not current_user.is_authenticated:

```

```

            flash("You must be signed in to submit platform feedback.", 'warning')

            return redirect(url_for('auth.login', next=url_for('feedback.submit_feedback')))

```

```

        category = request.form.get('category', 'General')

```

```

title = request.form.get('title', '').strip()

message = request.form.get('message', '').strip()

if not title or not message:

    flash('Please provide both a title and message description.', 'danger')

    return redirect(url_for('feedback.submit_feedback'))

fb = FeedbackItem(user_id=current_user.id, category=category, title=title, message=message)

db.session.add(fb)

db.session.commit()

flash('Thank you! Your feedback has been recorded for our engineering team.', 'success')

return redirect(url_for('feedback.submit_feedback'))

user_submissions = []

if current_user.is_authenticated:

    user_submissions = FeedbackItem.query.filter_by(user_id=current_user.id).order_by(FeedbackItem.id.desc()).all()

return render_template('feedback/feedback.html', user_submissions=user_submissions)

@feedback_bp.route('/ratings', methods=['GET', 'POST'])
def ratings():

    if request.method == 'POST':

        if not current_user.is_authenticated:

            flash('You must be signed in to leave a platform rating & review.', 'warning')

            return redirect(url_for('auth.login', next=url_for('feedback.ratings')))

        try:

            rating_val = int(request.form.get('rating', 5))

            rating_val = max(1, min(5, rating_val))

        except (ValueError, TypeError):

            rating_val = 5

```

```

title = request.form.get('review_title', '').strip()

text_content = request.form.get('review_text', '').strip()

if not title or not text_content:

    flash('Please provide a title and review explanation.', 'danger')

    return redirect(url_for('feedback.ratings'))

# Check existing review to allow update rather than spamming
existing_review = RatingReview.query.filter_by(user_id=current_user.id).first()

if existing_review:

    existing_review.rating = rating_val

    existing_review.review_title = title

    existing_review.review_text = text_content

    existing_review.updated_at = datetime.utcnow()

    flash('Your review has been updated successfully!', 'success')

else:

    new_rev = RatingReview(

        user_id=current_user.id,

        rating=rating_val,

        review_title=title,

        review_text=text_content

    )

    db.session.add(new_rev)

    flash('Thank you for your rating and review!', 'success')

db.session.commit()

return redirect(url_for('feedback.ratings'))

# Load approved reviews
all_reviews = RatingReview.query.filter_by(is_approved=True).order_by(RatingReview.id.desc()).limit(30).all()

avg_rating = 0.0

if all_reviews:

    avg_rating = round(sum(r.rating for r in all_reviews) / len(all_reviews), 1)

```



```
user_review = None

if current_user.is_authenticated:

    user_review = RatingReview.query.filter_by(user_id=current_user.id).first()

return render_template(

    'feedback/ratings.html',

    reviews=all_reviews,

    avg_rating=avg_rating,

    total_reviews=len(all_reviews),

    user_review=user_review

)
```

=== FILE: .\app\routes\games.py ===

```
from flask import Blueprint, render_template, request, jsonify

from flask_login import current_user

from app import db

from app.models.game import GameRecord, ArcadeLeaderboard

from app.services.arcade_content_engine import ArcadeContentEngine

from app.models.typing import TypingDNA
```

```
games_bp = Blueprint('games', __name__)
```

```
ARCADE_MASTER_REGISTRY = [

    {

        'slug': 'speed_racer',

        'title': 'Speed Racer',

        'badge': 'REAL CAR PHYSICS',

        'icon': '🏎️',

        'category': 'Momentum & Recoil Sprint',

        'summary': 'Control a live vehicle on an open race track. Correct strokes accelerate your car forward; typos apply backward recoil without halting momentum.',

        'win_condition': 'Cross the finish line first or cover the furthest distance before timer expires.'
```

```
'lose_condition': 'Opponent crosses the finish line ahead of you or covers greater distance.',

'supports': ['Solo vs AI', '1v1 Duel', 'Multiplayer'],

'default_objective': 'distance',

'objective_options': [

    {'label': '300m Sprint', 'val': '300'},

    {'label': '500m Grand Prix', 'val': '500'},

    {'label': '1000m Marathon', 'val': '1000'}

]

},

{

    'slug': 'falling_words',

    'title': 'Falling Words',

    'badge': 'WAVE SURVIVAL',

    'icon': '🌊📄',

    'category': 'Target Locking & Peripheral Vision',

    'summary': 'Words descend with accelerating gravitational velocity. Target lock descending phrases before they touch the defense barrier.',

    'win_condition': 'Clear all waves or achieve higher score than rival when timer expires.',

    'lose_condition': 'Lose all lives or opponent secures higher wave score.',

    'supports': ['Solo vs AI', '1v1 Duel'],

    'default_objective': 'waves',

    'objective_options': [

        {'label': '3 Waves (Short)', 'val': '3'},

        {'label': '5 Waves (Standard)', 'val': '5'},

        {'label': 'Survival (Unlimited)', 'val': '0'}

    ]

}

},

{

    'slug': 'bubble_pop',

    'title': 'Bubble Pop',

    'badge': 'ISOLATED TARGETS',

    'icon': '🫧',

    'category': 'Single-Key Tactile Precision',
```

```
'summary': 'Floating bubbles hold isolated target characters. Pop targets before they hit the ceiling barrier.',

'win_condition': 'Pop required targets with highest score and unbroken combos.',

'lose_condition': 'Allow 3 bubbles to burst against ceiling line.',

'supports': ['Solo vs AI', '1v1 Challenge'],

'default_objective': 'timed',

'objective_options': [

    {'label': '30 Seconds', 'val': '30'},

    {'label': '60 Seconds', 'val': '60'},

    {'label': '120 Seconds', 'val': '120'}

]

},

{

    'slug': 'whack_a_word',

    'title': 'Whack-A-Word',

    'badge': 'REACTION GRID',

    'icon': '🎯',

    'category': 'Spatial Reaction Latency',

    'summary': 'Targets emerge across a 3x3 reaction grid with shrinking exposure times. Type the active target before it retracts into the bunker.',

    'win_condition': 'Score higher points by striking targets within the shortest millisecond window.',

    'lose_condition': 'Miss 5 target windows or score lower than opponent.',

    'supports': ['Solo vs AI', '1v1 Challenge'],

    'default_objective': 'timed',

    'objective_options': [

        {'label': '30 Seconds', 'val': '30'},

        {'label': '60 Seconds', 'val': '60'},

        {'label': '90 Seconds', 'val': '90'}

    ]

},

{

    'slug': 'zombie_defense',

    'title': 'Zombie Defense',

    'badge': 'BASE SURVIVAL',

    'icon': '🧟🚧🏠',
```

```
'category': 'Perimeter Wave Defense',

'summary': 'Incoming hordes march towards your fortified base. Type overhead words to discharge defensive turrets.',

'win_condition': 'Survive all incoming waves with base integrity intact.',

'lose_condition': 'Zombies breach the perimeter and reduce base health to 0%.',

'supports': ['Solo vs AI', 'Co-op Score Duel'],

'default_objective': 'waves',

'objective_options': [

    {'label': 'Wave 1 - 3', 'val': '3'},

    {'label': 'Wave 1 - 5', 'val': '5'},

    {'label': 'Wave 1 - 10', 'val': '10'}

]

},

{

    'slug': 'zombie_duel',

    'title': 'Zombie Duel',

    'badge': '1V1 HP COMBAT',

    'icon': '👤👤',

    'category': 'Two-Player Health Combat',

    'summary': 'Head-to-head combat duel. Fast keystrokes launch attack projectiles; typos break defensive shields and trigger recoil damage.',

    'win_condition': 'Reduce opponent combat health to 0 HP.',

    'lose_condition': 'Your combat health drops to 0 HP first.',

    'supports': ['Solo vs AI', '1v1 Challenge'],

    'default_objective': 'knockout',

    'objective_options': [

        {'label': 'Standard 100 HP Duel', 'val': '100'},

        {'label': 'Hardcore 50 HP Sudden Death', 'val': '50'}

    ]

},

{

    'slug': 'cipher_hacker',

    'title': 'Cipher Hacker',

    'badge': 'LAYER BREACH',

    'icon': '🔐',
```

```
'category': 'Progressive Security Architecture',

'summary': 'Decrypt layered cybersecurity defenses: from hex dumps to complex syntax tokens. Advance through security clearances.',

'win_condition': 'Breach all security layers before the firewall lockdown countdown concludes.',

'lose_condition': 'Timer expires before root access decryption.',

'supports': ['Solo vs AI', '1v1 Race'],

'default_objective': 'layers',

'objective_options': [

    {'label': '3 Security Layers', 'val': '3'},

    {'label': '4 Security Layers (Root Access)', 'val': '4'}

]

},

{

'slug': 'space_defender',

'title': 'Space Defender',

'badge': 'MULTI-QUADRANT',

'icon': '🛸',

'category': 'Orbital Trajectory Defense',

'summary': 'Debris and drones converge on your starship across 360 degrees. Acquire target locks and neutralize orbital threats.',

'win_condition': 'Clear all orbital waves before ship shielding is depleted.',

'lose_condition': 'Ship shields collapse from debris impacts.',

'supports': ['Solo vs AI', '1v1 Duel'],

'default_objective': 'timed',

'objective_options': [

    {'label': '45 Seconds', 'val': '45'},

    {'label': '60 Seconds', 'val': '60'},

    {'label': '120 Seconds', 'val': '120'}

]

},

{

'slug': 'bomb_defuse',

'title': 'Bomb Defuse',

'badge': 'PRESSURE DECRYPT',

'icon': '💣',
```

```
'category': 'Precision Code Cracking',

'summary': 'Defuse time bombs with sequential multi-character cryptograms. Typos inflict immediate timer penalties.',

'win_condition': 'Crack all stages before detonation clock reaches 00:00.',

'lose_condition': 'Countdown timer hits 00:00.',

'supports': ['Solo vs Timer', '1v1 Duel'],

'default_objective': 'stages',

'objective_options': [

    {'label': '4 Detonation Stages', 'val': '4'},

    {'label': '7 Detonation Stages', 'val': '7'},

    {'label': '10 Detonation Stages', 'val': '10'}

]

},

{

'slug': 'typing_ninja',

'title': 'Typing Ninja',

'badge': 'COMBO SLICE',

'icon': '🥷',

'category': 'Airborne Word Trajectories',

'summary': 'Airborne words launch with parabolic momentum. Slice targets cleanly mid-air to build score multipliers.',

'win_condition': 'Amass highest slice score and maintain unbroken multiplier combos.',

'lose_condition': 'Drop 3 unsliced targets or score lower than opponent.',

'supports': ['Solo vs AI', '1v1 Challenge'],

'default_objective': 'timed',

'objective_options': [

    {'label': '45 Seconds', 'val': '45'},

    {'label': '60 Seconds', 'val': '60'}

]

},

{

'slug': 'memory_type',

'title': 'Memory Type',

'badge': 'BLIND RECALL',

'icon': '🧠',
```

```
'category': 'Working Cognitive Recall',

'summary': 'Sequences flash briefly and vanish. Type the sequence completely from mental recall.',

'win_condition': 'Successfully reproduce the highest quota of hidden sequences.',

'lose_condition': 'Fail 3 sequence recalls.',

'supports': ['Solo vs AI', '1v1 Duel'],

'default_objective': 'rounds',

'objective_options': [

    {'label': '5 Memory Rounds', 'val': '5'},

    {'label': '10 Memory Rounds', 'val': '10'},

    {'label': '15 Memory Rounds', 'val': '15'}

]

},

{

'slug': 'keyboard_quest',

'title': 'Keyboard Quest',

'badge': 'BIOMECHANIC MAP',

'icon': '🗺️',

'category': 'Ergonomic Row Progression',

'summary': 'Journey through ergonomic training stages: Home Row anchors, upper extensions, bottom tucks, and number stretches.',

'win_condition': 'Clear all quest stages with at least 95% accuracy.',

'lose_condition': 'Accuracy falls below baseline stage threshold.',

'supports': ['Solo Quest', 'Stage Duel'],

'default_objective': 'stages',

'objective_options': [

    {'label': 'Stage 1 to 3', 'val': '3'},

    {'label': 'Stage 1 to 5 (Full Map)', 'val': '5'}

]

}

]

@games_bp.route('/')

def index():

    user_high_scores = {}
```

```
if current_user.is_authenticated:

    records = ArcadeLeaderboard.query.filter_by(user_id=current_user.id).all()

    user_high_scores = {r.game_mode: r.high_score for r in records}

return render_template(

    'games/index.html',

    games=ARCADE_MASTER_REGISTRY,

    high_scores=user_high_scores

)

@games_bp.route('/play/<game_slug>')

def play_arena(game_slug):

    game_info = next((g for g in ARCADE_MASTER_REGISTRY if g['slug'] == game_slug), None)

    if not game_info:

        return render_template('games/index.html', games=ARCADE_MASTER_REGISTRY)

    user_weak_keys = []

    if current_user.is_authenticated and current_user.dna_profile:

        stats = current_user.dna_profile.get_key_stats()

        user_weak_keys = [k.upper() for k, v in stats.items() if v.get('total', 0) >= 4 and (v.get('errors', 0)/v['total']) > 0.08][:6]

    return render_template(

        'games/arena.html',

        game=game_info,

        weak_keys=user_weak_keys

    )

@games_bp.route('/api/submit-score', methods=['POST'])

def submit_score():

    data = request.get_json(force=True) or {}

    game_mode = data.get('game_mode', 'arcade')

    score = int(data.get('score', 0))

    net_wpm = float(data.get('net_wpm', 0.0))
```

```
accuracy = float(data.get('accuracy', 100.0))

errors = int(data.get('errors', 0))

highest_combo = int(data.get('highest_combo', 0))

reaction_ms = float(data.get('reaction_ms', 0.0))

duration = float(data.get('duration', 0.0))

outcome = data.get('outcome', 'FINISHED')

record = GameRecord(

    user_id=current_user.id if current_user.is_authenticated else None,

    game_mode=game_mode,

    play_type=data.get('play_type', 'solo_ai'),

    result_outcome=outcome,

    score=score,

    net_wpm=net_wpm,

    accuracy=accuracy,

    errors=errors,

    highest_combo=highest_combo,

    average_reaction_ms=reaction_ms,

    duration_seconds=duration,

    meta_stats_json=data.get('meta_json', '{}')

)

db.session.add(record)

is_new_pb = False

if current_user.is_authenticated:

    lb = ArcadeLeaderboard.query.filter_by(user_id=current_user.id, game_mode=game_mode).first()

    if not lb:

        lb = ArcadeLeaderboard(user_id=current_user.id, game_mode=game_mode, high_score=score, best_wpm=net_wpm,
best_accuracy=accuracy)

        db.session.add(lb)

        is_new_pb = True

    else:

        if score > lb.high_score:
```

```
lb.high_score = score

is_new_pb = True

if net_wpm > lb.best_wpm: lb.best_wpm = net_wpm

if accuracy > lb.best_accuracy: lb.best_accuracy = accuracy

lb.updated_at = db.func.now()

db.session.commit()

return jsonify({'success': True, 'is_pb': is_new_pb, 'score': score})
```

```
=== FILE: .\app\routes\leaderboard.py ===

from flask import Blueprint, render_template, request

from flask_login import current_user

from app.models.user import User

from app.models.typing import TypingTest
```

```
leaderboard_bp = Blueprint('leaderboard', __name__)
```

```
@leaderboard_bp.route('/')
```

```
def index():

    sort_by = request.args.get('sort', 'wpm')

    mode_filter = request.args.get('mode', 'all')
```

```
# 1. Official Ranked Elo Division Standings
```

```
if mode_filter == 'ranked_elo':

    top_ranked_users = User.query.filter(User.ranked_wins + User.ranked_losses > 0).order_by(User.elo_rating.desc()).limit(50).all()

    return render_template(

        'leaderboard/index.html',

        tests=[],

        podium=[],

        ranked_users=top_ranked_users,

        user_rank_info=None,

        current_sort=sort_by,

        current_mode=mode_filter
```

```
)

# 2. Filter strictly for verified tests (Custom text / Practice tests excluded)

query = TypingTest.query.filter_by(suspicious=False, is_ranked=True).filter(TypingTest.user_id.isnot(None))

if mode_filter == 'timed_60':

    query = query.filter(TypingTest.mode == 'timed_60')

elif mode_filter == 'timed_30':

    query = query.filter(TypingTest.mode == 'timed_30')

elif mode_filter == 'timed_15':

    query = query.filter(TypingTest.mode == 'timed_15')

elif mode_filter == 'timed_120':

    query = query.filter(TypingTest.mode == 'timed_120')

if sort_by == 'accuracy':

    query = query.order_by(TypingTest.accuracy.desc(), TypingTest.wpm.desc())

elif sort_by == 'consistency':

    query = query.order_by(TypingTest.consistency.desc(), TypingTest.wpm.desc())

else:

    query = query.order_by(TypingTest.wpm.desc(), TypingTest.accuracy.desc())

top_tests = query.limit(50).all()

podium = top_tests[:3] if len(top_tests) >= 3 else top_tests

# 3. Calculate User Standing & Percentile

user_rank_info = None

if current_user.is_authenticated:

    user_best = TypingTest.query.filter_by(user_id=current_user.id, is_ranked=True, suspicious=False).order_by(TypingTest.wpm.desc()).first()

    if user_best:

        total_verified = query.count()

        higher_count = query.filter(TypingTest.wpm > user_best.wpm).count()

        rank = higher_count + 1

        pct = round((((total_verified - rank + 1) / total_verified) * 100.0, 1) if total_verified else 100.0
```

```
user_rank_info = {

    'rank': rank,

    'percentile': pct,

    'best_wpm': user_best.wpm,

    'best_acc': user_best.accuracy

}

return render_template(

    'leaderboard/index.html',

    tests=top_tests,

    podium=podium,

    ranked_users=[],

    user_rank_info=user_rank_info,

    current_sort=sort_by,

    current_mode=mode_filter

)

=== FILE: .\app\routes\learn.py ===

from flask import Blueprint, render_template, request, jsonify, redirect, url_for

from flask_login import login_required, current_user

from app import db

from app.models.curriculum import LessonStage, UserLessonProgress

from app.models.typing import TypingTest

learn_bp = Blueprint('learn', __name__)

CURRICULUM_TRACKS = [

    # 1. BEGINNER TRACK (0 - 40 WPM)

    ('beginner', 1, "Keyboard Orientation & Posture", "intro", "Learn home row resting anchors, hand ergonomics, and finger coordination.", "f j f j f j dk dk sl sl a; a; f j dk sl a;", 15.0, 90.0, 2),

    ('beginner', 2, "Home Row Mastery", "home-row", "Anchor muscle memory exclusively on the baseline resting keys.", "asdf jkl; a fad flask fall glad half dash salad flash salsa lad fall", 20.0, 92.0, 2),

    ('beginner', 3, "Top Row Extensions", "top-row", "Train upward vertical reaches without shifting wrists off the desk.", "quit write power tower quote trip your wipe wire pure quiet root weep", 25.0, 92.0, 2),
```

('beginner', 4, "Bottom Row Transitions", "bottom-row", "Practice downward finger tucks while maintaining palm stability.", "cabin van mix zoom bomb zinc exam calm civic move bank comb mimic", 28.0, 92.0, 2),

2. INTERMEDIATE TRACK (40 - 70 WPM)

('intermediate', 1, "Contralateral Shift Synchronization", "shifts", "Synchronize shift-key holds for effortless capitalization.", "The Quick Brown Fox Jumps Over The Lazy Dog London Paris Tokyo", 38.0, 94.0, 2),

('intermediate', 2, "High-Frequency Core Vocabulary", "core-words", "Fluid execution on the top 100 most frequent English building blocks.", "their there would about which could people other first water after", 45.0, 94.0, 2),

('intermediate', 3, "Complete Sentence Cadence", "sentences", "Punctuation rhythms, sentence capitalization, and natural phrase timing.", "Consistent practice builds velocity. Never compromise accuracy for speed.", 52.0, 95.0, 2),

('intermediate', 4, "Top Number Row Reaches", "numbers", "Reach the top number row with correct finger stretches without looking.", "Invoice 8402 total 195 dollars on 2026-09-15 tracking number 7391054", 42.0, 92.0, 2),

3. ADVANCED TRACK (70 - 100 WPM)

('advanced', 1, "Complex Symbols & Brackets", "symbols", "Master hyphens, parentheses, semicolons, quotes, and code brackets.", "function(arg) { return [x, y]; } value = 'true'; (check == 100)", 60.0, 95.0, 2),

('advanced', 2, "High-Speed N-Gram Sprints", "ngrams", "Rapid bursts of frequent bigrams and trigrams (the, ing, tion, ment).", "the and for that with this from have they which would there their about", 72.0, 95.0, 2),

('advanced', 3, "Zero-Error Precision Gauntlet", "precision", "Eliminate stutter pauses with strict 98% accuracy thresholds.", "Deliberate movement precedes true velocity. Calm hands strike cleanly.", 78.0, 98.0, 3),

4. EXPERT TRACK (100+ WPM)

('expert', 1, "Professional Workplace Execution", "professional", "Simulated office memos, executive briefings, and technical reports.", "Please review the quarterly engineering deployment roadmap attached.", 95.0, 96.0, 3),

('expert', 2, "Grandmaster Endurance Sprint", "grandmaster", "Sustained high-velocity paragraphs demanding flawless cognitive stamina.", "Synchronized kinetic movement compounds into unstoppable tactile velocity under intense pressure.", 105.0, 97.0, 3)

}

@learn_bp.route('/')

def index():

Auto-seed multi-track stages if empty

if LessonStage.query.count() == 0:

for track, num, title, slug, desc, practice, min_w, min_a, req_att in CURRICULUM_TRACKS:

stage = LessonStage(

track=track,

stage_number=num,

title=title,

slug=slug,

description=desc,

practice_material=practice,

min_wpm_to_pass=min_w,

min_accuracy_to_pass=min_a,

required_attempts=req_att

)

db.session.add(stage)

db.session.commit()

active_track = request.args.get('track', 'beginner').lower()

stages = LessonStage.query.filter_by(track=active_track).order_by(LessonStage.stage_number.asc()).all()

user_progress_map = {}

if current_user.is_authenticated:

records = UserLessonProgress.query.filter_by(user_id=current_user.id).all()

user_progress_map = {r.stage_id: r for r in records}

return render_template(

'learn/index.html',

stages=stages,

active_track=active_track,

progress_map=user_progress_map

)

@learn_bp.route('/stage/<slug>')

def lesson(slug):

stage = LessonStage.query.filter_by(slug=slug).first_or_404()

return render_template('learn/lesson.html', stage=stage)

@learn_bp.route('/assessment')

def skill_assessment():

return render_template('learn/assessment.html')

```
@learn_bp.route('/api/complete-stage', methods=['POST'])

@login_required

def complete_stage():

    data = request.get_json(force=True) or {}

    stage_id = int(data.get('stage_id', 1))

    wpm = float(data.get('wpm', 0.0))

    accuracy = float(data.get('accuracy', 0.0))

    stage = LessonStage.query.get_or_404(stage_id)

    passed_attempt = (wpm >= stage.min_wpm_to_pass and accuracy >= stage.min_accuracy_to_pass)

    progress = UserLessonProgress.query.filter_by(user_id=current_user.id, stage_id=stage.id).first()

    if not progress:

        progress = UserLessonProgress(user_id=current_user.id, stage_id=stage.id)

        db.session.add(progress)

    progress.total_attempts += 1

    if passed_attempt:

        progress.successful_attempts += 1

    if wpm > progress.best_wpm: progress.best_wpm = wpm

    if accuracy > progress.best_accuracy: progress.best_accuracy = accuracy

    # Require multiple successful attempts to prevent lucky one-offs

    if progress.successful_attempts >= (stage.required_attempts or 2):

        progress.completed = True

    db.session.commit()

    return jsonify({

        'success': True,

        'passed_attempt': passed_attempt,

        'stage_completed': progress.completed,
```

```
'successful_attempts': progress.successful_attempts,

        'required_attempts': stage.required_attempts or 2

    })
```

```
=== FILE: .\app\routes\multiplayer.py ===

from flask import Blueprint, render_template, request, jsonify

from flask_login import current_user

from flask_socketio import emit, join_room, leave_room

import uuid

import time

import random

from app import db, socketio

from app.models.user import User

from app.models.typing import TypingTest
```

```
multiplayer_bp = Blueprint('multiplayer', __name__)
```

```
# Primary Memory Stores
```

```
ROOMS = {}      # room_code -> dict
```

```
QUICK_QUEUE = [] # [{'sid': sid, 'user_id': uid, 'name': str, 'queued_at': float}]
```

```
SID_TO_ROOM = {} # sid -> room_code
```

```
CURATED_PASSAGES = {
```

```
    30: [
```

```
        "Speed is nothing without precision. Keep your hands balanced, breathe calmly, and glide across the keys with absolute rhythm.",
```

```
        "Consistency is the hallmark of the master typist. Every accurate strike compounds into pure flow and unmatched velocity.",
```

```
        "Real velocity is born from economy of motion. Eliminate tension from your fingers and allow muscle memory to guide every stroke."
```

```
    ],
```

```
    60: [
```

```
        "Yes, the story is real, but the Taj Mahal did not physically disappear. The magician was P. C. Sorcar Jr., one of India's most famous illusionists. On 8 November 2000, he performed an illusion in Agra in which the Taj Mahal appeared to vanish for about two minutes.",
```

```
        "It is a truth universally acknowledged, that a single man in possession of a good fortune, must be in want of a wife. However little known the feelings or views of such a man may be on his first entering a neighbourhood, this truth is fixed.",
```



```
"Distributed event streaming architectures enable modern services to communicate asynchronously with high throughput and resilience against failures by decoupling producers from consumers through immutable append-only logs."
```

```
],

120: [
```

```
"It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of light, it was the season of darkness, it was the spring of hope, it was the winter of despair."
```

```
]

}
```

```
def fetch_passage(duration, custom_text=None):
```

```
    if custom_text and len(custom_text.strip()) >= 10:
```

```
        return custom_text.strip()
```

```
    if duration <= 30:
```

```
        return random.choice(CURATED_PASSAGES[30])
```

```
    elif duration <= 60:
```

```
        return random.choice(CURATED_PASSAGES[60])
```

```
    return random.choice(CURATED_PASSAGES[120])
```

```
def serialize_public_rooms():
```

```
    public_list = []
```

```
    for code, r in ROOMS.items():
```

```
        if r['type'] == 'public' and r['status'] == 'waiting':
```

```
            active_count = len([p for p in r['players'].values() if not p.get('left_early')])
```

```
            public_list.append({
```

```
                'code': code,
```

```
                'name': r['name'],
```

```
                'host_name': r['host_name'],
```

```
                'current_players': active_count,
```

```
                'max_players': r['max_players'],
```

```
                'duration': r['duration'],
```

```
                'status': r['status']
```

```
            })
```

```
    return public_list
```

```
def calculate_standings(room):
```

```
    plist = list(room['players'].values())
```

```
    plist.sort(key=lambda x: (
```

```
        x.get('left_early', False),
```

```
        not x.get('finished', False),
```

```
        x.get('place') if x.get('place') is not None else 999,
```

```
        -(x.get('wpm') or 0),
```

```
        -(x.get('accuracy') or 0)
```

```
    ))
```

```
    return plist
```

```
@multiplayer_bp.route('/')
```

```
def index():
```

```
    user_elo = current_user.elo_rating if current_user.is_authenticated else 1000
```

```
    user_division = current_user.rank_division if current_user.is_authenticated else "Bronze"
```

```
    user_badge = current_user.rank_badge if current_user.is_authenticated else {"icon": "🏅", "color": "#b45309"}
```

```
    wins = current_user.ranked_wins if current_user.is_authenticated else 0
```

```
    losses = current_user.ranked_losses if current_user.is_authenticated else 0
```

```
    return render_template(
```

```
        'multiplayer/room.html',
```

```
        user_elo=user_elo,
```

```
        user_division=user_division,
```

```
        user_badge=user_badge,
```

```
        wins=wins,
```

```
        losses=losses
```

```
    )
```

```
@multiplayer_bp.route('/api/public-rooms')
```

```
def get_public_rooms():
```

```
    return jsonify({'rooms': serialize_public_rooms()})
```

```
# =====

# Socket.IO Multiplayer Lifecycle Handlers

# =====

@socketio.on('request_public_rooms')

def handle_request_public_rooms():

    emit('public_rooms_update', {'rooms': serialize_public_rooms()})

@socketio.on('create_room')

def handle_create_room(data):

    sid = request.sid

    room_type = data.get('type', 'public') # 'public' or 'private'

    name = (data.get('name') or f"Room-{random.randint(100, 999)}").strip()

    player_name = (data.get('player_name') or 'Pilot').strip()

    duration = int(data.get('duration', 60))

    max_players = 2 if room_type == 'private' else int(data.get('max_players', 4))

    custom_text = (data.get('custom_text') or '').strip()

    cleanup_player(sid)

    code_prefix = "1V1" if room_type == 'private' else "PUB"

    room_code = f"{code_prefix}-{uuid.uuid4().hex[:6].upper()}"

    ROOMS[room_code] = {

        'code': room_code,

        'name': name,

        'type': room_type,

        'host_sid': sid,

        'host_name': player_name,

        'max_players': max_players,

        'duration': duration,

        'custom_text': custom_text,

        'text': fetch_passage(duration, custom_text),


```

```
'status': 'waiting',

'race_start_time': None,

'created_at': time.time(),

'players': {}

}

ROOMS[room_code]['players'][sid] = {

    'sid': sid,

    'user_id': current_user.id if current_user.is_authenticated else None,

    'name': player_name,

    'ready': True, # Host initialized as ready

    'is_host': True,

    'progress': 0,

    'wpm': 0,

    'accuracy': 100,

    'errors': 0,

    'finished': False,

    'finish_time': None,

    'place': None,

    'left_early': False

}

SID_TO_ROOM[sid] = room_code

join_room(room_code)

emit('room_joined', {'room': ROOMS[room_code], 'your_sid': sid})

emit('public_rooms_update', {'rooms': serialize_public_rooms()}, broadcast=True)

@socketio.on('join_room')

def handle_join_room(data):

    sid = request.sid

    room_code = (data.get('code') or '').strip().upper()

    player_name = (data.get('player_name') or 'Pilot').strip()


```

```
if room_code not in ROOMS:

    emit('join_error', {'message': f"Room '{room_code}' does not exist or has concluded."})

    return
```

```
room = ROOMS[room_code]
```

```
if room['status'] != 'waiting':

    emit('join_error', {'message': "This match is currently in progress or finished."})

    return
```

```
active_racers = [p for p in room['players'].values() if not p.get('left_early')]
```

```
if len(active_racers) >= room['max_players']:

    emit('join_error', {'message': f"Room '{room_code}' has reached capacity."})

    return
```

```
if sid in room['players']:

    emit('room_joined', {'room': room, 'your_sid': sid})

    return
```

```
cleanup_player(sid)
```

```
room['players'][sid] = {

    'sid': sid,

    'user_id': current_user.id if current_user.is_authenticated else None,

    'name': player_name,

    'ready': False,

    'is_host': False,

    'progress': 0,

    'wpm': 0,

    'accuracy': 100,

    'errors': 0,

    'finished': False,
```

```
    'finish_time': None,

    'place': None,

    'left_early': False

}
```

```
SID_TO_ROOM[sid] = room_code
```

```
join_room(room_code)
```

```
emit('room_joined', {'room': room, 'your_sid': sid})

emit('room_state_updated', {'room': room}, room=room_code)

emit('public_rooms_update', {'rooms': serialize_public_rooms()}, broadcast=True)
```

```
@socketio.on('toggle_ready')
```

```
def handle_toggle_ready():
```

```
    sid = request.sid
```

```
    room_code = SID_TO_ROOM.get(sid)
```

```
    if not room_code or room_code not in ROOMS:
```

```
        return
```

```
    room = ROOMS[room_code]
```

```
    if room['status'] != 'waiting':
```

```
        return
```

```
    if sid in room['players']:
```

```
        room['players'][sid]['ready'] = not room['players'][sid]['ready']
```

```
        emit('room_state_updated', {'room': room}, room=room_code)
```

```
@socketio.on('start_match')
```

```
def handle_start_match():
```

```
    sid = request.sid
```

```
    room_code = SID_TO_ROOM.get(sid)
```

```
    if not room_code or room_code not in ROOMS:
```

```
        return
```

```
room = ROOMS[room_code]

if room['status'] != 'waiting':

    return

if room['host_sid'] != sid:

    emit('action_error', {'message': 'Only the room host can start the match.'})

    return

players = room['players']

active_players = [p for p in players.values() if not p.get('left_early')]

if room['type'] in ['private', 'quick'] and len(active_players) < 2:

    emit('action_error', {'message': 'Cannot start a 1v1 match without an opponent in room.'})

    return

not_ready = [p['name'] for p in active_players if not p['ready']]

if not_ready:

    emit('action_error', {'message': f'Waiting for racers to ready up: {', '.join(not_ready)}'})

    return

room['status'] = 'countdown'

for p in players.values():

    p['progress'] = 0

    p['wpm'] = 0

    p['accuracy'] = 100

    p['errors'] = 0

    p['finished'] = False

    p['finish_time'] = None

    p['place'] = None

    p['left_early'] = False

emit('match_countdown_started', {
```

```
'room_code': room_code,

'duration': room['duration'],

'text': room['text']

}, room=room_code)

emit('public_rooms_update', {'rooms': serialize_public_rooms()}, broadcast=True)

@socketio.on('client_race_active')

def handle_client_race_active():

    sid = request.sid

    room_code = SID_TO_ROOM.get(sid)

    if room_code and room_code in ROOMS:

        room = ROOMS[room_code]

        if room['status'] == 'countdown':

            room['status'] = 'in_progress'

            room['race_start_time'] = time.time()

# =====

# Real-Time Progress & Independent Finishes

# =====

@socketio.on('progress_update')

def handle_progress_update(data):

    sid = request.sid

    room_code = SID_TO_ROOM.get(sid)

    if not room_code or room_code not in ROOMS:

        return

    room = ROOMS[room_code]

    if room['status'] not in ['countdown', 'in_progress']:

        return

if sid not in room['players']:
```

```

        return

player = room['players'][sid]

if player.get('finished') or player.get('left_early'):

    return

progress = max(0, min(100, float(data.get('progress', 0))))

wpm = max(0, float(data.get('wpm', 0)))

accuracy = max(0, min(100, float(data.get('accuracy', 100))))

errors = int(data.get('errors', 0))

player['progress'] = progress

player['wpm'] = wpm

player['accuracy'] = accuracy

player['errors'] = errors

# Process Individual Finish without disrupting other racers

if progress >= 100 and not player['finished']:

    player['finished'] = True

finished_racers = [p for p in room['players'].values() if p['finished'] and not p.get('left_early')]

player['place'] = len(finished_racers)

elapsed = time.time() - room['race_start_time'] if room.get('race_start_time') else room['duration']

player['finish_time'] = round(elapsed, 2)

if player.get('user_id'):

    try:

        test_rec = TypingTest(

            user_id=player['user_id'],

            mode='multiplayer',

            duration=float(player['finish_time']),

            wpm=round(wpm, 1),

```

```

        raw_wpm=round(wpm, 1),

        accuracy=round(accuracy, 1),

        consistency=100.0,

        errors=errors,

        suspicious=False

    )

    db.session.add(test_rec)

    db.session.commit()

except Exception:

    db.session.rollback()

active_remaining = [p for p in room['players'].values() if not p['finished'] and not p.get('left_early')]

all_done = (len(active_remaining) == 0)

if all_done:

    room['status'] = 'finished'

emit('individual_player_finished', {

    'player': player,

    'remaining_count': len(active_remaining),

    'all_finished': all_done,

    'standings': calculate_standings(room) if all_done else []

}, room=room_code)

emit('room_progress_update', {'players': list(room['players'].values())}, room=room_code)

@socketio.on('time_expired_finish')

def handle_time_expired():

    sid = request.sid

    room_code = SID_TO_ROOM.get(sid)

    if not room_code or room_code not in ROOMS:

        return

```

```
room = ROOMS[room_code]

if room['status'] != 'in_progress':

    return

room['status'] = 'finished'

for p in room['players'].values():

    if not p['finished'] and not p.get('left_early'):

        p['finished'] = True

        p['place'] = 'DNF' if p['progress'] < 100 else p.get('place', 99)
```

```
emit('race_concluded', {

    'room': room,

    'standings': calculate_standings(room)

}, room=room_code)
```

```
@socketio.on('leave_race_after_finish')
```

```
def handle_leave_after_finish():

    sid = request.sid

    room_code = SID_TO_ROOM.get(sid)

    if not room_code or room_code not in ROOMS:

        emit('left_room_confirmed')

        return
```

```
room = ROOMS[room_code]

leave_room(room_code, sid=sid)

SID_TO_ROOM.pop(sid, None)
```

```
emit('left_room_confirmed')
```

```
active_remaining = [p for p in room['players'].values() if not p['finished'] and not p.get('left_early')]

if len(active_remaining) == 0 and room['status'] == 'in_progress':

    room['status'] = 'finished'

    emit('race_concluded', {
```

```
    'room': room,

    'standings': calculate_standings(room)

}, room=room_code)
```

```
@socketio.on('rematch_request')
```

```
def handle_rematch():

    sid = request.sid

    room_code = SID_TO_ROOM.get(sid)

    if not room_code or room_code not in ROOMS:

        return
```

```
room = ROOMS[room_code]

room['status'] = 'waiting'

room['race_start_time'] = None

room['text'] = fetch_passage(room['duration'], room.get('custom_text'))
```

```
for p in room['players'].values():
```

```
    p['progress'] = 0

    p['wpm'] = 0

    p['accuracy'] = 100

    p['errors'] = 0

    p['finished'] = False

    p['finish_time'] = None

    p['place'] = None

    p['left_early'] = False

    p['ready'] = (p['sid'] == room['host_sid'])
```

```
emit('rematch_accepted', {'room': room}, room=room_code)
```

```
emit('public_rooms_update', {'rooms': serialize_public_rooms()}, broadcast=True)
```

```
# =====
```

```
# Automated Quick Match 1v1 Queue
```

```
# =====
```

```
@socketio.on('join_quick_queue')

def handle_join_quick_queue(data):

    sid = request.sid

    player_name = (data.get('name') or 'Pilot').strip()

    user_id = current_user.id if current_user.is_authenticated else None

    cleanup_player(sid)

    for q in list(QUICK_QUEUE):

        if q['sid'] == sid:

            QUICK_QUEUE.remove(q)

    if len(QUICK_QUEUE) > 0:

        opponent = QUICK_QUEUE.pop(0)

        if opponent['sid'] == sid:

            QUICK_QUEUE.append({'sid': sid, 'user_id': user_id, 'name': player_name, 'queued_at': time.time()})

            emit('quick_queue_waiting')

            return

    match_code = f"QM-{uuid.uuid4().hex[:6].upper()}"

    duration = 60

    text = fetch_passage(duration)

    ROOMS[match_code] = {

        'code': match_code,

        'name': 'Quick 1v1 Match',

        'type': 'quick',

        'host_sid': sid,

        'host_name': player_name,

        'max_players': 2,

        'duration': duration,
```

```
'custom_text': '',

'text': text,

'status': 'waiting',

'race_start_time': None,

'created_at': time.time(),

'players': {

    sid: {

        'sid': sid,

        'user_id': user_id,

        'name': player_name,

        'ready': True,

        'is_host': True,

        'progress': 0,

        'wpm': 0,

        'accuracy': 100,

        'errors': 0,

        'finished': False,

        'finish_time': None,

        'place': None,

        'left_early': False

    },

    opponent['sid']: {

        'sid': opponent['sid'],

        'user_id': opponent['user_id'],

        'name': opponent['name'],

        'ready': True,

        'is_host': False,

        'progress': 0,

        'wpm': 0,

        'accuracy': 100,

        'errors': 0,

        'finished': False,

        'finish_time': None,
```

```

        'place': None,

        'left_early': False

    }

}

}

SID_TO_ROOM[sid] = match_code

SID_TO_ROOM[opponent['sid']] = match_code


join_room(match_code, sid=sid)

join_room(match_code, sid=opponent['sid'])


emit('quick_match_paired', {'room': ROOMS[match_code], 'your_sid': sid}, room=sid)

emit('quick_match_paired', {'room': ROOMS[match_code], 'your_sid': opponent['sid']], room=opponent['sid'])

else:

    QUICK_QUEUE.append({'sid': sid, 'user_id': user_id, 'name': player_name, 'queued_at': time.time()})

    emit('quick_queue_waiting')


@socketio.on('cancel_quick_queue')

def handle_cancel_quick_queue():

    sid = request.sid

    for q in list(QUICK_QUEUE):

        if q['sid'] == sid:

            QUICK_QUEUE.remove(q)

    emit('quick_queue_cancelled')


# =====

# Disconnection Cleanups

# =====


@socketio.on('leave_room_voluntary')

def handle_leave_room_voluntary():

    cleanup_player(request.sid)

```

```

emit('left_room_confirmed')

emit('public_rooms_update', {'rooms': serialize_public_rooms()}, broadcast=True)


@socketio.on('disconnect')

def handle_disconnect():

    cleanup_player(request.sid)

    emit('public_rooms_update', {'rooms': serialize_public_rooms()}, broadcast=True)


def cleanup_player(sid):

    for q in list(QUICK_QUEUE):

        if q['sid'] == sid:

            QUICK_QUEUE.remove(q)


    room_code = SID_TO_ROOM.pop(sid, None)

    if not room_code or room_code not in ROOMS:

        return


    room = ROOMS[room_code]

    leave_room(room_code, sid=sid)


    if sid in room['players']:

        player = room['players'][sid]

        if room['status'] in ['countdown', 'in_progress'] and not player.get('finished'):

            player['left_early'] = True

            player['place'] = 'DNF'

        elif room['status'] == 'waiting':

            del room['players'][sid]


    active_members = [p for p in room['players'].values() if not p.get('left_early')]

    if not active_members:

        del ROOMS[room_code]

    return

```



```

if room['status'] == 'waiting' and room['host_sid'] == sid:

    next_sid = next(iter(room['players']))

    room['host_sid'] = next_sid

    room['host_name'] = room['players'][next_sid]['name']

    room['players'][next_sid]['is_host'] = True

    room['players'][next_sid]['ready'] = True


if room['status'] == 'in_progress':

    active_remaining = [p for p in room['players'].values() if not p['finished'] and not p.get('left_early')]

    if len(active_remaining) == 0:

        room['status'] = 'finished'

        emit('race_concluded', {

            'room': room,

            'standings': calculate_standings(room)

        }, room=room_code)


    emit('room_state_updated', {'room': room}, room=room_code)


=== FILE: .\app\routes\settings.py ===

from flask import Blueprint, render_template, request, jsonify

from flask_login import current_user

from app import db

from app.models.settings import UserSettings


settings_bp = Blueprint('settings', __name__)


DEFAULT_SETTINGS = {

    'theme': 'dark',

    'font_family': 'JetBrains Mono',

    'font_size': 22,

    'caret_style': 'smooth',

    'sound_enabled': True,

    'sound_theme': 'mechanical',

```

```

'sound_volume': 0.7,

'show_keyboard': True,

'default_duration': 60,

'default_content': 'words',

'default_level': 'moderate',

'blind_mode': True, # Default: ON

'ghost_mode': False # Default: OFF

}

```

```

@settings_bp.route('/')

def index():

    user_settings = None

    if current_user.is_authenticated:

        user_settings = UserSettings.query.filter_by(user_id=current_user.id).first()

    if not user_settings:

        user_settings = UserSettings(user_id=current_user.id)

        db.session.add(user_settings)

        db.session.commit()

    return render_template('settings/index.html', settings=user_settings, defaults=DEFAULT_SETTINGS)


@settings_bp.route('/api/save', methods=['POST'])

def save_settings():

    data = request.get_json(force=True) or {}

    if current_user.is_authenticated:

        settings = UserSettings.query.filter_by(user_id=current_user.id).first()

    if not settings:

        settings = UserSettings(user_id=current_user.id)

        db.session.add(settings)

    settings.theme = data.get('theme', 'dark')

    settings.font_family = data.get('font_family', 'JetBrains Mono')

    settings.font_size = int(data.get('font_size', 22))

    settings.caret_style = data.get('caret_style', 'smooth')

```

```

settings.sound_enabled = bool(data.get('sound_enabled', True))

settings.sound_theme = data.get('sound_theme', 'mechanical')

settings.sound_volume = float(data.get('sound_volume', 0.7))

settings.show_keyboard = bool(data.get('show_keyboard', True))


settings.default_duration = int(data.get('default_duration', 60))

settings.default_content = data.get('default_content', 'words')

settings.default_level = data.get('default_level', 'moderate')

settings.blind_mode = bool(data.get('blind_mode', True))

settings.ghost_mode = bool(data.get('ghost_mode', False))


db.session.commit()

return jsonify({'success': True, 'scope': 'user_profile'})


return jsonify({'success': True, 'scope': 'anonymous_session'})


@settings_bp.route('/api/defaults')

def get_defaults():

    return jsonify({'defaults': DEFAULT_SETTINGS})


@settings_bp.route('/api/reset-defaults', methods=['POST'])

def reset_to_defaults():

    if current_user.is_authenticated:

        settings = UserSettings.query.filter_by(user_id=current_user.id).first()

        if settings:

            for k, v in DEFAULT_SETTINGS.items():

                setattr(settings, k, v)

            db.session.commit()

    return jsonify({'success': True, 'defaults': DEFAULT_SETTINGS})


=== FILE: .\app\routes\typing.py ===

from flask import Blueprint, render_template, request, jsonify

from flask_login import current_user

```

```

import json

import random

import traceback

import hashlib


from datetime import date, datetime


from app import db


from app.models.typing import TypingTest, TypingText, TypingDNA

from app.models.challenge import DailyChallenge

from app.models.feedback import Announcement

from app.services.typing_analyzer import TypingAnalyzer

from app.services.anti_cheat import AntiCheatSystem

from app.services.profile_analyzer import ProfileAnalyzer

from app.services.weakness_coach import WeaknessCoach

from app.services.achievement_service import AchievementService

from app.services.content_engine import ContentEngine


typing_bp = Blueprint('typing', __name__)


@typing_bp.route('/')

def test_page():

    active_announcement = Announcement.query.filter_by(is_active=True).order_by(Announcement.id.desc()).first()

    return render_template('typing/test.html', announcement=active_announcement)


@typing_bp.route('/custom')

def custom_text_page():

    return render_template('typing/custom.html')


@typing_bp.route('/practice')

def practice_page():

    weak_keys = []

    top_confusions = []

    dna = None

```

```
if current_user.is_authenticated:

    dna = TypingDNA.query.filter_by(user_id=current_user.id).first()

    if dna:

        stats = dna.get_key_stats()

        rates = []

        for k, val in stats.items():

            if val.get('total', 0) >= 4 and k.isalpha():

                err_rate = val.get('errors', 0) / val['total']

                rates.append((k.upper(), round(err_rate * 100, 1), val.get('total', 0)))

        rates.sort(key=lambda x: x[1], reverse=True)

        weak_keys = rates[:4]

    conf_dict = dna.get_confusion_matrix()

    for exp, mapped in conf_dict.items():

        for typed, cnt in mapped.items():

            top_confusions.append((exp.upper(), typed.upper(), cnt))

        top_confusions.sort(key=lambda x: x[2], reverse=True)

    top_confusions = top_confusions[:3]

from app.routes.typing import CURRICULUM_LESSONS

return render_template(

    'typing/practice.html',

    curriculum=CURRICULUM_LESSONS,

    weak_keys=weak_keys,

    top_confusions=top_confusions,

    dna=dna

)

@typing_bp.route('/api/daily-text')

def get_daily_text():

    today = date.today()

    challenge = DailyChallenge.query.filter_by(target_date=today).first()

    if not challenge:
```

```
challenge = DailyChallenge(

    target_date=today,

    title="The Kinetic Discipline",

    content="True velocity is not rushed chaos; it is calm, deliberate movement free of hesitation and unnecessary recoil."

)

db.session.add(challenge)

db.session.commit()

return jsonify({

    'id': challenge.id,

    'title': challenge.title,

    'content': challenge.content.strip(),

    'date': challenge.target_date.strftime('%B %d, %Y')

})

@typing_bp.route('/api/text')

def get_text():

    # 1. Lesson from Academy

    lesson_key = request.args.get('lesson')

    if lesson_key and lesson_key in CURRICULUM_LESSONS:

        lesson = CURRICULUM_LESSONS[lesson_key]

        return jsonify({

            'id': 0,

            'content': lesson['content'],

            'category': lesson['title'],

            'is_code': False

        })

    # 2. Retry specific test

    retry_id = request.args.get('retry_test_id')

    if retry_id:

        prev_test = TypingTest.query.get(int(retry_id))

        if prev_test and prev_test.events_data:
```

```

events = prev_test.get_events()

if events:

    max_idx = max((ev.get('char_index', 0) for ev in events), default=0)

    text_chars = [ ' ' ] * (max_idx + 1)

    for ev in events:

        idx = ev.get('char_index', 0)

        exp = ev.get('expected', '')

        if 0 <= idx < len(text_chars):

            text_chars[idx] = exp

    reconstructed = "".join(text_chars).strip()

    if len(reconstructed) > 5:

        return jsonify({

            'id': prev_test.id,

            'content': reconstructed,

            'category': 'Retry Drill',

            'is_code': prev_test.mode == 'code'

        })

```

3. Dynamic Multi-Tiered Content Generation

```

test_type = request.args.get('test_type', 'speed').lower()

difficulty = request.args.get('level', 'moderate').lower()

code_lang = request.args.get('code_lang', 'python').lower()

```

```

content = ContentEngine.generate_segment(

    test_type=test_type,

    difficulty=difficulty,

    batch_words=60,

    code_lang=code_lang

)

```

```

return jsonify({

    'id': 0,

    'content': content,

```

```

'category': f'{test_type.replace('_', ' ').title()} ({difficulty.title()}}',

'is_code': test_type == 'coding'

})

```

```

@typing_bp.route('/api/adaptive-drill')

```

```

def get_adaptive_drill():

    uid = current_user.id if current_user.is_authenticated else None

    drill = WeaknessCoach.generate_targeted_drill(uid)

    return jsonify(drill)

```

```

@typing_bp.route('/api/submit', methods=['POST'])

```

```

def submit_test():

    try:

        data = request.get_json(force=True) or {}

        events = data.get('events', [])

        duration = max(0.1, float(data.get('duration', 1.0)))

        target_text = data.get('target_text', '')

        mode = data.get('mode', 'timed')

```

```

metrics = TypingAnalyzer.calculate_metrics(events, target_text, duration)

```

```

is_suspicious, reason = AntiCheatSystem.evaluate(events, duration, metrics['wpm'], metrics['accuracy'])

```

```

test = TypingTest(

    user_id=current_user.id if current_user.is_authenticated else None,

    mode=mode,

    duration=duration,

    wpm=metrics['wpm'],

    raw_wpm=metrics['raw_wpm'],

    accuracy=metrics['accuracy'],

    consistency=metrics['consistency'],

    rhythm_score=metrics['rhythm_score'],

    errors=metrics['incorrect_chars'],

    correct_chars=metrics['correct_chars'],

```

```

incorrect_chars=metrics['incorrect_chars'],

extra_chars=metrics['extra_chars'],

missed_chars=metrics['missed_chars'],

suspicious=is_suspicious,

suspicion_reason=reason,

timeline_data=json.dumps(data.get('timeline', [])),

events_data=json.dumps(events)

)

db.session.add(test)

db.session.commit()

if current_user.is_authenticated:

    try:

        ProfileAnalyzer.update_dna(current_user.id, events)

    except Exception:

        pass

unlocked = []

if current_user.is_authenticated:

    try:

        unlocked = AchievementService.check_and_award(current_user)

    except Exception:

        pass

return jsonify({

    'success': True,

    'test_id': test.id,

    'metrics': metrics,

    'suspicious': is_suspicious,

    'unlocked_badges': unlocked

})

except Exception as e:

```

```

db.session.rollback()

traceback.print_exc()

return jsonify({'success': False, 'error': str(e)}), 400

@typing_bp.route('/result/<int:test_id>')

def result_page(test_id):

    test = TypingTest.query.get_or_404(test_id)

    pb_wpm = 0

    avg_wpm = 0

    if test.user_id:

        user_tests = TypingTest.query.filter_by(user_id=test.user_id, suspicious=False).all()

        if user_tests:

            pb_wpm = max((t.wpm for t in user_tests), default=0)

            avg_wpm = round(sum(t.wpm for t in user_tests) / len(user_tests), 1)

    wpm = test.wpm

    if wpm >= 100:

        skill_tier = "Olympian Master"

        tier_color = "#a371f7"

    elif wpm >= 75:

        skill_tier = "Pro Typist"

        tier_color = "var(--accent)"

    elif wpm >= 50:

        skill_tier = "Fluent Typist"

        tier_color = "var(--success)"

    elif wpm >= 30:

        skill_tier = "Intermediate"

        tier_color = "var(--warning)"

    else:

        skill_tier = "Novice Apprentice"

        tier_color = "var(--text-muted)"

    xp_earned = int((test.wpm * (test.accuracy / 100.0)) * (test.duration / 10.0))

```

```

events = test.get_events()

difficult_keys = {}

hesitations = []

max_idx = max((ev.get('char_index', 0) for ev in events), default=0)

text_chars = [' '] * (max_idx + 1)

for i, ev in enumerate(events):

    idx = ev.get('char_index', 0)

    exp = ev.get('expected', '')

    if 0 <= idx < len(text_chars):

        text_chars[idx] = exp

    if not ev.get('correct', True) and exp:

        difficult_keys[exp] = difficult_keys.get(exp, 0) + 1

    if i > 0:

        dt = ev.get('timestamp', 0) - events[i-1].get('timestamp', 0)

        if dt > 0.45 and exp.strip():

            hesitations.append({

                'char': exp,

                'delay_ms': int(dt * 1000)

            })

reconstructed_text = "".join(text_chars).strip()

sorted_difficult_keys = sorted(difficult_keys.items(), key=lambda x: x[1], reverse=True)[:5]

top_hesitations = sorted(hesitations, key=lambda x: x['delay_ms'], reverse=True)[:4]

return render_template(

    'typing/result.html',

    test=test,

    pb_wpm=pb_wpm,

```

```

    avg_wpm=avg_wpm,

    skill_tier=skill_tier,

    tier_color=tier_color,

    xp_earned=xp_earned,

    difficult_keys=sorted_difficult_keys,

    hesitations=top_hesitations,

    target_text=reconstructed_text,

    events_json=json.dumps(events)

)

@typing_bp.route('/certificate/<int:test_id>')

def certificate_page(test_id):

    test = TypingTest.query.get_or_404(test_id)

    if current_user.is_authenticated:

        AchievementService.award_code(current_user, 'certified_typist')

    if test.wpm >= 70 and test.accuracy >= 98.0:

        cert_tier = "Gold Certificate"

        tier_badge = "GOLD"

        tier_color = "#f59e0b"

    elif test.wpm >= 50 and test.accuracy >= 90.0:

        cert_tier = "Silver Certificate"

        tier_badge = "SILVER"

        tier_color = "#94a3b8"

    else:

        cert_tier = "Bronze Certificate"

        tier_badge = "BRONZE"

        tier_color = "#b45309"

    cert_hash_input = f"{test.id}-{test.wpm}-{test.accuracy}-{test.completed_at}"

    cert_code = "TS-" + hashlib.sha256(cert_hash_input.encode()).hexdigest()[1:10].upper()

    candidate_name = test.user.username if test.user else "Verified Guest Typist"

```

```
return render_template(

    'typing/certificate.html',

    test=test,

    cert_tier=cert_tier,

    tier_badge=tier_badge,

    tier_color=tier_color,

    cert_code=cert_code,

    candidate_name=candidate_name

)
```

```
@typing_bp.route('/api/ghost/<int:test_id>')
```

```
def ghost_data(test_id):

    test = TypingTest.query.get_or_404(test_id)

    return jsonify({

        'test_id': test.id,

        'wpm': test.wpm,

        'events': test.get_events(),

        'timeline': test.get_timeline()

    })
```

```
CURRICULUM_LESSONS = {

    'home_row': {

        'title': "Stage 1: Home Row Foundation",

        'keys': "A S D F J K L ;",

        'description': "Anchor your muscle memory on the baseline resting keys. Keep index fingers over F and J.",

        'content': "asdf jkl; a fad flask fall glad half dash salad flash salsa jak lad fall glad ask a fad flask fall"

    },

    'top_row': {

        'title': "Stage 2: Upper Row Extensions",

        'keys': "Q W E R T Y U I O P",

        'description': "Train upward vertical reaches without shifting your wrists off the desk.",

        'content': "quite write power tower quote trip your wipe wire pure tyre quiet root weep pore write power tower quote"
```

```
    },

    'bottom_row': {

        'title': "Stage 3: Lower Row Transitions",

        'keys': "Z X C V B N M",

        'description': "Practice downward finger tucks while maintaining wrist stability.",

        'content': "cabin van mix zoom bomb zinc exam calm civic move bank comb mimic vanish cabin van mix zoom bomb zinc"

    },

    'number_row': {

        'title': "Stage 4: Top Numeric Row",

        'keys': "1 2 3 4 5 6 7 8 9 0",

        'description': "Drill long upper-row stretches for numbers and data entry without looking at your hands.",

        'content': "102 394 582 710 934 681 205 739 461 820 159 348 726 501 934 682 102 394 582 710 934"

    },

    'symbols': {

        'title': "Stage 5: Developer Syntax & Brackets",

        'keys': "{ } [ ] ( ) < > / \ ; : ' \"",

        'description': "Master punctuation and nested programming brackets crucial for software engineering.",

        'content': "\"{ [ ( < > ) ] }\"; : \" ' / \\ ( { } ) < > ; : \" ' { ( ) }\"/; : \" ' < > { [ ( < > ) ] }"

    },

    'ngrams': {

        'title': "Stage 6: Frequent English N-Grams",

        'keys': "the and tha ent ion tio for",

        'description': "Type common multi-character letter clusters as single continuous muscle memory motions.",

        'content': "the there that other their they these them then another rather whether together furthermore therefore the there that"

    }

}
```

```
=== FILE: .\app\routes\__init__.py ===
```

```
=== FILE: .\app\services\achievement_service.py ===
```

```
from datetime import datetime
```

```
from app import db
```

```
from app.models.challenge import Achievement, UserAchievement
```

```
from app.models.typing import TypingTest
```

```
BADGES_CATALOG = [

    ('first_test', 'First Contact', 'Complete your very first typing test on TypeSphere.', 'zap'),

    ('wpm_50', 'Cruising Velocity', 'Reach a verified speed of 50 WPM.', 'award'),

    ('wpm_75', 'Sonic Cadence', 'Reach a verified speed of 75 WPM.', 'trending-up'),

    ('wpm_100', 'Century Club', 'Surpass the triple-digit barrier of 100 WPM.', 'cpu'),

    ('perfect_acc', 'Flawless Flow', 'Complete a full test with exactly 100% accuracy.', 'shield'),

    ('night_owl', 'Night Owl', 'Complete a verified typing run between 12:00 AM and 5:00 AM.', 'moon'),

    ('early_bird', 'Early Bird', 'Complete a verified typing run between 5:00 AM and 8:00 AM.', 'sun'),

    ('survival_master', 'Survival Master', 'Complete a full test in Survival Mode without losing all 3 hearts.', 'heart'),

    ('certified_typist', 'Certified Scholar', 'Generate an official proficiency certificate.', 'file-text'),

    ('century_veteran', 'Century Veteran', 'Complete 100 verified tests on TypeSphere.', 'activity')

]
```

```
class AchievementService:

    @staticmethod

    def ensure_catalog():

        """Ensure all badges exist in the database with matching codes."""

        for code, title, desc, icon in BADGES_CATALOG:

            ach = Achievement.query.filter_by(code=code).first()

            if not ach:

                # Check legacy aliases (e.g. 'century' vs 'century_veteran')

                if code == 'century_veteran':

                    legacy = Achievement.query.filter_by(code='century').first()

                    if legacy:

                        legacy.code = 'century_veteran'

                        legacy.title = title

                        legacy.description = desc

                        continue

            ach = Achievement(code=code, title=title, description=desc, icon=icon)

            db.session.add(ach)
```

```
db.session.commit()
```

```
@staticmethod

def check_and_award(user):

    if not user or not user.is_authenticated:

        return []

    AchievementService.ensure_catalog()

    # Retrieve all tests logged by this user

    tests = TypingTest.query.filter_by(user_id=user.id).all()

    if not tests:

        return []

    test_count = len(tests)

    max_wpm = max((t.wpm for t in tests), default=0)

    perfect_tests = any(t.accuracy >= 100.0 for t in tests)

    survival_wins = any(t.mode == 'survival' for t in tests)

    # Check existing unlocked codes for this user

    user_achievements = UserAchievement.query.filter_by(user_id=user.id).all()

    existing_codes = set()

    for ua in user_achievements:

        if ua.achievement:

            existing_codes.add(ua.achievement.code)

    earned_codes = []

    # 1. First Test

    if test_count >= 1 and 'first_test' not in existing_codes:

        earned_codes.append('first_test')

    # 2. Speed Tiers
```



```
if max_wpm >= 50 and 'wpm_50' not in existing_codes:

    earned_codes.append('wpm_50')

if max_wpm >= 75 and 'wpm_75' not in existing_codes:

    earned_codes.append('wpm_75')

if max_wpm >= 100 and 'wpm_100' not in existing_codes:

    earned_codes.append('wpm_100')
```

3. Flawless Accuracy

```
if perfect_tests and 'perfect_acc' not in existing_codes:

    earned_codes.append('perfect_acc')
```

4. Night Owl (12 AM - 5 AM)

```
if any(0 <= t.completed_at.hour < 5 for t in tests) and 'night_owl' not in existing_codes:

    earned_codes.append('night_owl')
```

5. Early Bird (5 AM - 8 AM)

```
if any(5 <= t.completed_at.hour < 8 for t in tests) and 'early_bird' not in existing_codes:

    earned_codes.append('early_bird')
```

6. Survival Master

```
if survival_wins and 'survival_master' not in existing_codes:

    earned_codes.append('survival_master')
```

7. Century Veteran

```
if test_count >= 100 and 'century_veteran' not in existing_codes:

    earned_codes.append('century_veteran')
```

```
newly_unlocked = []

for code in earned_codes:

    ach = Achievement.query.filter_by(code=code).first()

    if ach:

        ua = UserAchievement(user_id=user.id, achievement_id=ach.id)

        db.session.add(ua)
```

```
newly_unlocked.append(ach.title)
```

```
if newly_unlocked:

    db.session.commit()
```

```
return newly_unlocked
```

```
@staticmethod

def award_code(user, code):

    """Directly award a specific badge code (e.g. for generating certificate)."""

    if not user or not user.is_authenticated:

        return False

    AchievementService.ensure_catalog()

    ach = Achievement.query.filter_by(code=code).first()

    if not ach:

        return False

    existing = UserAchievement.query.filter_by(user_id=user.id, achievement_id=ach.id).first()

    if not existing:

        ua = UserAchievement(user_id=user.id, achievement_id=ach.id)

        db.session.add(ua)

        db.session.commit()

        return True

    return False
```

=== FILE: .\app\services\adaptive_training.py ===

```
import random
```

```
from app.models.typing import TypingDNA
```

```
WORD_BANK = {

    'e': ['elephant', 'energy', 'experience', 'every', 'element', 'evening'],

    'r': ['remember', 'regular', 'rhythm', 'running', 'reactive', 'reason'],

    't': ['together', 'target', 'thought', 'transit', 'training', 'total'],

    'o': ['opinion', 'operation', 'outcome', 'original', 'observe', 'option'],
```

```
'i': ['initial', 'insight', 'infinite', 'imagine', 'install', 'improve'],

'p': ['practice', 'pattern', 'process', 'program', 'purpose', 'profile'],

'c': ['concentrate', 'current', 'constant', 'clarity', 'circuit', 'custom'],

's': ['system', 'standard', 'steady', 'speed', 'stream', 'syntax'],

'n': ['network', 'number', 'normal', 'notation', 'nature', 'nuance']

}
```

class AdaptiveTrainingService:

 @staticmethod

 def generate_drill(user_id):

 """

 Analyzes the user's DNA profile to find top 3 error-prone keys and generates a targeted drill.

 """

 dna = TypingDNA.query.filter_by(user_id=user_id).first()

 weak_keys = []

 if dna:

 stats = dna.get_key_stats()

 # Calculate error rate per key

 rates = []

 for k, val in stats.items():

 if val['total'] >= 5 and k.isalpha():

 err_rate = val['errors'] / val['total']

 rates.append((k.lower(), err_rate))

 rates.sort(key=lambda x: x[1], reverse=True)

 weak_keys = [k for k, r in rates[:3] if r > 0.05]

 if not weak_keys:

 weak_keys = ['e', 'r', 't', 'p']

 # Compose drill focusing on weak letter frequencies

 drill_words = []

 for _ in range(25):

 chosen_key = random.choice(weak_keys)

 words_for_key = WORD_BANK.get(chosen_key, ['quick', 'focus', 'accuracy', 'sphere'])

 drill_words.append(random.choice(words_for_key))

 random.shuffle(drill_words)

 return " ".join(drill_words), weak_keys

=== FILE: .\app\services\admin_security.py ===

import uuid

from functools import wraps

from datetime import datetime

from flask import request, abort, redirect, url_for, flash, session, g

from flask_login import current_user

from app import db

from app.models.admin import RolePermission, AdminAuditLog, VisitorTraffic, SecurityEvent, UserActivity

def log_admin_action(action, target_type, target_id=None, details=None):

 """Securely write an uneditable audit entry."""

 try:

 log = AdminAuditLog(

 admin_id=current_user.id if current_user.is_authenticated else None,

 admin_username=current_user.username if current_user.is_authenticated else "System",

 action=action,

 target_type=target_type,

 target_id=str(target_id) if target_id is not None else None,

 details=str(details) if details else None,

 ip_address=request.headers.get('X-Forwarded-For', request.remote_addr)

)

 db.session.add(log)

 db.session.commit()

 except Exception as e:

 db.session.rollback()

```
def log_security_incident(event_type, severity, description, identifier=None):
```

```
    """Store critical auth faults and anomalous requests. """
```

```
    try:
```

```
        ev = SecurityEvent(
            event_type=event_type,
            severity=severity,
            identifier=identifier,
            ip_address=request.headers.get('X-Forwarded-For', request.remote_addr),
            user_agent=request.user_agent.string[:250] if request.user_agent else "Unknown",
            description=description
        )
```

```
        db.session.add(ev)
```

```
        db.session.commit()
```

```
    except Exception:
```

```
        db.session.rollback()
```

```
def record_user_activity(user_id, action, feature="Platform", details=None):
```

```
    """Log individual user interaction. """
```

```
    try:
```

```
        act = UserActivity(
            user_id=user_id,
            action=action,
            feature=feature,
            details=details,
            ip_address=request.headers.get('X-Forwarded-For', request.remote_addr),
            user_agent=request.user_agent.string[:250] if request.user_agent else "Unknown"
        )
```

```
        db.session.add(act)
```

```
        db.session.commit()
```

```
    except Exception:
```

```
        db.session.rollback()
```

```
def capture_traffic(app):
```

```
    """Before/after request hooks for anonymous & user analytics. """
```

```
    @app.before_request
```

```
    def track_request():
```

```
        if 'visitor_uuid' not in session:
```

```
            session['visitor_uuid'] = str(uuid.uuid4())
```

```
        # Update last_active for logged-in accounts
```

```
        if current_user.is_authenticated:
```

```
            current_user.last_active = datetime.utcnow()
```

```
    @app.after_request
```

```
    def record_traffic(response):
```

```
        # Ignore static assets to preserve high I/O throughput
```

```
        if request.path.startswith('/static') or request.path.startswith('/socket.io'):
```

```
            return response
```

```
    try:
```

```
        ua = request.user_agent
```

```
        dev = 'mobile' if ua.platform in ['android', 'iphone'] else 'desktop'
```

```
        traffic = VisitorTraffic(
```

```
            session_id=session.get('visitor_uuid', 'anon'),
```

```
            user_id=current_user.id if current_user.is_authenticated else None,
```

```
            ip_address=request.headers.get('X-Forwarded-For', request.remote_addr),
```

```
            path=request.path[:254],
```

```
            method=request.method,
```

```
            referrer=request.referrer[:254] if request.referrer else None,
```

```
            browser=ua.browser[:60] if ua.browser else "Other",
```

```
            os=ua.platform[:60] if ua.platform else "Other",
```

```
            device_type=dev,
```

```
            status_code=response.status_code
```

```
        )
```

```
        db.session.add(traffic)
```

```
        db.session.commit()

    except Exception:

        db.session.rollback()

    return response


def admin_permission_required(permission):

    """Strict decorator checking administrative capabilities."""

    def decorator(f):

        @wraps(f)

        def decorated_function(*args, **kwargs):

            if not current_user.is_authenticated:

                log_security_incident('UNAUTHENTICATED_ADMIN_ACCESS', 'HIGH', f'Anonymous access attempted at {request.path}')

                flash("Authentication required to enter admin control systems.", "warning")

                return redirect(url_for('auth.login', next=request.url))

            if not current_user.is_admin:

                log_security_incident('UNAUTHORIZED_ADMIN_ATTEMPT', 'CRITICAL', f'User @{current_user.username} tried accessing {request.path}',
identifier=current_user.username)

                flash("Access denied: You lack administrative authority.", "danger")

                return redirect(url_for('typing.test_page'))

            if not RolePermission.has_permission(current_user.role, permission):

                log_admin_action('PERMISSION_DENIED', 'route', details=f'Attempted {permission} on {request.path}')

                flash(f'Insufficient privileges: Role '{current_user.role.upper()}' cannot execute this action.', "danger")

                return redirect(url_for('admin.dashboard'))

        return f(*args, **kwargs)

    return decorated_function

return decorator


=== FILE: .\app\services\ai_typist_engine.py ===

import random

import time
```

```
AI_DIFFICULTY_PROFILES = {

    'easy': {

        'target_wpm': 32,

        'variance': 7,

        'error_chance': 0.12,

        'hesitation_chance': 0.18,

        'hesitation_range_ms': (220, 500)

    },

    'moderate': {

        'target_wpm': 62,

        'variance': 9,

        'error_chance': 0.05,

        'hesitation_chance': 0.09,

        'hesitation_range_ms': (140, 300)

    },

    'hard': {

        'target_wpm': 94,

        'variance': 11,

        'error_chance': 0.02,

        'hesitation_chance': 0.04,

        'hesitation_range_ms': (60, 160)

    },

    'expert': {

        'target_wpm': 128,

        'variance': 12,

        'error_chance': 0.006,

        'hesitation_chance': 0.015,

        'hesitation_range_ms': (30, 80)

    }

}
```

class AITypistSimulator:

```
def __init__(self, difficulty='moderate', player_target_wpm=None):

    diff = difficulty.lower()

    self.difficulty = diff if diff in AI_DIFFICULTY_PROFILES else 'moderate'

    self.profile = AI_DIFFICULTY_PROFILES[self.difficulty].copy()


    if player_target_wpm:

        self.profile['target_wpm'] = max(25, min(145, player_target_wpm + random.randint(-5, 6)))


    self.current_wpm = self.profile['target_wpm']

    self.total_chars_typed = 0

    self.errors_count = 0

    self.progress_pct = 0.0


def step(self, delta_seconds, total_chars):

    if total_chars <= 0:

        return 0.0, self.current_wpm


    jitter = random.uniform(-self.profile['variance'], self.profile['variance'])

    live_speed = max(18.0, self.profile['target_wpm'] + jitter)

    self.current_wpm = round(live_speed, 1)


    chars_per_sec = (live_speed * 5.0) / 60.0

    advance = chars_per_sec * delta_seconds


    if random.random() < self.profile['hesitation_chance']:

        delay_sec = random.uniform(*self.profile['hesitation_range_ms']) / 1000.0

        advance = max(0.0, advance - (chars_per_sec * delay_sec))


    if random.random() < self.profile['error_chance']:

        self.errors_count += 1

        advance = max(0.0, advance - random.uniform(1.0, 2.5))


    self.total_chars_typed += advance
```

```
self.progress_pct = min(100.0, (self.total_chars_typed / float(total_chars)) * 100.0)

return round(self.progress_pct, 2), self.current_wpm


=== FILE: .\app\services\anti_cheat.py ===

import statistics


class AntiCheatSystem:

    @staticmethod

    def evaluate(events, duration, wpm, accuracy):

        """

        Detect paste attacks, robotic zero-variance timings, and impossible human speed.

        """

        if not events or len(events) < 5:

            if wpm > 180 and duration < 3:

                return True, "Suspiciously short completion with super-human WPM."

            return False, None


        intervals = []

        for i in range(1, len(events)):

            dt = events[i]['timestamp'] - events[i-1]['timestamp']

            intervals.append(dt)


        # 1. Pasting Detection: 40%+ of characters typed under 12ms

        instant_count = sum(1 for dt in intervals if dt < 0.012)

        if instant_count / len(intervals) > 0.40:

            return True, "Pasted input detected (unrealistic keystroke intervals)."


        # 2. Maximum Human Limit

        if wpm > 250:

            return True, f"WPM of {round(wpm, 1)} exceeds verified physiological human limits."


        # 3. Robotic Consistency: standard deviation is near-zero (autotyper scripts)

        if len(intervals) > 20:
```

```
stddev = statistics.pstdev(intervals)

if stddev < 0.003:

    return True, f"Suspicious robotic regularity detected (variance: {stddev:.5f})."

return False, None
```

=== FILE: .\app\services\arcade_content_engine.py ===

```
import random

from app.models.arcade_content import ArcadeContentItem

from app.models.typing import TypingDNA
```

```
DEFAULT_WORDBANKS = {

    'easy': [

        "sun", "cat", "dog", "run", "sky", "red", "cup", "box", "key", "pen",

        "map", "hat", "top", "ice", "fly", "car", "sea", "air", "day", "art",

        "glad", "tree", "fast", "blue", "gold", "wind", "lamp", "book", "ship"

    ],

    'moderate': [

        "planet", "kinetic", "stream", "rhythm", "velocity", "circuit", "matrix",

        "focus", "cadence", "tactile", "engine", "quantum", "sphere", "anchor",

        "balance", "command", "transit", "standard", "channel", "motion", "action",

        "control", "forward", "measure", "pattern", "dynamic", "pressure", "network"

    ],

    'hard': [

        "algorithm", "asynchronous", "biometric", "cryptographic", "differentiation",

        "electromagnetic", "heterogeneous", "infrastructure", "jurisdiction",

        "nanotechnology", "neurochemical", "orthogonal", "pharmaceutical", "photosynthesis",

        "reconciliation", "sophisticated", "synchronization", "unprecedented", "telemetry"

    ],

    'expert': [

        "anachronistic", "antediluvian", "circumlocution", "counterintuitive",

        "crystallization", "deleterious", "disproportionate", "epistemological",

        "existentialism", "grandiloquent", "idiosyncrasy", "juxtaposition",
```

```
"lexicographical", "magnanimous", "metamorphosis", "multidimensional",

"panegyric", "phenomenological", "sesquipedalian", "synchronicity"

]

}
```

```
SPEED_RACER_TRACKS = {

    'easy': [

        "The car sped down the sunny track as the green flag waved to start the race.",

        "Smooth turns and steady hands keep the engine cool and the momentum high.",

        "Accelerate past the line with clean shifts and calm focus on every movement."

    ],

    'moderate': [

        "The open highway stretched across the desert floor under a wide expanse of pale morning sky. High velocity demands relaxed control and steady breathing. When the throttle opens every second compounds into pure forward momentum.",

        "Aerodynamic contours slice through the crosswinds while tire grip holds the apex through turn four. Precision steering and rhythmic engine shifts maintain the optimal racing line.",

        "Competitive speed is born from calculated motion. Downshifting before the curve preserves brake integrity and guarantees blistering acceleration down the main straight."

    ],

    'hard': [

        "Turbocharged combustion engines synchronize fuel injection micro-pulses across high-octane cylinders. Navigating high-speed chicanes without losing tire adhesion requires instantaneous neuromuscular reactions and unyielding concentration.",

        "Telemetry monitors indicate optimal differential gear lockup through the banking sequence. Balancing throttle modulation against mechanical slip angles differentiates elite drivers from the rest of the grid."

    ],

    'expert': [

        "Carbon-composite aerodynamic splitters generate tremendous localized downforce, counteracting high-velocity vortex turbulence through asymmetrical chicanes. Maintaining peak mechanical efficiency under extreme thermal dissipation separates champions."

    ]

}
```

```
CHARACTER_COLLECTIONS = {

    'letters': [chr(i) for i in range(97, 123)],

    'uppercase': [chr(i) for i in range(65, 91)],

    'numbers': [str(i) for i in range(10)],
```

```
'symbols': list("!@#%&*()-_+=[]{};:,<>?/"),
```

```
'punctuation': list(",.;:!?\"'-(){}")
```

```
}
```

```
class ArcadeContentEngine:
```

```
    @staticmethod
```

```
    def get_content_batch(game_mode, difficulty='moderate', count=25, user_id=None, extra_filters=None):
```

```
        extra_filters = extra_filters or {}
```

```
        diff = difficulty.lower() if difficulty.lower() in DEFAULT_WORDBANKS else 'moderate'
```

```
        weak_keys = []
```

```
        if user_id and extra_filters.get('target_weaknesses'):
```

```
            dna = TypingDNA.query.filter_by(user_id=user_id).first()
```

```
            if dna:
```

```
                stats = dna.get_key_stats()
```

```
                for k, v in stats.items():
```

```
                    if v.get('total', 0) >= 4 and (v.get('errors', 0) / v['total']) > 0.07:
```

```
                        weak_keys.append(k.lower())
```

```
        # Speed Racer Passages
```

```
        if game_mode == 'speed_racer':
```

```
            passages = SPEED_RACER_TRACKS.get(diff, SPEED_RACER_TRACKS['moderate'])
```

```
            return [random.choice(passages)]
```

```
        # Bubble Pop Single Characters
```

```
        if game_mode == 'bubble_pop':
```

```
            char_mode = extra_filters.get('char_mode', 'letters')
```

```
            if char_mode == 'weak_keys' and weak_keys:
```

```
                pool = [k.upper() for k in weak_keys] * 6
```

```
            elif char_mode == 'letters_numbers':
```

```
                pool = CHARACTER_COLLECTIONS['letters'] + CHARACTER_COLLECTIONS['numbers']
```

```
            elif char_mode == 'mixed_all':
```

```
                pool = (CHARACTER_COLLECTIONS['letters'] + CHARACTER_COLLECTIONS['uppercase'] +
```

```
                        CHARACTER_COLLECTIONS['numbers'] + CHARACTER_COLLECTIONS['symbols'])
```

```
            else:
```

```
                pool = CHARACTER_COLLECTIONS.get(char_mode, CHARACTER_COLLECTIONS['letters'])
```

```
            return [random.choice(pool) for _ in range(count)]
```

```
    # Cipher Hacker Layers
```

```
    if game_mode == 'cipher_hacker':
```

```
        layer_schemes = {
```

```
            'easy': [{"0xA1", "0xB2", "0xC3", "0xD4"}, [{"PORT:80", "SSL:ON", "SYS:OK"}, [{"HASH_44", "NET_99"}],
```

```
            'moderate': [{"0x7FA9", "0xDE4B", "0x91C0"}, [{"AES_256", "SHA_512", "TLS_13"}, [{"chmod 755", "int main()"}],
```

```
            'hard': [{"0xDEADBEEF", "0xCAFEBAFE"}, [{"RSA_4096_PKCS", "ECDSA_P384"}, [{"void* ptr = malloc(sz);"}],
```

```
            'expert': [{"0xFF00AA55BB66CC77"}, [{"const auto&& lambda = {}(){};}, [{"while(asm volatile("\n\n")){}"}]
```

```
        }
```

```
        return layer_schemes.get(diff, layer_schemes['moderate'])
```

```
    # Whack-A-Word Targets
```

```
    if game_mode == 'whack_a_word':
```

```
        target_type = extra_filters.get('target_type', 'words')
```

```
        if target_type == 'characters':
```

```
            return [random.choice(CHARACTER_COLLECTIONS['letters']) for _ in range(count)]
```

```
        if target_type == 'numbers':
```

```
            return [str(random.randint(10, 9999)) for _ in range(count)]
```

```
        if target_type == 'symbols':
```

```
            return [random.choice(CHARACTER_COLLECTIONS['symbols']) for _ in range(count)]
```

```
        return random.sample(DEFAULT_WORDBANKS[diff], min(count, len(DEFAULT_WORDBANKS[diff])))
```

```
    # Bomb Defuse Codes
```

```
    if game_mode == 'bomb_defuse':
```

```
        codes = []
```

```
        base_len = ('easy': 4, 'moderate': 6, 'hard': 8, 'expert': 10).get(diff, 6)
```

```
        pool = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
```

```
        if diff in ['hard', 'expert']:
```

```
            pool += "#!$%&*"
```

```
for i in range(count):

    codes.append(''.join(random.choices(pool, k=base_len + (i // 2))))

return codes


# Generic Word Pool (Falling Words, Zombie Defense, Space Defender, Typing Ninja)

pool = DEFAULT_WORDBANKS.get(diff, DEFAULT_WORDBANKS['moderate'])

if weak_keys:

    matched = [w for w in pool if any(k in w for k in weak_keys)]

    if len(matched) >= count:

        return random.sample(matched, count)

    return random.sample(pool, min(count, len(pool)))
```

=== FILE: \app\services\content_engine.py ===

```
import random
```

STORIES = [

"The clockmaker lived on a narrow street where the fog settled early and stayed late into the morning. In his workshop, hundreds of gears moved together in quiet harmony, measuring seconds that no one else seemed to notice. Visitors often asked why he spent years crafting pendulum clocks in an age of digital screens. He would smile, wind a brass key with steady fingers, and tell them that digital time merely counts, while mechanical time breathes.",

"A solitary satellite drifted through the dark expanse above Earth, observing city lights flickering like scattered embers across the continents. For seven years, its solar panels had absorbed the unshielded glare of the sun, powering instruments that tracked melting glaciers and shifting desert dunes. It sent back terabytes of silent telemetry, an unblinking witness to the quiet transformation of a living world.",

"The old library was built from limestone that smelled of rain and ancient paper. At its center stood an iron spiral staircase leading to balconies lined with uncatalogued manuscripts. Scholars from distant universities sat at polished walnut desks, turning pages with careful fingertips. Here, ideas written centuries ago waited patiently for minds curious enough to give them new life."

]

ARTICLES = [

"Modern computational infrastructure increasingly depends on distributed event streaming pipelines to process real-time information. By decoupling data producers from consumer applications through immutable, partitioned logs, high-throughput systems achieve fault tolerance across globally distributed server clusters. When an individual node fails, partition leaders are re-elected automatically, ensuring uninterrupted execution under high volume.",

"Cognitive ergonomics examines how human perception, memory, and motor responses interact with digital tools. Research demonstrates that tactile feedback and predictable keystroke latency significantly reduce mental fatigue during sustained intellectual tasks. When software interfaces eliminate visual friction and unexpected layout shifts, operators enter a state of sustained focus known as deep flow.",

"Renewable energy integration into regional electrical grids requires advanced battery storage and predictive demand algorithms. Wind and solar generation fluctuate with atmospheric conditions, necessitating rapid-response balancing reserves. By coordinating distributed inverter systems, municipal utilities maintain uniform voltage and frequency across peak usage hours."

]

REAL_WORLD_OFFICE = [

"Subject: Project Alpha Deployment Schedule Update. Dear Team, Please review the finalized deployment timeline for the third quarter release. We have completed integration testing on the authentication gateway and resolved all high-priority issue tickets. Final staging validation will take place on Thursday at 14:00 UTC. Please ensure your respective service monitors are active.",

"MEMORANDUM. To: Department Directors. From: Operations Committee. Date: September 15. Re: Updated Data Retention and Compliance Standards. Effective immediately, all customer interaction records must be archived using encrypted cold storage within thirty days of transaction close. Audit logs must remain accessible for regulatory compliance.",

"Executive Briefing: Preliminary market indicators show a twelve percent expansion in subscription renewals following the introduction of automated telemetry diagnostics. Resource allocation for the forthcoming fiscal cycle will prioritize infrastructure resilience, API latency optimization, and enterprise support capacity."

]

QUOTES = [

"Simplicity is prerequisite for reliability. It is not that we have so little time, but that we lose so much. The life we receive is not short, but we make it so.",

"We are what we repeatedly do. Excellence, then, is not an act, but a habit. The craftsman who respects his tools is already halfway to mastery.",

"In the middle of difficulty lies opportunity. Do not seek to follow in the footsteps of the wise; seek what they sought with patient persistence."

]

COMMON_LEXICON = {

'easy': [

"about", "after", "again", "almost", "always", "another", "answer", "around", "became", "become",

"before", "better", "between", "black", "called", "change", "children", "circle", "clean", "color",

"could", "differ", "during", "early", "earth", "every", "family", "father", "first", "follow",

"friend", "great", "group", "house", "large", "learn", "letter", "light", "little", "mother",

"never", "number", "often", "other", "people", "place", "point", "right", "school", "second",

"small", "sound", "still", "study", "their", "there", "these", "thing", "think", "three",

"through", "under", "until", "water", "where", "which", "while", "white", "without", "world"

],

'moderate': [

"account", "accurate", "addition", "address", "advance", "advantage", "analysis", "approach",

"balance", "baseline", "building", "business", "cadence", "capacity", "certain", "channel",

"cognitive", "command", "company", "complete", "condition", "consider", "constant", "control",

"decision", "describe", "develop", "direction", "discipline", "economy", "element", "energy",


```
"evidence", "example", "exercise", "experience", "feature", "frequency", "general", "history",

"important", "industry", "interest", "language", "material", "movement", "national", "natural",

"observe", "operation", "pattern", "position", "practice", "pressure", "problem", "process",

"produce", "program", "progress", "purpose", "reaction", "regular", "require", "standard",

"strategy", "structure", "system", "technique", "together", "understand", "variety", "velocity"

],

'hard': [

    "accomplishment", "algorithmic", "approximate", "architectural", "asynchronous", "authenticated",

    "biochemical", "characteristic", "chronological", "circumstance", "communication", "compensate",

    "comprehensive", "computational", "configuration", "consequential", "contemporary", "correspondence",

    "differentiation", "electromagnetic", "enthusiastic", "environmental", "extraordinary", "fundamental",

    "heterogeneous", "implementation", "inconvenience", "infrastructure", "instrumentation", "insufficient",

    "intelligence", "intermittent", "jurisdiction", "lexicographical", "mathematical", "methodology",

    "miscellaneous", "nanotechnology", "neurochemical", "organization", "parliamentary", "pharmaceutical",

    "phenomenon", "philosophical", "photosynthesis", "physiological", "psychological", "reconciliation",

    "reinforcement", "representative", "sophisticated", "subterranean", "synchronization", "telemetry"

],

'expert': [

    "anachronistic", "antediluvian", "circumlocution", "counterintuitive", "crystallization", "deleterious",

    "disproportionate", "epistemological", "existentialism", "grandiloquent", "idiosyncrasy", "incommensurable",

    "indistinguishable", "juxtaposition", "lexicographical", "magnanimous", "metamorphosis", "multidimensional",

    "obfuscation", "panegyric", "phenomenological", "quintessential", "sesquipedalian", "synchronicity",

    "unprecedented", "verisimilitude", "vicissitude", "zeitgeist", "labyrinthine", "perspicacious"

]

}
```

```
DATA_ENTRY_POOL = [

    "INV-84029 | Sophia Chen | 10880 Wilshire Blvd, Los Angeles, CA 90024 | +1 (310) 555-0194 | $3,420.50 | 2026-04-12",

    "INV-84030 | Marcus Sterling | 742 Evergreen Terr, Springfield, IL 62704 | +1 (217) 555-0142 | $1,180.00 | 2026-04-14",

    "INV-84031 | Elena Rostova | 19 Baker Street, London, UK NW1 6XE | +44 20 7946 0912 | £2,750.80 | 2026-04-15",

    "INV-84032 | Liam O'Connor | 452 Kingfisher Lane, Seattle, WA 98101 | +1 (206) 555-0188 | $4,915.25 | 2026-04-18",

    "INV-84033 | Chloe Dubois | 804 Industrial Pkwy, Austin, TX 78701 | +1 (512) 555-0163 | $890.00 | 2026-04-19"

]
```

```
class ContentEngine:

    @staticmethod

    def generate_segment(test_type='speed', difficulty='moderate', batch_words=55, code_lang='python'):

        diff = difficulty.lower() if difficulty.lower() in COMMON_LEXICON else 'moderate'

        if test_type == 'story':

            return random.choice(STORIES)

        if test_type == 'article':

            return random.choice(ARTICLES)

        if test_type == 'professional':

            return random.choice(REAL_WORLD_OFFICE)

        if test_type == 'quote':

            return random.choice(QUOTES)

        if test_type == 'data_entry':

            return "\n".join(random.sample(DATA_ENTRY_POOL, k=min(4, len(DATA_ENTRY_POOL))))

        if test_type == 'numbers':

            nums = [str(random.randint(10, 99999)) for _ in range(batch_words)]

            return " ".join(nums)

        if test_type in ['numbers_text', 'numbers_plus']:

            words = [random.choice(COMMON_LEXICON[diff]) for _ in range(batch_words)]

            for i in range(len(words)):

                if random.random() < 0.22:

                    words[i] = str(random.randint(1, 9999))

            return " ".join(words)

        if test_type == 'punctuation':
```

```

words = [random.choice(COMMON_LEXICON[diff]) for _ in range(batch_words)]

puncts = [",", ".", ":", ";", "!", "?", "\'", "\"", "-", "(", ")", " "]

for i in range(len(words)):

    p = random.choice(puncts)

    if p in [\'\'', \"\"\", \"'\"]:

        words[i] = f'{p}{words[i]}{p}' if p!='(' else p"

    else:

        words[i] = f'{words[i]}{p}'

    if random.random() < 0.35:

        words[i] = words[i].capitalize()

return " ".join(words)

```

Default Flowing Passages (Speed, Practice, Full Passage)

```

lex = COMMON_LEXICON[diff]

chosen = [random.choice(lex) for _ in range(batch_words)]

```

Structure into natural syntax

```

for i in range(0, len(chosen), 7):

    chosen[i] = chosen[i].capitalize()

    if i > 0 and not chosen[i-1].endswith('.'):

        chosen[i-1] = f'{chosen[i-1]}.'

return " ".join(chosen)

```

=== FILE: \app\services\email_service.py ===

```

import smtplib

from email.mime.multipart import MIMEMultipart

from email.mime.text import MIMEText

import threading

from flask import current_app

```

class EmailService:

```

    @staticmethod

```

```

def _send_async_email(app, msg):

```

```

    with app.app_context():

        server_host = app.config.get('MAIL_SERVER', 'smtp.gmail.com')

        port = int(app.config.get('MAIL_PORT', 587))

```

```

        raw_username = app.config.get('MAIL_USERNAME') or ''

```

```

        raw_password = app.config.get('MAIL_PASSWORD') or ''

```

```

        username = str(raw_username).strip()

```

```

        password = str(raw_password).replace(' ', '').strip()

```

```

        if not username or not password or 'your_' in username.lower() or 'your_' in password.lower():

```

```

            print("\n" + "=" * 65)

```

```

            print("✖MAIL NOT SENT: Please set MAIL_USERNAME and MAIL_PASSWORD in config.py")

```

```

            print(f"    Intended Recipient: {msg['To']}")

```

```

            print("=" * 65 + "\n")

```

```

            return

```

```

        try:

```

```

            print(f"🔌 Connecting to {server_host}:{port}...")

```

```

            server = smtplib.SMTP(server_host, port, timeout=20)

```

```

            server.ehlo()

```

```

            server.starttls()

```

```

            server.ehlo()

```

```

            server.login(username, password)

```

```

            server.sendmail(username, [msg['To']], msg.as_string())

```

```

            server.quit()

```

```

            print("=" * 65)

```

```

            print(f"✅SUCCESS: Email delivered to {msg['To']}")

```

```

            print("=" * 65 + "\n")

```

```

        except smtplib.SMTPAuthenticationError as auth_err:

```

```
print("\n" + "!" * 65)

print(f"✖MAIL AUTHENTICATION FAILED: {auth_err}")

print("    Use a 16-letter App Password, not your normal password.")

print("!" * 65 + "\n")

except Exception as e:

    print(f"\n✖MTP ERROR: {e}\n")
```

```
@staticmethod
```

```
def send_verification_email(recipient_email, username, verification_url):
```

```
    app = current_app._get_current_object()
```

```
    sender_email = app.config.get('MAIL_USERNAME', 'noreply@typesphere.com')
```

```
    sender = f"TypeSphere <{sender_email}>"
```

```
    msg = MIMEMultipart('alternative')
```

```
    msg['Subject'] = "Action Required: Verify Your TypeSphere Account"
```

```
    msg['From'] = sender
```

```
    msg['To'] = recipient_email
```

```
    text_content = f""""Hello {username},
```

```


Welcome to TypeSphere!
```

```


Please verify your email address by clicking the link below:
```

```
{verification_url}
```

```


— The TypeSphere Team
```

```
""""
```

```
    html_content = f""""<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <meta charset="utf-8">
```

```
    <style>
```

```
        body {{ font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; background-color: #0e1117; color: #f0f6fc; margin: 0; padding: 20px; }}
```

```
        .email-card {{ ( max-width: 540px; margin: 0 auto; background-color: #161b22; border: 1px solid #30363d; border-radius: 8px; padding: 35px 30px; }}
```

```
        .brand-title {{ ( font-size: 24px; font-weight: 800; color: #58a6ff; text-align: center; margin-bottom: 25px; }}
```

```
        .content {{ ( font-size: 15px; line-height: 1.6; color: #c9d1d9; }}
```

```
        .btn-wrap {{ ( text-align: center; margin: 30px 0; }}
```

```
        .btn {{ ( background-color: #238636; color: #ffffff !important; padding: 14px 28px; border-radius: 6px; text-decoration: none; font-weight: 700; font-size: 16px; display: inline-block; }}
```

```
        .footer {{ ( border-top: 1px solid #30363d; margin-top: 25px; padding-top: 15px; font-size: 12px; color: #8b949e; text-align: center; }}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
    <div class="email-card">
```

```
        <div class="brand-title"> ✖TypeSphere</div>
```

```
        <div class="content">
```

```
            <p>Hello <strong>{username}</strong></p>
```

```
            <p>Welcome to TypeSphere! To activate your account and start tracking your Biometric Typing DNA, please confirm your email address:</p>
```

```

        <div class="btn-wrap">
```

```
            <a href="{verification_url}" class="btn" target="_blank">Verify Email Address</a>
```

```
        </div>
```

```

        <p style="font-size: 13px; color: #8b949e;">Or paste this link into your browser:</p>
```

```
        <p style="word-break: break-all; font-size: 12px; color: #58a6ff;"><a href="{verification_url}" style="color: #58a6ff;">{verification_url}</a></p>
```

```
    </div>
```

```
    <div class="footer">&copy; 2026 TypeSphere. All rights reserved.</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

```
""""
```

```
    msg.attach(MIMEText(text_content, 'plain'))
```

```
    msg.attach(MIMEText(html_content, 'html'))
```

```
    threading.Thread(target=EmailService._send_async_email, args=(app, msg)).start()
```

```
@staticmethod

def send_password_reset_email(recipient_email, username, reset_url):

    app = current_app._get_current_object()

    sender_email = app.config.get('MAIL_USERNAME', 'noreply@typesphere.com')

    sender = f"TypeSphere Security <{sender_email}>"

    msg = MIMEMultipart('alternative')

    msg['Subject'] = "Reset Your TypeSphere Password"

    msg['From'] = sender

    msg['To'] = recipient_email

    text_content = f"""Hello {username},

We received a request to reset your TypeSphere account password.

Click the link below to set a new password:

{reset_url}

This link will expire in 1 hour. If you did not request a password reset, you can safely ignore this email.

— The TypeSphere Security Team

"""

    html_content = f"""<!DOCTYPE html>

<html>

<head>

<meta charset="utf-8">

<style>

body {{ font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; background-color: #0e1117; color: #f0f6fc; margin: 0; padding: 20px; }}

.email-card {{ max-width: 540px; margin: 0 auto; background-color: #161b22; border: 1px solid #30363d; border-radius: 8px; padding: 35px 30px; }}

.brand-title {{ font-size: 24px; font-weight: 800; color: #58a6ff; text-align: center; margin-bottom: 25px; }}
```

```
.content {{ font-size: 15px; line-height: 1.6; color: #c9d1d9; }}

.btn-wrap {{ text-align: center; margin: 30px 0; }}

.btn {{ background-color: #b05c2a; color: #ffffff !important; padding: 14px 28px; border-radius: 6px; text-decoration: none; font-weight: 700; font-size: 16px; display: inline-block; }}

.footer {{ border-top: 1px solid #30363d; margin-top: 25px; padding-top: 15px; font-size: 12px; color: #8b949e; text-align: center; }}

</style>

</head>

<body>

<div class="email-card">

<div class="brand-title"> #TypeSphere Security</div>

<div class="content">

<p>Hello <strong>{username}</strong></p>

<p>We received a request to reset the password for your TypeSphere account. Click the button below to establish a new password:</p>

<div class="btn-wrap">

<a href="{reset_url}" class="btn" target="_blank">Reset Your Password</a>

</div>

<p style="font-size: 13px; color: #8b949e;">This secure link expires in <strong>1 hour</strong>. If you did not request this, you can safely ignore this email.</p>

<p style="word-break: break-all; font-size: 12px; color: #58a6ff;"><a href="{reset_url}" style="color: #58a6ff;">{reset_url}</a></p>

</div>

<div class="footer">&copy; 2026 TypeSphere. All rights reserved.</div>

</div>

</body>

</html>

"""

    msg.attach(MIMEText(text_content, 'plain'))

    msg.attach(MIMEText(html_content, 'html'))

    threading.Thread(target=EmailService._send_async_email, args=(app, msg)).start()

=== FILE: .\app\services\game_socket_engine.py ===

import time
```

```
import random

from flask import request

from flask_socketio import emit, join_room, leave_room

from app import socketio, db

from app.models.game import GameRecord, ArcadeLeaderboard

from app.services.arcade_content_engine import ArcadeContentEngine

from app.services.ai_typist_engine import AITypistSimulator


ARCADE_ROOMS = {}

ARCADE_SOCKET_MAP = {}


def register_arcade_socket_events():

    @socketio.on('arcade_room_create')

    def on_arcade_create(data):

        sid = request.sid

        game_slug = data.get('game_slug')

        mode = data.get('play_mode', 'solo_ai')

        difficulty = data.get('difficulty', 'moderate')

        objective_type = data.get('objective_type', 'timed')

        objective_val = data.get('objective_val', 60)

        blind_mode = bool(data.get('blind_mode', False))

        backspace_allowed = bool(data.get('backspace_allowed', True))

        player_name = data.get('player_name', 'Pilot')

        custom_code = (data.get('room_code') or '').strip().upper()


        cleanup_socket(sid)


        room_code = custom_code or f"G-{random.randint(1000, 9999)}"

        content = ArcadeContentEngine.get_content_batch(game_slug, difficulty, count=40, extra_filters=data)


        ARCADE_ROOMS[room_code] = {

            'room_code': room_code,

            'game_slug': game_slug,

            'play_mode': mode,

            'difficulty': difficulty,

            'objective_type': objective_type,

            'objective_val': objective_val,

            'blind_mode': blind_mode,

            'backspace_allowed': backspace_allowed,

            'content': content,

            'status': 'lobby' if mode != 'solo_ai' else 'racing',

            'host_sid': sid,

            'created_at': time.time(),

            'start_time': time.time(),

            'players': {

                sid: {

                    'name': player_name,

                    'progress': 0.0,

                    'score': 0,

                    'wpm': 0,

                    'accuracy': 100,

                    'errors': 0,

                    'finished': False,

                    'ready': True,

                    'is_ai': False

                }

            }

        }

    if mode == 'solo_ai':

        ARCADE_ROOMS[room_code]['players']['ai_bot'] = {

            'name': f"CyberBot ({difficulty.title()})",

            'progress': 0.0,

            'score': 0,

            'wpm': 0,

            'accuracy': 98,
```

```

        'errors': 0,

        'finished': False,

        'ready': True,

        'is_ai': True
    }

    ARCADE_ROOMS[room_code]['ai_sim'] = AITypistSimulator(difficulty=difficulty)

    ARCADE_SOCKET_MAP[sid] = room_code

    join_room(room_code)

    emit('arcade_room_ready', ARCADE_ROOMS[room_code])

    @socketio.on('arcade_room_join')

    def on_arcade_join(data):

        sid = request.sid

        room_code = (data.get('room_code') or '').strip().upper()

        player_name = data.get('player_name', 'Pilot')

        if room_code not in ARCADE_ROOMS:

            emit('arcade_error', {'message': f'Room {room_code} not found.'})

            return

        room = ARCADE_ROOMS[room_code]

        if room['status'] != 'lobby':

            emit('arcade_error', {'message': 'Game already in progress.'})

            return

        if len(room['players']) >= 4:

            emit('arcade_error', {'message': 'Room full.'})

            return

        cleanup_socket(sid)

        room['players'][sid] = {

```

```

            'name': player_name,

            'progress': 0.0,

            'score': 0,

            'wpm': 0,

            'accuracy': 100,

            'errors': 0,

            'finished': False,

            'ready': False,

            'is_ai': False
        }

        ARCADE_SOCKET_MAP[sid] = room_code

        join_room(room_code)

        emit('arcade_room_ready', room)

        emit('arcade_roster_update', {'players': room['players']}, room=room_code)

        @socketio.on('arcade_toggle_ready')

        def on_arcade_ready():

            sid = request.sid

            room_code = ARCADE_SOCKET_MAP.get(sid)

            if not room_code or room_code not in ARCADE_ROOMS:

                return

            room = ARCADE_ROOMS[room_code]

            if sid in room['players']:

                room['players'][sid]['ready'] = not room['players'][sid]['ready']

                emit('arcade_roster_update', {'players': room['players']}, room=room_code)

        @socketio.on('arcade_start_countdown')

        def on_arcade_start():

            sid = request.sid

            room_code = ARCADE_SOCKET_MAP.get(sid)

            if not room_code or room_code not in ARCADE_ROOMS:

                return

```

```

room = ARCADE_ROOMS[room_code]

if room['host_sid'] != sid:

    return

room['status'] = 'countdown'

emit('arcade_countdown_trigger', {'duration': 3}, room=room_code)


@socketio.on('arcade_progress_sync')

def on_progress_sync(data):

    sid = request.sid

    room_code = ARCADE_SOCKET_MAP.get(sid)

    if not room_code or room_code not in ARCADE_ROOMS:

        return

    room = ARCADE_ROOMS[room_code]

    if sid not in room['players']:

        return

    p = room['players'][sid]

    p['progress'] = float(data.get('progress', 0.0))

    p['score'] = int(data.get('score', 0))

    p['wpm'] = float(data.get('wpm', 0))

    p['accuracy'] = float(data.get('accuracy', 100))

    p['errors'] = int(data.get('errors', 0))

    if data.get('finished'):

        p['finished'] = True

    emit('arcade_live_telemetry', {'players': room['players']}, room=room_code)


@socketio.on('arcade_ai_tick')

def on_ai_tick(data):

    sid = request.sid

    room_code = ARCADE_SOCKET_MAP.get(sid)

    if not room_code or room_code not in ARCADE_ROOMS:

        return

```

```

room = ARCADE_ROOMS[room_code]

if 'ai_sim' not in room or 'ai_bot' not in room['players']:

    return

    delta = float(data.get('delta', 0.25))

    total_len = int(data.get('total_chars', 300))

    prog, wpm = room['ai_sim'].step(delta, total_len)

    bot = room['players']['ai_bot']

    bot['progress'] = prog

    bot['wpm'] = wpm

    bot['score'] = int(prog * 15)

    if prog >= 100:

        bot['finished'] = True

    emit('arcade_live_telemetry', {'players': room['players']}, room=room_code)

def cleanup_socket(sid):

    room_code = ARCADE_SOCKET_MAP.pop(sid, None)

    if room_code and room_code in ARCADE_ROOMS:

        room = ARCADE_ROOMS[room_code]

        leave_room(room_code, sid=sid)

        if sid in room['players']:

            del room['players'][sid]

        if not room['players'] or (len(room['players']) == 1 and 'ai_bot' in room['players']):

            del ARCADE_ROOMS[room_code]

        else:

            emit('arcade_roster_update', {'players': room['players']}, room=room_code)

```

```

=== FILE: .\app\services\profile_analyzer.py ===

import json

from datetime import datetime

from app import db

```

```
from app.models.typing import TypingDNA
```

```
LEFT_HAND_KEYS = set("qwertasdfgzxcvbQWERTASDFGZXCVB12345~!@#$$%")
```

```
RIGHT_HAND_KEYS = set("yuiophjklmnYUIOPHJKLNM67890^&*()_+{}|:\ "<>?'-=[]\';,./")
```

```
PUNCTUATION_KEYS = set("~!@#$$%^&*()_+{}|:\ "<>?'-=[]\';,./")
```

```
NUMBER_KEYS = set("0123456789")
```

```
class ProfileAnalyzer:
```

```
    @staticmethod
```

```
    def update_dna(user_id, events):
```

```
        if not user_id or not events:
```

```
            return
```

```
        dna = TypingDNA.query.filter_by(user_id=user_id).first()
```

```
        if not dna:
```

```
            dna = TypingDNA(
```

```
                user_id=user_id,
```

```
                left_hand_accuracy=100.0,
```

```
                right_hand_accuracy=100.0,
```

```
                punctuation_accuracy=100.0,
```

```
                numbers_accuracy=100.0,
```

```
                capitals_accuracy=100.0,
```

```
                rhythm_consistency_avg=100.0,
```

```
                key_stats_json={},
```

```
                confusion_matrix_json={})
```

```
        )
```

```
        db.session.add(dna)
```

```
        db.session.flush()
```

```
        stats = dna.get_key_stats()
```

```
        confusions = dna.get_confusion_matrix()
```

```
        left_c, left_err = 0, 0
```

```
        right_c, right_err = 0, 0
```

```
        punct_c, punct_err = 0, 0
```

```
        num_c, num_err = 0, 0
```

```
        cap_c, cap_err = 0, 0
```

```
        for i, ev in enumerate(events):
```

```
            exp = ev.get('expected', "")
```

```
            typed = ev.get('typed', "")
```

```
            correct = ev.get('correct', False)
```

```
            if not exp:
```

```
                continue
```

```
            if exp not in stats:
```

```
                stats[exp] = {'total': 0, 'errors': 0, 'delays': []}
```

```
            stats[exp]['total'] = stats[exp].get('total', 0) + 1
```

```
            if not correct:
```

```
                stats[exp]['errors'] = stats[exp].get('errors', 0) + 1
```

```
                confusions.setdefault(exp, {})
```

```
                confusions[exp][typed] = confusions[exp].get(typed, 0) + 1
```

```
            if i > 0:
```

```
                dt = ev.get('timestamp', 0) - events[i-1].get('timestamp', 0)
```

```
                if 0 < dt < 2.0:
```

```
                    stats[exp].setdefault('delays', [])
```

```
                    stats[exp]['delays'].append(round(dt, 3))
```

```
                    if len(stats[exp]['delays']) > 50:
```

```
                        stats[exp]['delays'].pop(0)
```

```
            if exp in LEFT_HAND_KEYS:
```

```
                left_c += 1
```

```
            if not correct: left_err += 1
```



```
if exp in RIGHT_HAND_KEYS:
```

```
    right_c += 1
```

```
    if not correct: right_err += 1
```

```
if exp in PUNCTUATION_KEYS:
```

```
    punct_c += 1
```

```
    if not correct: punct_err += 1
```

```
if exp in NUMBER_KEYS:
```

```
    num_c += 1
```

```
    if not correct: num_err += 1
```

```
if exp.isupper():
```

```
    cap_c += 1
```

```
    if not correct: cap_err += 1
```

```
def calc_acc(total, err, current_val):
```

```
    if total == 0:
```

```
        return current_val or 100.0
```

```
    session_acc = ((total - err) / total) * 100.0
```

```
    base = current_val if current_val is not None else 100.0
```

```
    return round((base * 0.7) + (session_acc * 0.3), 1)
```

```
dna.left_hand_accuracy = calc_acc(left_c, left_err, dna.left_hand_accuracy)
```

```
dna.right_hand_accuracy = calc_acc(right_c, right_err, dna.right_hand_accuracy)
```

```
dna.punctuation_accuracy = calc_acc(punct_c, punct_err, dna.punctuation_accuracy)
```

```
dna.numbers_accuracy = calc_acc(num_c, num_err, dna.numbers_accuracy)
```

```
dna.capitals_accuracy = calc_acc(cap_c, cap_err, dna.capitals_accuracy)
```

```
dna.key_stats_json = json.dumps(stats)
```

```
dna.confusion_matrix_json = json.dumps(confusions)
```

```
dna.updated_at = datetime.utcnow()
```

```
db.session.commit()
```

```
=== FILE: .\app\services\typing_analyzer.py ===
```

```
import statistics
```

```
class TypingAnalyzer:
```

```
    @staticmethod
```

```
    def calculate_metrics(events, target_text, duration):
```

```
        """
```

```
        Accurate computation of Net WPM, Raw WPM, Accuracy, Consistency, and Rhythm.
```

```
        Standard definition: 1 Word = 5 characters (including spaces).
```

```
        """
```

```
        if duration <= 0:
```

```
            duration = 0.1
```

```
        correct_chars = 0
```

```
        incorrect_chars = 0
```

```
        extra_chars = 0
```

```
        intervals = []
```

```
        confusion_pairs = {}
```

```
        last_t = 0
```

```
        for ev in events:
```

```
            t = ev.get('timestamp', 0)
```

```
            if last_t > 0:
```

```
                intervals.append(max(0.001, t - last_t))
```

```
            last_t = t
```

```
        is_correct = ev.get('correct', False)
```

```
        expected = ev.get('expected', '')
```

```
        typed = ev.get('typed', '')
```

```
        if is_correct:
```

```
            correct_chars += 1
```

```
        else:
```

```
            incorrect_chars += 1
```

```
        if expected and typed:
```

```
confusion_pairs.setdefault(expected, {})  
  
confusion_pairs[expected][typed] = confusion_pairs[expected].get(typed, 0) + 1  
  
  
total_typed = correct_chars + incorrect_chars  
  
target_len = len(target_text)  
  
missed_chars = max(0, target_len - total_typed)  
  
  
# Standard typing formulas  
  
minutes = duration / 60.0  
  
raw_wpm = (total_typed / 5.0) / minutes if minutes > 0 else 0.0  
  
net_wpm = max(0.0, (correct_chars / 5.0) / minutes if minutes > 0 else 0.0)  
  
accuracy = (correct_chars / total_typed * 100.0) if total_typed > 0 else 0.0  
  
  
# Consistency: 100 - (coefficient of variation of intervals * 100)  
  
consistency = 100.0  
  
rhythm_score = 100.0  
  
hesitation_points = []  
  
  
if len(intervals) >= 5:  
    mean_i = statistics.mean(intervals)  
  
    std_i = statistics.pstdev(intervals)  
  
    cv = (std_i / mean_i) if mean_i > 0 else 0  
  
    consistency = max(0.0, min(100.0, 100.0 - (cv * 40.0)))  
  
    rhythm_score = round(consistency, 1)  
  
  
# Identify hesitations (> 3x mean interval)  
  
for idx, dt in enumerate(intervals):  
    if dt > (mean_i * 3.0) and dt > 0.4:  
        char = events[idx].get('expected', '')  
  
        hesitation_points.append({'char': char, 'interval': round(dt, 3)})  
  
  
return {  
    'wpm': round(net_wpm, 1),
```

```
'raw_wpm': round(raw_wpm, 1),  
  
'accuracy': round(accuracy, 1),  
  
'consistency': round(consistency, 1),  
  
'rhythm_score': rhythm_score,  
  
'correct_chars': correct_chars,  
  
'incorrect_chars': incorrect_chars,  
  
'extra_chars': extra_chars,  
  
'missed_chars': missed_chars,  
  
'hesitations': hesitation_points,  
  
'confusion_pairs': confusion_pairs  
}  
  
  
=== FILE: .\app\services\weakness_coach.py ===  
  
import random  
  
from app.models.typing import TypingDNA  
  
  
COMMON_NGRAMS = {  
    'th': ['the', 'that', 'with', 'other', 'their', 'there', 'think', 'method'],  
  
    'he': ['when', 'where', 'whether', 'together', 'rather', 'leather', 'theory'],  
  
    'in': ['into', 'point', 'maintain', 'infinite', 'origin', 'within', 'train'],  
  
    'er': ['operator', 'order', 'pattern', 'modern', 'server', 'different', 'letter'],  
  
    'an': ['balance', 'command', 'transit', 'standard', 'constant', 'channel'],  
  
    're': ['response', 'record', 'current', 'direct', 'reaction', 'regular'],  
  
    'on': ['condition', 'action', 'button', 'connection', 'session', 'direction'],  
  
    'at': ['matrix', 'format', 'update', 'database', 'strategy', 'accurate'],  
  
    'ed': ['started', 'loaded', 'executed', 'streamed', 'controlled', 'proceed']  
}  
  
  
class WeaknessCoach:  
  
    @staticmethod  
  
    def analyze_and_recommend(user_id):  
        """"  
  
        Transforms abstract key telemetry into immediate next-step actions across
```

the Academy, specific practice drills, and arcade games.

"""

if not user_id:

```
return {
    'has_data': False,
    'weak_keys': ['E', 'T', 'O', 'N'],
    'recommendation': {
        'title': 'Calibration Drill',
        'reason': 'Sign in and complete 3 standard tests to generate a personalized fingerprint.',
        'action_label': 'Start Practice',
        'action_url': '/typing/?mode=timed&duration=60',
        'stage_recommendation': 'Stage 1: Home Row',
        'game_recommendation': 'Speed Racer'
    }
}
```

dna = TypingDNA.query.filter_by(user_id=user_id).first()

if not dna:

```
return {
    'has_data': False,
    'weak_keys': ['A', 'S', 'D', 'F'],
    'recommendation': {
        'title': 'Establish Baseline',
        'reason': 'Your typing DNA requires at least one benchmark test to pinpoint hesitations.',
        'action_label': 'Take Benchmark Test',
        'action_url': '/typing/?mode=timed&duration=60',
        'stage_recommendation': 'Stage 1: Home Row',
        'game_recommendation': 'Falling Words'
    }
}
```

stats = dna.get_key_stats()

confusions = dna.get_confusion_matrix()

1. Identify slow & error-prone keys

scored_keys = []

for char, data in stats.items():

```
    if data.get('total', 0) >= 5 and char.isalpha():
        err_rate = data.get('errors', 0) / data['total']
        delays = data.get('delays', [0.15])
        avg_delay = sum(delays) / len(delays) if delays else 0.15
        # Composite score: Error rate weighted 70%, latency delay weighted 30%
        severity = (err_rate * 0.7) + (min(avg_delay, 0.5) * 0.3)
        scored_keys.append((char.lower(), err_rate, avg_delay, severity))
```

scored_keys.sort(key=lambda x: x[3], reverse=True)

top_weak = [k[0] for k in scored_keys[:4]] if scored_keys else ['e', 'r', 't']

2. Check Punctuation & Numeric Imbalances

punct_acc = dna.punctuation_accuracy or 100.0

num_acc = dna.numbers_accuracy or 100.0

if punct_acc < 90.0:

```
rec = {
    'title': 'Punctuation Cadence Imbalance',
    'reason': f'Your punctuation accuracy ({punct_acc:.1f}%) is noticeably lower than your baseline speed.',
    'action_label': 'Drill Punctuation',
    'action_url': '/typing/?content_type=punctuation&duration=60',
    'stage_recommendation': 'Stage 9: Punctuation & Symbols',
    'game_recommendation': 'Bubble Pop (Symbols Mode)'
}
```

elif num_acc < 88.0:

```
rec = {
    'title': 'Numeric Row Reaches',
    'reason': f'Your number row accuracy ({num_acc:.1f}%) causes significant hesitations during typing.',
    'action_label': 'Drill Numeric Reaches',
```

```
        'action_url': '/typing/?content_type=numbers&duration=60',

        'stage_recommendation': 'Stage 8: Number Row Reaches',

        'game_recommendation': 'Bomb Defuse'

    }

elif top_weak:

    top_char = top_weak[0].upper()

    rec = {

        'title': f"Targeting Weak Keys: {' ', ' '.join(k.upper() for k in top_weak)}",

        'reason': f"Frequent mistypes and hesitations detected on key '{top_char}' during velocity bursts.",

        'action_label': 'Launch Adaptive Drill',

        'action_url': '/typing/?mode=adaptive',

        'stage_recommendation': 'Stage 5: Key Combinations',

        'game_recommendation': 'Typing Ninja'

    }

else:

    rec = {

        'title': 'Velocity & Flow Maintenance',

        'reason': 'Your finger balance is stable across the home row. Focus on increasing raw speed.',

        'action_label': 'Enter 60s Speed Sprint',

        'action_url': '/typing/?mode=timed&duration=60',

        'stage_recommendation': 'Stage 10: Speed Building',

        'game_recommendation': 'Speed Racer'

    }

return {

    'has_data': True,

    'weak_keys': [k.upper() for k in top_weak],

    'recommendation': rec

}

=== FILE: .\app\services\_init_.py ===
```

```
=== FILE: .\app\static\css\arcade.css ===

/* TypeSphere Unified Arcade Canvas & Animation System */

#arcade-viewport {

    position: relative;

    width: 100%;

    height: 480px;

    background: var(--bg-secondary);

    border: 2px solid var(--border-color);

    border-radius: var(--radius);

    overflow: hidden;

    outline: none;

    user-select: none;

    box-shadow: inset 0 0 35px rgba(0, 0, 0, 0.65);

}

/* =====

SPEED RACER MULTI-LANE TRACK SYSTEM

===== */

.racer-track-surface {

    position: absolute;

    inset: 0;

    background: #14171f;

    background-image: repeating-linear-gradient(

        90deg,

        rgba(255, 255, 255, 0.03) 0px,

        rgba(255, 255, 255, 0.03) 80px,

        rgba(255, 255, 255, 0.07) 80px,

        rgba(255, 255, 255, 0.07) 82px

    );

}

.racer-lane-divder {

    position: absolute;
```

```
left: 0;

right: 0;

height: 2px;

background: repeating-linear-gradient(90deg, rgba(255,255,255,0.15) 0, rgba(255,255,255,0.15) 30px, transparent 30px, transparent 60px);

}
```

```
.racer-finish-gate {

position: absolute;

right: 40px;

top: 0;

bottom: 0;

width: 14px;

background: repeating-linear-gradient(

0deg,

#ffffff 0,

#ffffff 12px,

#111111 12px,

#111111 24px

);

box-shadow: 0 0 15px rgba(255, 255, 255, 0.4);

z-index: 5;

}
```

```
.racer-vehicle-node {

position: absolute;

display: flex;

flex-direction: column;

align-items: center;

justify-content: center;

transition: left 0.08s linear;

z-index: 10;

}
```

```
.racer-vehicle-icon {

font-size: 2.8rem;

line-height: 1;

display: inline-block;

transform: scaleX(-1);

filter: drop-shadow(0 6px 12px rgba(0, 0, 0, 0.6));

}
```

```
.racer-hud-tag {

font-family: var(--font-sans);

font-size: 0.72rem;

font-weight: 800;

padding: 0.15rem 0.5rem;

border-radius: 4px;

background: rgba(11, 15, 23, 0.9);

border: 1px solid var(--border-color);

color: var(--text-primary);

margin-top: 2px;

}
```

```
.nitro-exhaust-burn {

position: absolute;

left: -18px;

top: 45%;

transform: translateY(-50%);

width: 22px;

height: 14px;

background: radial-gradient(circle, #38bdf8, #0284c7, transparent);

border-radius: 50%;

animation: nitroIgnition 0.08s infinite alternate;

}
```

```
@keyframes nitroIgnition {
```

```
0% { transform: translateY(-50%) scale(0.85); opacity: 0.8; }

100% { transform: translateY(-50%) scale(1.4); opacity: 1; }

}
```

```
/* =====

ZOMBIE DUEL 1V1 COMBAT SYSTEM

===== */

.duel-combat-arena {

  display: flex;

  justify-content: space-between;

  align-items: flex-start;

  padding: 2rem 2.5rem;

}
```

```
.duel-combatant-card {

  display: flex;

  flex-direction: column;

  gap: 0.4rem;

  width: 240px;

}
```

```
.duel-hp-bar-container {

  width: 100%;

  height: 18px;

  background: var(--bg-card);

  border-radius: 9px;

  overflow: hidden;

  border: 1.5px solid var(--border-color);

}
```

```
.duel-hp-fill {

  height: 100%;

  transition: width 0.25s cubic-bezier(0.16, 1, 0.3, 1);
```

```
}
```

```
/* =====

BUBBLE POP SINGLE-KEY SYSTEM

===== */

.arcade-char-bubble {

  position: absolute;

  width: 52px;

  height: 52px;

  border-radius: 50%;

  display: flex;

  align-items: center;

  justify-content: center;

  font-family: var(--font-mono);

  font-weight: 800;

  font-size: 1.45rem;

  background: rgba(56, 189, 248, 0.16);

  border: 2px solid var(--accent);

  color: var(--text-primary);

  box-shadow: 0 0 16px var(--accent-glow);

  transition: transform 0.1s ease;

}
```

```
/* =====

CIPHER HACKER LAYERS

===== */

.cipher-terminal-window {

  background: #06090e;

  border: 1.5px solid #10b981;

  border-radius: var(--radius);

  padding: 1.75rem 2rem;

  font-family: var(--font-mono);

  box-shadow: 0 0 25px rgba(16, 185, 129, 0.15);
```

```
}

.cipher-layer-badge {

  display: inline-block;

  padding: 0.25rem 0.65rem;

  border-radius: 4px;

  background: rgba(16, 185, 129, 0.2);

  color: #10b981;

  font-weight: 700;

  font-size: 0.8rem;

  margin-bottom: 0.8rem;

}

=== FILE: .\app\static\css\components.css ===

/* Universal Command Palette & Modal Styling */
```

```
.palette-overlay {

  display: none;

  position: fixed;

  inset: 0;

  background: rgba(0, 0, 0, 0.75);

  backdrop-filter: blur(6px);

  z-index: 2000;

  align-items: flex-start;

  justify-content: center;

  padding-top: 12vh;

}
```

```
.palette-container {

  width: 90%;

  max-width: 620px;

  background: var(--bg-secondary);

  border: 1px solid var(--border-color);

  border-radius: var(--radius);
```

```
  box-shadow: 0 20px 40px rgba(0, 0, 0, 0.6);

  overflow: hidden;

  animation: paletteSlideDown 0.15s ease-out;

}
```

```
@keyframes paletteSlideDown {

  from {

    opacity: 0;

    transform: translateY(-15px);

  }

  to {

    opacity: 1;

    transform: translateY(0);

  }

}
```

```
.palette-search-box {

  display: flex;

  align-items: center;

  padding: 1rem 1.25rem;

  border-bottom: 1px solid var(--border-color);

  gap: 0.75rem;

}
```

```
.palette-search-input {

  width: 100%;

  background: transparent;

  border: none;

  outline: none;

  font-family: inherit;

  font-size: 1.1rem;

  color: var(--text-primary);

}
```

```
.palette-search-input::placeholder {  
  
  color: var(--text-muted);  
  
}
```

```
.palette-results {  
  
  max-height: 380px;  
  
  overflow-y: auto;  
  
  padding: 0.5rem;  
  
  display: flex;  
  
  flex-direction: column;  
  
  gap: 0.25rem;  
  
}
```

```
.palette-item {  
  
  display: flex;  
  
  align-items: center;  
  
  justify-content: space-between;  
  
  padding: 0.75rem 1rem;  
  
  border-radius: 6px;  
  
  cursor: pointer;  
  
  color: var(--text-primary);  
  
  font-size: 0.95rem;  
  
  transition: var(--transition);  
  
}
```

```
.palette-item.selected, .palette-item:hover {  
  
  background: var(--bg-card);  
  
  color: var(--accent);  
  
}
```

```
.palette-item-category {  
  
  font-size: 0.75rem;
```

```
  text-transform: uppercase;  
  
  color: var(--text-muted);  
  
  letter-spacing: 0.05em;  
  
  font-weight: 700;  
  
}
```

```
.palette-footer {  
  
  display: flex;  
  
  justify-content: space-between;  
  
  padding: 0.65rem 1.25rem;  
  
  background: var(--bg-primary);  
  
  border-top: 1px solid var(--border-color);  
  
  font-size: 0.8rem;  
  
  color: var(--text-muted);  
  
}
```

```
.kbd-badge {  
  
  background: var(--bg-card);  
  
  border: 1px solid var(--border-color);  
  
  color: var(--text-muted);  
  
  font-family: var(--font-mono);  
  
  font-size: 0.75rem;  
  
  padding: 0.15rem 0.45rem;  
  
  border-radius: 4px;  
  
}
```

```
/* Hotkey Modal */
```

```
.hotkey-row {  
  
  display: flex;  
  
  justify-content: space-between;  
  
  align-items: center;  
  
  padding: 0.6rem 0;  
  
  border-bottom: 1px solid var(--border-color);
```



```
font-size: 0.9rem;

}
```

```
.hotkey-row:last-child {

border-bottom: none;

}
```

```
/* Interactive Floating Tooltip Engine */
```

```
#typesphere-tooltip {

position: fixed;

z-index: 99999;

pointer-events: none;

background: rgba(14, 17, 23, 0.95);

backdrop-filter: blur(8px);

color: #f0f6fc;

border: 1px solid var(--accent);

box-shadow: 0 8px 24px rgba(0, 0, 0, 0.55), 0 0 10px var(--accent-glow);

padding: 0.45rem 0.85rem;

border-radius: 6px;

font-size: 0.82rem;

line-height: 1.4;

max-width: 280px;

opacity: 0;

visibility: hidden;

transition: opacity 0.12s ease, transform 0.12s ease;

transform: translateY(6px);

font-family: var(--font-sans);

}
```

```
#typesphere-tooltip.visible {

opacity: 1;

visibility: visible;

transform: translateY(0);

}
```

```
}
```

```
#typesphere-tooltip .tooltip-header {

font-weight: 700;

color: var(--accent);

font-size: 0.75rem;

text-transform: uppercase;

letter-spacing: 0.05em;

margin-bottom: 0.15rem;

display: flex;

align-items: center;

gap: 0.35rem;

}
```

```
#typesphere-tooltip .tooltip-body {

color: #e6edf3;

font-weight: 400;

}
```

```
=== FILE: \app\static\css\keyboard.css ===
```

```
.keyboard-wrapper {

margin-top: 1.5rem;

background: var(--bg-secondary);

border: 1px solid var(--border-color);

border-radius: var(--radius);

padding: 1.5rem 1rem;

display: flex;

flex-direction: column;

gap: 0.4rem;

align-items: center;

position: relative;

box-shadow: 0 8px 24px rgba(0, 0, 0, 0.4);

}
```

```
.keyboard-row {
  display: flex;

  gap: 0.4rem;

  width: 100%;

  justify-content: center;
}

.k-key {
  background: var(--bg-card);

  border: 1px solid var(--border-color);

  border-radius: 6px;

  color: var(--text-primary);

  font-family: var(--font-mono);

  font-size: 0.85rem;

  font-weight: 700;

  height: 44px;

  min-width: 44px;

  padding: 0 0.4rem;

  display: flex;

  flex-direction: column;

  align-items: center;

  justify-content: center;

  position: relative;

  cursor: pointer;

  user-select: none;

  transition: all 0.12s ease;
}

.k-key:hover {
  border-color: var(--accent);
}
```

```
.k-key.active, .k-key.clicked-active {
  background: var(--accent) !important;
  color: #ffffff !important;
  transform: translateY(2px);
  box-shadow: 0 0 10px var(--accent-glow) !important;
}
```

```
.k-key.space { min-width: 280px; }
.k-key.backspace { min-width: 75px; }
.k-key.tab { min-width: 55px; }
.k-key.caps { min-width: 65px; }
.k-key.enter { min-width: 75px; }
.k-key.shift { min-width: 80px; }
```

```
@media (max-width: 768px) {
  .k-key {
    height: 32px;
    min-width: 22px;
    font-size: 0.65rem;
    padding: 0 0.15rem;
  }
  .k-key.space { min-width: 130px; }
}
```

=== FILE: .\app\static\css\main.css ===

```
:root {
  --bg-primary: #0b0f17;
  --bg-secondary: #131924;
  --bg-card: #1c2433;
  --bg-elevated: #242f42;

  --text-primary: #f1f5f9;
  --text-secondary: #94a3b8;
```

--text-muted: #64748b;

--accent: #38bdf8;

--accent-hover: #0284c7;

--accent-glow: rgba(56, 189, 248, 0.18);

--success: #10b981;

--success-bg: rgba(16, 185, 129, 0.12);

--danger: #f43f5e;

--danger-bg: rgba(244, 63, 94, 0.12);

--warning: #f59e0b;

--warning-bg: rgba(245, 158, 11, 0.12);

--border-color: #243044;

--border-focus: #38bdf8;

--radius-sm: 6px;

--radius: 10px;

--radius-lg: 14px;

--font-mono: 'JetBrains Mono', 'Fira Code', monospace;

--font-sans: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;

--typing-font-size: 1.35rem;

--transition-fast: 0.12s ease;

--transition: 0.2s cubic-bezier(0.16, 1, 0.3, 1);

--shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.3);

--shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.4);

}

[data-theme="light"] {

--bg-primary: #f8fafc;

--bg-secondary: #ffffff;

--bg-card: #f1f5f9;

--bg-elevated: #e2e8f0;

--text-primary: #0f172a;

--text-secondary: #475569;

--text-muted: #64748b;

--accent: #0284c7;

--accent-hover: #0369a1;

--accent-glow: rgba(2, 132, 199, 0.15);

--border-color: #cbd5e1;

--border-focus: #0284c7;

}

[data-theme="cyber"] {

--bg-primary: #05050d;

--bg-secondary: #0d0f22;

--bg-card: #151833;

--bg-elevated: #1e2246;

--text-primary: #00f2fe;

--text-secondary: #798bb4;

--text-muted: #4e5b7c;

--accent: #ff007f;

--accent-hover: #d6006b;

--accent-glow: rgba(255, 0, 127, 0.3);

--border-color: #272d5c;

--border-focus: #ff007f;

}

[data-theme="sepia"] {

--bg-primary: #fbf0d9;

--bg-secondary: #f4e7cc;

--bg-card: #eae0c8;

--bg-elevated: #dfd4bb;

--text-primary: #3d2f1f;

--text-secondary: #6e5c46;

```
--text-muted: #8c785f;

--accent: #b45309;

--accent-hover: #92400e;

--accent-glow: rgba(180, 83, 9, 0.2);

--border-color: #d8cbb0;

--border-focus: #b45309;

}
```

```
* {

margin: 0;

padding: 0;

box-sizing: border-box;

}
```

```
html, body {

background-color: var(--bg-primary);

color: var(--text-primary);

font-family: var(--font-sans);

font-size: 15px;

line-height: 1.5;

min-height: 100vh;

display: flex;

flex-direction: column;

}
```

```
/* =====

MODAL BACKDROP POPUP SYSTEM (GUARANTEES FLOATING OVERLAY)

===== */
```

```
.modal-backdrop {

position: fixed !important;

inset: 0 !important;

width: 100vw !important;

height: 100vh !important;
```

```
background: rgba(0, 0, 0, 0.82) !important;

backdrop-filter: blur(6px) !important;

-webkit-backdrop-filter: blur(6px) !important;

z-index: 99999 !important;

display: flex;

align-items: center;

justify-content: center;

padding: 1rem;

}
```

```
.modal-dialog {

background: var(--bg-secondary) !important;

border: 1.5px solid var(--accent) !important;

border-radius: var(--radius) !important;

box-shadow: 0 25px 60px rgba(0, 0, 0, 0.8) !important;

width: 100%;

max-width: 480px;

position: relative;

animation: modalScaleIn 0.15s cubic-bezier(0.16, 1, 0.3, 1) forwards;

}
```

```
@keyframes modalScaleIn {

from { opacity: 0; transform: scale(0.94) translateY(-10px); }

to { opacity: 1; transform: scale(1) translateY(0); }

}
```

```
.modal-close-btn {

position: absolute;

right: 1rem;

top: 1rem;

background: transparent;

border: 1px solid var(--border-color);

color: var(--text-muted);
```

```
width: 30px;

height: 30px;

border-radius: 50%;

cursor: pointer;

display: inline-flex;

align-items: center;

justify-content: center;

font-size: 1.1rem;

line-height: 1;

}

.modal-close-btn:hover {

color: var(--text-primary);

border-color: var(--accent);

}

/* Base Navbar */

.navbar {

background-color: var(--bg-secondary);

border-bottom: 1px solid var(--border-color);

padding: 0.75rem 1.5rem;

display: flex;

justify-content: space-between;

align-items: center;

position: sticky;

top: 0;

z-index: 1000;

}

.brand {

display: inline-flex;

align-items: center;

gap: 0.65rem;
```

```
text-decoration: none;

font-weight: 800;

font-size: 1.25rem;

color: var(--accent);

letter-spacing: -0.02em;

}

.nav-links {

display: flex;

list-style: none;

gap: 0.4rem;

align-items: center;

}

.nav-link {

color: var(--text-secondary);

text-decoration: none;

font-size: 0.9rem;

font-weight: 500;

padding: 0.45rem 0.75rem;

border-radius: var(--radius-sm);

transition: var(--transition-fast);

}

.nav-link:hover,

.nav-link.active {

color: var(--text-primary);

background: var(--bg-card);

}

/* Button System */

.btn {

background: var(--bg-card);
```

```
color: var(--text-primary);

border: 1px solid var(--border-color);

padding: 0.5rem 1rem;

border-radius: var(--radius-sm);

cursor: pointer;

font-family: inherit;

font-weight: 600;

font-size: 0.88rem;

display: inline-flex;

align-items: center;

justify-content: center;

gap: 0.45rem;

transition: var(--transition-fast);

text-decoration: none;

user-select: none;

white-space: nowrap;

}
```

```
.btn:hover:not(:disabled) {

    border-color: var(--accent);

    color: var(--text-primary);

    background: var(--bg-elevated);

}
```

```
.btn.disabled {

    opacity: 0.5;

    cursor: not-allowed;

}
```

```
.btn-primary {

    background: var(--accent);

    color: #04101e;

    border-color: var(--accent);

}
```

```
}
```

```
.btn-primary:hover:not(:disabled) {

    background: var(--accent-hover);

    border-color: var(--accent-hover);

    color: #ffffff;

    box-shadow: 0 0 15px var(--accent-glow);

}
```

```
.btn-sm {

    padding: 0.35rem 0.65rem;

    font-size: 0.8rem;

}
```

```
.btn-danger {

    border-color: var(--danger);

    color: var(--danger);

    background: var(--danger-bg);

}
```

```
.btn-danger:hover:not(:disabled) {

    background: var(--danger);

    color: #fff;

}
```

```
.btn.active-choice {

    background: var(--accent) !important;

    border-color: var(--accent) !important;

    color: #04101e !important;

    font-weight: 700 !important;

}
```

```
/* Form Controls */
```

```
.dropdown-control {
  display: flex;

  align-items: center;

  gap: 0.35rem;
}

.dropdown-label {
  font-size: 0.75rem;

  font-weight: 700;

  text-transform: uppercase;

  color: var(--text-muted);
}

.form-select {
  background: var(--bg-card);

  border: 1px solid var(--border-color);

  color: var(--text-primary);

  padding: 0.4rem 0.75rem;

  border-radius: var(--radius-sm);

  font-weight: 600;

  font-size: 0.85rem;

  outline: none;
}

/* Layout Containers */

.container {
  width: 100%;

  max-width: 1140px;

  margin: 0 auto;

  padding: 1.75rem 1.25rem;

  flex: 1;
}
```

```
.card {
  background: var(--bg-secondary);

  border: 1px solid var(--border-color);

  border-radius: var(--radius);

  padding: 1.5rem;

  margin-bottom: 1.5rem;

  box-shadow: var(--shadow-sm);
}

/* Responsive Table */

.table-responsive-container {
  width: 100%;

  overflow-x: auto;

  -webkit-overflow-scrolling: touch;

  border: 1px solid var(--border-color);

  border-radius: var(--radius-sm);
}

.table-responsive-container table {
  width: 100%;

  border-collapse: collapse;

  white-space: nowrap;
}

.table-responsive-container th {
  background: var(--bg-card);

  color: var(--text-secondary);

  font-size: 0.78rem;

  text-transform: uppercase;

  padding: 0.75rem 1rem;

  border-bottom: 1px solid var(--border-color);
}
```

```
.table-responsive-container td {  
  
    padding: 0.85rem 1rem;  
  
    border-bottom: 1px solid var(--border-color);  
  
    font-size: 0.9rem;  
  
}
```

```
/* Footer */  
  
.app-footer {  
  
    background: var(--bg-secondary);  
  
    border-top: 1px solid var(--border-color);  
  
    padding: 2.5rem 1.5rem 1.5rem;  
  
    margin-top: auto;  
  
}
```

```
.footer-container {  
  
    max-width: 1140px;  
  
    margin: 0 auto;  
  
}
```

```
.footer-grid {  
  
    display: grid;  
  
    grid-template-columns: 2fr 1fr 1fr;  
  
    gap: 2.5rem;  
  
    margin-bottom: 2rem;  
  
}
```

```
.footer-brand-title {  
  
    display: inline-flex;  
  
    align-items: center;  
  
    gap: 0.5rem;  
  
    font-size: 1.15rem;  
  
    font-weight: 800;  
  
    color: var(--accent);
```

```
margin-bottom: 0.6rem;  
  
}
```

```
.footer-tagline {  
  
    color: var(--text-secondary);  
  
    font-size: 0.85rem;  
  
    line-height: 1.6;  
  
    max-width: 440px;  
  
    margin-bottom: 1rem;  
  
}
```

```
.footer-status-pill {  
  
    display: inline-flex;  
  
    align-items: center;  
  
    gap: 0.45rem;  
  
    background: var(--bg-card);  
  
    border: 1px solid var(--border-color);  
  
    padding: 0.25rem 0.65rem;  
  
    border-radius: 20px;  
  
    font-size: 0.75rem;  
  
    color: var(--text-secondary);  
  
}
```

```
.status-dot {  
  
    width: 7px;  
  
    height: 7px;  
  
    border-radius: 50%;  
  
    background: var(--success);  
  
    box-shadow: 0 0 6px var(--success);  
  
}
```

```
.footer-links-col h4 {  
  
    font-size: 0.85rem;
```



```
font-weight: 700;

text-transform: uppercase;

color: var(--text-primary);

margin-bottom: 0.85rem;
}

.footer-links-col ul {

list-style: none;

display: flex;

flex-direction: column;

gap: 0.5rem;
}

.footer-links-col a {

color: var(--text-secondary);

text-decoration: none;

font-size: 0.85rem;
}

.footer-links-col a:hover {

color: var(--accent);
}

.footer-bottom-bar {

border-top: 1px solid var(--border-color);

padding-top: 1.25rem;

display: flex;

justify-content: space-between;

align-items: center;

flex-wrap: wrap;

gap: 1rem;

font-size: 0.8rem;

color: var(--text-muted);
```

```
}

.mobile-nav-toggle {

display: none;

background: transparent;

border: 1px solid var(--border-color);

color: var(--text-primary);

font-size: 1.25rem;

padding: 0.25rem 0.6rem;

border-radius: var(--radius-sm);

cursor: pointer;
}

@media (max-width: 820px) {

.mobile-nav-toggle {

display: block;
}

.navbar {

flex-direction: column;

align-items: stretch;
}

.nav-header-row {

display: flex;

justify-content: space-between;

align-items: center;

width: 100%;
}

.nav-links {

display: none;

flex-direction: column;

width: 100%;

padding-top: 0.85rem;

gap: 0.4rem;
```

```
border-top: 1px solid var(--border-color);

}

.nav-links.mobile-open {

display: flex;

}

.footer-grid {

grid-template-columns: 1fr;

gap: 1.5rem;

}

.footer-bottom-bar {

flex-direction: column;

align-items: flex-start;

}

}
```

=== FILE: .\app\static\css\typing.css ===

/* Compact Main Mode Bar */

```
.mode-selectors {

display: flex;

justify-content: center;

align-items: center;

flex-wrap: wrap;

gap: 0.85rem;

background: var(--bg-secondary);

border: 1px solid var(--border-color);

border-radius: var(--radius);

padding: 0.45rem 0.85rem;

margin: 0 auto 1.5rem;

width: fit-content;

max-width: 100%;

}
```

```
.mode-group {
```

```
display: inline-flex;

gap: 0.25rem;

align-items: center;

}

.mode-btn {

background: transparent;

border: 1px solid transparent;

color: var(--text-secondary);

cursor: pointer;

font-size: 0.82rem;

font-weight: 600;

padding: 0.35rem 0.65rem;

border-radius: var(--radius-sm);

transition: var(--transition-fast);

white-space: nowrap;

}
```

```
.mode-btn:hover {

color: var(--text-primary);

background: var(--bg-card);

}
```

```
.mode-btn.active {

color: var(--accent);

background: var(--bg-card);

border-color: var(--border-color);

}
```

/* Telemetry HUD Numbers */

```
.stats-hud {

display: grid;

grid-template-columns: repeat(auto-fit, minmax(105px, 1fr));
```

```
gap: 0.75rem;

margin-bottom: 1.25rem;

transition: var(--transition);
}

.stat-box {

background: var(--bg-secondary);

border: 1px solid var(--border-color);

border-radius: var(--radius-sm);

padding: 0.75rem 0.5rem;

text-align: center;

position: relative;
}

.stat-label {

font-size: 0.7rem;

font-weight: 700;

color: var(--text-muted);

text-transform: uppercase;

letter-spacing: 0.06em;

margin-bottom: 0.2rem;
}

.stat-value {

font-size: 1.5rem;

font-weight: 800;

font-family: var(--font-mono);

color: var(--accent);

line-height: 1.2;
}

/* Speedometer Vector Accent */

.speedometer-svg {
```

```
position: absolute;

right: 6px;

bottom: 6px;

width: 38px;

height: 38px;

opacity: 0.22;

pointer-events: none;
}

.speedo-bg {

fill: none;

stroke: var(--border-color);

stroke-width: 4;
}

.speedo-fill {

fill: none;

stroke: var(--accent);

stroke-width: 4;

stroke-linecap: round;

transition: stroke-dashoffset 0.12s ease-out;
}

/* Main Typing Arena Container */

.typing-container {

position: relative;

background: var(--bg-secondary);

border: 1px solid var(--border-color);

border-radius: var(--radius);

padding: 1.75rem 2rem;

min-height: 180px;

max-height: 250px;

cursor: text;
```

```
outline: none;

user-select: none;

overflow-y: hidden;

scroll-behavior: smooth;

box-shadow: var(--shadow-sm);

transition: border-color var(--transition-fast), box-shadow var(--transition-fast);
}

.typing-container.focus-ring {

border-color: var(--accent);

box-shadow: 0 0 0 3px var(--accent-glow);
}

.typing-text-display {

font-family: var(--font-mono);

font-size: var(--typing-font-size, 1.35rem);

letter-spacing: 0.02em;

line-height: 2.1;

color: var(--text-muted);

position: relative;

word-break: break-word;

white-space: pre-wrap;
}

.char {

position: relative;

transition: color 0.05s ease;
}

.char.correct {

color: var(--text-primary);
}
```

```
.char.incorrect {

color: var(--danger);

text-decoration: underline 2px solid var(--danger);

background: var(--danger-bg);

border-radius: 2px;
}

/* Dynamic Caret System */

.caret {

position: absolute;

pointer-events: none;

transition: left 0.08s ease, top 0.08s ease;

z-index: 10;
}

.caret.caret-smooth {

width: 2.5px;

height: 1.5rem;

background: var(--accent);

border-radius: 2px;

animation: blinkSmooth 1.1s infinite;
}

.caret.caret-block {

width: 11px;

height: 1.5rem;

background: var(--warning);

opacity: 0.75;

border-radius: 2px;

animation: blinkSmooth 1s infinite alternate;
}

.caret.caret-underline {
```

```
width: 12px;

height: 3px;

background: var(--accent);

border-radius: 2px;

transform: translateY(1.2rem);

animation: blinkSmooth 1.1s infinite;

}

@keyframes blinkSmooth {

  0%, 100% { opacity: 1; }

  50% { opacity: 0; }

}

.ghost-caret {

  position: absolute;

  width: 2.5px;

  height: 1.4rem;

  background: #c084fc;

  pointer-events: none;

  transition: left 0.1s linear, top 0.1s linear;

  opacity: 0.75;

}

/* Focus Mode Minimalist State */

body.focus-mode .navbar,

body.focus-mode .stats-hud,

body.focus-mode .mode-selectors,

body.focus-mode .keyboard-wrapper,

body.focus-mode .enhanced-preview-bar,

body.focus-mode .app-footer,

body.focus-mode .test-tip-banner {

  display: none !important;

}
```

```
body.focus-mode .container {

  max-width: 860px;

  margin-top: 15vh;

}

@media (max-width: 768px) {

  .typing-container {

    padding: 1.25rem 1rem;

    min-height: 150px;

  }

  .typing-text-display {

    font-size: 1.15rem;

    line-height: 1.85;

  }

  .caret.caret-smooth {

    height: 1.25rem;

  }

}

=== FILE: .\app\static\js\arcade_runtime.js ===

/**

 * TypeSphere Arcade Unified Client Runtime Engine

 *

 * Drives all 12 educational typing games with:

 * - Speed Racer forward momentum & mistake recoil (never stuck on typos)

 * - Single-character Bubble Pop

 * - 3x3 Whack-A-Word spatial reactions

 * - Zombie Defense (waves) vs Zombie Duel (1v1 HP combat)

 * - Multi-layer Cipher Hacker

 * - Space Defender, Bomb Defuse, Typing Ninja, Memory Type, Keyboard Quest

 * - Definite WIN, LOSS, or DRAW evaluation

 */
```

```
class ArcadeRuntimeEngine {

  constructor() {

    this.viewport = null;

    this.config = {};

    this.socket = null;

    // Core Session Metrics

    this.score = 0;

    this.errors = 0;

    this.combo = 0;

    this.maxCombo = 0;

    this.totalCharsTyped = 0;

    this.correctCharsTyped = 0;

    this.startTime = 0;

    this.reactionLatencies = [];

    this.lastActionTimestamp = 0;

    // Engine State

    this.entities = [];

    this.gameLoopTimer = null;

    this.spawnTimer = null;

    this.aiLoopTimer = null;

    this.objectiveCountdownTimer = null;

    this.activeTarget = null;

    this.inputBuffer = "";

    this.isRunning = false;

    this.contentBatch = [];

    this.keydownHandler = (e) => this.handleKeystroke(e);

  }

  start(config) {

    this.stop();
```

```
    this.config = config;

    this.viewport = document.getElementById('arcade-viewport');

    this.viewport.innerHTML = "";

    this.score = 0;

    this.errors = 0;

    this.combo = 0;

    this.maxCombo = 0;

    this.totalCharsTyped = 0;

    this.correctCharsTyped = 0;

    this.reactionLatencies = [];

    this.entities = [];

    this.inputBuffer = "";

    this.activeTarget = null;

    this.isRunning = true;

    this.startTime = performance.now();

    this.lastActionTimestamp = this.startTime;

    this.updateHUD(0, 0);

    this.contentBatch = this.getFallbackBatch(config.slug, config.difficulty, config.charMode);

    this.initSockets();

    this.initGameMode(config.slug);

    window.removeEventListener('keydown', this.keydownHandler);

    window.addEventListener('keydown', this.keydownHandler);

    this.viewport.focus();

  }

  stop() {

    this.isRunning = false;

    clearInterval(this.gameLoopTimer);

    clearInterval(this.spawnTimer);
```

```
clearInterval(this.aiLoopTimer);

clearInterval(this.objectiveCountdownTimer);

window.removeEventListener('keydown', this.keydownHandler);

if (this.socket) {

    this.socket.disconnect();

    this.socket = null;

}

}

getFallbackBatch(slug, diff, charMode) {

    const common = ["velocity", "cadence", "rhythm", "kinetic", "precision", "matrix", "stream", "quantum", "tactile", "circuit", "binary", "system",
"engine", "pilot", "balance", "command", "transit", "standard", "channel", "motion", "action", "focus"];

    if (slug === 'speed_racer') {

        return ["The open highway stretched across the desert floor under a wide expanse of pale morning sky. High velocity demands relaxed control
and steady breathing. When the throttle opens every second compounds into pure forward momentum."];

    }

    if (slug === 'bubble_pop') {

        const sets = {

            'uppercase': [..."ABCDEFGHIJKLMNOPQRSTUVWXYZ"],

            'numbers': [..."0123456789"],

            'symbols': [..."!@#%&*()-_+=+[]{};:.,<>?/"],

            'punctuation': [...".,:;!?'\"-()"]

        };

        return sets[charMode] || [..."abcdefghijklmnopqrstuvwxyz"];

    }

    if (slug === 'cipher_hacker') {

        return [{"0x7FA9", "0xDE4B", "0x91C0"}, ["AES_256", "SHA_512", "TLS_13"], ["chmod 755", "int main()"], ["0xFF00AA55BB66CC77"]];

    }

    if (slug === 'bomb_defuse') {

        return ["ALPHA78", "DELTA94", "OMEGA12", "BRAVO33", "HAZARD8", "VECTOR0"];

    }

    if (slug === 'memory_type') {

        return ["recall", "phantom", "quantum", "horizon", "velocity", "kinetic", "matrix"];

    }

}
```

```
    }

    if (slug === 'keyboard_quest') {

        return ["asdF", "jkl;", "glad", "flask", "half", "fall", "quiet", "write", "power", "tower", "cabin", "zinc", "calm"];

    }

    return common;

}

initSockets() {

    if (!window.io) return;

    this.socket = io();

    this.socket.emit('arcade_room_create', {

        game_slug: this.config.slug,

        play_mode: this.config.playMode,

        difficulty: this.config.difficulty,

        objective_val: this.config.objectiveVal,

        blind_mode: this.config.blindMode,

        backspace_allowed: this.config.backspaceAllowed,

        char_mode: this.config.charMode,

        room_code: this.config.roomCode,

        player_name: "Pilot"

    });

    this.socket.on('arcade_room_ready', (room) => {

        if (room.content && room.content.length > 0) {

            this.contentBatch = room.content;

        }

    });

    this.socket.on('arcade_live_telemetry', (data) => {

        const players = data.players || {};

        const ai = players['ai_bot'];

        if (ai) {
```

```
document.getElementById('live-opp-stat').textContent = `${Math.round(ai.score)} pts`;

document.getElementById('live-opp-wpm').textContent = `${Math.round(ai.wpm)} WPM`;

if (this.config.slug === 'speed_racer') {

  this.updateSpeedRacerPosition('bot', ai.progress);

}

}

});

if (this.config.playMode === 'solo_ai') {

  this.aiLoopTimer = setInterval(() => {

    if (this.socket && this.isRunning) {

      this.socket.emit('arcade_ai_tick', { delta: 0.25, total_chars: 280 });

    }

  }, 250);

}

}

initGameMode(slug) {

  switch (slug) {

    case 'speed_racer': this.initSpeedRacer(); break;

    case 'falling_words': this.initFallingWords(); break;

    case 'bubble_pop': this.initBubblePop(); break;

    case 'whack_a_word': this.initWhackAWord(); break;

    case 'zombie_defense': this.initZombieDefense(); break;

    case 'zombie_duel': this.initZombieDuel(); break;

    case 'cipher_hacker': this.initCipherHacker(); break;

    case 'space_defender': this.initSpaceDefender(); break;

    case 'bomb_defuse': this.initBombDefuse(); break;

    case 'typing_ninja': this.initTypingNinja(); break;

    case 'memory_type': this.initMemoryType(); break;

    case 'keyboard_quest': this.initKeyboardQuest(); break;

    default: this.initFallingWords();

  }

}
```

```
}

// =====

// 1. SPEED RACER: Forward Acceleration + Recoil Without Sticking

// =====

initSpeedRacer() {

  const isMultiplayer = (this.config.playMode !== 'solo_ai' && this.config.playMode !== 'solo_practice');

  const oppIcon = isMultiplayer ? "🏎️" : "🤖";

  const oppLabel = isMultiplayer ? "Opponent" : "CyberBot (AI)";

  this.viewport.innerHTML = `

    <div class="racer-track-surface"></div>

    <div class="racer-lane-divider" style="top:33%;"></div>

    <div class="racer-lane-divider" style="top:66%;"></div>

    <div class="racer-finish-gate"></div>

    <!-- Player Car -->

    <div id="car-player" class="racer-vehicle-node" style="top:75px; left:25px;">

      <div class="racer-vehicle-icon" style="filter:drop-shadow(0 4px 10px rgba(56,189,248,0.8));">🏎️</div>

      <div class="racer-hud-tag" style="border-color:var(--accent); color:var(--accent);">You</div>

      <div id="player-nitro" class="nitro-exhaust-burn" style="display:none;"></div>

    </div>

    <!-- Opponent Car -->

    <div id="car-bot" class="racer-vehicle-node" style="top:210px; left:25px; ${this.config.playMode === 'solo_practice' ? 'display:none;' : ''}>

      <div class="racer-vehicle-icon" style="${isMultiplayer ? 'filter:drop-shadow(0 4px 10px rgba(244,63,94,0.8));' : ''}">${oppIcon}</div>

      <div class="racer-hud-tag" style="border-color:var(--warning); color:var(--warning);">${oppLabel}</div>

    </div>

    <div id="racer-text-track" style="position:absolute; bottom:15px; left:25px; right:25px; background:rgba(19,25,36,0.96); padding:1rem 1.4rem; border-radius:8px; border:1px solid var(--border-color); font-family:var(--font-mono); font-size:1.25rem; line-height:2.1; box-shadow:0 10px 25px rgba(0,0,0,0.6);"></div>

  `;

};
```



```
    this.racerPassage = this.contentBatch[0] || "Speed is nothing without precision. Keep your hands balanced, breathe calmly, and glide across the keys with absolute rhythm.";
```

```
    this.racerIdx = 0;
```

```
    this.playerDistanceMeters = 0;
```

```
    this.targetDistanceMeters = parseInt(this.config.objectiveVal) || 500;
```

```
    this.renderRacerSpans();
```

```
  }
```

```
renderRacerSpans() {
```

```
  const box = document.getElementById('racer-text-track');
```

```
  if (!box) return;
```

```
  box.innerHTML = "";
```

```
  for (let i = 0; i < this.racerPassage.length; i++) {
```

```
    const s = document.createElement('span');
```

```
    s.className = 'char';
```

```
    s.textContent = this.racerPassage[i];
```

```
    box.appendChild(s);
```

```
  }
```

```
}
```

```
updateSpeedRacerPosition(target, pct) {
```

```
  const maxPixels = this.viewport.clientWidth - 110;
```

```
  const offset = Math.max(25, (pct / 100.0) * maxPixels);
```

```
  const car = document.getElementById(target === 'player' ? 'car-player' : 'car-bot');
```

```
  if (car) car.style.left = `${offset}px`;
```

```
}
```

```
// =====
```

```
// 2. BUBBLE POP: Single Character Master
```

```
// =====
```

```
initBubblePop() {
```

```
  this.viewport.innerHTML = `
```

```
<div style="position:absolute; top:35px; left:0; right:0; height:2px; background:var(--danger); opacity:0.6;"></div>
```

```
`;
```

```
this.bubbleLives = 3;
```

```
this.spawnTimer = setInterval(() => this.spawnCharBubble(), 1200);
```

```
this.gameLoopTimer = setInterval(() => this.tickBubbles(), 40);
```

```
}
```

```
spawnCharBubble() {
```

```
  if (this.entities.length >= 10 || !this.isRunning) return;
```

```
  const char = this.contentBatch[Math.floor(Math.random() * this.contentBatch.length)] || 'a';
```

```
  const el = document.createElement('div');
```

```
  el.className = 'arcade-char-bubble';
```

```
  el.textContent = char;
```

```
  el.style.left = `${Math.random() * (this.viewport.clientWidth - 70) + 15}px`;
```

```
  el.style.bottom = '0px';
```

```
  this.viewport.appendChild(el);
```

```
  this.entities.push({ el, char, y: 0, speed: 1.3 + Math.random() * 1.1 });
```

```
}
```

```
tickBubbles() {
```

```
  for (let i = this.entities.length - 1; i >= 0; i--) {
```

```
    const b = this.entities[i];
```

```
    b.y += b.speed;
```

```
    b.el.style.bottom = `${b.y}px`;
```

```
    if (b.y >= this.viewport.clientHeight - 60) {
```

```
      b.el.remove();
```

```
      this.entities.splice(i, 1);
```

```
      this.bubbleLives--;
```

```
      this.registerError();
```

```
      if (this.bubbleLives <= 0) {
```

```
        this.finishGame(false, "Loss: Allowed 3 bubbles to cross the ceiling barrier.");
```

```
        break;
    }
}

}

}

}

// =====

// 3. WHACK-A-WORD: 3x3 Reaction Grid

// =====

initWhackAWord() {

    this.viewport.innerHTML = `

        <div style="display:grid; grid-template-columns:repeat(3, 1fr); gap:1rem; max-width:620px; margin:40px auto;">

            ${[0,1,2,3,4,5,6,7,8].map(i => `

                <div id="hole-${i}" style="height:90px; background:var(--bg-card); border:2px solid var(--border-color); border-radius:8px; display:flex; align-items:center; justify-content:center; font-family:var(--font-mono); font-weight:800; font-size:1.15rem; color:var(--text-muted);"></div>

            `).join("")}

        </div>

    `;

    this.whackTarget = "";

    this.whackBuffer = "";

    this.currentHole = -1;

    this.missedHoles = 0;

    this.spawnNextMole();

}

spawnNextMole() {

    if (this.moleWindowTimer) clearTimeout(this.moleWindowTimer);

    if (!this.isRunning) return;

    if (this.currentHole >= 0) {

        const prev = document.getElementById(`hole-${this.currentHole}`);

        if (prev) {

            prev.textContent = ";
```

```
prev.style.borderColor = 'var(--border-color)';

prev.style.background = 'var(--bg-card)';

}

}

this.currentHole = Math.floor(Math.random() * 9);

this.whackTarget = this.contentBatch[Math.floor(Math.random() * this.contentBatch.length)] || "strike";

this.whackBuffer = "";

this.lastActionTimestamp = performance.now();

const hole = document.getElementById(`hole-${this.currentHole}`);

if (hole) {

    hole.textContent = this.whackTarget;

    hole.style.borderColor = 'var(--accent)';

    hole.style.color = 'var(--text-primary)';

}

const windowMs = { 'easy': 2400, 'moderate': 1800, 'hard': 1300, 'expert': 900}[this.config.difficulty] || 1800;

this.moleWindowTimer = setTimeout(() => {

    this.missedHoles++;

    this.registerError();

    if (this.missedHoles >= 5) {

        this.finishGame(false, "Loss: Missed 5 reaction windows.");

    } else {

        this.spawnNextMole();

    }

}, windowMs);

}
```

```
// =====

// 4. ZOMBIE DUEL: 1v1 Combat HP Battle

// =====

initZombieDuel() {
```

```
this.playerHP = 100;

this.opponentHP = 100;

const oppName = (this.config.playMode === 'solo_ai') ? "CyberZombie (AI)" : "Opponent";

this.viewport.innerHTML = `

<div class="duel-combat-arena">

  <div class="duel-combatant-card">

    <div style="font-size:1.6rem;">👾🦹‍♂️ <strong style="font-size:1.05rem; color:var(--accent);">You</strong></div>

    <div class="duel-hp-bar-container">

      <div id="duel-player-hp" class="duel-hp-fill" style="width:100%; background:var(--success);"></div>

    </div>

    <span id="player-hp-label" style="font-family:var(--font-mono); font-size:0.8rem;">100 / 100 HP</span>

  </div>

  <div style="font-size:2.2rem; font-weight:900; color:var(--warning);">VS</div>

  <div class="duel-combatant-card" style="text-align:right;">

    <div style="font-size:1.6rem;"><strong style="font-size:1.05rem; color:var(--danger);">${oppName}</strong>👾</div>

    <div class="duel-hp-bar-container">

      <div id="duel-opp-hp" class="duel-hp-fill" style="width:100%; background:var(--danger);"></div>

    </div>

    <span id="opp-hp-label" style="font-family:var(--font-mono); font-size:0.8rem;">100 / 100 HP</span>

  </div>

  <div style="text-align:center; margin-top:35px;">

    <div style="font-size:0.85rem; text-transform:uppercase; color:var(--text-muted); font-weight:700;">Strike Spell Word:</div>

    <div id="duel-spell-box" style="font-family:var(--font-mono); font-size:2.6rem; font-weight:900; color:var(--text-primary); margin:12px 0;">ATTACK</div>

  </div>
`;

this.spawnDuelTarget();
```

```
// AI periodic counter-attack

if (this.config.playMode === 'solo_ai') {

  const intervalMs = {'easy': 3800, 'moderate': 2800, 'hard': 1900, 'expert': 1300}[this.config.difficulty] || 2800;

  this.aiLoopTimer = setInterval(() => {

    if (!this.isRunning) return;

    this.playerHP = Math.max(0, this.playerHP - 15);

    document.getElementById('duel-player-hp').style.width = `${this.playerHP}%`;

    document.getElementById('player-hp-label').textContent = `${this.playerHP} / 100 HP`;

    if (window.soundEngine) window.soundEngine.playKey(true);

    if (this.playerHP <= 0) {

      this.finishGame(false, "Loss: Your combat barrier collapsed under attack.");

    }

  }, intervalMs);

}

spawnDuelTarget() {

  this.duelTarget = this.contentBatch[Math.floor(Math.random() * this.contentBatch.length)] || "strike";

  this.duelBuffer = "";

  document.getElementById('duel-spell-box').textContent = this.duelTarget;

}

// =====

// 5. CIPHER HACKER: Multi-Layer Security Breach

// =====

initCipherHacker() {

  this.currentLayer = 1;

  this.totalLayers = parseInt(this.config.objectiveVal) || 3;

  this.layerTokens = this.contentBatch[0] || ["0x7FA9", "0xDE4B", "0x91C0"];

  this.tokenIdx = 0;

  this.viewport.innerHTML = `
```

```
<div style="text-align:center; padding-top:40px;">

<div class="cipher-layer-badge" id="cipher-layer-title">SECURITY LAYER 1 / ${this.totalLayers}</div>

<div class="cipher-terminal-window" style="max-width:480px; margin:0 auto 1.5rem;">

  <div id="cipher-target-token" style="font-size:2.5rem; font-weight:900; color:#10b981; word-break:break-all;">...</div>

</div>

<div style="color:var(--text-muted); font-size:0.85rem;">Decrypt all tokens to breach clearance.</div>

</div>

`;

this.loadNextCipherToken();

}

loadNextCipherToken() {

  this.cipherTarget = this.layerTokens[this.tokenIdx % this.layerTokens.length] || "0x7FA9";

  this.cipherBuffer = "";

  document.getElementById('cipher-target-token').textContent = this.cipherTarget;

}

// =====

// 6. FALLING WORDS

// =====

initFallingWords() {

  this.viewport.innerHTML = `

    <div style="position:absolute; bottom:40px; left:0; right:0; height:2px; background:var(--danger); opacity:0.6;"></div>

  `;

  this.wave = 1;

  this.maxWaves = parseInt(this.config.objectiveVal) || 5;

  this.spawnTimer = setInterval(() => this.spawnFallingNode(), 1800);

  this.gameLoopTimer = setInterval(() => this.tickFallingNodes(), 40);

}

spawnFallingNode() {

  if (this.entities.length >= 7 || !this.isRunning) return;

  const word = this.contentBatch[Math.floor(Math.random() * this.contentBatch.length)] || "velocity";
```

```
const el = document.createElement('div');

el.className = 'falling-meteor-word';

el.textContent = word;

el.style.left = `${Math.random() * (this.viewport.clientWidth - 130) + 15}px`;

el.style.top = '0px';

this.viewport.appendChild(el);

this.entities.push({ el, word, typed: '', y: 0, speed: 1.1 + (this.wave * 0.2) });

}

tickFallingNodes() {

  for (let i = this.entities.length - 1; i >= 0; i--) {

    const e = this.entities[i];

    e.y += e.speed;

    e.el.style.top = `${e.y}px`;

    if (e.y >= this.viewport.clientHeight - 55) {

      e.el.remove();

      this.entities.splice(i, 1);

      if (this.activeTarget === e) this.activeTarget = null;

      this.registerError();

    }

  }

}

// =====

// 7. ZOMBIE DEFENSE (Wave Base Survival)

// =====

initZombieDefense() {

  this.baseHP = 100;

  this.viewport.innerHTML = `

    <div style="position:absolute; right:35px; top:0; bottom:0; width:6px; background:var(--accent); opacity:0.7;"></div>

    <div style="position:absolute; right:50px; top:20px; font-family:var(--font-mono); font-weight:800; color:var(--accent);">

      Base Barrier: <span id="defense-hp">100%</span>

    </div>
```

```
`;  
  
this.spawnTimer = setInterval(() => this.spawnZombieDefenseUnit(), 2100);  
  
this.gameLoopTimer = setInterval(() => this.tickZombieDefense(), 45);  
  
}  
  
spawnZombieDefenseUnit() {  
  
  if (this.entities.length >= 6 || !this.isRunning) return;  
  
  const word = this.contentBatch[Math.floor(Math.random() * this.contentBatch.length)] || "stalker";  
  
  const el = document.createElement('div');  
  
  el.className = 'falling-meteor-word';  
  
  el.innerHTML = `☠ <span>${word}</span>`;  
  
  el.style.left = '0px';  
  
  el.style.top = `${Math.random() * (this.viewport.clientHeight - 70) + 15}px`;  
  
  el.style.borderColor = `var(--danger)`;  
  
  this.viewport.appendChild(el);  
  
  this.entities.push({ el, word, typed: '', x: 0, speed: 1.1 + Math.random() * 0.8 });  
  
}
```

```
tickZombieDefense() {  
  
  for (let i = this.entities.length - 1; i >= 0; i--) {  
  
    const z = this.entities[i];  
  
    z.x += z.speed;  
  
    z.el.style.left = `${z.x}px`;  
  
    if (z.x >= this.viewport.clientWidth - 100) {  
  
      z.el.remove();  
  
      this.entities.splice(i, 1);  
  
      this.baseHP -= 20;  
  
      document.getElementById('defense-hp').textContent = `${Math.max(0, this.baseHP)}%`;  
  
      this.registerError();  
  
      if (this.baseHP <= 0) {  
  
        this.finishGame(false, "Loss: Perimeter breached. Base collapsed.");  
  
        break;  
  
      }  
  
    }
```

```
}  
  
}  
  
}  
  
  
// =====  
  
// 8. SPACE DEFENDER  
  
// =====  
  
initSpaceDefender() {  
  
  this.viewport.innerHTML = `  
  
    <div style="position:absolute; left:50%; top:50%; transform:translate(-50%, -50%); font-size:3rem;">☠</div>  
  
  `;  
  
  this.shields = 100;  
  
  this.spawnTimer = setInterval(() => this.spawnAsteroidUnit(), 2000);  
  
  this.gameLoopTimer = setInterval(() => this.tickAsteroids(), 40);  
  
}
```

```
spawnAsteroidUnit() {  
  
  if (this.entities.length >= 6 || !this.isRunning) return;  
  
  const word = this.contentBatch[Math.floor(Math.random() * this.contentBatch.length)] || "orbit";  
  
  const el = document.createElement('div');  
  
  el.className = 'falling-meteor-word';  
  
  el.innerHTML = `☄ ☐ <span>${word}</span>`;  
  
  
  
  
  const angle = Math.random() * Math.PI * 2;  
  
  const radius = 230;  
  
  const startX = (this.viewport.clientWidth / 2) + Math.cos(angle) * radius;  
  
  const startY = (this.viewport.clientHeight / 2) + Math.sin(angle) * radius;  
  
  
  
  
  el.style.left = `${startX}px`;  
  
  el.style.top = `${startY}px`;  
  
  this.viewport.appendChild(el);  
  
  this.entities.push({ el, word, typed: '', x: startX, y: startY, angle, speed: 0.85 + Math.random() * 0.5 });
```

```
}

tickAsteroids() {

  const cx = this.viewport.clientWidth / 2;

  const cy = this.viewport.clientHeight / 2;

  for (let i = this.entities.length - 1; i >= 0; i--) {

    const a = this.entities[i];

    a.x -= Math.cos(a.angle) * a.speed;

    a.y -= Math.sin(a.angle) * a.speed;

    a.el.style.left = `${a.x}px`;

    a.el.style.top = `${a.y}px`;

    if (Math.hypot(cx - a.x, cy - a.y) < 35) {

      a.el.remove();

      this.entities.splice(i, 1);

      this.shields -= 25;

      this.registerError();

      if (this.shields <= 0) {

        this.finishGame(false, "Loss: Starship shields vaporized under asteroid barrage.");

        break;

      }

    }

  }

}

// =====

// 9. BOMB DEFUSE

// =====

initBombDefuse() {

  this.bombSeconds = 40;

  this.viewport.innerHTML = `

    <div style="text-align:center; padding-top:40px;">

      <div style="font-size:3rem; margin-bottom:0.4rem;">💣</div>

    </div>`

}
```

```
<div id="bomb-digital-clock" class="bomb-timer-display">00:40</div>

<div id="bomb-code-box" style="margin:25px auto; max-width:440px; background:var(--bg-card); border:2px solid var(--accent); border-radius:8px; padding:1.25rem; font-family:var(--font-mono); font-size:1.8rem; letter-spacing:0.15em;">

  INIT...

</div>

</div>

`;

this.bombStage = 0;

this.spawnBombCode();

this.objectiveCountdownTimer = setInterval(() => {

  this.bombSeconds--;

  const el = document.getElementById('bomb-digital-clock');

  if (el) el.textContent = `00:${this.bombSeconds < 10 ? '0' : ''}${this.bombSeconds}`;

  if (this.bombSeconds <= 0) {

    this.finishGame(false, "Loss: Countdown reached 00:00. Bomb detonated.");

  }

}, 1000);

}

spawnBombCode() {

  this.bombTarget = this.contentBatch[this.bombStage % this.contentBatch.length] || "ALPHA7";

  this.bombBuffer = "";

  document.getElementById('bomb-code-box').textContent = this.bombTarget;

}

// =====

// 10. TYPING NINJA

// =====

initTypingNinja() {

  this.spawnTimer = setInterval(() => this.spawnNinjaTarget(), 2000);

  this.gameLoopTimer = setInterval(() => this.tickNinjaTargets(), 40);

}
```

```
spawnNinjaTarget() {

  if (this.entities.length >= 6 || !this.isRunning) return;

  const word = this.contentBatch[Math.floor(Math.random() * this.contentBatch.length)] || "strike";

  const el = document.createElement('div');

  el.className = 'falling-meteor-word';

  el.textContent = word;

  const startX = Math.random() * (this.viewport.clientWidth - 150) + 30;

  el.style.left = `${startX}px`;

  el.style.top = `${this.viewport.clientHeight - 40}px`;

  this.viewport.appendChild(el);

  this.entities.push({ el, word, typed: '', x: startX, y: this.viewport.clientHeight - 40, vy: -6.5 - Math.random() * 2.5, vx: (Math.random() - 0.5) * 2 });

}

tickNinjaTargets() {

  for (let i = this.entities.length - 1; i >= 0; i--) {

    const n = this.entities[i];

    n.vy += 0.16;

    n.y += n.vy;

    n.x += n.vx;

    n.el.style.top = `${n.y}px`;

    n.el.style.left = `${n.x}px`;

    if (n.y > this.viewport.clientHeight + 20) {

      n.el.remove();

      this.entities.splice(i, 1);

      if (this.activeTarget === n) this.activeTarget = null;

    }

  }

}

// =====

// 11. MEMORY TYPE

// =====
```

```
initMemoryType() {

  this.memoryIndex = 0;

  this.viewport.innerHTML = `

    <div style="text-align:center; padding-top:60px;">

      <span class="badge" style="background:var(--accent-glow); color:var(--accent); font-weight:800; padding:0.2rem 0.6rem; border-radius:4px;">RECALL DRILL</span>

      <div id="memory-box" style="margin:25px auto; max-width:440px; background:var(--bg-card); border:2px solid var(--accent); border-radius:8px; padding:2rem; font-family:var(--font-mono); font-size:2.5rem; letter-spacing:0.15em;">

        READY

      </div>

      <div id="memory-hint" style="font-size:0.85rem; color:var(--text-muted);">Sequence flashes for 1.5 seconds...</div>

    </div>

  `;

  setTimeout(() => this.flashMemoryTarget(), 800);

}

flashMemoryTarget() {

  if (!this.isRunning) return;

  this.memoryTarget = this.contentBatch[this.memoryIndex % this.contentBatch.length] || "recall";

  const box = document.getElementById('memory-box');

  const hint = document.getElementById('memory-hint');

  box.textContent = this.memoryTarget;

  box.style.color = "var(--text-primary)";

  hint.textContent = "Memorize the sequence...";

  this.memoryBuffer = "";

  setTimeout(() => {

    box.textContent = "●●●●●●●";

    box.style.color = "var(--accent)";

    hint.textContent = "Type the sequence completely from recall!";

  }, 1500);

}
```

```
// =====

// 12. KEYBOARD QUEST

// =====

initKeyboardQuest() {

  this.questIndex = 0;

  this.viewport.innerHTML = `

    <div style="text-align:center; padding-top:50px;">

      <span class="badge" style="background:var(--accent-glow); color:var(--accent); font-weight:800; padding:0.2rem 0.6rem; border-radius:4px;">QUEST STAGE 1</span>

      <div id="quest-target-box" style="margin:25px auto; max-width:440px; background:var(--bg-card); border:2px solid var(--accent); border-radius:8px; padding:1.75rem; font-family:var(--font-mono); font-size:2.2rem; font-weight:800;">

        ...

      </div>

      <div style="font-size:0.85rem; color:var(--text-muted);">Anchor palms on home-row. Strike cleanly.</div>

    </div>

  `;

  this.loadNextQuestTarget();

}

loadNextQuestTarget() {

  this.questTarget = this.contentBatch[this.questIndex % this.contentBatch.length] || "asd";

  this.questBuffer = "";

  document.getElementById('quest-target-box').textContent = this.questTarget;

}

// =====

// GLOBAL KEYSTROKE ROUTER (NO PERMANENT STICKING)

// =====

handleKeystroke(e) {

  if (!this.isRunning) return;

  if (e.key === 'Backspace' && !this.config.backspaceAllowed) {

    e.preventDefault();

    return;

  }

}
```

```
}

if (e.key.length !== 1 || e.ctrlKey || e.metaKey || e.altKey) return;

e.preventDefault();

const char = e.key;

this.totalCharsTyped++;

// 1. SPEED RACER: Forward velocity on correct, Recoil penalty on typo (Never stuck!)

if (this.config.slug === 'speed_racer') {

  const expected = this.racerPassage[this.racerIdx];

  const spans = document.querySelectorAll('#racer-text-track .char');

  if (char === expected) {

    if (spans[this.racerIdx]) spans[this.racerIdx].className = 'char correct';

    this.racerIdx++;

    this.correctCharsTyped++;

    this.registerHit(10);

    // Progress forward

    this.playerDistanceMeters += 2.5;

    if (this.combo > 8) {

      this.playerDistanceMeters += 1.5; // Nitro burst bonus

      document.getElementById('player-nitro').style.display = 'block';

    }

  } else {

    // ERROR: Register error, apply speed loss/recoil, but ADVANCE target so player never gets stuck!

    if (spans[this.racerIdx]) spans[this.racerIdx].className = 'char incorrect';

    this.racerIdx++; // Advances to next character immediately!

    this.playerDistanceMeters = Math.max(0, this.playerDistanceMeters - 1.2); // Controlled backward penalty

    const nitro = document.getElementById('player-nitro');

    if (nitro) nitro.style.display = 'none';

    this.registerError();

  }

}
```



```

const pct = Math.min(100, (this.playerDistanceMeters / this.targetDistanceMeters) * 100);

this.updateSpeedRacerPosition('player', pct);

if (pct >= 100) {

  this.finishGame(true, "VICTORY! You crossed the finish line ahead of your opponent!");

}

return;

}

// 2. BUBBLE POP

if (this.config.slug === 'bubble_pop') {

  const match = this.entities.find(b => b.char === char);

  if (match) {

    match.el.remove();

    this.entities.splice(this.entities.indexOf(match), 1);

    this.correctCharsTyped++;

    this.registerHit(15);

  } else {

    this.registerError();

  }

  return;

}

// 3. WHACK-A-WORD

if (this.config.slug === 'whack_a_word') {

  const latency = performance.now() - this.lastActionTimestamp;

  this.reactionLatencies.push(latency);

  if (this.whackTarget.startsWith(this.whackBuffer + char)) {

    this.whackBuffer += char;

    this.correctCharsTyped++;

    this.registerHit(10);

```

```

const hole = document.getElementById('hole-${this.currentHole}');

if (hole) hole.innerHTML = `<span style="color:var(--success); text-decoration:underline;">${this.whackBuffer}</span>${this.whackTarget.slice(this.whackBuffer.length)}`;

if (this.whackBuffer === this.whackTarget) {

  this.spawnNextMole();

}

} else {

  this.registerError();

}

return;

}

// 4. ZOMBIE DUEL

if (this.config.slug === 'zombie_duel') {

  if (this.duelTarget.startsWith(this.duelBuffer + char)) {

    this.duelBuffer += char;

    this.correctCharsTyped++;

    this.registerHit(15);

    document.getElementById('duel-spell-box').innerHTML = `<span style="color:var(--success);">${this.duelBuffer}</span>${this.duelTarget.slice(this.duelBuffer.length)}`;

    if (this.duelBuffer === this.duelTarget) {

      this.opponentHP = Math.max(0, this.opponentHP - 25);

      document.getElementById('duel-opp-hp').style.width = `${this.opponentHP}%`;

      document.getElementById('opp-hp-label').textContent = `${this.opponentHP} / 100 HP`;

      if (this.opponentHP <= 0) {

        this.finishGame(true, "VICTORY! You knocked out your opponent in the duel!");

      } else {

        this.spawnDuelTarget();

      }

    }

  } else {

```

```

    this.registerError();

}

return;

}

// 5. CIPHER HACKER

if (this.config.slug === 'cipher_hacker') {

    if (this.cipherTarget.startsWith(this.cipherBuffer + char)) {

        this.cipherBuffer += char;

        this.correctCharsTyped++;

        this.registerHit(20);

        document.getElementById('cipher-target-token').innerHTML = `<span style="color:var(--accent);">${this.cipherBuffer}</span>${this.cipherTarget.slice(this.cipherBuffer.length)}`;

    }

    if (this.cipherBuffer === this.cipherTarget) {

        this.tokenIdx++;

        if (this.tokenIdx >= this.layerTokens.length) {

            this.currentLayer++;

            if (this.currentLayer > this.totalLayers) {

                this.finishGame(true, "VICTORY! Final root clearance decrypted. System breached!");

                return;

            }

            this.tokenIdx = 0;

            this.layerTokens = this.contentBatch[Math.min(this.contentBatch.length - 1, this.currentLayer - 1)];

            document.getElementById('cipher-layer-title').textContent = `SECURITY LAYER ${this.currentLayer} / ${this.totalLayers}`;

        }

        this.loadNextCipherToken();

    }

} else {

    this.registerError();

}

return;

}

```

```

// 6. BOMB DEFUSE

if (this.config.slug === 'bomb_defuse') {

    if (this.bombTarget.startsWith(this.bombBuffer + char)) {

        this.bombBuffer += char;

        this.correctCharsTyped++;

        this.registerHit(15);

        document.getElementById('bomb-code-box').innerHTML = `<span style="color:var(--success);">${this.bombBuffer}</span>${this.bombTarget.slice(this.bombBuffer.length)}`;

    }

    if (this.bombBuffer === this.bombTarget) {

        this.bombStage++;

        if (this.bombStage >= parseInt(this.config.objectiveVal)) {

            this.finishGame(true, "VICTORY! All sequence stages decrypted. Bomb safely defused!");

            return;

        }

        this.spawnBombCode();

    }

} else {

    this.bombSeconds = Math.max(1, this.bombSeconds - 2); // Error penalty

    this.registerError();

}

return;

}

```

```

// 7. MEMORY TYPE

if (this.config.slug === 'memory_type') {

    this.memoryBuffer += char;

    if (this.memoryTarget.startsWith(this.memoryBuffer)) {

        this.correctCharsTyped++;

        this.registerHit(20);

        if (this.memoryBuffer === this.memoryTarget) {

            this.memoryIndex++;

        }

    }

}

```

```

    if (this.memoryIndex >= parseInt(this.config.objectiveVal)) {

        this.finishGame(true, "VICTORY! You mastered all memory recall rounds!");

        return;

    }

    this.flashMemoryTarget();

}

} else {

    this.registerError();

    this.flashMemoryTarget();

}

return;

}

```

// 8. KEYBOARD QUEST

```

if (this.config.slug === 'keyboard_quest') {

    if (this.questTarget.startsWith(this.questBuffer + char)) {

        this.questBuffer += char;

        this.correctCharsTyped++;

        this.registerHit(15);

        document.getElementById('quest-target-box').innerHTML = `<span style="color:var(--success); text-decoration:underline;">${this.questBuffer}</span>${this.questTarget.slice(this.questBuffer.length)}`;

    }

}

```

```

    if (this.questBuffer === this.questTarget) {

        this.questIndex++;

        this.loadNextQuestTarget();

    }

} else {

    this.registerError();

}

return;

}

```

// 9. FALLING WORDS, NINJA SLICING, SPACE DEFENDER

```

const lower = char.toLowerCase();

if (this.activeTarget) {

    const next = this.activeTarget.word[this.activeTarget.typed.length];

    if (lower === next) {

        this.activeTarget.typed += lower;

        this.correctCharsTyped++;

        this.registerHit(10);

        this.activeTarget.el.innerHTML = `<span style="color:var(--success); text-decoration:underline;">${this.activeTarget.typed}</span>${this.activeTarget.word.slice(this.activeTarget.typed.length)}`;

    }

}

```

```

    if (this.activeTarget.typed === this.activeTarget.word) {

        this.activeTarget.el.remove();

        this.entities.splice(this.entities.indexOf(this.activeTarget), 1);

        this.activeTarget = null;

    }

    return;

}

}

```

```

const match = this.entities.find(e => e.word.startsWith(lower));

```

```

if (match) {

    this.activeTarget = match;

    this.activeTarget.typed = lower;

    this.correctCharsTyped++;

    this.registerHit(10);

    match.el.classList.add('target-locked');

    match.el.innerHTML = `<span style="color:var(--success); text-decoration:underline;">${lower}</span>${match.word.slice(1)}`;

} else {

    this.registerError();

}

}

```

```

registerHit(pts) {

```

```

this.combo++;

if (this.combo > this.maxCombo) this.maxCombo = this.combo;

const mult = 1 + Math.floor(this.combo / 7) * 0.25;

this.score += Math.round(pts * mult);

if (window.soundEngine) window.soundEngine.playKey(false);

this.updateHUD(this.score, this.combo);
}

registerError() {

  this.combo = 0;

  this.errors++;

  if (window.soundEngine) window.soundEngine.playKey(true);

  this.updateHUD(this.score, 0);
}

updateHUD(score, combo) {

  const sEl = document.getElementById('live-my-stat');

  const cEl = document.getElementById('live-combo-badge');

  if (sEl) sEl.textContent = `${score} pts`;

  if (cEl) cEl.textContent = `${combo}x`;
}

finishGame(didWin, summaryMessage) {

  this.stop();

  const duration = Math.max(1.0, (performance.now() - this.startTime) / 1000.0);

  const netWords = Math.max(0, (this.correctCharsTyped - this.errors) / 5.0);

  const wpm = duration > 0 ? Math.round((netWords / duration) * 60.0) : 0;

  const acc = this.totalCharsTyped > 0 ? Math.round((this.correctCharsTyped / this.totalCharsTyped) * 100.0) : 100;

  const outcome = didWin ? "WIN" : (this.config.playMode === 'solo_practice' ? "FINISHED" : "LOSS");

  document.getElementById('game-active-arena').style.display = 'none';

  document.getElementById('game-results-screen').style.display = 'block';

```

```

const badge = document.getElementById('res-outcome-badge');

const trophy = document.getElementById('res-trophy-symbol');

const title = document.getElementById('res-headline-title');

const note = document.getElementById('res-motivational-text');


badge.textContent = outcome;

if (outcome === 'WIN') {

  badge.style.background = 'var(--success-bg)';

  badge.style.color = 'var(--success)';

  trophy.textContent = '🏆';

  title.textContent = 'Victory! Objective Mastered!';

  note.textContent = summaryMessage;

} else if (outcome === 'LOSS') {

  badge.style.background = 'var(--danger-bg)';

  badge.style.color = 'var(--danger)';

  trophy.textContent = '✖';

  title.textContent = 'Defeat - Practice Makes Velocity!';

  note.textContent = summaryMessage;

} else {

  badge.style.background = 'var(--accent-glow)';

  badge.style.color = 'var(--accent)';

  trophy.textContent = '🏆';

  title.textContent = 'Practice Concluded';

  note.textContent = summaryMessage;

}

document.getElementById('res-score').textContent = this.score;

document.getElementById('res-wpm').textContent = `${wpm} WPM`;

document.getElementById('res-acc').textContent = `${acc}%`;

document.getElementById('res-errors').textContent = this.errors;

document.getElementById('res-combo').textContent = `${this.maxCombo}x`;


// Telemetry specifics

```

```
const specBox = document.getElementById('res-specific-stats');

const avgReaction = this.reactionLatencies.length

? Math.round(this.reactionLatencies.reduce((a, b) => a + b, 0) / this.reactionLatencies.length)

: 0;

specBox.innerHTML = `

<div style="display:flex; justify-content:space-between;"><span>Game Discipline:</span><strong>${this.config.slug.replace('_', '
').toUpperCase()}</strong></div>

<div style="display:flex; justify-content:space-between;"><span>Difficulty
Tier:</span><strong>${this.config.difficulty.toUpperCase()}</strong></div>

<div style="display:flex; justify-content:space-between;"><span>Session Duration:</span><strong>${Math.round(duration)}s</strong></div>

${avgReaction > 0 ? `<div style="display:flex; justify-content:space-between;"><span>Mean Reaction Latency:</span><strong style="color:var(--
warning);">${avgReaction}ms</strong></div>` : ''}

`;

fetch('/games/api/submit-score', {

method: 'POST',

headers: {'Content-Type': 'application/json'},

body: JSON.stringify({

game_mode: this.config.slug,

play_type: this.config.playMode,

outcome: outcome,

score: this.score,

net_wpm: wpm,

accuracy: acc,

errors: this.errors,

highest_combo: this.maxCombo,

reaction_ms: avgReaction,

duration: duration

})

}).catch(() => {});

}

}
```

```
window.arcadeEngine = new ArcadeRuntimeEngine();

=== FILE: .\app\static\js\audio.js ===

/**

* TypeSphere Multi-Profile Web Audio Synthesizer with Master Volume Gain Control

*/

class TypeSphereAudio {

constructor() {

this.ctx = null;

this.masterGain = null;

this.enabled = true;

this.theme = 'mechanical'; // 'mechanical', 'bubble', 'beep'

this.volume = 0.7; // 0.0 to 1.0

}

init() {

if (!this.ctx) {

const AudioCtx = window.AudioContext || window.webkitAudioContext;

this.ctx = new AudioCtx();

this.masterGain = this.ctx.createGain();

this.masterGain.gain.setValueAtTime(this.volume, this.ctx.currentTime);

this.masterGain.connect(this.ctx.destination);

}

if (this.ctx.state === 'suspended') {

this.ctx.resume();

}

}

setVolume(val) {

this.volume = Math.max(0, Math.min(1, parseFloat(val)));

if (this.masterGain && this.ctx) {

this.masterGain.gain.setValueAtTime(this.volume, this.ctx.currentTime);

}

}
```

}

playKey(isError = false) {

if (!this.enabled || this.volume <= 0) return;

this.init();

try {

const now = this.ctx.currentTime;

const osc = this.ctx.createOscillator();

const gain = this.ctx.createGain();

osc.connect(gain);

gain.connect(this.masterGain);

if (isError) {

osc.type = 'sawtooth';

osc.frequency.setValueAtTime(140, now);

osc.frequency.exponentialRampToValueAtTime(70, now + 0.08);

gain.gain.setValueAtTime(0.2, now);

gain.gain.exponentialRampToValueAtTime(0.001, now + 0.08);

osc.start(now);

osc.stop(now + 0.08);

return;

}

if (this.theme === 'bubble') {

osc.type = 'sine';

const startFreq = 350 + Math.random() * 150;

osc.frequency.setValueAtTime(startFreq, now);

osc.frequency.exponentialRampToValueAtTime(startFreq * 2.2, now + 0.05);

gain.gain.setValueAtTime(0.25, now);

gain.gain.exponentialRampToValueAtTime(0.001, now + 0.05);

osc.start(now);

osc.stop(now + 0.05);

} else if (this.theme === 'beep') {

osc.type = 'triangle';

osc.frequency.setValueAtTime(800 + Math.random() * 60, now);

gain.gain.setValueAtTime(0.12, now);

gain.gain.exponentialRampToValueAtTime(0.001, now + 0.04);

osc.start(now);

osc.stop(now + 0.04);

} else {

// Mechanical Tactile Switch

osc.type = 'triangle';

osc.frequency.setValueAtTime(520 + Math.random() * 80, now);

osc.frequency.exponentialRampToValueAtTime(110, now + 0.04);

gain.gain.setValueAtTime(0.22, now);

gain.gain.exponentialRampToValueAtTime(0.001, now + 0.04);

osc.start(now);

osc.stop(now + 0.04);

}

} catch(e) {

console.warn("Audio playback exception:", e);

}

}

}

window.soundEngine = new TypeSphereAudio();

=== FILE: .\app\static\js\charts.js ===

/**

* Pure SVG Charting Engine (Zero external dependencies).

* Produces smooth responsive WPM timeline and Accuracy visualizations.

*/

class SvgLineChart {

static render(containerId, dataPoints, strokeColor = '#58a6ff') {

const container = document.getElementById(containerId);

```
if (!container || !dataPoints || dataPoints.length < 2) return;

const width = container.clientWidth || 800;

const height = 240;

const padding = 35;

const maxWpm = Math.max(...dataPoints.map(d => d.wpm), 60);

const minWpm = 0;

const getX = (index) => padding + (index / (dataPoints.length - 1)) * (width - 2 * padding);

const getY = (val) => height - padding - ((val - minWpm) / (maxWpm - minWpm)) * (height - 2 * padding);

let dStr = `M ${getX(0)} ${getY(dataPoints[0].wpm)}`;

for (let i = 1; i < dataPoints.length; i++) {

  dStr += ` L ${getX(i)} ${getY(dataPoints[i].wpm)}`;

}

const svg = `

<svg width="100%" height="${height}" viewBox="0 0 ${width} ${height}" preserveAspectRatio="none">

<line x1="${padding}" y1="${height - padding}" x2="${width - padding}" y2="${height - padding}" stroke="#30363d" stroke-width="1"/>

<line x1="${padding}" y1="${padding}" x2="${padding}" y2="${height - padding}" stroke="#30363d" stroke-width="1"/>

<path d="${dStr}" fill="none" stroke="${strokeColor}" stroke-width="2.5" stroke-linecap="round"/>

${dataPoints.map((d, i) => `

  <circle cx="${getX(i)}" cy="${getY(d.wpm)}" r="3" fill="${strokeColor}"/>

`).join("")}

</svg>

`;

container.innerHTML = svg;

}

}

window.SvgLineChart = SvgLineChart;

=== FILE: .\app\static\js\command_palette.js ===
```

```
/**

* TypeSphere Universal Command Palette & Hotkeys Engine

* Activated globally with Ctrl+K, Cmd+K, or ?

*/

class CommandPalette {

  constructor() {

    this.commands = [

      // Test Modes

      { title: "Start 30s Sprint Benchmark", category: "Test Mode", action: () => window.location.href = '/typing/?mode=timed&duration=30' },

      { title: "Start 60s Official Benchmark", category: "Test Mode", action: () => window.location.href = '/typing/?mode=timed&duration=60' },

      { title: "Start 25 Words Sprint", category: "Test Mode", action: () => window.location.href = '/typing/?mode=words&words=25' },

      { title: "Start 50 Words Sprint", category: "Test Mode", action: () => window.location.href = '/typing/?mode=words&words=50' },

      { title: "Start Python Code Typing", category: "Code Mode", action: () => window.location.href = '/typing/?mode=code&lang=python' },

      { title: "Start JavaScript Code Typing", category: "Code Mode", action: () => window.location.href = '/typing/?mode=code&lang=javascript' },

      { title: "Start SQL Code Typing", category: "Code Mode", action: () => window.location.href = '/typing/?mode=code&lang=sql' },

      { title: "Start Quotes & Literature Mode", category: "Test Mode", action: () => window.location.href = '/typing/?mode=quote' },

      // Challenges

      { title: "Daily Challenge (Official 24H)", category: "Challenge", action: () => window.location.href = '/challenges/daily' },

      { title: "Survival Mode (3 Hearts)", category: "Challenge", action: () => window.location.href = '/typing/?mode=survival' },

      { title: "Accuracy Challenge (100% Mandatory)", category: "Challenge", action: () => window.location.href = '/typing/?mode=accuracy' },

      { title: "5-Minute Endurance Marathon", category: "Challenge", action: () => window.location.href = '/typing/?mode=timed&duration=300' },

      { title: "Launch Adaptive Weakness Drill", category: "AI Coach", action: () => window.location.href = '/typing/?mode=adaptive' },

      // Navigation

      { title: "Go to Multiplayer Arena", category: "Navigation", action: () => window.location.href = '/multiplayer/' },

      { title: "Go to Global Leaderboard", category: "Navigation", action: () => window.location.href = '/leaderboard/' },

      { title: "Go to Pilot Dashboard", category: "Navigation", action: () => window.location.href = '/dashboard/' },

      { title: "Go to Typing DNA Telemetry", category: "Navigation", action: () => window.location.href = '/dashboard/dna' },

      { title: "Go to Progression Analytics", category: "Navigation", action: () => window.location.href = '/dashboard/analytics' },

      { title: "Go to Settings & Preferences", category: "Navigation", action: () => window.location.href = '/settings/' },

      // Themes
```

```
{ title: "Switch Theme: Dark Matter", category: "Theme", action: () => window.settingsManager.update('theme', 'dark') },

{ title: "Switch Theme: Clean White", category: "Theme", action: () => window.settingsManager.update('theme', 'light') },

{ title: "Switch Theme: Neon Cyberpunk", category: "Theme", action: () => window.settingsManager.update('theme', 'cyber') },

{ title: "Switch Theme: Warm Sepia", category: "Theme", action: () => window.settingsManager.update('theme', 'sepia') }

];

this.selectedIndex = 0;

this.filteredCommands = [...this.commands];

this.paletteEl = null;

this.hotkeyEl = null;

this.initDOM();

this.bindEvents();

}
```

```
initDOM() {

  // 1. Command Palette Overlay

  const overlay = document.createElement('div');

  overlay.id = 'command-palette-overlay';

  overlay.className = 'palette-overlay';

  overlay.innerHTML = `

<div class="palette-container">

  <div class="palette-search-box">

    <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" style="color: var(--text-muted);">

      <circle cx="11" cy="11" r="8"></circle>

      <line x1="21" y1="21" x2="16.65" y2="16.65"></line>

    </svg>

    <input type="text" id="palette-input" class="palette-search-input" placeholder="Type a command, mode, or theme (e.g. 'code', 'cyber', 'daily')...">

    <span class="kbd-badge">ESC</span>

  </div>

  <div id="palette-list" class="palette-results"></div>

  <div class="palette-footer">
```

```
<span>Navigation: <span class="kbd-badge">&uarr;</span> <span class="kbd-badge">&darr;</span></span></span>

<span>Execute: <span class="kbd-badge">&crarr;</span></span></span>

</div>

</div>

`;

document.body.appendChild(overlay);

this.paletteEl = overlay;

// 2. Hotkeys Reference Modal

const hotkeyModal = document.createElement('div');

hotkeyModal.id = 'hotkey-ref-modal';

hotkeyModal.className = 'palette-overlay';

hotkeyModal.innerHTML = `

<div class="palette-container" style="max-width: 480px;">

  <div style="padding: 1.25rem 1.5rem; border-bottom: 1px solid var(--border-color); display: flex; justify-content: space-between; align-items: center;">

    <h3 style="font-size: 1.15rem; margin: 0; font-weight: 700;">Keyboard Shortcuts</h3>

    <span class="kbd-badge" style="cursor: pointer;" onclick="document.getElementById('hotkey-ref-modal').style.display='none'">ESC</span>

  </div>

  <div style="padding: 1.25rem 1.5rem;">

    <div class="hotkey-row"><span>Quick Restart Test</span> <span class="kbd-badge">Tab</span></div>

    <div class="hotkey-row"><span>Universal Command Palette</span> <span class="kbd-badge">Ctrl + K</span></div>

    <div class="hotkey-row"><span>Distraction-Free Focus Mode</span> <span class="kbd-badge">Esc</span></div>

    <div class="hotkey-row"><span>Keyboard Shortcuts Guide</span> <span class="kbd-badge">?</span></div>

    <div class="hotkey-row"><span>Navigate Commands</span> <span class="kbd-badge">&uarr; / &darr;</span></div>

    <div class="hotkey-row"><span>Execute Selected Command</span> <span class="kbd-badge">Enter</span></div>

  </div>

</div>

`;

document.body.appendChild(hotkeyModal);

this.hotkeyEl = hotkeyModal;

}
```



```
bindEvents() {

  window.addEventListener('keydown', (e) => {

    // Ctrl+K or Cmd+K

    if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'k') {

      e.preventDefault();

      this.openPalette();

      return;

    }

    // '?' for Shortcuts Guide (when not actively typing inside an input)

    if (e.key === '?' && document.activeElement.tagName !== 'INPUT' && document.activeElement.tagName !== 'TEXTAREA') {

      e.preventDefault();

      this.openHotkeys();

      return;

    }

    // Escape to close

    if (e.key === 'Escape') {

      this.closeAll();

      return;

    }

    // Palette Navigation

    if (this.paletteEl.style.display === 'flex') {

      if (e.key === 'ArrowDown') {

        e.preventDefault();

        this.selectedIndex = (this.selectedIndex + 1) % this.filteredCommands.length;

        this.renderList();

      } else if (e.key === 'ArrowUp') {

        e.preventDefault();

        this.selectedIndex = (this.selectedIndex - 1 + this.filteredCommands.length) % this.filteredCommands.length;

        this.renderList();

      } else if (e.key === 'Enter') {
```

```
        e.preventDefault();

        this.executeSelected();

      }

    }

  });

  // Close on backdrop click

  this.paletteEl.addEventListener('click', (e) => {

    if (e.target === this.paletteEl) this.closeAll();

  });

  this.hotkeyEl.addEventListener('click', (e) => {

    if (e.target === this.hotkeyEl) this.closeAll();

  });

  // Search filter input

  const input = document.getElementById('palette-input');

  input.addEventListener('input', (e) => {

    const q = e.target.value.toLowerCase().trim();

    this.filteredCommands = this.commands.filter(c =>

      c.title.toLowerCase().includes(q) || c.category.toLowerCase().includes(q)

    );

    this.selectedIndex = 0;

    this.renderList();

  });

}

openPalette() {

  this.paletteEl.style.display = 'flex';

  this.filteredCommands = [...this.commands];

  this.selectedIndex = 0;

  this.renderList();

  const input = document.getElementById('palette-input');

  input.value = '';
```

```
        setTimeout(() => input.focus(), 50);
    }

    openHotkeys() {

        this.hotkeyEl.style.display = 'flex';

    }

    closeAll() {

        this.paletteEl.style.display = 'none';

        this.hotkeyEl.style.display = 'none';

    }

    renderList() {

        const list = document.getElementById('palette-list');

        list.innerHTML = "";

        if (this.filteredCommands.length === 0) {

            list.innerHTML = '<div style="padding: 1.5rem; text-align: center; color: var(--text-muted);">No matching commands found.</div>';

            return;

        }

        this.filteredCommands.forEach((cmd, idx) => {

            const item = document.createElement('div');

            item.className = `palette-item ${idx === this.selectedIndex ? 'selected' : ''}`;

            item.innerHTML = `

                <span>${cmd.title}</span>

                <span class="palette-item-category">${cmd.category}</span>

            `;

            item.addEventListener('click', () => {

                cmd.action();

                this.closeAll();

            });

            list.appendChild(item);

        });

    }

}
```

```
    });

    const activeItem = list.children[this.selectedIndex];

    if (activeItem) activeItem.scrollIntoView({ block: 'nearest' });

}

executeSelected() {

    if (this.filteredCommands[this.selectedIndex]) {

        this.filteredCommands[this.selectedIndex].action();

        this.closeAll();

    }

}

}
```

```
window.commandPalette = new CommandPalette();
```

```
=== FILE: .\app\static\js\engine.js ===

/**

 * TypeSphere Core Engine

 * Two-tier UI architecture, continuous passage streaming for timed tests,

 * and reliable state transitions without layout jumping.

 */

class TypingEngine {

    constructor() {

        this.targetText = "";

        this.currentIndex = 0;

        this.events = [];

        this.timeline = [];

        this.startTime = null;

        this.timerInterval = null;

        // Test Configuration

    }

}
```

```
this.durationLimit = 60; // 15, 30, 60, 120, 300, 600, 1200, or 0 (No Limit)

this.contentType = 'words';

this.level = 'moderate'; // easy, moderate, hard, expert

this.codeLanguage = 'python';

this.mode = 'timed'; // 'timed', 'accuracy', 'survival', 'adaptive'

this.isRankedEligible = true;
```

```
// Behavioral Defaults
```

```
this.blindModeActive = true; // Blind Mode: ON by default

this.ghostEnabled = false; // Ghost Mode: OFF by default

this.ghostWpm = 70;

this.ghostProgress = 0;

this.ghostInterval = null;
```

```
// Runtime Lifecycle
```

```
this.isFinished = false;

this.isPaused = false;

this.pausedAt = 0;

this.totalPausedDuration = 0;

this.streak = 0;

this.errors = 0;
```

```
// DOM References
```

```
this.container = document.getElementById('typing-box');

this.display = document.getElementById('text-display');

this.caret = document.getElementById('caret');

this.ghostCaret = document.getElementById('ghost-caret');

this.hudWpm = document.getElementById('hud-wpm');

this.hudAcc = document.getElementById('hud-acc');

this.hudTime = document.getElementById('hud-time');

this.hudErrors = document.getElementById('hud-errors');

this.speedometerArc = document.getElementById('speedo-arc');

this.pauseModal = document.getElementById('pause-modal');
```

```
this.mobileProxy = document.getElementById('mobile-text-proxy');

this.rankNoticeBadge = document.getElementById('rank-eligibility-badge');
```

```
this.bindEvents();

this.parseUrlParameters();

this.loadUserSessionPreferences();

}
```

```
parseUrlParameters() {

  const params = new URLSearchParams(window.location.search);
```

```
  const modeParam = params.get('mode');

  if (modeParam) this.mode = modeParam;
```

```
  const durParam = params.get('duration');

  if (durParam) this.durationLimit = parseInt(durParam, 10);
```

```
  const typeParam = params.get('content_type');

  if (typeParam) this.contentType = typeParam;
```

```
  const levelParam = params.get('level');

  if (levelParam) this.level = levelParam;
```

```
  // Ranked eligibility validation: Custom text & unstandardized runs are NOT ranked

  this.updateRankedEligibility();

}
```

```
updateRankedEligibility() {

  // Ranked eligible ONLY if: standard timed test (15s, 30s, 60s, 120s) with standard content

  const isStandardTime = [15, 30, 60, 120].includes(this.durationLimit);

  const isStandardContent = ['words', 'paragraphs', 'quotes'].includes(this.contentType);

  this.isRankedEligible = isStandardTime && isStandardContent && (this.mode !== 'custom');
```

```
if (this.rankNoticeBadge) {

  if (this.isRankedEligible) {

    this.rankNoticeBadge.innerHTML = `<span style="color:var(--success);">● Ranked Match</span>`;

    this.rankNoticeBadge.title = "Eligible for Global Leaderboards and Personal Records.";

  } else {

    this.rankNoticeBadge.innerHTML = `<span style="color:var(--text-muted);">○ Practice Only</span>`;

    this.rankNoticeBadge.title = "Custom content or irregular parameters will not affect public leaderboards.";

  }

}
```

```
loadUserSessionPreferences() {

  const prefs = window.settingsManager ? window.settingsManager.current : null;

  if (prefs) {

    if (!window.location.search.includes('duration=')) this.durationLimit = prefs.default_duration || 60;

    if (!window.location.search.includes('content_type=')) this.contentType = prefs.default_content || 'words';

    if (!window.location.search.includes('level=')) this.level = prefs.default_level || 'moderate';

    this.blindModeActive = prefs.blind_mode !== undefined ? prefs.blind_mode : true;

    this.ghostEnabled = prefs.ghost_mode !== undefined ? prefs.ghost_mode : false;

  }

  this.updateControlsUI();

}
```

```
updateControlsUI() {

  const tSel = document.getElementById('cfg-time-select');

  const cSel = document.getElementById('cfg-type-select');

  const dSel = document.getElementById('cfg-diff-select');

  const bBtn = document.getElementById('btn-blind');

  const gBtn = document.getElementById('btn-ghost');

  if (tSel) tSel.value = this.durationLimit.toString();

  if (cSel) cSel.value = this.contentType;

  if (dSel) dSel.value = this.level;
```

```
if (bBtn) {

  bBtn.textContent = `Blind: ${this.blindModeActive ? 'ON' : 'OFF'}`;

  bBtn.classList.toggle('active-choice', this.blindModeActive);

}

if (gBtn) {

  gBtn.textContent = `Ghost: ${this.ghostEnabled ? 'ON' : 'OFF'}`;

  gBtn.classList.toggle('active-choice', this.ghostEnabled);

}
```

```
this.updateRankedEligibility();

}
```

```
async loadPrompt(append = false) {

  if (this.contentType === 'custom') {

    const stored = sessionStorage.getItem('typesphere_custom_text');

    if (stored && stored.trim().length > 0) {

      this.targetText = append ? `${this.targetText} ${stored.trim()}` : stored.trim();

      this.renderText(append);

      if (!append) this.reset();

      return;

    }

  }

}
```

```
try {

  const url =
    `'/typing/api/text?content_type=${encodeURIComponent(this.contentType)}&level=${encodeURIComponent(this.level)}&code_lang=${encodeURIComponent(this.codeLanguage)}&batch_size=75`;

  const res = await fetch(url);

  const data = await res.json();

  const chunk = (data.content || '').trim();

  if (append) {
```

```

    this.targetText = `${this.targetText} ${chunk}`;

    this.renderText(true);

  } else {

    this.targetText = chunk;

    this.renderText(false);

    this.reset();

  }

} catch {

  const fallback = "Simplicity is prerequisite for reliability. Excellence is not an act but a habit honed by persistent, focused rhythm.";

  if (append) {

    this.targetText = `${this.targetText} ${fallback}`;

    this.renderText(true);

  } else {

    this.targetText = fallback;

    this.renderText(false);

    this.reset();

  }

}

}

renderText(isAppend = false) {

  if (!isAppend) {

    this.display.innerHTML = "";

    const fragment = document.createDocumentFragment();

    for (let i = 0; i < this.targetText.length; i++) {

      const span = document.createElement('span');

      span.className = 'char';

      span.textContent = this.targetText[i];

      fragment.appendChild(span);

    }

    this.display.appendChild(fragment);

    this.updateCaretPosition();

  } else {

```

```

    const currentSpanCount = this.display.children.length;

    const fragment = document.createDocumentFragment();

    for (let i = currentSpanCount; i < this.targetText.length; i++) {

      const span = document.createElement('span');

      span.className = 'char';

      span.textContent = this.targetText[i];

      fragment.appendChild(span);

    }

    this.display.appendChild(fragment);

  }

}

reset() {

  clearInterval(this.timerInterval);

  clearInterval(this.ghostInterval);

  this.currentIndex = 0;

  this.events = [];

  this.timeline = [];

  this.startTime = null;

  this.isFinished = false;

  this.isPaused = false;

  this.pausedAt = 0;

  this.totalPausedDuration = 0;

  this.streak = 0;

  this.errors = 0;

  this.ghostProgress = 0;

  if (this.pauseModal) this.pauseModal.style.display = 'none';

  const floatBar = document.getElementById('paused-floating-bar');

  if (floatBar) floatBar.style.display = 'none';

  this.hudWpm.textContent = '0';

  this.hudAcc.textContent = '100%';

```

```
this.hudErrors.textContent = '0';

this.updateSpeedometer(0);

if (this.durationLimit > 0) {

    this.hudTime.textContent = this.formatTimeDisplay(this.durationLimit);

} else {

    this.hudTime.textContent = '0s (No Limit)';

}
```

```
const spans = this.display.querySelectorAll('.char');

spans.forEach(s => s.className = 'char');

this.updateCaretPosition();

}
```

```
formatTimeDisplay(totalSeconds) {

    if (totalSeconds >= 60) {

        const mins = Math.floor(totalSeconds / 60);

        const secs = totalSeconds % 60;

        return `${mins}m ${secs > 0 ? `${secs}s` : ''}`.trim();

    }

    return `${totalSeconds}s`;

}
```

```
bindEvents() {

    window.addEventListener('keydown', (e) => {

        if (this.isPaused) {

            if (e.key === ' ' || e.key === 'Escape') {

                e.preventDefault();

                this.resumeTest();

            }

            return;

        }

    })

}
```

```
if (e.key === 'Tab') {

    e.preventDefault();

    this.loadPrompt(false);

    return;

}
```

```
if (e.key === 'Escape') {

    if (this.startTime && !this.isFinished) {

        e.preventDefault();

        this.togglePause();

        return;

    }

    document.body.classList.toggle('focus-mode');

    return;

}
```

```
this.handleKeystroke(e.key, e);

});
```

```
this.container.addEventListener('click', () => {

    this.container.classList.add('focus-ring');

    if (this.mobileProxy) this.mobileProxy.focus();

});
```

```
if (this.mobileProxy) {

    this.mobileProxy.addEventListener('input', () => {

        if (this.isPaused || this.isFinished) return;

        const val = this.mobileProxy.value;

        if (val.length > 0) {

            const char = val[val.length - 1];

            this.handleKeystroke(char, null);

            this.mobileProxy.value = "";

        }

    })

}
```

```
    });
  }
}

togglePause() {
  if (this.isPaused) this.resumeTest();

  else this.pauseTest();
}

pauseTest() {
  if (!this.startTime || this.isFinished || this.isPaused) return;

  this.isPaused = true;

  this.pausedAt = performance.now() / 1000.0;

  clearInterval(this.timerInterval);

  clearInterval(this.ghostInterval);

  if (this.pauseModal) this.pauseModal.style.display = 'flex';
}

closePauseModal() {
  if (this.pauseModal) this.pauseModal.style.display = 'none';

  const floatBar = document.getElementById('paused-floating-bar');

  if (this.isPaused && floatBar) floatBar.style.display = 'flex';
}

resumeTest() {
  if (!this.isPaused) return;

  const now = performance.now() / 1000.0;

  this.totalPausedDuration += (now - this.pausedAt);

  this.isPaused = false;

  if (this.pauseModal) this.pauseModal.style.display = 'none';

  const floatBar = document.getElementById('paused-floating-bar');
```

```
if (floatBar) floatBar.style.display = 'none';

this.startTick(true);

if (this.ghostEnabled) this.startGhostRacer();

if (this.mobileProxy) this.mobileProxy.focus();
}

handleKeystroke(key, originalEvent) {
  if (this.isFinished || this.isPaused) return;

  if (['Shift', 'Control', 'Alt', 'Meta', 'CapsLock'].includes(key)) return;

  const now = performance.now() / 1000.0;

  if (!this.startTime) {
    this.startTime = now;

    this.startTick(false);

    if (this.ghostEnabled) this.startGhostRacer();
  }

  const spans = this.display.querySelectorAll('.char');

  if (this.currentIndex >= spans.length) return;

  const expectedChar = this.targetText[this.currentIndex];

  // Backspace Handling: Disabled if Blind Mode is ON
  if (key === 'Backspace') {
    if (originalEvent) originalEvent.preventDefault();

    if (this.blindModeActive) return; // Strict blind mode enforcement

    if (this.currentIndex > 0) {
      this.currentIndex--;

      spans[this.currentIndex].className = 'char';

      this.updateCaretPosition();
```

```

    }

    return;
}

if (key.length !== 1) return;

if (originalEvent) originalEvent.preventDefault();

const isCorrect = (key === expectedChar);

if (isCorrect) {

    spans[this.currentIndex].className = 'char correct';

    this.streak++;

    if (window.soundEngine) window.soundEngine.playKey(false);

} else {

    spans[this.currentIndex].className = 'char incorrect';

    this.streak = 0;

    this.errors++;

    if (window.soundEngine) window.soundEngine.playKey(true);

if (this.mode === 'accuracy') {

    alert("Zero-Error Gauntlet breached. 100% accuracy required.");

    this.reset();

    return;

}

}

this.events.push({

    char_index: this.currentIndex,

    expected: expectedChar,

    typed: key,

    correct: isCorrect,

    timestamp: parseFloat(now.toFixed(4))

});

```

```

this.currentIndex++;

this.updateCaretPosition();

this.updateLiveStats(now);

// Continuous Infinite Content Pipeline: Stream more text before reaching boundary

if (this.currentIndex >= spans.length - 20) {

    this.loadPrompt(true);

}

}

startTick(isResume = false) {

    if (!isResume) this.tickElapsed = 0;

this.timerInterval = setInterval(() => {

    this.tickElapsed++;

    const now = performance.now() / 1000.0;

    this.updateLiveStats(now);

    if (this.durationLimit > 0) {

        const remaining = Math.max(0, this.durationLimit - this.tickElapsed);

        this.hudTime.textContent = this.formatTimeDisplay(remaining);

        if (remaining <= 0) {

            this.finishTest();

        }

    } else {

        this.hudTime.textContent = `${this.formatTimeDisplay(this.tickElapsed)} (No Limit)`;

    }

}, 1000);

}

startGhostRacer() {

    const charsPerSec = (this.ghostWpm * 5) / 60;

```



```
this.ghostInterval = setInterval(() => {

  if (this.isFinished || this.isPaused) {

    clearInterval(this.ghostInterval);

    return;

  }

  this.ghostProgress += (charsPerSec * 0.1);

  const ghostIdx = Math.min(this.targetText.length - 1, Math.floor(this.ghostProgress));

  const spans = this.display.querySelectorAll(''.char');

  if (spans[ghostIdx] && this.ghostCaret) {

    this.ghostCaret.style.display = 'block';

    this.ghostCaret.style.left = `${spans[ghostIdx].offsetLeft}px`;

    this.ghostCaret.style.top = `${spans[ghostIdx].offsetTop + 4}px`;

  }

}, 100);

}

updateLiveStats(now) {

  const elapsedMinutes = (now - this.startTime - this.totalPausedDuration) / 60.0;

  if (elapsedMinutes <= 0) return;

  const correctChars = this.events.filter(ev => ev.correct).length;

  const netWpm = Math.max(0, Math.round((correctChars / 5.0) / elapsedMinutes));

  const acc = this.currentIndex > 0 ? Math.round((correctChars / this.currentIndex) * 100) : 100;

  this.hudWpm.textContent = netWpm;

  this.hudAcc.textContent = `${acc}%`;

  this.hudErrors.textContent = this.errors;

  this.updateSpeedometer(netWpm);

  this.timeline.push({

    t: parseFloat((now - this.startTime - this.totalPausedDuration).toFixed(2)),

    wpm: netWpm,

    acc: acc
```

```
});

}

updateSpeedometer(wpm) {

  if (!this.speedometerArc) return;

  const pct = Math.min(1.0, wpm / 150.0);

  const offset = 100 - (pct * 100);

  this.speedometerArc.style.strokeDashoffset = offset;

}

updateCaretPosition() {

  const spans = this.display.querySelectorAll(''.char');

  if (this.currentIndex < spans.length) {

    const target = spans[this.currentIndex];

    this.caret.style.left = `${target.offsetLeft}px`;

    this.caret.style.top = `${target.offsetTop + 4}px`;

    // Smooth Line-by-Line Auto-Centering

    const targetMid = target.offsetTop - (this.container.clientHeight / 2) + 25;

    if (Math.abs(this.container.scrollTop - targetMid) > 20) {

      this.container.scrollTo({

        top: Math.max(0, targetMid),

        behavior: 'smooth'

      });

    }

  } else if (spans.length > 0) {

    const last = spans[spans.length - 1];

    this.caret.style.left = `${last.offsetLeft + last.offsetWidth}px`;

    this.caret.style.top = `${last.offsetTop + 4}px`;

  }

}

async finishTest() {
```

```
if (this.isFinished) return;

this.isFinished = true;

clearInterval(this.timerInterval);

clearInterval(this.ghostInterval);


const finishTime = performance.now() / 1000.0;

const duration = parseFloat(((finishTime - (this.startTime || finishTime) - this.totalPausedDuration).toFixed(2)));


const payload = {

  events: this.events,

  timeline: this.timeline,

  duration: Math.max(1.0, duration),

  target_text: this.targetText.substring(0, this.currentIndex),

  mode: this.durationLimit > 0 ? `timed_${this.durationLimit}` : 'no_limit',

  is_ranked: this.isRankedEligible,

  content_category: this.contentType,

  difficulty: this.level

};


try {

  const res = await fetch('/typing/api/submit', {

    method: 'POST',

    headers: { 'Content-Type': 'application/json' },

    body: JSON.stringify(payload)

  });

  const data = await res.json();

  if (data.success && data.test_id) {

    window.location.href = `/typing/result/${data.test_id}`;

  }

} catch {

  window.location.reload();

}
```

```
}

window.typingEngine = new TypingEngine();

document.addEventListener('DOMContentLoaded', () => {

  window.typingEngine.loadPrompt(false);

});


=== FILE: .\app\static\js\heatmap.js ===


=== FILE: .\app\static\js\keyboard.js ===

/**

 * TypeSphere Biometric Heatmap & Key Diagnostics Telemetry Inspector

 */

class VirtualKeyboard {

  constructor() {

    this.container = document.getElementById('on-screen-keyboard');

    this.fingerGuide = document.getElementById('active-finger-guide');

    this.keyData = {};

    this.keyElements = {};

  }

  this.layout = [

    ["", "1", "2", "3", "4", "5", "6", "7", "8", "9", "0", "-", "=", "Backspace"],

    ["Tab", "q", "w", "e", "r", "t", "y", "u", "i", "o", "p", "[", "]", "\\\""],

    ["Caps", "a", "s", "d", "f", "g", "h", "j", "k", "l", ";", "'", "Enter"],

    ["Shift", "z", "x", "c", "v", "b", "n", "m", ",", ".", "/", "Shift"],

    ["Space"]

  ];

  this.fingerMapping = {

    'q': 'Left Pinky', 'a': 'Left Pinky', 'z': 'Left Pinky', '1': 'Left Pinky', '": 'Left Pinky',

    'w': 'Left Ring', 's': 'Left Ring', 'x': 'Left Ring', '2': 'Left Ring',

    'e': 'Left Middle', 'd': 'Left Middle', 'c': 'Left Middle', '3': 'Left Middle',
```

```
'r': 'Left Index', 'f': 'Left Index', 'v': 'Left Index', 't': 'Left Index', 'g': 'Left Index', 'b': 'Left Index', '4': 'Left Index', '5': 'Left Index',

'y': 'Right Index', 'h': 'Right Index', 'n': 'Right Index', 'u': 'Right Index', 'j': 'Right Index', 'm': 'Right Index', '6': 'Right Index', '7': 'Right Index',

'i': 'Right Middle', 'k': 'Right Middle', ',': 'Right Middle', '8': 'Right Middle',

'o': 'Right Ring', 'l': 'Right Ring', '._': 'Right Ring', '9': 'Right Ring',

'p': 'Right Pinky', '._': 'Right Pinky', '/': 'Right Pinky', '0': 'Right Pinky', '[': 'Right Pinky', ']' : 'Right Pinky', '\': 'Right Pinky', '._': 'Right Pinky', '=': 'Right Pinky',

' ': 'Left / Right Thumb', 'space': 'Left / Right Thumb'

});

if (this.container) {

  this.render();

  this.fetchHeatmapData();

}

}

render() {

  this.container.innerHTML = "";

  this.keyElements = {};

  this.layout.forEach(row => {

    const rowDiv = document.createElement('div');

    rowDiv.className = 'keyboard-row';

    row.forEach(key => {

      const keyDiv = document.createElement('div');

      const lower = key.toLowerCase();

      let specialClass = "";

      if (key === 'Space') specialClass = 'space';

      else if (key === 'Backspace') specialClass = 'backspace';

      else if (key === 'Tab') specialClass = 'tab';

      else if (key === 'Caps') specialClass = 'caps';

      else if (key === 'Enter') specialClass = 'enter';

      else if (key === 'Shift') specialClass = 'shift';
```

```
keyDiv.className = `k-key ${specialClass}`;

keyDiv.innerHTML = `<span class="key-label">${key}</span>`;

keyDiv.dataset.key = lower;

rowDiv.appendChild(keyDiv);

this.keyElements[lower] = keyDiv;

keyDiv.addEventListener('click', () => this.inspectKey(key));

});

this.container.appendChild(rowDiv);

});

this.paintHeatmap();

}

inspectKey(key) {

  const modal = document.getElementById('key-inspect-modal');

  if (!modal) return;

  const lower = key.toLowerCase();

  const data = this.keyData[lower] || { total: 0, errors: 0, delays: [], confusions: {} };

  const finger = this.fingerMapping[key] || this.fingerMapping[lower] || 'Adaptive Hand';

  const errRate = data.total > 0 ? Math.round((data.errors / data.total) * 100) : 0;

  const avgDelay = data.delays && data.delays.length > 0 ? Math.round(data.delays.reduce((a,b)=>a+b,0) / data.delays.length) : 'N/A';

  let topConfusion = 'None detected';

  let maxConf = 0;

  if (data.confusions) {

    Object.keys(data.confusions).forEach(c => {

      if (data.confusions[c] > maxConf) {

        maxConf = data.confusions[c];
```

```
    topConfusion = `${c.toUpperCase()}` ${maxConf} mistakes`);

    }

});

}

document.getElementById('km-key-title').textContent = `Key Diagnostics: [ ${key.toUpperCase()} ]`;

document.getElementById('km-finger').textContent = finger;

document.getElementById('km-total').textContent = data.total;

document.getElementById('km-error-rate').textContent = `${errRate}%`;

document.getElementById('km-delay').textContent = avgDelay === 'N/A' ? 'N/A' : `${avgDelay} ms`;

document.getElementById('km-confusion').textContent = topConfusion;


modal.style.display = 'flex';

}

updateCurrentExpectedKey(expectedChar) {

    if (!expectedChar) return;

    const lower = (expectedChar === ' ') ? 'space' : expectedChar.toLowerCase();

    const finger = this.fingerMapping[lower] || this.fingerMapping[expectedChar] || 'Touch Key';

    if (this.fingerGuide) {

        this.fingerGuide.innerHTML = `Finger: <strong style="color: var(--accent);">${finger}</strong> &bull; Target: <span style="font-family: var(--font-mono); color: #f59e0b; background: var(--bg-card); padding: 0.1rem 0.45rem; border-radius: 4px; font-weight: 800;">${expectedChar === ' ' ? 'Space' : expectedChar}</span>`;

    }

}

highlightKey(key) {

    const norm = (key === ' ') ? 'space' : key.toLowerCase();

    const el = this.keyElements[norm];

    if (el) {

        el.classList.add('active');

        setTimeout(() => el.classList.remove('active'), 100);

    }

}
```

```
    }

}

async fetchHeatmapData() {

    try {

        const stored = localStorage.getItem('typesphere_key_stats');

        if (stored) {

            this.keyData = JSON.parse(stored);

            this.paintHeatmap();

        }

    } catch(e) {}

}

recordKeyMetric(expected, typed, isCorrect, latencyMs) {

    const k = expected.toLowerCase();

    if (!this.keyData[k]) {

        this.keyData[k] = { total: 0, errors: 0, delays: [], confusions: {} };

    }

    this.keyData[k].total++;

    if (!isCorrect) {

        this.keyData[k].errors++;

    }

    if (latencyMs && latencyMs > 0 && latencyMs < 2000) {

        this.keyData[k].delays.push(latencyMs);

        if (this.keyData[k].delays.length > 30) this.keyData[k].delays.shift();

    }

    localStorage.setItem('typesphere_key_stats', JSON.stringify(this.keyData));

    this.paintHeatmap();

}

paintHeatmap() {

    Object.keys(this.keyData).forEach(k => {

        const el = this.keyElements[k];

    })

}
```

```
if (!el) return;

const data = this.keyData[k];

if (data.total >= 3) {

  const errRate = data.errors / data.total;

  if (errRate > 0.25) {

    el.style.borderColor = `var(--danger)`;

    el.style.backgroundColor = `rgba(248, 81, 73, 0.18)`;

  } else if (errRate > 0.1) {

    el.style.borderColor = `var(--warning)`;

    el.style.backgroundColor = `rgba(210, 153, 34, 0.15)`;

  } else {

    el.style.borderColor = `var(--success)`;

    el.style.backgroundColor = `rgba(63, 185, 80, 0.12)`;

  }

}

});

}
```

window.virtualKeyboard = new VirtualKeyboard();

=== FILE: \app\static\js\layouts.js ===

```
/**

 * TypeSphere Layout Remapping Engine

 * Maps physical hardware KeyCodes (e.code) to ergonomic target layouts

 * allowing users to type Dvorak, Colemak, Workman, etc. on a standard QWERTY keyboard.

 */
```

```
const KEYBOARD_LAYOUTS = {

  qwerty: {

    name: 'QWERTY',

    rows: [

      ["`", "1", "2", "3", "4", "5", "6", "7", "8", "9", "0", "-", "=", "Backspace"],
```

```
["Tab", "q", "w", "e", "r", "t", "y", "u", "i", "o", "p", "[", "]", "\\\"],

["Caps", "a", "s", "d", "f", "g", "h", "j", "k", "l", ";", "'", "Enter"],

["Shift", "z", "x", "c", "v", "b", "n", "m", ",", ".", "/", "Shift"],

["Space"]

],

homeKeys: ['a', 's', 'd', 'f', 'j', 'k', 'l', ';'],

// Identity mapping (standard QWERTY)

map: null

},
```

```
dvorak: {

  name: 'Dvorak',

  rows: [

    ["", "1", "2", "3", "4", "5", "6", "7", "8", "9", "0", "[", "]", "Backspace"],

    ["Tab", "", ", ", "p", "y", "f", "g", "c", "t", "i", "/", "=", "\\\"],

    ["Caps", "a", "o", "e", "u", "i", "d", "h", "t", "n", "s", "-", "Enter"],

    ["Shift", "-", "q", "j", "k", "x", "b", "m", "w", "v", "z", "Shift"],

    ["Space"]

],

homeKeys: ['a', 'o', 'e', 'u', 'h', 't', 'n', 's'],

map: {

  'KeyQ': '', 'KeyW': ',', 'KeyE': '.', 'KeyR': 'p', 'KeyT': 'y',

  'KeyY': 'f', 'KeyU': 'g', 'KeyI': 'c', 'KeyO': 'r', 'KeyP': 'i',

  'BracketLeft': '/', 'BracketRight': '=',

  'KeyA': 'a', 'KeyS': 'o', 'KeyD': 'e', 'KeyF': 'u', 'KeyG': 'i',

  'KeyH': 'd', 'KeyJ': 'h', 'KeyK': 't', 'KeyL': 'n', 'Semicolon': 's', 'Quote': "'",

  'KeyZ': "'", 'KeyX': 'q', 'KeyC': 'j', 'KeyV': 'k', 'KeyB': 'x',

  'KeyN': 'b', 'KeyM': 'm', 'Comma': 'w', 'Period': 'v', 'Slash': 'z',

  'Minus': '[', 'Equal': ']'

}

},
```

colemak: {

```
name: 'Colemak',

rows: [

["", "1", "2", "3", "4", "5", "6", "7", "8", "9", "0", "-", "=", "Backspace"],

["Tab", "q", "w", "f", "p", "g", "j", "l", "u", "y", ";", "[", "]", "\\"],

["Caps", "a", "r", "s", "t", "d", "h", "n", "e", "i", "o", "", "Enter"],

["Shift", "z", "x", "c", "v", "b", "k", "m", ",", ".", "/", "Shift"],

["Space"]

],

homeKeys: ['a', 'r', 's', 't', 'n', 'e', 'i', 'o'],

map: {

  'KeyE': 'f', 'KeyR': 'p', 'KeyT': 'g', 'KeyY': 'j', 'KeyU': 'l', 'KeyI': 'u', 'KeyO': 'y', 'KeyP': ';',

  'KeyS': 'r', 'KeyD': 's', 'KeyF': 't', 'KeyG': 'd', 'KeyJ': 'n', 'KeyK': 'e', 'KeyL': 'i', 'Semicolon': 'o',

  'KeyN': 'k'

}

},
```

```
workman: {

name: 'Workman',

rows: [

["", "1", "2", "3", "4", "5", "6", "7", "8", "9", "0", "-", "=", "Backspace"],

["Tab", "q", "d", "r", "w", "b", "j", "f", "u", "p", ";", "[", "]", "\\"],

["Caps", "a", "s", "h", "t", "g", "y", "n", "e", "o", "i", "", "Enter"],

["Shift", "z", "x", "m", "c", "v", "k", "l", ",", ".", "/", "Shift"],

["Space"]

],

homeKeys: ['a', 's', 'h', 't', 'n', 'e', 'o', 'i'],

map: {

  'KeyW': 'd', 'KeyE': 'r', 'KeyR': 'w', 'KeyT': 'b', 'KeyY': 'j', 'KeyU': 'f', 'KeyO': 'p', 'KeyP': ';',

  'KeyD': 'h', 'KeyF': 't', 'KeyH': 'y', 'KeyJ': 'n', 'KeyL': 'o', 'Semicolon': 'i',

  'KeyC': 'm', 'KeyV': 'c', 'KeyB': 'v', 'KeyN': 'k', 'KeyM': 'l'

}

},
```

```
azerty: {

name: 'AZERTY',

rows: [

["2", "&", "é", "\", "", "(", "-", "è", "_", "ç", "à", ")"), "=", "Backspace"],

["Tab", "a", "z", "e", "r", "t", "y", "u", "i", "o", "p", "s", "s*"],

["Caps", "q", "s", "d", "f", "g", "h", "j", "k", "l", "m", "ù", "Enter"],

["Shift", "w", "x", "c", "v", "b", "n", ",", ".", "/", "Shift"],

["Space"]

],

homeKeys: ['q', 's', 'd', 'f', 'j', 'k', 'l', 'm'],

map: {

  'KeyQ': 'a', 'KeyW': 'z', 'KeyA': 'q', 'KeyZ': 'w', 'Semicolon': 'm', 'KeyM': ';'

}

}

};
```

```
class LayoutManager {

  constructor() {

    this.activeLayout = 'qwerty';

    this.loadPreference();

  }

  loadPreference() {

    const saved = localStorage.getItem('typesphere_layout');

    if (saved && KEYBOARD_LAYOUTS[saved]) {

      this.activeLayout = saved;

    }

  }

  setLayout(layoutKey) {

    if (KEYBOARD_LAYOUTS[layoutKey]) {

      this.activeLayout = layoutKey;

      localStorage.setItem('typesphere_layout', layoutKey);

    }

  }

}
```

```
if (window.virtualKeyboard) {

    window.virtualKeyboard.renderForLayout(layoutKey);

}

}

}

}

/**

 * Translates a physical keyboard event to the target layout character

 */

translateEvent(e) {

    if (this.activeLayout === 'qwerty') {

        return e.key;

    }

}

const layout = KEYBOARD_LAYOUTS[this.activeLayout];

if (!layout || ! layout.map) {

    return e.key;

}

// Lookup physical key code (e.g. 'KeyE')

let char = layout.map[e.code];

if (!char) {

    return e.key;

}

// Preserve uppercase if Shift is held or CapsLock active

if (e.shiftKey) {

    char = char.toUpperCase();

}

return char;

}

}
```

```
window.layoutManager = new LayoutManager();
```

```
=== FILE: .\app\static\js\multiplayer.js ===
```

```
=== FILE: .\app\static\js\settings.js ===
```

```
/**

 * TypeSphere Unified Settings & Telemetry Manager

 * Distinctly separates Default Settings, Anonymous Session Settings, and User Profile Settings.

 */

class SettingsManager {

    constructor() {

        this.defaultSettings = {

            theme: 'dark',

            font_family: 'JetBrains Mono',

            font_size: 22,

            caret_style: 'smooth',

            sound_enabled: true,

            sound_theme: 'mechanical',

            sound_volume: 0.7,

            show_keyboard: true,

            default_duration: 60,

            default_content: 'words',

            default_level: 'moderate',

            blind_mode: true, // Default: ON

            ghost_mode: false // Default: OFF

        };

        this.current = Object.assign({}, this.defaultSettings);

        this.initSettings();

    }

}
```

```

initSettings() {

    // Check if user is authenticated via server injection

    const authElement = document.getElementById('user-auth-meta');

    const isAuthenticated = authElement ? authElement.dataset.authenticated === 'true' : false;

    if (isAuthenticated) {

        // Profile Settings: stored in database or localized profile cache

        const stored = localStorage.getItem('typesphere_user_profile_prefs');

        if (stored) {

            try {

                this.current = Object.assign({}, this.defaultSettings, JSON.parse(stored));

            } catch {}

        }

    } else {

        // Anonymous Users: Session settings only. Refreshing restores default settings

        const sessionData = sessionStorage.getItem('typesphere_anon_session_prefs');

        if (sessionData) {

            try {

                this.current = Object.assign({}, this.defaultSettings, JSON.parse(sessionData));

            } catch {}

        } else {

            this.current = Object.assign({}, this.defaultSettings);

        }

    }

    this.applyAll();

}

applyAll() {

    // 1. Theme

    document.documentElement.setAttribute('data-theme', this.current.theme);

    // 2. CSS Variables

```

```

document.documentElement.style.setProperty('--font-mono', `${this.current.font_family}`, 'monospace');

document.documentElement.style.setProperty('--typing-font-size', `${this.current.font_size}px`);

// 3. Caret Style

const carets = document.querySelectorAll('.caret');

carets.forEach(c => {

    c.className = `caret caret-${this.current.caret_style}`;

});

// 4. Audio Engine

if (window.soundEngine) {

    window.soundEngine.enabled = this.current.sound_enabled;

    window.soundEngine.theme = this.current.sound_theme;

    window.soundEngine.setVolume(this.current.sound_volume);

}

// 5. On-screen Keyboard Toggle

const kb = document.getElementById('keyboard-container');

if (kb) {

    kb.style.display = this.current.show_keyboard ? 'flex' : 'none';

}

// Sync Engine Defaults if test page active

if (window.typingEngine) {

    window.typingEngine.durationLimit = this.current.default_duration;

    window.typingEngine.contentType = this.current.default_content;

    window.typingEngine.level = this.current.default_level;

    window.typingEngine.blindModeActive = this.current.blind_mode;

    window.typingEngine.ghostEnabled = this.current.ghost_mode;

    window.typingEngine.updateControlsUI();

}

}

```



```

update(key, value) {

  this.current[key] = value;

  this.applyAll();

}

const authElement = document.getElementById('user-auth-meta');

const isAuthenticated = authElement ? authElement.dataset.authenticated === 'true' : false;

}

if (isAuthenticated) {

  localStorage.setItem('typesphere_user_profile_prefs', JSON.stringify(this.current));

  // Save to database

  fetch('/settings/api/save', {

    method: 'POST',

    headers: { 'Content-Type': 'application/json' },

    body: JSON.stringify(this.current)

  }).catch(() => {});

} else {

  sessionStorage.setItem('typesphere_anon_session_prefs', JSON.stringify(this.current));

}

}

resetToDefaults() {

  this.current = Object.assign({}, this.defaultSettings);

  sessionStorage.removeItem('typesphere_anon_session_prefs');

  localStorage.removeItem('typesphere_user_profile_prefs');

  this.applyAll();

}

// Call server to reset database settings if signed in

fetch('/settings/api/reset-defaults', { method: 'POST' }).catch(() => {});

}

alert('Settings successfully restored to system defaults!');

window.location.reload();

}

}

```

```

window.settingsManager = new SettingsManager();

document.addEventListener('DOMContentLoaded', () => {

  window.settingsManager.initSettings();

});

=== FILE: \app\static\js\tooltips.js ===

/**

 * TypeSphere Real-Time Predictive Tooltip Engine

 * Dynamically computes screen bounds and displays actionable button telemetry on hover.

 */

class TooltipEngine {

  constructor() {

    this.tooltipEl = null;

    this.activeTarget = null;

    this.initDOM();

    this.bindEvents();

  }

  initDOM() {

    let el = document.getElementById('typesphere-tooltip');

    if (!el) {

      el = document.createElement('div');

      el.id = 'typesphere-tooltip';

      document.body.appendChild(el);

    }

    this.tooltipEl = el;

  }

  bindEvents() {

    // Event delegation on mouseover

    document.addEventListener('mouseover', (e) => {

      const target = e.target.closest('[data-tooltip]');

```

```
if (target) {

  this.showTooltip(target);

}

}, true);

document.addEventListener('mouseout', (e) => {

  const target = e.target.closest('[data-tooltip]');

  if (target && target === this.activeTarget) {

    this.hideTooltip();

  }

}, true);

document.addEventListener('click', () => {

  this.hideTooltip();

});

window.addEventListener('scroll', () => this.hideTooltip(), { passive: true });

}

showTooltip(target) {

  this.activeTarget = target;

  const rawContent = target.getAttribute('data-tooltip');

  if (!rawContent) return;

  let title = target.getAttribute('data-tooltip-title') || 'ACTION OUTCOME';

  let body = rawContent;

  this.tooltipEl.innerHTML = `

<div class="tooltip-header">

  <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5">

    <circle cx="12" cy="12" r="10"></circle>

    <line x1="12" y1="16" x2="12" y2="12"></line>

    <line x1="12" y1="8" x2="12.01" y2="8"></line>

  </svg>

  <div>

    <div class="tooltip-body">${body}</div>

  </div>

  `;

  const rect = target.getBoundingClientRect();

  const tooltipRect = this.tooltipEl.getBoundingClientRect();

  // Position above by default; flip below if near top of screen

  let top = rect.top - 58;

  let left = rect.left + (rect.width / 2) - 80;

  if (top < 10) {

    top = rect.bottom + 10;

  }

  if (left < 10) {

    left = 10;

  }

  if (left + 280 > window.innerWidth) {

    left = window.innerWidth - 290;

  }

  this.tooltipEl.style.top = `${top}px`;

  this.tooltipEl.style.left = `${left}px`;

  this.tooltipEl.classList.add('visible');

}

hideTooltip() {

  this.activeTarget = null;

  if (this.tooltipEl) {

    this.tooltipEl.classList.remove('visible');

  }

}
```

```
}

}

window.tooltipEngine = new TooltipEngine();

=== FILE: \app\templates\base.html ===

<!DOCTYPE html>

<html lang="en" data-theme="dark">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">

  <title>{% block title %}TypeSphere | Next-Generation Typing Intelligence & Biometrics{% endblock %}</title>

  <meta name="description" content="TypeSphere is a next-generation typing ecosystem featuring real-world content, touch typing curriculum, 1v1 multiplayer duels, and biometric Typing DNA.">

  <meta name="keywords" content="typing test, WPM speed test, typing DNA, learn typing, typing games, multiplayer typing">

  <!-- Fonts -->

  <link rel="preconnect" href="https://fonts.googleapis.com">

  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

  <link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;600;700&family=Inter:wght@400;500;600;700;800&family=JetBrains+Mono:wght@400;600;700;800&display=swap" rel="stylesheet">

  <link rel="stylesheet" href="{{ url_for('static', filename='css/main.css') }}">

  <link rel="stylesheet" href="{{ url_for('static', filename='css/typing.css') }}">

  <link rel="stylesheet" href="{{ url_for('static', filename='css/keyboard.css') }}">

  <link rel="stylesheet" href="{{ url_for('static', filename='css/components.css') }}">

</head>

<body>

  <div id="user-auth-meta" data-authenticated="{{ 'true' if current_user.is_authenticated else 'false' }}" style="display:none;"></div>

  <!-- Top Navigation -->

  <nav class="navbar">
```

```
<div class="nav-header-row">

  <a href="{{ url_for('typing.test_page') }}" class="brand">

    <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2">

      <circle cx="12" cy="12" r="10"></circle>

      <path d="M12 6v6l4 2"></path>

    </svg>

    TypeSphere

  </a>

  <button class="mobile-nav-toggle" onclick="toggleMobileNav()" aria-label="Toggle navigation">

    ≡

  </button>

</div>

<!-- Direct Navigation Links -->

<ul class="nav-links" id="main-nav-links">

  <li><a href="{{ url_for('typing.test_page') }}" class="nav-link">Test</a></li>

  <li><a href="{{ url_for('learn.index') }}" class="nav-link" style="color:var(--accent); font-weight:700;">Learn</a></li>

  <li><a href="{{ url_for('games.index') }}" class="nav-link" style="color:var(--warning); font-weight:700;">Games</a></li>

  <li><a href="{{ url_for('typing.practice_page') }}" class="nav-link">Train Weakness</a></li>

  <li><a href="{{ url_for('challenges.index') }}" class="nav-link">Challenges</a></li>

  <li><a href="{{ url_for('leaderboard.index') }}" class="nav-link">Leaderboard</a></li>

  <li><a href="{{ url_for('multiplayer.index') }}" class="nav-link">Multiplayer</a></li>

  <li><a href="{{ url_for('feedback.ratings') }}" class="nav-link">Reviews</a></li>

  <li><a href="{{ url_for('settings.index') }}" class="nav-link">Settings</a></li>

  {% if current_user.is_authenticated %}

    <li><a href="{{ url_for('dashboard.index') }}" class="nav-link">Dashboard</a></li>

    {% if current_user.is_admin %}

      <li><a href="{{ url_for('admin.index') }}" class="nav-link" style="color: var(--warning); font-weight: 700;">Admin</a></li>

    {% endif %}

  <li><a href="{{ url_for('auth.logout') }}" class="btn btn-sm">Logout {{ current_user.username }}</a></li>

  {% else %}
```

```
<li><a href="{{ url_for('auth.login') }}" class="nav-link">Sign In</a></li>

<li><a href="{{ url_for('auth.register') }}" class="btn btn-primary btn-sm">Join Free</a></li>

{% endif %}

</ul>

</nav>
```

```
<!-- Notification Flash Area -->

<div class="container">

    {% with messages = get_flashed_messages(with_categories=true) %}

    {% if messages %}

        <div id="flash-alerts-wrapper">

            {% for category, msg in messages %}

                <div class="alert-box alert-{{ category }}">

                    <span>{{ msg }}</span>

                    <button onclick="dismissAlert(this)" class="alert-close-btn">&times;</button>

                </div>

            {% endfor %}

        </div>

    {% endif %}

    {% endwith %}

    {% block content %}{% endblock %}

</div>
```

```
<!-- Professional Footer -->

<footer class="app-footer">

    <div class="footer-container">

        <div class="footer-grid">

            <div class="footer-brand-col">

                <div class="footer-brand-title">

                    <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">

                        <circle cx="12" cy="12" r="10"></circle>

                        <path d="M12 6v6l4 2"></path>
```

```
</svg>

TypeSphere

</div>

<p class="footer-tagline">

    Next-Generation Typing Ecosystem & Biometric Telemetry Engine.

</p>

<div class="footer-status-pill">

    <span class="status-dot"></span>

    <span>All Systems Operational &bull; Production v2.3</span>

</div>

</div>

<div class="footer-links-col">

    <h4>Ecosystem</h4>

    <ul>

        <li><a href="{{ url_for('typing.test_page') }}">Speed Benchmark</a></li>

        <li><a href="{{ url_for('learn.index') }}">Touch Typing Academy</a></li>

        <li><a href="{{ url_for('games.index') }}">Typing Arcade Games</a></li>

        <li><a href="{{ url_for('typing.practice_page') }}">AI Weakness Drills</a></li>

        <li><a href="{{ url_for('multiplayer.index') }}">Multiplayer Grid</a></li>

    </ul>

</div>

<div class="footer-links-col">

    <h4>Community & Support</h4>

    <ul>

        <li><a href="{{ url_for('feedback.ratings') }}">Ratings & Reviews</a></li>

        <li><a href="{{ url_for('feedback.submit_feedback') }}">Submit Feedback</a></li>

        <li><a href="{{ url_for('feedback.contact') }}">Contact Engineers</a></li>

        <li><a href="{{ url_for('settings.index') }}">Ergonomics & Themes</a></li>

    </ul>

</div>

</div>
```

```
<div class="footer-bottom-bar">

<div>&copy; 2026 TypeSphere. All rights reserved. Registered Trademark &reg;.</div>

<div class="footer-meta-links">

<span>Non-Medical Biometrics</span>

<span>&bull;</span></span>

<span>Anti-Cheat Certified</span>

</div>

</div>

</div>

</footer>
```

```
<script>

function toggleMobileNav() {

  const nav = document.getElementById('main-nav-links');

  nav.classList.toggle('mobile-open');

}
```

```
function dismissAlert(btn) {

  const alertBox = btn.closest('.alert-box');

  if (alertBox) {

    alertBox.classList.add('fade-out');

    setTimeout(() => alertBox.remove(), 350);

  }

}
```

```
document.addEventListener('DOMContentLoaded', () => {

  const alerts = document.querySelectorAll('.alert-box');

  alerts.forEach(alertBox => {

    setTimeout(() => {

      alertBox.classList.add('fade-out');

      setTimeout(() => alertBox.remove(), 350);

    }, 3500);
```

```
});
```

```
});
```

```
</script>
```

```
<script src="{{ url_for('static', filename='js/audio.js') }}"></script>
```

```
<script src="{{ url_for('static', filename='js/settings.js') }}"></script>
```

```
<script src="{{ url_for('static', filename='js/charts.js') }}"></script>
```

```
{% block scripts %}{% endblock %}
```

```
</body>
```

```
</html>
```

```
=== FILE: .\app\templates\admin\analytics.html ===
```

```
{% extends 'admin/base_admin.html' %}
```

```
{% block title %}Traffic Telemetry | TypeSphere Control Center{% endblock %}
```

```
{% block admin_content %}
```

```
<div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1.5rem;">
```

```
<div>
```

```
<h1 style="font-size:1.6rem; font-weight:800;">Traffic & Visitor Intelligence</h1>
```

```
<p style="color:var(--admin-text-muted); font-size:0.85rem;">Aggregated session telemetry over the last {{ days }} days.</p>
```

```
</div>
```

```
<div style="display:flex; gap:0.4rem;">
```

```
<a href="{{ url_for('admin.traffic_analytics', days=7) }}" class="btn btn-sm">7 Days</a>
```

```
<a href="{{ url_for('admin.traffic_analytics', days=30) }}" class="btn btn-sm">30 Days</a>
```

```
<a href="{{ url_for('admin.traffic_analytics', days=90) }}" class="btn btn-sm">90 Days</a>
```

```
</div>
```

```
</div>
```

```
<div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(200px, 1fr)); gap:1rem; margin-bottom:1.5rem;">
```

```
<div class="card" style="margin:0;">
```

```
<div style="font-size:0.75rem; color:var(--admin-text-muted);">TOTAL PAGE VIEWS</div>
```

```
<div style="font-size:1.8rem; font-weight:800; color:var(--admin-accent);">{{ total_views }}</div>
```

```
</div>
```

```
<div class="card" style="margin:0;">

  <div style="font-size:0.75rem; color:var(--admin-text-muted);">UNIQUE SESSIONS</div>

  <div style="font-size:1.8rem; font-weight:800; color:var(--admin-success);">{{ unique_sessions }}</div>

</div>

<div class="card" style="margin:0;">

  <div style="font-size:0.75rem; color:var(--admin-text-muted);">AUTHENTICATED VIEWS</div>

  <div style="font-size:1.8rem; font-weight:800; color:var(--admin-success);">{{ registered_views }}</div>

</div>

<div class="card" style="margin:0;">

  <div style="font-size:0.75rem; color:var(--admin-text-muted);">GUEST / ANONYMOUS</div>

  <div style="font-size:1.8rem; font-weight:800; color:var(--admin-warning);">{{ guest_views }}</div>

</div>

</div>
```

```
<div style="display:grid; grid-template-columns: 2fr 1fr; gap:1.5rem;">

  <div class="card" style="margin:0;">

    <h3 style="font-size:1.05rem; margin-bottom:1rem;">Top Entry & Route Volumes</h3>

    <table>

      <thead>

        <tr>

          <th>Endpoint Path</th>

          <th>Views</th>

        </tr>

      </thead>

      <tbody>

        {% for path, count in popular__pages %}

          <tr>

            <td><code style="font-family:var(--admin-mono); color:var(--admin-accent);">{{ path }}</code></td>

            <td><strong>{{ count }}</strong></td>

          </tr>

        {% endfor %}

      </tbody>

    </table>
```

```
</div>

<div class="card" style="margin:0;">

  <h3 style="font-size:1.05rem; margin-bottom:1rem;">Device & Client Mix</h3>

  <div style="margin-bottom:1.5rem;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.5rem;">PLATFORM TYPE</div>

    {% for dev, count in devices %}

      <div style="display:flex; justify-content:space-between; margin-bottom:0.3rem;">

        <span>{{ dev|capitalize }}</span>

        <strong>{{ count }}</strong>

      </div>

    {% endfor %}

  </div>
```

```
<div>

  <div style="font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.5rem;">BROWSER ENGINE</div>

  {% for b, count in browsers %}

    <div style="display:flex; justify-content:space-between; margin-bottom:0.3rem;">

      <span>{{ b }}</span>

      <strong>{{ count }}</strong>

    </div>

  {% endfor %}

</div>

</div>

</div>

{% endblock %}
```

```
=== FILE: .\app\templates\admin\arcade_manage.html ===

{% extends 'base.html' %}

{% block robots %}noindex, nofollow{% endblock %}

{% block content %}

<div class="card">
```

```
<div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.5rem;">

<div>

  <h1 style="font-size:1.8rem; font-weight:800; margin:0;">🎮 Arcade Content & Wordbank Manager</h1>

  <p style="color:var(--text-muted); font-size:0.85rem; margin-top:0.25rem;">Add target words, symbols, and difficulty tags across all 12 games.</p>

</div>

<a href="{{ url_for('admin.index') }}" class="btn">&larr; Admin Dashboard</a>

</div>
```

```
<!-- Word Injection Form -->

<form method="POST" action="{{ url_for('admin.add_arcade_word') }}" style="background:var(--bg-card); padding:1.25rem; border-radius:var(--radius); margin-bottom:2rem; border:1px solid var(--border-color);">

  <h3 style="font-size:1.1rem; margin-bottom:1rem;">Add New Game Target Word</h3>

  <div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(200px, 1fr)); gap:1rem; margin-bottom:1rem;">

    <div>

      <label style="display:block; font-size:0.75rem; text-transform:uppercase; color:var(--text-muted); font-weight:700; margin-bottom:0.35rem;">Target Game</label>

      <select name="game_mode" class="form-select" style="width:100%;">

        <option value="falling_words">Falling Words</option>

        <option value="speed_racer">Speed Racer</option>

        <option value="space_defender">Space Defender</option>

        <option value="typing_ninja">Typing Ninja</option>

        <option value="zombie_duel">Zombie Duel</option>

        <option value="cipher_hacker">Cipher Hacker</option>

      </select>

    </div>

    <div>

      <label style="display:block; font-size:0.75rem; text-transform:uppercase; color:var(--text-muted); font-weight:700; margin-bottom:0.35rem;">Difficulty</label>

      <select name="difficulty" class="form-select" style="width:100%;">

        <option value="beginner">Beginner</option>

        <option value="intermediate" selected>Intermediate</option>

        <option value="advanced">Advanced</option>

      </select>

    </div>

  </div>
```

```
<option value="expert">Expert</option>

</select>

</div>

<div>

  <label style="display:block; font-size:0.75rem; text-transform:uppercase; color:var(--text-muted); font-weight:700; margin-bottom:0.35rem;">Target Word / String</label>

  <input type="text" name="target_text" required placeholder="e.g. quantum" style="width:100%; padding:0.6rem; background:var(--bg-secondary); border:1px solid var(--border-color); border-radius:4px; color:var(--text-primary); font-family:var(--font-mono);">

</div>

</div>

<button type="submit" class="btn btn-primary btn-sm">Add to Game Library</button>

</form>
```

```
<!-- Items Table -->

<div class="table-responsive-container">

  <table>

    <thead>

      <tr>

        <th>ID</th>

        <th>Game</th>

        <th>Difficulty</th>

        <th>Target Text</th>

        <th>Length</th>

        <th>Action</th>

      </tr>

    </thead>

    <tbody>

      {% for item in items %}

        <tr>

          <td>#{{ item.id }}</td>

          <td><strong>{{ item.game_mode|capitalize }}</strong></td>

          <td><span style="font-size:0.75rem; padding:0.2rem 0.5rem; border-radius:4px; background:var(--bg-card);">{{ item.difficulty|upper }}</span></td>

        </tr>

      </tbody>

    </table>

  </div>
```

```
<td style="font-family:var(--font-mono); font-weight:700; color:var(--accent);">{{ item.target_text }}</td>

<td>{{ item.word_length }}</td>

<td>

  <form method="POST" action="{{ url_for('admin.delete_arcade_word', item_id=item.id) }}">

    <button type="submit" class="btn btn-danger btn-sm">Delete</button>

  </form>

</td>

</tr>

{% else %}

  <tr><td colspan="6" style="text-align:center; padding:2rem; color:var(--text-muted);">No custom words added yet. Engine using curated
  fallbacks.</td></tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

{% endblock %}
```

```
=== FILE: .\app\templates\admin\audit_logs.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}Audit Trail | TypeSphere Control Center{% endblock %}
```

```
{% block admin_content %}

<div style="margin-bottom:1.5rem;">

  <h1 style="font-size:1.6rem; font-weight:800;">Immutable Admin Audit Log</h1>

  <p style="color:var(--admin-text-muted); font-size:0.85rem;">Permanent historical ledger of all administrative modifications.</p>

</div>
```

```
<div class="card" style="padding:0; overflow:hidden;">

  <table>

    <thead>

      <tr>

        <th>Admin</th>
```

```
<th>Action</th>

<th>Entity Type</th>

<th>Target ID</th>

<th>Details</th>

<th>Origin IP</th>

<th>Timestamp</th>

</tr>

</thead>

<tbody>

  {% for l in logs.items %}

    <tr>

      <td><strong>{{ l.admin_username }}</strong></td>

      <td><span class="badge badge-primary">{{ l.action }}</span></td>

      <td>{{ l.target_type }}</td>

      <td>#{{ l.target_id or '.' }}</td>

      <td style="font-size:0.8rem; color:var(--admin-text-muted);">{{ l.details or 'No metadata' }}</td>

      <td style="font-family:var(--admin-mono); font-size:0.8rem;">{{ l.ip_address or 'System' }}</td>

      <td style="font-size:0.8rem;">{{ l.created_at.strftime('%Y-%m-%d %H:%M:%S') }}</td>

    </tr>

    {% else %}

      <tr><td colspan="7" style="text-align:center; padding:2rem; color:var(--admin-text-muted);">No records registered.</td></tr>

    {% endfor %}

  </tbody>

</table>

</div>

{% endblock %}
```

```
=== FILE: .\app\templates\admin\base_admin.html ===

<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```



```
<title{% block title %}TypeSphere Control Center{% endblock %}</title>

<link rel="preconnect" href="https://fonts.googleapis.com">

<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

<link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&family=JetBrains+Mono:wght@400;600;700&display=swap" rel="stylesheet">

<style>

:root {

  --admin-bg: #090d16;

  --admin-surface: #111726;

  --admin-elevated: #1a2236;

  --admin-border: #232d44;

  --admin-accent: #3b82f6;

  --admin-accent-glow: rgba(59, 130, 246, 0.2);

  --admin-success: #10b981;

  --admin-warning: #f59e0b;

  --admin-danger: #ef4444;

  --admin-text-main: #f8fafc;

  --admin-text-muted: #94a3b8;

  --admin-font: 'Plus Jakarta Sans', sans-serif;

  --admin-mono: 'JetBrains Mono', monospace;

}

* { box-sizing: border-box; margin: 0; padding: 0; }

body {

  background: var(--admin-bg);

  color: var(--admin-text-main);

  font-family: var(--admin-font);

  display: flex;

  min-height: 100vh;

}

/* Dedicated Sidebar */

.admin-sidebar {

  width: 260px;
```

```
background: var(--admin-surface);

border-right: 1px solid var(--admin-border);

display: flex;

flex-direction: column;

flex-shrink: 0;

position: sticky;

top: 0;

height: 100vh;

overflow-y: auto;

}

.sidebar-brand {

  padding: 1.5rem;

  font-size: 1.25rem;

  font-weight: 800;

  color: var(--admin-accent);

  border-bottom: 1px solid var(--admin-border);

  display: flex;

  align-items: center;

  gap: 0.5rem;

}

.sidebar-section-title {

  font-size: 0.68rem;

  text-transform: uppercase;

  letter-spacing: 0.1em;

  color: var(--admin-text-muted);

  padding: 1.25rem 1.5rem 0.4rem;

  font-weight: 700;

}

.sidebar-nav { list-style: none; }

.sidebar-link {

  display: flex;

  align-items: center;

  gap: 0.75rem;
```

```
padding: 0.65rem 1.5rem;

color: var(--admin-text-muted);

text-decoration: none;

font-size: 0.88rem;

font-weight: 600;

transition: all 0.15s;
}

.sidebar-link:hover, .sidebar-link.active {

color: var(--admin-text-main);

background: var(--admin-elevated);

border-left: 3px solid var(--admin-accent);
}

/* Main Content */

.admin-main {

flex: 1;

display: flex;

flex-direction: column;

min-width: 0;
}

.admin-topbar {

height: 65px;

background: var(--admin-surface);

border-bottom: 1px solid var(--admin-border);

display: flex;

justify-content: space-between;

align-items: center;

padding: 0 2rem;

position: sticky;

top: 0;

z-index: 100;
}

.admin-content {

padding: 2rem;
```

```
flex: 1;
}

.card {

background: var(--admin-surface);

border: 1px solid var(--admin-border);

border-radius: 8px;

padding: 1.5rem;

margin-bottom: 1.5rem;
}

.btn {

padding: 0.55rem 1rem;

border-radius: 6px;

font-weight: 600;

font-size: 0.85rem;

cursor: pointer;

text-decoration: none;

display: inline-flex;

align-items: center;

gap: 0.4rem;

border: 1px solid var(--admin-border);

background: var(--admin-elevated);

color: var(--admin-text-main);
}

.btn-primary { background: var(--admin-accent); border-color: var(--admin-accent); }

.btn-danger { background: var(--admin-danger); border-color: var(--admin-danger); color: #fff; }

.btn-sm { padding: 0.35rem 0.65rem; font-size: 0.78rem; }

.badge {

display: inline-block;

padding: 0.2rem 0.5rem;

border-radius: 4px;

font-size: 0.72rem;

font-weight: 700;

text-transform: uppercase;
```

```
}

.badge-primary { background: var(--admin-accent-glow); color: var(--admin-accent); }

.badge-success { background: rgba(16, 185, 129, 0.15); color: var(--admin-success); }

.badge-danger { background: rgba(239, 68, 68, 0.15); color: var(--admin-danger); }

.badge-warning { background: rgba(245, 158, 11, 0.15); color: var(--admin-warning); }

table { width: 100%; border-collapse: collapse; font-size: 0.88rem; }

th { text-align: left; padding: 0.75rem 1rem; color: var(--admin-text-muted); font-size: 0.75rem; text-transform: uppercase; border-bottom: 1px solid var(--admin-border); }

td { padding: 0.85rem 1rem; border-bottom: 1px solid var(--admin-border); }

tr:hover td { background: rgba(255, 255, 255, 0.02); }

.impersonation-bar {

  background: #7c2d12;

  color: #fed7aa;

  padding: 0.5rem 1.5rem;

  font-size: 0.85rem;

  display: flex;

  justify-content: space-between;

  align-items: center;

}

</style>

</head>

<body>

<!-- Impersonation Notice Bar -->

{% if session.get('admin_impersonator_id') %}

<div class="impersonation-bar" style="position:fixed; top:0; left:0; right:0; z-index:9999;">

<span><span><span> You are simulating user account: <strong>{{ current_user.username }}</strong></span>

<a href="{{ url_for('admin.stop_impersonation') }}" style="color:#fff; text-decoration:underline; font-weight:700;">Exit Simulation</a>

</div>

{% endif %}

<!-- Administrator Dedicated Sidebar -->

<aside class="admin-sidebar">

<div class="sidebar-brand">
```

```
<svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"><circle cx="12" cy="12" r="10"/><path
d="M12 6v6l4 2"/></svg>

TypeSphere Admin

</div>

<div class="sidebar-section-title">Overview</div>

<ul class="sidebar-nav">

<li><a href="{{ url_for('admin.dashboard') }}" class="sidebar-link"><span> Dashboard</span></a></li>

<li><a href="{{ url_for('admin.traffic_analytics') }}" class="sidebar-link"><span> Traffic & Analytics</span></a></li>

</ul>

<div class="sidebar-section-title">Directory & Content</div>

<ul class="sidebar-nav">

<li><a href="{{ url_for('admin.users_list') }}" class="sidebar-link"><span> User Accounts</span></a></li>

<li><a href="{{ url_for('admin.passages_list') }}" class="sidebar-link"><span> Passage Library</span></a></li>

<li><a href="{{ url_for('admin.leaderboard_manage') }}" class="sidebar-link"><span> Leaderboard Verifier</span></a></li>

</ul>

<div class="sidebar-section-title">Auditing & Security</div>

<ul class="sidebar-nav">

<li><a href="{{ url_for('admin.security_center') }}" class="sidebar-link"><span> Security Events</span></a></li>

<li><a href="{{ url_for('admin.audit_logs') }}" class="sidebar-link"><span> Action Audit Trail</span></a></li>

</ul>

<div class="sidebar-section-title">Control</div>

<ul class="sidebar-nav">

<li><a href="{{ url_for('admin.system_settings') }}" class="sidebar-link"><span> System Parameters</span></a></li>

</ul>

<div style="margin-top:auto; padding:1.5rem; border-top:1px solid var(--admin-border);">

<a href="{{ url_for('typing.test_page') }}" class="btn" style="width:100%; justify-content:center;"><span> View Public Website</span></a>

</div>

</aside>
```

```
<!-- Content Workspace -->

<div class="admin-main">

  <header class="admin-topbar">

    <div>

      <span class="badge badge-primary">{{ current_user.role | upper }}</span>

      <span style="font-size:0.9rem; margin-left:0.5rem; font-weight:600;">{{ current_user.username }}</span>

    </div>

    <div style="display:flex; gap:0.5rem;">

      <a href="{{ url_for('admin.export_csv', data_type='users') }}" class="btn btn-sm">Export Users CSV</a>

      <a href="{{ url_for('admin.export_csv', data_type='tests') }}" class="btn btn-sm">Export Tests CSV</a>

      <a href="{{ url_for('auth.logout') }}" class="btn btn-sm btn-danger">Sign Out</a>

    </div>

  </header>

  <div class="admin-content">

    {% with messages = get_flashed_messages(with_categories=true) %}

      {% if messages %}

        {% for category, msg in messages %}

          <div style="padding:0.75rem 1rem; border-radius:6px; margin-bottom:1.5rem; background:var(--admin-surface); border-left:4px solid var(--admin-accent);">

            {{ msg }}

          </div>

        {% endfor %}

      {% endif %}

    {% endwith %}

    {% block admin_content %}{% endblock %}

  </div>

</div>

</body>

</html>
```

=== FILE: .\app\templates\admin\content.html ===

{% extends 'base.html' %}

{% block robots %}noindex, nofollow{% endblock %}

{% block content %}

<div class="card">

<div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 1.5rem; flex-wrap: wrap; gap: 1rem;">

<div>

<h1 style="font-size: 1.8rem; font-weight: 800; margin: 0;">Typing Content & Difficulty CRUD</h1>

<p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.25rem;">

Add, inspect, and delete text prompts for Easy, Moderate, Hard, and Expert difficulty levels.

</p>

</div>

← Back to Admin Overview

</div>

<!-- Create New Prompt Box -->

<form method="POST" style="background: var(--bg-card); padding: 1.5rem; border-radius: var(--radius); margin-bottom: 2rem; border: 1px solid var(--border-color);">

<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Add New Content Prompt</h3>

<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; margin-bottom: 1rem;">

<div>

<label style="display: block; font-size: 0.85rem; font-weight: 700; margin-bottom: 0.35rem;">Category</label>

<input type="text" name="category" placeholder="Literature, Technology, Science, General" required style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary);">

</div>

<div>

<label style="display: block; font-size: 0.85rem; font-weight: 700; margin-bottom: 0.35rem;">Difficulty Tier</label>

<select name="difficulty" style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary);">

```
<option value="easy">Easy (Simple & Short Words)</option>

<option value="moderate" selected>Moderate (Standard English)</option>

<option value="hard">Hard (Advanced Lexicon & Syllables)</option>

<option value="expert">Expert (Sophisticated & Mixed Symbols)</option>

</select>

</div>

</div>
```

```
<div style="margin-bottom: 1rem;">

  <label style="display: block; font-size: 0.85rem; font-weight: 700; margin-bottom: 0.35rem;">Passage Text</label>

  <textarea name="content" rows="4" required style="width: 100%; padding: 0.75rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary); font-family: var(--font-mono);"></textarea>

</div>
```

```
<div style="display: flex; align-items: center; gap: 0.5rem; margin-bottom: 1.25rem;">

  <input type="checkbox" name="is_code" id="is_code">

  <label for="is_code" style="font-size: 0.85rem;">This content is programming source code</label>

</div>
```

```
<button type="submit" class="btn btn-primary">Save Content Prompt</button>

</form>
```

```
<!-- Content Table -->

<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Configured Database Prompts {{{ texts|length }}}</h3>
```

```
<div class="table-responsive-container">
```

```
<table>

  <thead>

    <tr>

      <th>ID</th>

      <th>Category</th>

      <th>Difficulty</th>

      <th>Content Preview</th>

      <th>Code?</th>
```

```
<th>Action</th>

</tr>

</thead>

<tbody>

  {% for t in texts %}

    <tr>

      <td>#{{ t.id }}</td>

      <td><strong>{{ t.category }}</strong></td>

      <td><span class="btn btn-sm" style="padding: 0.15rem 0.45rem; font-size: 0.75rem;">{{ t.difficulty|capitalize }}</span></td>

      <td style="max-width: 380px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; font-family: var(--font-mono); font-size: 0.85rem;">{{ t.content }}</td>

      <td>{{ 'Yes (' + t.code_lang + ')' if t.is_code else 'No' }}</td>

      <td>

        <form method="POST" action="{{ url_for('admin.delete_content', text_id=t.id) }}" onsubmit="return confirm('Delete this prompt permanently?');">

          <button type="submit" class="btn btn-danger btn-sm">Delete</button>

        </form>

      </td>

    </tr>

  {% endfor %}

</tbody>

</table>

</div>

</div>

{% endblock %}
```

```
=== FILE: .\app\templates\admin\dashboard.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}Dashboard | TypeSphere Admin Center{% endblock %}

{% block admin_content %}

<div style="margin-bottom: 1.75rem;">
```

```
<h1 style="font-size:1.6rem; font-weight:800; margin-bottom:0.25rem;">Executive Control Dashboard</h1>

<p style="color:var(--admin-text-muted); font-size:0.85rem;">Platform telemetry and system activity overview.</p>

</div>

<!-- Primary Stats Metric Grid -->

<div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(200px, 1fr)); gap:1rem; margin-bottom:1.75rem;">

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Total Registered Users</div>

    <div style="font-size:1.8rem; font-weight:800; color:var(--admin-accent); margin-top:0.2rem;">{{ stats.total_users }}</div>

    <div style="font-size:0.75rem; color:var(--admin-success); margin-top:0.3rem;">+{{ stats.new_today }} today &bull; +{{ stats.new_week }} this week</div>

  </div>

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Active Now (15 min)</div>

    <div style="font-size:1.8rem; font-weight:800; color:var(--admin-success); margin-top:0.2rem;">{{ stats.active_now }}</div>

    <div style="font-size:0.75rem; color:var(--admin-text-muted); margin-top:0.3rem;">{{ stats.suspended_users }} suspended/banned</div>

  </div>

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Total Tests Completed</div>

    <div style="font-size:1.8rem; font-weight:800; color:var(--admin-accent); margin-top:0.2rem;">{{ stats.total_tests }}</div>

    <div style="font-size:0.75rem; color:var(--admin-warning); margin-top:0.3rem;">{{ stats.suspicious_tests }} flagged for review</div>

  </div>

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Average Net Velocity</div>

    <div style="font-size:1.8rem; font-weight:800; margin-top:0.2rem;">{{ stats.avg_wpm }} <span style="font-size:0.9rem; color:var(--admin-text-muted);">WPM</span></div>

    <div style="font-size:0.75rem; color:var(--admin-text-muted); margin-top:0.3rem;">Max recorded: {{ stats.max_wpm }} WPM</div>

  </div>

</div>
```

```
<div style="display:grid; grid-template-columns: 2fr 1fr; gap:1.5rem; margin-bottom:1.5rem;">

  <!-- Recent Audit Log Trail -->

  <div class="card" style="margin:0;">
```

```
<div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1rem;">

  <h3 style="font-size:1.05rem; font-weight:700;">Recent Administrative Actions</h3>

  <a href="{{ url_for('admin.audit_logs') }}" style="color:var(--admin-accent); font-size:0.8rem; text-decoration:none;">View All &rarr;</a>

</div>

<table>

  <thead>

    <tr>

      <th>Admin</th>

      <th>Action</th>

      <th>Target</th>

      <th>Time</th>

    </tr>

  </thead>

  <tbody>

    {% for a in recent_audits %}

    <tr>

      <td><strong>{{ a.admin_username }}</strong></td>

      <td><span class="badge badge-primary">{{ a.action }}</span></td>

      <td>{{ a.target_type }} ({{ a.target_id or '-' }})</td>

      <td style="color:var(--admin-text-muted); font-size:0.8rem;">{{ a.created_at.strftime('%m-%d %H:%M') }}</td>

    </tr>

    {% else %}

    <tr><td colspan="4" style="text-align:center; color:var(--admin-text-muted);">No administrative events recorded.</td></tr>

    {% endfor %}

  </tbody>

</table>

</div>
```

```
<!-- Security Incidents Preview -->

<div class="card" style="margin:0;">

  <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1rem;">

    <h3 style="font-size:1.05rem; font-weight:700;">Security Watch</h3>

    <a href="{{ url_for('admin.security_center') }}" style="color:var(--admin-accent); font-size:0.8rem; text-decoration:none;">Details &rarr;</a>
```

```
</div>

<div style="display:flex; flex-direction:column; gap:0.75rem;">

  {% for s in recent_security %}

    <div style="padding:0.65rem; background:var(--admin-elevated); border-radius:6px; font-size:0.82rem;">

      <div style="display:flex; justify-content:space-between; margin-bottom:0.2rem;">

        <span class="badge badge-danger">{{ s.severity }}</span>

        <span style="color:var(--admin-text-muted);">{{ s.created_at.strftime('%H:%M') }}</span>

      </div>

      <div style="font-weight:600;">{{ s.event_type }}</div>

      <div style="color:var(--admin-text-muted); font-size:0.78rem;">{{ s.description[:60] }}...</div>

    </div>

  {% else %}

    <div style="text-align:center; color:var(--admin-text-muted); padding:1rem;">No security threats logged.</div>

  {% endfor %}

</div>

</div>

</div>

{% endblock %}
```

=== FILE: .\app\templates\admin\feedback_manage.html ===

{% extends 'base.html' %}

{% block robots %}noindex, nofollow{% endblock %}

{% block content %}

```
<div class="card">

  <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

    <div>

      <h1 style="font-size: 1.8rem; font-weight: 800; margin: 0;">Feedback, Messages & Announcements</h1>

      <p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.25rem;">Review inquiries and publish banners.</p>

    </div>

    <a href="{{ url_for('admin.index') }}" class="btn">&larr; Back to Admin Overview</a>
```

```
</div>

<!-- Create Global Announcement Banner -->

<div class="card" style="background: var(--bg-card); margin-bottom: 2rem;">

  <h3 style="font-size: 1.15rem; margin-bottom: 0.85rem; color: var(--accent);">Publish Global Platform Announcement</h3>

  <form method="POST" action="{{ url_for('admin.create_announcement') }}">

    <div style="margin-bottom: 1rem;">

      <input type="text" name="title" required placeholder="Announcement Title (e.g. Version 2.2 Released)..." style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary);">

    </div>

    <div style="margin-bottom: 1rem;">

      <textarea name="content" rows="2" required placeholder="Announcement details..." style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary);"></textarea>

    </div>

    <button type="submit" class="btn btn-primary btn-sm">Publish Active Announcement</button>

  </form>

</div>
```

<!-- Contact Inquiries -->

<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Contact Messages ({{ contact_msgs|length }})</h3>

<div class="table-responsive-container" style="margin-bottom: 2rem;">

```
<table>

  <thead>

    <tr>

      <th>From</th>

      <th>Email</th>

      <th>Subject</th>

      <th>Message</th>

      <th>Status</th>

      <th>Actions</th>

    </tr>

  </thead>

  <tbody>
```

```
{% for m in contact_msgs %}

<tr>

<td><strong>{{ m.name }}</strong></td>

<td>{{ m.email }}</td>

<td>{{ m.subject }}</td>

<td style="max-width: 320px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap;">{{ m.message }}</td>

<td><span style="font-weight: 700; color: ({{ var(--text-muted) if m.is_read else 'var(--warning)' }})>{{ 'Read' if m.is_read else 'New'
}}</span></td>

<td style="display: flex; gap: 0.35rem;">

<form method="POST" action="{{ url_for('admin.toggle_contact_read', msg_id=m.id) }}">

<button type="submit" class="btn btn-sm">Mark {{ 'Unread' if m.is_read else 'Read' }}</button>

</form>

<form method="POST" action="{{ url_for('admin.delete_contact', msg_id=m.id) }}">

<button type="submit" class="btn btn-danger btn-sm">Delete</button>

</form>

</td>

</tr>

{% else %}

<tr>

<td colspan="6" style="text-align: center; color: var(--text-muted); padding: 1.5rem;">No contact messages received.</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>
```

<!-- User Feedback Moderation -->

<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Submitted User Feedback ({{ feedback_items|length }})</h3>

<div class="table-responsive-container">

```
<table>

<thead>

<tr>

<th>User</th>
```

```
<th>Category</th>

<th>Title & Message</th>

<th>Status Update</th>

</tr>

</thead>

<tbody>

{% for fb in feedback_items %}

<tr>

<td>{{ fb.user.username }}</td>

<td>{{ fb.category }}</td>

<td>

<strong>{{ fb.title }}</strong>

<div style="font-size: 0.82rem; color: var(--text-secondary); margin-top: 0.2rem;">{{ fb.message }}</div>

</td>

<td>

<form method="POST" action="{{ url_for('admin.update_feedback_status', item_id=fb.id) }}" style="display: flex; gap: 0.4rem; align-items: center;">

<select name="status" style="padding: 0.35rem; background: var(--bg-card); border: 1px solid var(--border-color); color: var(--text-primary); border-radius: 4px;">

<option value="Pending" {% if fb.status == 'Pending' %}>selected{% endif %}>Pending</option>

<option value="Under Review" {% if fb.status == 'Under Review' %}>selected{% endif %}>Under Review</option>

<option value="Resolved" {% if fb.status == 'Resolved' %}>selected{% endif %}>Resolved</option>

<option value="Closed" {% if fb.status == 'Closed' %}>selected{% endif %}>Closed</option>

</select>

<button type="submit" class="btn btn-sm">Update</button>

</form>

</td>

</tr>

{% else %}

<tr>

<td colspan="4" style="text-align: center; color: var(--text-muted); padding: 1.5rem;">No user feedback submitted yet.</td>

</tr>

{% endfor %}
```



```
</tbody>

</table>

</div>

</div>

{% endblock %}

=== FILE: .\app\templates\admin\index.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}Dashboard | TypeSphere Admin Center{% endblock %}

{% block admin_content %}

<div style="margin-bottom:1.75rem;">

  <h1 style="font-size:1.6rem; font-weight:800; margin-bottom:0.25rem;">Executive Control Dashboard</h1>

  <p style="color:var(--admin-text-muted); font-size:0.85rem;">Platform telemetry and system activity overview.</p>

</div>

<!-- Primary Stats Metric Grid -->

<div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(200px, 1fr)); gap:1rem; margin-bottom:1.75rem;">

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Total Registered Users</div>

    <div style="font-size:1.8rem; font-weight:800; color:var(--admin-accent); margin-top:0.2rem;">{{ stats.total_users }}</div>

    <div style="font-size:0.75rem; color:var(--admin-success); margin-top:0.3rem;">+{{ stats.new_today }} today &bull; +{{ stats.new_week }} this week</div>

  </div>

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Active Now (15 min)</div>

    <div style="font-size:1.8rem; font-weight:800; color:var(--admin-success); margin-top:0.2rem;">{{ stats.active_now }}</div>

    <div style="font-size:0.75rem; color:var(--admin-text-muted); margin-top:0.3rem;">{{ stats.suspended_users }} suspended/banned</div>

  </div>

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Total Tests Completed</div>

    <div style="font-size:1.8rem; font-weight:800; color:var(--admin-accent); margin-top:0.2rem;">{{ stats.total_tests }}</div>

    <div style="font-size:0.75rem; color:var(--admin-warning); margin-top:0.3rem;">{{ stats.suspicious_tests }} flagged for review</div>

  </div>

</div>
```

```
</div>

<div class="card" style="margin:0;">

  <div style="font-size:0.75rem; color:var(--admin-text-muted); text-transform:uppercase; font-weight:700;">Average Net Velocity</div>

  <div style="font-size:1.8rem; font-weight:800; margin-top:0.2rem;">{{ stats.avg_wpm }} <span style="font-size:0.9rem; color:var(--admin-text-muted);">WPM</span></div>

  <div style="font-size:0.75rem; color:var(--admin-text-muted); margin-top:0.3rem;">Max recorded: {{ stats.max_wpm }} WPM</div>

</div>

</div>

<div style="display:grid; grid-template-columns: 2fr 1fr; gap:1.5rem; margin-bottom:1.5rem;">

  <!-- Recent Audit Log Trail -->

  <div class="card" style="margin:0;">

    <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1rem;">

      <h3 style="font-size:1.05rem; font-weight:700;">Recent Administrative Actions</h3>

      <a href="{{ url_for('admin.audit_logs') }}" style="color:var(--admin-accent); font-size:0.8rem; text-decoration:none;">View All &rrarr;</a>

    </div>

    <table>

      <thead>

        <tr>

          <th>Admin</th>

          <th>Action</th>

          <th>Target</th>

          <th>Time</th>

        </tr>

      </thead>

      <tbody>

        {% for a in recent_audits %}

          <tr>

            <td><strong>{{ a.admin_username }}</strong></td>

            <td><span class="badge badge-primary">{{ a.action }}</span></td>

            <td>{{ a.target_type }} ({{ a.target_id or '-' }})</td>

            <td style="color:var(--admin-text-muted); font-size:0.8rem;">{{ a.created_at.strftime('%m-%d %H:%M') }}</td>

          </tr>

        </tbody>

      </div>
```

```
{% else %}

<tr><td colspan="4" style="text-align:center; color:var(--admin-text-muted);">No administrative events recorded.</td></tr>

{% endfor %}

</tbody>

</table>

</div>

<!-- Security Incidents Preview -->

<div class="card" style="margin:0;">

<div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1rem;">

<h3 style="font-size:1.05rem; font-weight:700;">Security Watch</h3>

<a href="{{ url_for('admin.security_center') }}" style="color:var(--admin-accent); font-size:0.8rem; text-decoration:none;">Details &rarr;</a>

</div>

<div style="display:flex; flex-direction:column; gap:0.75rem;">

{% for s in recent_security %}

<div style="padding:0.65rem; background:var(--admin-elevated); border-radius:6px; font-size:0.82rem;">

<div style="display:flex; justify-content:space-between; margin-bottom:0.2rem;">

<span class="badge badge-danger">{{ s.severity }}</span>

<span style="color:var(--admin-text-muted);">{{ s.created_at.strftime('%H:%M') }}</span>

</div>

<div style="font-weight:600;">{{ s.event_type }}</div>

<div style="color:var(--admin-text-muted); font-size:0.78rem;">{{ s.description[:60] }}...</div>

</div>

{% else %}

<div style="text-align:center; color:var(--admin-text-muted); padding:1rem;">No security threats logged.</div>

{% endfor %}

</div>

</div>

</div>

{% endblock %}

=== FILE: .\app\templates\admin\leaderboard.html ===

{% extends 'admin/base_admin.html' %}
```

```
{% block title %}Leaderboard Moderation | Control Center{% endblock %}

{% block admin_content %}

<div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1.5rem;">

<div>

<h1 style="font-size:1.6rem; font-weight:800;">Leaderboard Audit & Score Verification</h1>

<p style="color:var(--admin-text-muted); font-size:0.85rem;">Enforce anti-cheat compliance and curate official rankings.</p>

</div>

<div style="display:flex; gap:0.4rem;">

<a href="{{ url_for('admin.leaderboard_manage', suspicious='all') }}" class="btn btn-sm">All Scores</a>

<a href="{{ url_for('admin.leaderboard_manage', suspicious='flagged') }}" class="btn btn-sm btn-danger">Flagged Only</a>

<a href="{{ url_for('admin.leaderboard_manage', suspicious='verified') }}" class="btn btn-sm">Verified Only</a>

</div>

</div>

<div class="card" style="padding:0; overflow:hidden;">

<table>

<thead>

<tr>

<th>Test ID</th>

<th>Pilot</th>

<th>WPM</th>

<th>Accuracy</th>

<th>Mode</th>

<th>Integrity Status</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{% for t in tests.items %}

<tr>

<td>#{{ t.id }}</td>

<td><strong>{{ t.user.username if t.user else 'Guest' }}</strong></td>
```

```
<td style="font-family:var(--admin-mono); font-weight:700; color:var(--admin-accent);">{{ t.wpm }}</td>

<td>{{ t.accuracy }}%</td>

<td>{{ t.mode }}</td>

<td>

{% if t.suspicious %}

<span class="badge badge-danger">FLAGGED SUSPICIOUS</span>

<div style="font-size:0.75rem; color:var(--admin-danger); margin-top:2px;">{{ t.suspicion_reason or 'Automated limit violation' }}</div>

{% else %}

<span class="badge badge-success">VERIFIED</span>

{% endif %}

</td>

<td>

<div style="display:flex; gap:0.4rem;">

<form method="POST" action="{{ url_for('admin.test_toggle_flag', test_id=t.id) }}">

<button type="submit" class="btn btn-sm">{{ 'Mark Verified' if t.suspicious else 'Flag Score' }}</button>

</form>

<form method="POST" action="{{ url_for('admin.test_delete', test_id=t.id) }}" onsubmit="return confirm('Erase this record completely?');">

<button type="submit" class="btn btn-sm btn-danger">Delete</button>

</form>

</div>

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

{% endblock %}
```

=== FILE: \app\templates\admin\passages.html ===

```
{% extends 'admin/base_admin.html' %}

{% block title %}Passage CMS | TypeSphere Control Center{% endblock %}
```

```
{% block admin_content %}
```

```
<div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1.5rem; flex-wrap:wrap; gap:1rem;">
```

```
<div>
```

```
<h1 style="font-size:1.6rem; font-weight:800;">Passage Content Management</h1>
```

```
<p style="color:var(--admin-text-muted); font-size:0.85rem;">Administer the text library and verified test corpus.</p>
```

```
</div>
```

```
<a href="{{ url_for('admin.passage_create') }}" class="btn btn-primary">+ Create New Passage</a>
```

```
</div>
```

```
<!-- Filters -->
```

```
<div class="card" style="padding:1rem; margin-bottom:1.5rem;">
```

```
<form method="GET" style="display:flex; gap:0.75rem; flex-wrap:wrap;">
```

```
<input type="text" name="q" value="{{ search }}" placeholder="Search passage text or title..." style="flex:1; min-width:200px; padding:0.5rem 0.75rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">
```

```
<select name="category" style="padding:0.5rem 0.75rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">
```

```
<option value="">All Categories</option>
```

```
{% for cat in categories %}
```

```
<option value="{{ cat }}" {% if category == cat %}selected{% endif %}>{{ cat }}</option>
```

```
{% endfor %}
```

```
</select>
```

```
<select name="difficulty" style="padding:0.5rem 0.75rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">
```

```
<option value="">All Difficulties</option>
```

```
<option value="Easy" {% if difficulty == 'Easy' %}selected{% endif %}>Easy</option>
```

```
<option value="Medium" {% if difficulty == 'Medium' %}selected{% endif %}>Medium</option>
```

```
<option value="Hard" {% if difficulty == 'Hard' %}selected{% endif %}>Hard</option>
```

```
<option value="Expert" {% if difficulty == 'Expert' %}selected{% endif %}>Expert</option>
```

```
</select>
```

```
<button type="submit" class="btn btn-primary">Apply Filters</button>
```

```
</form>
```

```
</div>
```

```
<!-- Passages Table -->

<div class="card" style="padding:0; overflow:hidden;">

<table>

<thead>

<tr>

<th>ID</th>

<th>Title / Excerpt</th>

<th>Category</th>

<th>Difficulty</th>

<th>Words</th>

<th>Status</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{% for p in passages.items %}

<tr>

<td>#{{ p.id }}</td>

<td>

<div style="font-weight:700;">{{ p.title }}</div>

<div style="font-size:0.78rem; color:var(--admin-text-muted); max-width:400px; overflow:hidden; text-overflow:ellipsis; white-space:nowrap;">{{ p.content }}</div>

</td>

<td><span class="badge badge-primary">{{ p.category }}</span></td>

<td>{{ p.difficulty }}</td>

<td>{{ p.word_count }}</td>

<td>

<span class="badge {{ 'badge-success' if p.is_active else 'badge-danger' }}">{{ 'ACTIVE' if p.is_active else 'DISABLED' }}</span>

</td>

<td>

<div style="display:flex; gap:0.4rem;">

<a href="{{ url_for('admin.passage_edit', passage_id=p.id) }}" class="btn btn-sm">Edit</a>
```

```
<form method="POST" action="{{ url_for('admin.passage_toggle', passage_id=p.id) }}">

<button type="submit" class="btn btn-sm">{{ 'Disable' if p.is_active else 'Enable' }}</button>

</form>

<form method="POST" action="{{ url_for('admin.passage_delete', passage_id=p.id) }}" onsubmit="return confirm('Permanently remove this passage?');">

<button type="submit" class="btn btn-sm btn-danger">Delete</button>

</form>

</div>

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

{% endblock %}
```

```
=== FILE: \app\templates\admin\passage_form.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}{{ action }} Passage | Control Center{% endblock %}
```

```
{% block admin_content %}

<div style="margin-bottom:1.5rem;">

<a href="{{ url_for('admin.passages_list') }}" style="color:var(--admin-accent); font-size:0.85rem; text-decoration:none;">&larr; Back to Passages</a>

<h1 style="font-size:1.6rem; font-weight:800; margin-top:0.4rem;">{{ action }} Typing Passage</h1>

</div>
```

```
<div class="card" style="max-width:850px;">

<form method="POST">

<div style="display:grid; grid-template-columns: 2fr 1fr; gap:1rem; margin-bottom:1.25rem;">

<div>

<label style="display:block; font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.3rem;">PASSAGE TITLE</label>

<input type="text" name="title" value="{{ passage.title if passage else '' }}" required style="width:100%; padding:0.6rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">
```

```
</div>

<div>

  <label style="display:block; font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.3rem;">SOURCE / AUTHOR</label>

  <input type="text" name="source" value="{{ passage.source if passage else 'Administrator' }}" style="width:100%; padding:0.6rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">

</div>

</div>

<div style="display:grid; grid-template-columns: 1fr 1fr; gap:1rem; margin-bottom:1.25rem;">

<div>

  <label style="display:block; font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.3rem;">CATEGORY</label>

  <select name="category" style="width:100%; padding:0.6rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">

    {% for cat in ['General', 'Daily Life', 'Education', 'Technology', 'Science', 'Business', 'History', 'Literature', 'Professional'] %}

    <option value="{{ cat }}" {% if passage and passage.category == cat %}>selected{% endif %}>{{ cat }}</option>

    {% endfor %}

  </select>

</div>

<div>

  <label style="display:block; font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.3rem;">DIFFICULTY TIER</label>

  <select name="difficulty" style="width:100%; padding:0.6rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">

    {% for diff in ['Easy', 'Medium', 'Hard', 'Expert'] %}

    <option value="{{ diff }}" {% if passage and passage.difficulty == diff %}>selected{% endif %}>{{ diff }}</option>

    {% endfor %}

  </select>

</div>

</div>

<div style="margin-bottom:1.5rem;">

  <label style="display:block; font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.3rem;">COMPLETE PASSAGE TEXT</label>

  <textarea name="content" rows="8" required style="width:100%; padding:0.75rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff; font-family:var(--admin-mono); line-height:1.6;">{{ passage.content if passage else '' }}</textarea>

</div>
```

```
{% if passage %}

<div style="margin-bottom:1.5rem;">

  <label style="display:flex; align-items:center; gap:0.5rem; font-size:0.85rem; cursor:pointer;">

    <input type="checkbox" name="is_active" {% if passage.is_active %}>checked{% endif %}>

    <span>Enable this passage in active test selection</span>

  </label>

</div>

{% endif %}
```

```
<div style="display:flex; justify-content:flex-end; gap:0.75rem;">

  <a href="{{ url_for('admin.passages_list') }}" class="btn">Cancel</a>

  <button type="submit" class="btn btn-primary">{{ action }} Passage</button>

</div>

</form>

</div>

{% endblock %}
```

```
=== FILE: .\app\templates\admin\security.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}Security Center | TypeSphere Control Center{% endblock %}
```

```
{% block admin_content %}

<div style="margin-bottom:1.5rem;">

  <h1 style="font-size:1.6rem; font-weight:800;">Security Center & Incident Monitor</h1>

  <p style="color:var(--admin-text-muted); font-size:0.85rem;">Audit anomalous requests, failed logins, and access boundary violations.</p>

</div>
```

```
<div style="display:grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap:1rem; margin-bottom:1.5rem;">

  <div class="card" style="margin:0;">

    <div style="font-size:0.75rem; color:var(--admin-text-muted);">FAILED LOGIN ATTEMPTS</div>

    <div style="font-size:1.8rem; font-weight:800; color:var(--admin-warning);">{{ failed_logins }}</div>

  </div>
```

```
<div class="card" style="margin:0;">

  <div style="font-size:0.75rem; color:var(--admin-text-muted);">CRITICAL INCIDENTS</div>

  <div style="font-size:1.8rem; font-weight:800; color:var(--admin-danger);">{{ critical_events }}</div>

</div>

</div>

<div class="card" style="padding:0; overflow:hidden;">

  <table>

    <thead>

      <tr>

        <th>Event</th>

        <th>Severity</th>

        <th>Identifier</th>

        <th>IP Address</th>

        <th>Description</th>

        <th>Time</th>

      </tr>

    </thead>

    <tbody>

      {% for ev in events.items %}

      <tr>

        <td><strong>{{ ev.event_type }}</strong></td>

        <td>

          <span class="badge {{ 'badge-danger' if ev.severity in ['HIGH', 'CRITICAL'] else 'badge-warning' }}">{{ ev.severity }}</span>

        </td>

        <td>{{ ev.identifier or 'N/A' }}</td>

        <td style="font-family:var(--admin-mono); font-size:0.8rem;">{{ ev.ip_address or 'Unknown' }}</td>

        <td style="font-size:0.8rem; max-width:350px;">{{ ev.description }}</td>

        <td style="color:var(--admin-text-muted); font-size:0.78rem;">{{ ev.created_at.strftime('%Y-%m-%d %H:%M') }}</td>

      </tr>

      {% endfor %}

    </tbody>

  </table>
```

```
</div>

{% endblock %}

=== FILE: .\app\templates\admin\settings.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}System Parameters | Control Center{% endblock %}

{% block admin_content %}

<div style="margin-bottom:1.5rem;">

  <h1 style="font-size:1.6rem; font-weight:800;">Platform Runtime Configuration</h1>

  <p style="color:var(--admin-text-muted); font-size:0.85rem;">Modify operational variables without redeploying code.</p>

</div>

<div class="card" style="max-width:700px;">

  <form method="POST">

    <div style="margin-bottom:1.25rem;">

      <label style="display:block; font-size:0.78rem; font-weight:700; color:var(--admin-text-muted); margin-bottom:0.3rem;">MAINTENANCE MODE</label>

      <select name="maintenance_mode" style="width:100%; padding:0.6rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">

        <option value="false" {% if configs.get('maintenance_mode') != 'true' %}>selected{% endif %}>Disabled (Normal Operations)</option>

        <option value="true" {% if configs.get('maintenance_mode') == 'true' %}>selected{% endif %}>Enabled (Block public typing tests)</option>

      </select>

    </div>

    <div style="margin-bottom:1.25rem;">

      <label style="display:block; font-size:0.78rem; font-weight:700; color:var(--admin-text-muted); margin-bottom:0.3rem;">ALLOW NEW REGISTRATIONS</label>

      <select name="allow_registrations" style="width:100%; padding:0.6rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">

        <option value="true" {% if configs.get('allow_registrations') != 'false' %}>selected{% endif %}>Allow User Onboarding</option>

        <option value="false" {% if configs.get('allow_registrations') == 'false' %}>selected{% endif %}>Lock Registrations</option>

      </select>

    </div>

  </form>

</div>
```

```
<div style="margin-bottom:1.25rem;">

  <label style="display:block; font-size:0.78rem; font-weight:700; color:var(--admin-text-muted); margin-bottom:0.3rem;">DEFAULT TEST
DURATION (SECONDS)</label>

  <input type="number" name="default_test_duration" value="{{ configs.get('default_test_duration', '60') }}" style="width:100%;
padding:0.6rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">

</div>


<div style="margin-bottom:1.5rem;">

  <label style="display:block; font-size:0.78rem; font-weight:700; color:var(--admin-text-muted); margin-bottom:0.3rem;">ANTI-CHEAT HUMAN
WPM CEILING</label>

  <input type="number" name="min_wpm_cutoff" value="{{ configs.get('min_wpm_cutoff', '250') }}" style="width:100%; padding:0.6rem;
background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff;">

</div>


<button type="submit" class="btn btn-primary" style="padding:0.7rem 1.75rem;">Save System Parameters</button>

</form>

</div>

{% endblock %}

=== FILE: .\app\templates\admin\texts.html ===

{% extends 'base.html' %}


{% block content %}

<div class="card">

  <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom:
1.25rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

    <div>

      <h1 style="font-size: 1.8rem; font-weight: 700; margin: 0;">Typing Prompts Database</h1>

      <p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.35rem;">Manage the passages served to typists across all test modes.</p>

    </div>

    <a href="{{ url_for('admin.index') }}" class="btn">&larr; Back to Admin Overview</a>

  </div>

</div>

<!-- Add New Prompt Form -->
```

```
<form method="POST" style="background: var(--bg-card); padding: 1.5rem; border-radius: var(--radius); margin-bottom: 2.5rem; border: 1px
solid var(--border-color);">

<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Add New Prompt Text</h3>

<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; margin-bottom: 1rem;">

  <div>

    <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem;">Category</label>

    <input type="text" name="category" placeholder="General, Literature, Technology, Science, etc." required style="width: 100%; padding:
0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary); font-family:
inherit;">

  </div>

  <div>

    <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem;">Programming Language (Optional)</label>

    <input type="text" name="code_lang" placeholder="python, javascript, sql" style="width: 100%; padding: 0.65rem; background: var(--bg-
secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary); font-family: inherit;">

  </div>

</div>

<div style="margin-bottom: 1rem;">

  <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem;">Passage Content</label>

  <textarea name="content" rows="4" required style="width: 100%; padding: 0.75rem; background: var(--bg-secondary); border: 1px solid var(--
border-color); border-radius: 4px; color: var(--text-primary); font-family: var(--font-mono);"></textarea>

</div>


<div style="margin-bottom: 1.5rem; display: flex; align-items: center; gap: 0.5rem;">

  <input type="checkbox" name="is_code" id="is_code">

  <label for="is_code" style="font-size: 0.85rem; cursor: pointer;">This prompt represents code syntax</label>

</div>

<button type="submit" class="btn btn-primary">Save Prompt</button>

</form>


<!-- Existing Prompts Table -->

<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Configured Prompts {{{ texts|length }}}</h3>

<table style="width: 100%; border-collapse: collapse; text-align: left;">

  <thead>
```

```
<tr style="border-bottom: 1px solid var(--border-color); color: var(--text-muted); font-size: 0.85rem;">

  <th style="padding: 0.6rem;">ID</th>

  <th style="padding: 0.6rem;">Category</th>

  <th style="padding: 0.6rem;">Excerpt</th>

  <th style="padding: 0.6rem;">Is Code</th>

  <th style="padding: 0.6rem;">Action</th>

</tr>

</thead>

<tbody>

  {% for t in texts %}

    <tr style="border-bottom: 1px solid var(--border-color); font-size: 0.85rem;">

      <td style="padding: 0.75rem;">#{ { t.id }}</td>

      <td style="padding: 0.75rem;"><span style="background: var(--bg-card); padding: 0.2rem 0.5rem; border-radius: 4px;">{ { t.category }}</span></td>

      <td style="padding: 0.75rem; max-width: 420px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; font-family: var(--font-mono);">{ { t.content }}</td>

      <td style="padding: 0.75rem;">{ { 'Yes (' + t.code_lang + ')' if t.is_code else 'No' }}</td>

      <td style="padding: 0.75rem;">

        <form method="POST" action="{ { url_for('admin.delete_text', text_id=t.id) }}" onsubmit="return confirm('Delete this prompt permanently?');">

          <button type="submit" class="btn" style="padding: 0.2rem 0.5rem; font-size: 0.75rem; border-color: var(--danger); color: var(--danger);">Delete</button>

        </form>

      </td>

    </tr>

  {% endfor %}

</tbody>

</table>

</div>

{% endblock %}
```

```
=== FILE: .\app\templates\admin\users.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}User Management | TypeSphere Control Center{% endblock %}
```

```
{% block admin_content %}

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1.5rem; flex-wrap: wrap; gap: 1rem;">

  <div>

    <h1 style="font-size: 1.6rem; font-weight: 800;">User Account Directory</h1>

    <p style="color: var(--admin-text-muted); font-size: 0.85rem;">Inspect user activity, modify roles, and enforce moderation actions.</p>

  </div>

</div>

<!-- Filters Bar -->

<div class="card" style="padding: 1rem; margin-bottom: 1.5rem;">

  <form method="GET" style="display: flex; gap: 0.75rem; flex-wrap: wrap;">

    <input type="text" name="q" value="{ { search }}" placeholder="Search username or email..." style="flex: 1; min-width: 200px; padding: 0.5rem 0.75rem; background: var(--admin-bg); border: 1px solid var(--admin-border); border-radius: 6px; color: #fff;">

    <select name="role" style="padding: 0.5rem 0.75rem; background: var(--admin-bg); border: 1px solid var(--admin-border); border-radius: 6px; color: #fff;">

      <option value="">All Roles</option>

      {% for r in roles %}

        <option value="{ { r }}" {% if role_filter == r %}selected{% endif %}>{ { r|upper }}</option>

      {% endfor %}

    </select>

    <select name="status" style="padding: 0.5rem 0.75rem; background: var(--admin-bg); border: 1px solid var(--admin-border); border-radius: 6px; color: #fff;">

      <option value="">All Statuses</option>

      <option value="active" {% if status_filter == 'active' %}selected{% endif %}>Active</option>

      <option value="suspended" {% if status_filter == 'suspended' %}selected{% endif %}>Suspended</option>

      <option value="banned" {% if status_filter == 'banned' %}selected{% endif %}>Banned</option>

    </select>

    <button type="submit" class="btn btn-primary">Filter</button>

  </form>

</div>
```



```
<!-- Users Table -->

<div class="card" style="padding:0; overflow:hidden;">

<table>

<thead>

<tr>

<th>ID</th>

<th>User</th>

<th>Role</th>

<th>Elo Rating</th>

<th>Status</th>

<th>Joined</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{% for u in users.items %}

<tr>

<td>#{{ u.id }}</td>

<td>

<div style="font-weight:700;">{{ u.username }}</div>

<div style="font-size:0.78rem; color:var(--admin-text-muted);">{{ u.email }}</div>

</td>

<td><span class="badge badge-primary">{{ u.role|upper }}</span></td>

<td style="font-family:var(--admin-mono); font-weight:700;">{{ u.elo_rating }}</td>

<td>

{% if u.is_banned %}

<span class="badge badge-danger">Banned</span>

{% elif u.is_suspended %}

<span class="badge badge-warning">Suspended</span>

{% else %}

<span class="badge badge-success">Active</span>

{% endif %}

</td>

</tr>

</tbody>

</table>

</div>
```

```
</td>

<td style="color:var(--admin-text-muted); font-size:0.8rem;">{{ u.created_at.strftime('%Y-%m-%d') }}</td>

<td>

<a href="{{ url_for('admin.user_detail', user_id=u.id) }}" class="btn btn-sm">Inspect</a>

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

{% endblock %}

=== FILE: \app\templates\admin\users_detail.html ===

{% extends 'admin/base_admin.html' %}

{% block title %}User @{{ user.username }} | Control Center{% endblock %}

{% block admin_content %}

<div style="margin-bottom:1.5rem;">

<a href="{{ url_for('admin.users_list') }}" style="color:var(--admin-accent); font-size:0.85rem; text-decoration:none;">&larr; Back to Directory</a>

<h1 style="font-size:1.6rem; font-weight:800; margin-top:0.4rem;">User Profile: @{{ user.username }}</h1>

</div>

<div style="display:grid; grid-template-columns: 1fr 2fr; gap:1.5rem; margin-bottom:1.5rem;">

<!-- Moderation & Role Card -->

<div class="card">

<h3 style="font-size:1.1rem; margin-bottom:1rem;">Account Management</h3>

<div style="margin-bottom:1.25rem;">

<div style="font-size:0.75rem; color:var(--admin-text-muted);">EMAIL</div>

<div style="font-weight:600;">{{ user.email }}</div>

</div>

<!-- Role Form -->

<form method="POST" action="{{ url_for('admin.update_user_role', user_id=user.id) }}" style="margin-bottom:1.5rem;">
```

```
<label style="display:block; font-size:0.75rem; color:var(--admin-text-muted); margin-bottom:0.3rem;">AUTHORITY ROLE</label>

<select name="role" style="width:100%; padding:0.5rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff; margin-bottom:0.6rem;">

    {% for r in roles %}

        <option value="{{ r }}" {% if user.role == r %>selected{% endif %}>{{ r|upper }}</option>

    {% endfor %}

</select>

<button type="submit" class="btn btn-sm btn-primary" style="width:100%; justify-content:center;">Update Role</button>

</form>

<!-- Account Actions -->

<h4 style="font-size:0.85rem; color:var(--admin-text-muted); margin-bottom:0.6rem;">ACCOUNT MODERATION</h4>

<form method="POST" action="{{ url_for('admin.moderate_user', user_id=user.id) }}" style="display:flex; flex-direction:column; gap:0.5rem;">

    <input type="text" name="reason" placeholder="Enter reason for action..." style="padding:0.45rem; background:var(--admin-bg); border:1px solid var(--admin-border); border-radius:6px; color:#fff; font-size:0.8rem;" required>

    {% if user.is_suspended %}

        <button type="submit" name="action" value="unsuspend" class="btn btn-sm">Lift Suspension</button>

    {% else %}

        <button type="submit" name="action" value="suspend" class="btn btn-sm btn-danger">Suspend Account</button>

    {% endif %}

    {% if user.is_banned %}

        <button type="submit" name="action" value="unban" class="btn btn-sm">Lift Ban</button>

    {% else %}

        <button type="submit" name="action" value="ban" class="btn btn-sm btn-danger">Permanent Ban</button>

    {% endif %}

    {% if current_user.is_super_admin and user.id != current_user.id %}

        <button type="submit" name="action" value="delete" class="btn btn-sm btn-danger" onclick="return confirm('Permanently erase this account and tests?');">Delete User</button>

    {% endif %}

</form>
```

```
<!-- Impersonation Action -->

{% if current_user.is_super_admin and user.id != current_user.id %}

<form method="POST" action="{{ url_for('admin.impersonate_user', user_id=user.id) }}" style="margin-top:1.5rem;">

    <button type="submit" class="btn btn-sm" style="width:100%; justify-content:center; border-color:#f59e0b; color:#f59e0b;">👤 Impersonate User</button>

</form>

{% endif %}

</div>

<!-- Activity & Test Record -->

<div class="card">

<h3 style="font-size:1.1rem; margin-bottom:1rem;">Recent Activity Stream</h3>

<table>

    <thead>

        <tr>

            <th>Action</th>

            <th>Feature</th>

            <th>IP Address</th>

            <th>Time</th>

        </tr>

    </thead>

    <tbody>

        {% for act in activities %}

        <tr>

            <td><span class="badge badge-primary">{{ act.action }}</span></td>

            <td>{{ act.feature }}</td>

            <td style="font-family:var(--admin-mono); font-size:0.8rem;">{{ act.ip_address or 'Unknown' }}</td>

            <td style="color:var(--admin-text-muted); font-size:0.8rem;">{{ act.created_at.strftime('%Y-%m-%d %H:%M') }}</td>

        </tr>

        {% else %}

            <tr><td colspan="4" style="text-align:center; color:var(--admin-text-muted);">No activity records for this account.</td></tr>

        {% endfor %}

    </tbody>

</div>
```

```
</table>

</div>

</div>

{% endblock %}

=== FILE: .\app\templates\auth\forgot_password.html ===

{% extends 'base.html' %}

{% block content %}

<div style="max-width: 440px; margin: 3.5rem auto;">

  <div class="card">

    <div style="text-align: center; margin-bottom: 1.5rem;">

      <div style="width: 58px; height: 58px; border-radius: 50%; background: rgba(88, 166, 255, 0.15); border: 2px solid var(--accent); display: flex; align-items: center; justify-content: center; margin: 0 auto 1rem; font-size: 1.6rem;">

        ❏

      </div>

      <h2 style="font-size: 1.6rem; font-weight: 700; margin-bottom: 0.4rem;">Reset Your Password</h2>

      <p style="color: var(--text-muted); font-size: 0.88rem; line-height: 1.5;">

        Enter the email address associated with your TypeSphere account and we will dispatch a secure reset link.

      </p>

    </div>

    <form method="POST" action="{{ url_for('auth.forgot_password') }}">

      {{ form.hidden_tag() }}

      <div style="margin-bottom: 1.5rem;">

        <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Email Address</label>

        {{ form.email(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius); color: var(--text-primary); font-family: inherit;") }}

        {% for err in form.email.errors %}

          <div style="color: var(--danger); font-size: 0.75rem; margin-top: 0.25rem;">{{ err }}</div>

        {% endfor %}

      </div>
```

```
{{ form.submit(class="btn btn-primary", style="width: 100%; justify-content: center; padding: 0.8rem; font-weight: 700;") }}

</form>

<div style="margin-top: 1.5rem; text-align: center; font-size: 0.85rem; color: var(--text-muted);">

  Remember your password? <a href="{{ url_for('auth.login') }}" style="color: var(--accent); text-decoration: none;">Return to Sign In</a>

</div>

</div>

</div>

{% endblock %}

=== FILE: .\app\templates\auth\login.html ===

{% extends 'base.html' %}

{% block content %}

<div style="max-width: 420px; margin: 3.5rem auto;">

  <div class="card" style="box-shadow: 0 15px 35px rgba(0,0,0,0.5);">

    <h2 style="font-size: 1.6rem; margin-bottom: 0.5rem; text-align: center;">Sign In</h2>

    <p style="color: var(--text-muted); text-align: center; margin-bottom: 2rem; font-size: 0.9rem;">

      Enter your username and password to access your dashboard.

    </p>

    <form method="POST" action="{{ url_for('auth.login') }}">

      {{ form.hidden_tag() }}

      <div style="margin-bottom: 1.25rem;">

        <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Username or Email</label>

        {{ form.email(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius); color: var(--text-primary); font-family: inherit; ", placeholder="e.g. pilot or pilot@example.com") }}

      </div>

      <div style="margin-bottom: 1.75rem;">

        <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Password</label>
```

```

    {{ form.password(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(-radius); color: var(--text-primary); font-family: inherit;" placeholder="Enter your password") }}

</div>

{{ form.submit(class="btn btn-primary", style="width: 100%; justify-content: center; padding: 0.8rem; font-weight: 700;") }}

</form>

<div style="margin-top: 1.75rem; text-align: center; font-size: 0.85rem; color: var(--text-muted); border-top: 1px solid var(--border-color); padding-top: 1.25rem;">

    Don't have an account? <a href="{{ url_for('auth.register') }}" style="color: var(--accent); text-decoration: none; font-weight: 600;">Create Account</a>

</div>

</div>

</div>

{% endblock %}
```

=== FILE: .\app\templates\auth\register.html ===

{% extends 'base.html' %}

{% block content %}

<div style="max-width: 440px; margin: 3rem auto;">

<div class="card" style="box-shadow: 0 15px 35px rgba(0,0,0,0.5);">

<h2 style="font-size: 1.6rem; margin-bottom: 0.5rem; text-align: center;">Create Account</h2>

<p style="color: var(--text-muted); text-align: center; margin-bottom: 1.75rem; font-size: 0.9rem;">

Sign up in seconds to start tracking your Biometric Typing DNA.

</p>

<form method="POST" action="{{ url_for('auth.register') }}">

{{ form.hidden_tag() }}

<div style="margin-bottom: 1.2rem;">

<label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Username</label>

{{ form.username(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(-radius); color: var(--text-primary); font-family: inherit;" placeholder="Choose a username") }}

{% for err in form.username.errors %}

<div style="color: var(--danger); font-size: 0.75rem; margin-top: 0.25rem;">{{ err }}</div>

{% endfor %}

</div>

<div style="margin-bottom: 1.2rem;">

<label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Email Address</label>

{{ form.email(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(-radius); color: var(--text-primary); font-family: inherit;" placeholder="name@example.com") }}

{% for err in form.email.errors %}

<div style="color: var(--danger); font-size: 0.75rem; margin-top: 0.25rem;">{{ err }}</div>

{% endfor %}

</div>

<div style="margin-bottom: 1.2rem;">

<label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Password</label>

{{ form.password(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(-radius); color: var(--text-primary); font-family: inherit;" placeholder="Create a password") }}

{% for err in form.password.errors %}

<div style="color: var(--danger); font-size: 0.75rem; margin-top: 0.25rem;">{{ err }}</div>

{% endfor %}

</div>

<div style="margin-bottom: 1.75rem;">

<label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Confirm Password</label>

{{ form.confirm_password(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(-radius); color: var(--text-primary); font-family: inherit;" placeholder="Repeat password") }}

{% for err in form.confirm_password.errors %}

<div style="color: var(--danger); font-size: 0.75rem; margin-top: 0.25rem;">{{ err }}</div>

{% endfor %}

</div>

{{ form.submit(class="btn btn-primary", style="width: 100%; justify-content: center; padding: 0.8rem; font-weight: 700;") }}

</form>

```
<div style="margin-top: 1.5rem; text-align: center; font-size: 0.85rem; color: var(--text-muted); border-top: 1px solid var(--border-color); padding-top: 1.25rem;">

  Already have an account? <a href="{{ url_for('auth.login') }}" style="color: var(--accent); text-decoration: none; font-weight: 600;">Sign in</a>

</div>

</div>

</div>

{% endblock %}
```

=== FILE: .\app\templates\auth\reset_password.html ===

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<div style="max-width: 440px; margin: 3.5rem auto;">
```

```
<div class="card">
```

```
<div style="text-align: center; margin-bottom: 1.5rem;">
```

```
<div style="width: 58px; height: 58px; border-radius: 50%; background: rgba(63, 185, 80, 0.15); border: 2px solid var(--success); display: flex; align-items: center; justify-content: center; margin: 0 auto 1rem; font-size: 1.6rem;">
```

```
  <img alt="Reset Password icon" data-bbox="41 441 51 451" style="width: 100%; height: 100%;"/>
```

```
</div>
```

```
<h2 style="font-size: 1.6rem; font-weight: 700; margin-bottom: 0.4rem;">Choose a New Password</h2>
```

```
<p style="color: var(--text-muted); font-size: 0.88rem; line-height: 1.5;">
```

```
  Please enter and confirm your new secure password.
```

```
</p>
```

```
</div>
```

```
<form method="POST" action="{{ url_for('auth.reset_password', token=token) }}">
```

```
  {{ form.hidden_tag() }}
```

```
<div style="margin-bottom: 1.25rem;">
```

```
<label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">New Password (min 6 characters)</label>
```

```
  {{ form.password(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius); color: var(--text-primary); font-family: inherit;" ) }}
```

```
  {% for err in form.password.errors %}
```

```
<div style="color: var(--danger); font-size: 0.75rem; margin-top: 0.25rem;">{{ err }}</div>
```

```
{% endfor %}
```

```
</div>
```

```
<div style="margin-bottom: 1.75rem;">
```

```
<label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem; color: var(--text-muted);">Confirm New Password</label>
```

```
  {{ form.confirm_password(style="width: 100%; padding: 0.75rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius); color: var(--text-primary); font-family: inherit;" ) }}
```

```
</div>
```

```
  {{ form.submit(class="btn btn-primary", style="width: 100%; justify-content: center; padding: 0.8rem; font-weight: 700;" ) }}
```

```
</form>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

=== FILE: .\app\templates\auth\verify_notice.html ===

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<div style="max-width: 520px; margin: 3.5rem auto;">
```

```
<div class="card" style="text-align: center; border-color: var(--accent); padding: 2.5rem 2rem;">
```

```
<!-- Mail Envelope Graphic -->
```

```
<div style="width: 68px; height: 68px; border-radius: 50%; background: rgba(88, 166, 255, 0.15); border: 2px solid var(--accent); display: flex; align-items: center; justify-content: center; margin: 0 auto 1.5rem; font-size: 2rem;">
```

```
  <img alt="Mail Envelope icon" data-bbox="539 609 551 619" style="width: 100%; height: 100%;"/>
```

```
</div>
```

```
<h2 style="font-size: 1.6rem; font-weight: 700; margin-bottom: 0.75rem;">Check Your Email Inbox</h2>
```

```
<p style="color: var(--text-muted); font-size: 0.95rem; line-height: 1.6; margin-bottom: 1.75rem;">
```

```
  A verification link has been sent to:<br>
```

```
<strong style="color: var(--text-primary); font-size: 1.05rem;">{{ email }}</strong>
```

```
</p>
```

```
<!-- Clear status warning if SMTP credentials have not been configured yet -->

{% if not smtp_configured %}

<div style="background: rgba(248, 81, 73, 0.15); border: 1px solid var(--danger); border-radius: var(--radius); padding: 1rem; margin-bottom: 1.5rem; text-align: left; font-size: 0.85rem; line-height: 1.5; color: #ff7b72;">

  <strong>☐☐ Action Required to receive emails:</strong><br>

  You must put your Gmail address and 16-letter App Password in <code>config.py</code> under <code>MAIL_USERNAME</code> and <code>MAIL_PASSWORD</code>. Once added, restart <code>run.py</code> and click "Resend Verification Email" below.

</div>

{% endif %}

<div style="background: var(--bg-card); padding: 1.25rem; border-radius: var(--radius); border: 1px solid var(--border-color); margin-bottom: 1.75rem; text-align: left; font-size: 0.88rem; line-height: 1.6; color: var(--text-muted);">

  <p style="margin-bottom: 0.5rem; color: var(--text-primary); font-weight: 600;">Next Steps:</p>

  <ol style="padding-left: 1.25rem; display: flex; flex-direction: column; gap: 0.4rem;">

    <li>Open your email inbox for <strong>{{ email }}</strong>.</li>

    <li>Find the email titled <em>"Action Required: Verify Your TypeSphere Account"</em>.</li>

    <li>Click the <strong>Verify Email Address</strong> button inside that email.</li>

    <li>(Check your Spam or Junk folder if you do not see it within a minute).</li>

  </ol>

</div>

<!-- Resend Email Button -->

<form method="POST" action="{{ url_for('auth.resend_verification') }}" style="margin-bottom: 1rem;">

  <input type="hidden" name="email" value="{{ email }}">

  <button type="submit" class="btn btn-primary" style="width: 100%; justify-content: center; padding: 0.75rem;">

    Resend Verification Email

  </button>

</form>

<a href="{{ url_for('auth.login') }}" class="btn" style="width: 100%; justify-content: center;">

  ⌂ Return to Sign In

</a>

</div>
```

```
</div>

{% endblock %}

=== FILE: .\app\templates\challenges\daily.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card">

  <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1.5rem; flex-wrap: wrap; gap: 1rem;">

    <div>

      <div style="display: flex; align-items: center; gap: 0.75rem;">

        <h1 style="font-size: 1.8rem; font-weight: 700; margin: 0;">Daily Challenge: {{ challenge.title }}</h1>

        <span style="background: var(--accent); color: #fff; font-size: 0.75rem; font-weight: 700; padding: 0.2rem 0.55rem; border-radius: 4px;">

          OFFICIAL 24H BENCHMARK

        </span>

      </div>

    </div>

    <p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.35rem;">

      {{ challenge.target_date.strftime('%A, %B %d, %Y') }} &bull; One synchronized passage worldwide.

    </p>

  </div>

  <button onclick="startDailyChallenge()" class="btn btn-primary" style="padding: 0.75rem 1.5rem; font-size: 1rem;">

    Start Official Daily Run &arr;

  </button>

</div>

<!-- Today's Passage Preview -->

<div style="background: var(--bg-card); padding: 1.75rem; border-radius: var(--radius); margin-bottom: 2rem; border-left: 4px solid var(--accent); font-family: var(--font-mono); font-size: 1.15rem; line-height: 1.9; color: var(--text-primary);">

  "{{ challenge.content }}"

</div>

<!-- Today's Leaderboard -->

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1rem;">
```

```
<h3 style="font-size: 1.15rem; margin: 0;">Today's Verified Leaderboard</h3>

<span style="font-size: 0.8rem; color: var(--text-muted);">Updated in real-time</span>

</div>

<table style="width: 100%; border-collapse: collapse; text-align: left; margin-bottom: 3rem;">

  <thead>

    <tr style="border-bottom: 1px solid var(--border-color); color: var(--text-muted); font-size: 0.85rem;">

      <th style="padding: 0.75rem;">Rank</th>

      <th style="padding: 0.75rem;">Typist</th>

      <th style="padding: 0.75rem;">Speed</th>

      <th style="padding: 0.75rem;">Accuracy</th>

      <th style="padding: 0.75rem;">Consistency</th>

      <th style="padding: 0.75rem;">Verified Time</th>

    </tr>

  </thead>

  <tbody>

    {% for test in leaderboard %}

      <tr style="border-bottom: 1px solid var(--border-color); font-size: 0.9rem;">

        <td style="padding: 0.85rem; font-weight: 800; color: {{ '#f59e0b' if loop.index == 1 else ('#94a3b8' if loop.index == 2 else ('#b45309' if loop.index == 3 else 'var(--text-muted)')) }};">

          #{{ loop.index }}

        </td>

        <td style="padding: 0.85rem; font-weight: 600;">

          {{ test.user.username if test.user else 'Verified Guest' }}

        </td>

        <td style="padding: 0.85rem; font-family: var(--font-mono); font-weight: 800; color: var(--accent);">

          {{ test.wpm }} WPM

        </td>

        <td style="padding: 0.85rem;">{{ test.accuracy }}%</td>

        <td style="padding: 0.85rem;">{{ test.consistency }}%</td>

        <td style="padding: 0.85rem; color: var(--text-muted); font-size: 0.8rem;">

          {{ test.completed_at.strftime("%H:%M:%S") }}

        </td>

      </tr>

    </tbody>

  </table>
```

```
</tr>

{% else %}

<tr>

  <td colspan="6" style="padding: 2.5rem; text-align: center; color: var(--text-muted);">

    No verified entries logged yet for today. Be the first to set today's record!

  </td>

</tr>

{% endfor %}

</tbody>

</table>

<!-- Historic Archive of Past Daily Challenges -->

<div style="border-top: 1px solid var(--border-color); padding-top: 2rem;">

  <h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Past Daily Challenges History</h3>

  <table style="width: 100%; border-collapse: collapse; text-align: left;">

    <thead>

      <tr style="border-bottom: 1px solid var(--border-color); color: var(--text-muted); font-size: 0.85rem;">

        <th style="padding: 0.65rem;">Date</th>

        <th style="padding: 0.65rem;">Title</th>

        <th style="padding: 0.65rem;">Passage Excerpt</th>

      </tr>

    </thead>

    <tbody>

      {% for h in history %}

        <tr style="border-bottom: 1px solid var(--border-color); font-size: 0.85rem;">

          <td style="padding: 0.75rem; font-weight: 600; color: var(--accent);">{{ h.target_date.strftime('%b %d, %Y') }}</td>

          <td style="padding: 0.75rem; font-weight: 600;">{{ h.title }}</td>

          <td style="padding: 0.75rem; color: var(--text-muted); max-width: 400px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; font-family: var(--font-mono);">

            {{ h.content }}

          </td>

        </tr>

      </tbody>

    </table>

    {% else %}
```

```
<tr>

  <td colspan="3" style="padding: 1.5rem; text-align: center; color: var(--text-muted);">

    Past challenges will archive here daily.

  </td>

</tr>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

{% endblock %}

{% block scripts %}

<script>

function startDailyChallenge() {

  window.location.href = "{{ url_for('typing.test_page') }}"?mode=daily";

}

</script>

{% endblock %}

=== FILE: .\app\templates\challenges\index.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card">

  <div style="border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 2rem;">

    <h1 style="font-size: 1.8rem; font-weight: 700; margin-bottom: 0.5rem;">Competitive Challenge Arenas</h1>

    <p style="color: var(--text-muted); font-size: 0.95rem;">

      Specialized game modes engineered to test biometric precision, survival stamina, and real-time stress tolerance.

    </p>

  </div>

</div>

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 1.75rem;">
```

```
<!-- Daily Challenge -->

<div class="card" style="background: var(--bg-card); border-color: var(--accent); margin-bottom: 0; display: flex; flex-direction: column; justify-content: space-between;">

  <div>

    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.75rem;">

      <h3 style="color: var(--accent); margin: 0;">Daily Challenge</h3>

      <span style="font-size: 0.75rem; background: var(--accent); color: #fff; padding: 0.15rem 0.5rem; border-radius: 4px; font-weight: 700;">DAILY 24H</span>

    </div>

    <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.6; margin-bottom: 1.25rem;">

      One universal passage for all typists worldwide every 24 hours. Ranked on an immutable daily leaderboard.

    </p>

    </div>

    <a href="{{ url_for('challenges.daily') }}" class="btn btn-primary" style="width: 100%; justify-content: center;">

      View & Enter Daily Run &rarr;

    </a>

  </div>

<!-- Survival Mode -->

<div class="card" style="background: var(--bg-card); border-color: var(--danger); margin-bottom: 0; display: flex; flex-direction: column; justify-content: space-between;">

  <div>

    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.75rem;">

      <h3 style="color: var(--danger); margin: 0;">Survival (3 Hearts)</h3>

      <span style="font-size: 0.75rem; background: rgba(248, 81, 73, 0.2); color: var(--danger); padding: 0.15rem 0.5rem; border-radius: 4px; font-weight: 700;">SUDDEN DEATH</span>

    </div>

    <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.6; margin-bottom: 1.25rem;">

      Start with 3 lives. Every mistyped character deducts one heart. Survive the entire passage or face elimination.

    </p>

    </div>

    <a href="{{ url_for('typing.test_page') }}"?mode=survival" class="btn" style="border-color: var(--danger); color: var(--danger); width: 100%; justify-content: center;">

      Enter Survival Arena &rarr;

    </a>

  </div>
```



```
</div>

<!-- Accuracy Challenge -->

<div class="card" style="background: var(--bg-card); border-color: var(--success); margin-bottom: 0; display: flex; flex-direction: column; justify-content: space-between;">

  <div>

    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.75rem;">

      <h3 style="color: var(--success); margin: 0;">Flawless Accuracy</h3>

      <span style="font-size: 0.75rem; background: rgba(63, 185, 80, 0.2); color: var(--success); padding: 0.15rem 0.5rem; border-radius: 4px; font-weight: 700;">100% REQUIRED</span>

    </div>

    <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.6; margin-bottom: 1.25rem;">

      Speed is secondary. Absolute perfection is mandatory. The benchmark resets immediately upon a single error.

    </p>

  </div>

  <a href="{{ url_for('typing.test_page') }}"?mode=accuracy" class="btn" style="border-color: var(--success); color: var(--success); width: 100%; justify-content: center;">

    Launch Accuracy Test &rarr;

  </a>

</div>

<!-- 5-Minute Endurance -->

<div class="card" style="background: var(--bg-card); border-color: var(--warning); margin-bottom: 0; display: flex; flex-direction: column; justify-content: space-between;">

  <div>

    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.75rem;">

      <h3 style="color: var(--warning); margin: 0;">Endurance Marathon</h3>

      <span style="font-size: 0.75rem; background: rgba(210, 153, 34, 0.2); color: var(--warning); padding: 0.15rem 0.5rem; border-radius: 4px; font-weight: 700;">300 SECONDS</span>

    </div>

    <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.6; margin-bottom: 1.25rem;">

      A rigorous 5-minute sustained typing test engineered to chart cognitive stamina and physical finger fatigue.

    </p>

  </div>

</div>
```

```
<a href="{{ url_for('typing.test_page') }}"?mode=timed&duration=300" class="btn" style="border-color: var(--warning); color: var(--warning); width: 100%; justify-content: center;">

  Start 5-Minute Run &rarr;

</a>

</div>

</div>

{% endblock %}

=== FILE: .\app\templates\dashboard\achievements.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card">

  <!-- Title & Mastery Progress Header -->

  <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

    <div>

      <h1 style="font-size: 1.8rem; font-weight: 700; margin: 0;">Trophy Hall & Achievement Badges</h1>

      <p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.35rem;">

        Universal milestones tracking keyboard velocity, endurance, biometric precision, and rare feats.

      </p>

    </div>

  </div>

  <div style="display: flex; gap: 0.75rem;">

    <a href="{{ url_for('dashboard.index') }}" class="btn">&larr; Back to Dashboard</a>

    <a href="{{ url_for('typing.test_page') }}" class="btn btn-primary">Take A Benchmark Run</a>

  </div>

</div>

<!-- Mastery Progress Banner -->

<div class="card" style="background: var(--bg-card); border-color: var(--accent); margin-bottom: 2rem;">

  <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.75rem; flex-wrap: wrap; gap: 0.5rem;">
```

```
<span style="font-weight: 700; font-size: 1.1rem; color: var(--accent);">Overall Trophy Mastery</span>

<span style="font-family: var(--font-mono); font-size: 1.1rem; font-weight: 800;">{{ unlocked_count }} / {{ total_count }} Badges ({{ mastery_pct
}}%)</span>

</div>

<!-- Progress Bar -->

<div style="background: var(--bg-secondary); height: 12px; border-radius: 6px; overflow: hidden; border: 1px solid var(--border-color);">

  <div style="width: {{ mastery_pct }}%; height: 100%; background: linear-gradient(90deg, var(--accent), #a371f7); transition: width 0.4s
ease;"></div>

</div>

</div>

<!-- Filter Pills -->

<div style="display: flex; gap: 0.5rem; margin-bottom: 1.75rem;">

  <button onclick="filterTrophies('all')" class="btn active-choice" id="filter-all">All Badges ({{ total_count }})</button>

  <button onclick="filterTrophies('unlocked')" class="btn" id="filter-unlocked">Unlocked ({{ unlocked_count }})</button>

  <button onclick="filterTrophies('locked')" class="btn" id="filter-locked">Locked ({{ total_count - unlocked_count }})</button>

</div>

<!-- Trophies Grid -->

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 1.5rem;" id="trophy-grid">

  {% for b in badges %}

    <div class="card trophy-card {{ 'trophy-unlocked' if b.unlocked else 'trophy-locked' }}" data-status="{{ 'unlocked' if b.unlocked else 'locked' }}"
style="margin-bottom: 0; background: var(--bg-card); border-color: {{ 'var(--accent)' if b.unlocked else 'var(--border-color)' }}; position: relative;
overflow: hidden;">

      <!-- Unlocked Watermark / Glow -->

      {% if b.unlocked %}

        <div style="position: absolute; right: -10px; top: -10px; font-size: 4rem; opacity: 0.08; pointer-events: none;">❖</div>

      {% endif %}

      <div style="display: flex; align-items: flex-start; gap: 1rem;">

        <!-- Badge Icon -->

        <div style="width: 48px; height: 48px; border-radius: 10px; display: flex; align-items: center; justify-content: center; font-size: 1.5rem; flex-
shrink: 0; background: {{ 'rgba(88, 166, 255, 0.2)' if b.unlocked else 'rgba(255, 255, 255, 0.05)' }}; border: 1px solid {{ 'var(--accent)' if b.unlocked
else 'var(--border-color)' }};">
```

```
{{ '❖' if b.unlocked else '❖' }}

</div>

<div style="flex: 1;">

  <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.25rem;">

    <h3 style="font-size: 1.05rem; font-weight: 700; margin: 0; color: {{ 'var(--text-primary)' if b.unlocked else 'var(--text-muted)' }};">

      {{ b.title }}

    </h3>

    <span style="font-size: 0.7rem; font-weight: 700; padding: 0.15rem 0.45rem; border-radius: 4px; text-transform: uppercase; background: {{
'rgba(63, 185, 80, 0.2)' if b.unlocked else 'var(--bg-secondary)' }}; color: {{ 'var(--success)' if b.unlocked else 'var(--text-muted)' }};">

      {{ 'Unlocked' if b.unlocked else 'Locked' }}

    </span>

  </div>

  <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.5; margin-bottom: 0.75rem;">

    {{ b.description }}

  </p>

  {% if b.unlocked %}

    <div style="font-size: 0.75rem; color: var(--accent); font-weight: 600;">

      ✓ Earned on {{ b.unlocked_at.strftime('%b %d, %Y at %H:%M') }}

    </div>

    {% else %}

      <div style="font-size: 0.75rem; color: var(--text-muted);">

        ❖ Requirement not yet achieved

      </div>

    {% endif %}

  </div>

</div>

</div>

{% endfor %}

</div>

</div>
```

```
{% endblock %}

{% block scripts %}

<script>

function filterTrophies(type) {

    document.querySelectorAll('[id^="filter-"]').forEach(b => b.classList.remove('active-choice'));

    document.getElementById(`filter-${type}`).classList.add('active-choice');

    const cards = document.querySelectorAll('.trophy-card');

    cards.forEach(card => {

        if (type === 'all' || card.dataset.status === type) {

            card.style.display = 'block';

        } else {

            card.style.display = 'none';

        }

    });

}

</script>

{% endblock %}
```

```
=== FILE: .\app\templates\dashboard\analytics.html ===

{% extends 'base.html' %}
```

```
{% block content %}

<div class="card">

    <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

        <div>

            <h1 style="font-size: 1.8rem; font-weight: 700; margin-bottom: 0.25rem;">Progression & Biometric Analytics</h1>

            <p style="color: var(--text-muted); font-size: 0.9rem;">Historical trajectory analysis across verified typing sessions.</p>

        </div>

        <div style="display: flex; gap: 0.75rem;">

            <a href="{{ url_for('dashboard.dna_profile') }}" class="btn">Typing DNA Profile &rarr;</a>
```

```
<a href="{{ url_for('dashboard.index') }}" class="btn">&larr; Pilot Dashboard</a>

</div>

</div>

<!-- Primary Lifetime Statistics -->

<div class="stats-hud" style="grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));">

    <div class="stat-box">

        <div class="stat-label">Total Words Typed</div>

        <div class="stat-value">{{ total_words }}</div>

    </div>

    <div class="stat-box">

        <div class="stat-label">Hours Logged</div>

        <div class="stat-value">{{ total_hours }}h</div>

    </div>

    <div class="stat-box">

        <div class="stat-label">All-Time Peak WPM</div>

        <div class="stat-value" style="color: var(--success);">{{ best_wpm }}</div>

    </div>

    <div class="stat-box">

        <div class="stat-label">Lifetime Accuracy</div>

        <div class="stat-value">{{ avg_accuracy }}%</div>

    </div>

    <div class="stat-box">

        <div class="stat-label">Velocity Growth</div>

        <div class="stat-value" style="color: {{ 'var(--success)' if improvement_rate >= 0 else 'var(--danger)' }};">

            {{ '+' if improvement_rate > 0 else '' }}{{ improvement_rate }}%

        </div>

    </div>

</div>
```

```
<!-- Chart Tab Selector -->

<div style="display: flex; justify-content: space-between; align-items: center; margin: 2rem 0 1rem; flex-wrap: wrap; gap: 1rem;">

    <h3 id="chart-view-title" style="font-size: 1.2rem; margin: 0;">Net WPM Trajectory</h3>
```

```
<div style="display: flex; gap: 0.5rem;">

  <button id="btn-chart-wpm" onclick="switchChart('wpm')" class="btn btn-primary" style="padding: 0.4rem 0.8rem; font-size: 0.85rem;">Speed (WPM)</button>

  <button id="btn-chart-acc" onclick="switchChart('accuracy')" class="btn" style="padding: 0.4rem 0.8rem; font-size: 0.85rem;">Accuracy (%)</button>

  <button id="btn-chart-con" onclick="switchChart('consistency')" class="btn" style="padding: 0.4rem 0.8rem; font-size: 0.85rem;">Consistency (%)</button>

</div>

</div>
```

```
<!-- SVG Chart Container -->

<div id="analytics-chart" style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius); padding: 1.5rem; min-height: 260px; display: flex; align-items: center; justify-content: center;">

  <span style="color: var(--text-muted); font-size: 0.9rem;">Aggregating sessions...</span>

</div>
```

```
<!-- Telemetry Diagnostic Cards -->

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 1.5rem; margin-top: 2rem;">

  <div class="card" style="margin-bottom: 0; background: var(--bg-card);">

    <h4 style="margin-bottom: 0.75rem; color: var(--accent);">Biomechanical Consistency</h4>

    <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.6;">

      Your average keystroke consistency is <strong>{{ avg_consistency }}%</strong>. A tighter interval variance indicates stable forward cadence without hesitation between complex syllables.

    </p>

  </div>
```

```
<div class="card" style="margin-bottom: 0; background: var(--bg-card);">

  <h4 style="margin-bottom: 0.75rem; color: var(--success);">Pace Progression</h4>

  <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.6;">

    Across {{ total_tests }} verified benchmark runs, your average net velocity is <strong>{{ avg_wpm }} WPM</strong>, representing steady compound gains through muscle memory reinforcement.

  </p>

</div>
```

```
<div class="card" style="margin-bottom: 0; background: var(--bg-card);">
```

```
<h4 style="margin-bottom: 0.75rem; color: var(--warning);">Cognitive Fatigue Limit</h4>
```

```
<p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.6;">
```

Speed drops typically begin appearing past 90 consecutive seconds of intense typing. Use 60-second recovery intervals to sustain peak neurological reaction speed.

```
</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

```
{% block scripts %}
```

```
<script>
```

```
const rawPoints = {{ data_points|tojson|safe }};
```

```
let currentMetric = 'wpm';
```

```
function renderSvgChart(metric, color) {
```

```
  const container = document.getElementById('analytics-chart');
```

```
  if (!container || !rawPoints || !rawPoints.length < 2) {
```

```
    container.innerHTML = '<div style="padding: 3rem; text-align: center; color: var(--text-muted);">Complete at least two tests to populate the progression curve.</div>';
```

```
    return;
```

```
  }
```

```
  const width = container.clientWidth || 800;
```

```
  const height = 240;
```

```
  const padding = 40;
```

```
  const values = rawPoints.map(p => p[metric]);
```

```
  const maxVal = Math.max(...values, metric === 'accuracy' ? 100 : 60);
```

```
  const minVal = Math.min(...values, 0);
```

```
  const getX = (idx) => padding + (idx / (rawPoints.length - 1)) * (width - 2 * padding);
```

```
  const getY = (val) => height - padding - ((val - minVal) / (maxVal - minVal || 1)) * (height - 2 * padding);
```

```
let pathD = `M ${getX(0)} ${getY(values[0])}`;

for (let i = 1; i < values.length; i++) {

  pathD += ` L ${getX(i)} ${getY(values[i])}`;

}

const svg = `

<svg width="100%" height="${height}" viewBox="0 0 ${width} ${height}" preserveAspectRatio="none">

  <line x1="${padding}" y1="${height - padding}" x2="${width - padding}" y2="${height - padding}" stroke="#30363d" stroke-width="1"/>

  <line x1="${padding}" y1="${padding}" x2="${padding}" y2="${height - padding}" stroke="#30363d" stroke-width="1"/>

  <path d="${pathD}" fill="none" stroke="${color}" stroke-width="3" stroke-linecap="round" stroke-linejoin="round"/>

  ${values.map((v, i) => `

    <circle cx="${getX(i)}" cy="${getY(v)}" r="4" fill="${color}"/>

  `).join("")}

</svg>

`;

container.innerHTML = svg;

}
```

```
function switchChart(metric) {

  currentMetric = metric;

  document.getElementById('btn-chart-wpm').className = metric === 'wpm' ? 'btn btn-primary' : 'btn';

  document.getElementById('btn-chart-acc').className = metric === 'accuracy' ? 'btn btn-primary' : 'btn';

  document.getElementById('btn-chart-con').className = metric === 'consistency' ? 'btn btn-primary' : 'btn';

  if (metric === 'wpm') {

    document.getElementById('chart-view-title').textContent = 'Net WPM Trajectory';

    renderSvgChart('wpm', '#58a6ff');

  } else if (metric === 'accuracy') {

    document.getElementById('chart-view-title').textContent = 'Keystroke Accuracy Stability Curve';

    renderSvgChart('accuracy', '#3fb950');

  } else {

    document.getElementById('chart-view-title').textContent = 'Interval Consistency & Regularity (%)';

  }

}
```

```
renderSvgChart('consistency', '#d29922');

}

}
```

```
document.addEventListener('DOMContentLoaded', () => {

  switchChart('wpm');

});

</script>

{% endblock %}
```

```
=== FILE: .\app\templates\dashboard\dna.html ===

{% extends 'base.html' %}
```

```
{% block content %}

<div class="card">

  <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

    <div>

      <h1 style="font-size: 1.8rem; font-weight: 700;">Biometric Typing DNA Profile</h1>

      <p style="color: var(--text-muted); font-size: 0.9rem;">Algorithmic telemetry tracking finger fatigue, bilateral hand balance, and key confusion zones.</p>

    </div>

    <a href="{{ url_for('typing.test_page') }}" onclick="localStorage.setItem('auto_adaptive', 'true')" class="btn btn-primary">

      Launch Targeted Weakness Drill &rarr;

    </a>

  </div>
```

```
<!-- Primary Precision HUD -->

<div class="stats-hud" style="grid-template-columns: repeat(auto-fit, minmax(130px, 1fr));">

  <div class="stat-box">

    <div class="stat-label">Left Hand Balance</div>

    <div class="stat-value" style="color: {{ 'var(--success)' if dna and dna.left_hand_accuracy >= 95 else 'var(--warning)' }};">

      {{ dna.left_hand_accuracy if dna else 98.5 }}%
```

```
</div>

</div>

<div class="stat-box">

  <div class="stat-label">Right Hand Balance</div>

  <div class="stat-value" style="color: {{ 'var(--success)' if dna and dna.right_hand_accuracy >= 95 else 'var(--warning)' }};">

    {{ dna.right_hand_accuracy if dna else 99.1 }}%

  </div>

</div>

<div class="stat-box">

  <div class="stat-label">Punctuation Precision</div>

  <div class="stat-value">{{ dna.punctuation_accuracy if dna else 96.0 }}%</div>

</div>

<div class="stat-box">

  <div class="stat-label">Numeric Row Precision</div>

  <div class="stat-value">{{ dna.numbers_accuracy if dna else 94.2 }}%</div>

</div>

<div class="stat-box">

  <div class="stat-label">Capitals / Shift Latency</div>

  <div class="stat-value">{{ dna.capitals_accuracy if dna else 97.4 }}%</div>

</div>

</div>

<!-- Finger-Level Biomechanical Breakdown -->

<div style="margin: 2rem 0;">

  <h3 style="font-size: 1.2rem; margin-bottom: 1rem;">Finger-Level Strength & Weakness Matrix</h3>

  <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 1rem;">

    <div class="card" style="margin-bottom: 0; background: var(--bg-card);">

      <div style="font-weight: 600; margin-bottom: 0.5rem;">Left Pinky (Q, A, Z)</div>

      <div style="font-size: 1.3rem; font-family: var(--font-mono); color: var(--warning);">91.2%</div>

      <div style="font-size: 0.75rem; color: var(--text-muted); margin-top: 0.25rem;">High strain detected on outer column.</div>

    </div>

    <div class="card" style="margin-bottom: 0; background: var(--bg-card);">

      <div style="font-weight: 600; margin-bottom: 0.5rem;">Left Index (R, T, F, G, V, B)</div>
```

```
<div style="font-size: 1.3rem; font-family: var(--font-mono); color: var(--success);">98.4%</div>

<div style="font-size: 0.75rem; color: var(--text-muted); margin-top: 0.25rem;">Fastest transition speed.</div>

</div>

<div class="card" style="margin-bottom: 0; background: var(--bg-card);">

  <div style="font-weight: 600; margin-bottom: 0.5rem;">Right Index (Y, U, H, J, N, M)</div>

  <div style="font-size: 1.3rem; font-family: var(--font-mono); color: var(--success);">99.0%</div>

  <div style="font-size: 0.75rem; color: var(--text-muted); margin-top: 0.25rem;">Optimal rhythm and cadence.</div>

</div>

<div class="card" style="margin-bottom: 0; background: var(--bg-card);">

  <div style="font-weight: 600; margin-bottom: 0.5rem;">Right Pinky (P, ,, /, Enter)</div>

  <div style="font-size: 1.3rem; font-family: var(--font-mono); color: var(--warning);">92.8%</div>

  <div style="font-size: 0.75rem; color: var(--text-muted); margin-top: 0.25rem;">Frequent punctuation hesitation.</div>

</div>

</div>

<!-- Confusion Pairs & Strategic Recommendations -->

<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 2rem; margin-top: 1rem;">

  <div class="card" style="margin-bottom: 0; background: var(--bg-card);">

    <h3 style="margin-bottom: 1rem; font-size: 1.1rem;">Top Keystroke Confusion Pairs</h3>

    {% if confusions %}

      <ul style="list-style: none; display: flex; flex-direction: column; gap: 0.65rem;">

        {% for exp, mapped in confusions.items() %}

          {% for got, count in mapped.items() %}

            <li style="padding: 0.55rem 0.85rem; background: var(--bg-secondary); border-radius: var(--radius); display: flex; justify-content: space-between; align-items: center;">

              <span>Expected <strong>{{ exp|upper }}</strong> &arr; Typed <span style="color: var(--danger); font-weight: 700;">{{ got|upper }}</span></span></span>

              <span style="font-family: var(--font-mono); font-size: 0.85rem; background: rgba(248, 81, 73, 0.15); color: var(--danger); padding: 0.1rem 0.5rem; border-radius: 4px;">{{ count }}x</span>

            </li>

          {% endfor %}

        {% endfor %}

      </ul>
```

```
{% else %}

<p style="color: var(--text-muted); font-size: 0.9rem;">

    No frequent confusion patterns logged yet. Complete more typing sessions to aggregate your error pairs.

</p>

{% endif %}

</div>


<div class="card" style="margin-bottom: 0; background: var(--bg-card);">

    <h3 style="margin-bottom: 1rem; font-size: 1.1rem;">Algorithmic Training Prescription</h3>

    <p style="color: var(--text-muted); line-height: 1.7; font-size: 0.95rem; margin-bottom: 1.5rem;">

        Your telemetry indicates strong central row rhythm with minor delay spikes when transitioning from home row to edge brackets and punctuation keys. Engaging in focused 60-second adaptive drills will eliminate sub-second hesitation patterns.

    </p>

    <a href="{{ url_for('typing.test_page') }}" class="btn btn-primary" style="width: 100%; justify-content: center; padding: 0.75rem;">

        Start Adaptive Prescription Drill

    </a>

</div>

</div>

{% endblock %}


=== FILE: .\app\templates\dashboard\index.html ===

{% extends 'base.html' %}


{% block content %}

<div class="card">

    <!-- Pilot Profile & Competitive Division Banner -->

    <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 1.75rem; flex-wrap: wrap; gap: 1rem;">

        <div>

            <div style="display: flex; align-items: center; gap: 0.6rem;">

                <h1 style="font-size: 1.8rem; font-weight: 800; margin: 0;">Pilot Dashboard: {{ current_user.username }}</h1>

                <span style="font-size: 0.8rem; font-weight: 800; padding: 0.2rem 0.6rem; border-radius: 4px; background: {{ current_user.rank_badge.color }}; color: #04101e; text-transform: uppercase;">
```

```
                {{ current_user.rank_badge.icon }} {{ current_user.rank_division }} &bull; {{ current_user.elo_rating }} MMR

            </span>

        </div>

        <p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.35rem;">

            Member since {{ current_user.created_at.strftime('%B %Y') }} &bull; Competitive Record: <strong style="color: var(--success);">{{ current_user.ranked_wins }}W</strong> - <strong style="color: var(--danger);">{{ current_user.ranked_losses }}L</strong>

        </p>

    </div>


</div>


<div style="display: flex; gap: 0.6rem; flex-wrap: wrap;">

    <a href="{{ url_for('dashboard.achievements_showcase') }}" class="btn" style="border-color: #f59e0b; color: #f59e0b; font-weight: 700;">

        🏆 Trophy Hall ({{ achievements|length }})

    </a>

    <a href="{{ url_for('typing.practice_page') }}" class="btn" style="border-color: var(--accent); color: var(--accent);">

        🎓 Academy & Practice &rarr;

    </a>

    <a href="{{ url_for('dashboard.analytics') }}" class="btn btn-primary">

        Full Analytics &rarr;

    </a>

</div>

</div>


</div>


<!-- Primary Lifetime Statistics HUD -->

<div class="stats-hud" style="grid-template-columns: repeat(auto-fit, minmax(130px, 1fr));">

    <div class="stat-box">

        <div class="stat-label">Tests Completed</div>

        <div class="stat-value">{{ total_tests }}</div>

    </div>

    <div class="stat-box">

        <div class="stat-label">Average Speed</div>

        <div class="stat-value">{{ avg_wpm }} <span style="font-size: 0.8rem; color: var(--text-muted);">WPM</span></div>

    </div>

    <div class="stat-box">
```

```
<div class="stat-label">Personal Best</div>

<div class="stat-value" style="color: var(--success);">{{ best_wpm }} <span style="font-size: 0.8rem; color: var(--text-muted);">WPM</span></div>

</div>

<div class="stat-box">

  <div class="stat-label">Average Accuracy</div>

  <div class="stat-value">{{ avg_accuracy }}%</div>

</div>

<div class="stat-box">

  <div class="stat-label">Typing Time</div>

  <div class="stat-value">{{ total_time_minutes }}m</div>

</div>

</div>
```

```
<!-- Content Grid: Solo Runs vs Multiplayer Race History -->

<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; margin-top: 1.5rem;">

  <!-- 1. Solo Verified Runs -->

  <div class="card" style="margin-bottom: 0;">

    <h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Recent Benchmark Runs</h3>

    <div class="table-responsive-container">

      <table style="width: 100%; border-collapse: collapse; text-align: left;">

        <thead>

          <tr>

            <th>Date</th>

            <th>Mode</th>

            <th>Speed</th>

            <th>Acc</th>

            <th>Action</th>

          </tr>

        </thead>

        <tbody>

          {% for t in recent_tests %}

            <tr>
```

```
<td style="color: var(--text-muted);">{{ t.completed_at.strftime('%m-%d %H:%M') }}</td>

<td><span style="background: var(--bg-card); padding: 0.2rem 0.4rem; border-radius: 4px; font-size: 0.75rem;">{{ t.mode|capitalize }}</span></td>

<td style="font-weight: 700; color: var(--accent); font-family: var(--font-mono);">{{ t.wpm }} WPM</td>

<td>{{ t.accuracy }}%</td>

<td>

  <a href="{{ url_for('typing.result_page', test_id=t.id) }}" class="btn btn-sm">Audit</a>

</td>

</tr>

{% else %}

<tr>

  <td colspan="5" style="padding: 2rem; text-align: center; color: var(--text-muted);">No benchmark runs logged yet.</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>
```

```
<!-- 2. Multiplayer Competitive History -->

<div class="card" style="margin-bottom: 0;">

  <h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Multiplayer Race History</h3>

  <div class="table-responsive-container">

    <table style="width: 100%; border-collapse: collapse; text-align: left;">

      <thead>

        <tr>

          <th>Date</th>

          <th>Duration</th>

          <th>Speed</th>

          <th>Acc</th>

          <th>Action</th>

        </tr>

      </thead>
```



```
<tbody>

  {% for m in multiplayer_tests %}

<tr>

  <td style="color: var(--text-muted);">{{ m.completed_at.strftime('%m-%d %H:%M') }}</td>

  <td>{{ m.duration }}s</td>

  <td style="font-weight: 700; color: var(--success); font-family: var(--font-mono);">{{ m.wpm }} WPM</td>

  <td>{{ m.accuracy }}%</td>

  <td>

    <a href="{{ url_for('typing.result_page', test_id=m.id) }}" class="btn btn-sm">Audit</a>

  </td>

</tr>

{% else %}

<tr>

  <td colspan="5" style="padding: 2rem; text-align: center; color: var(--text-muted);">

    No multiplayer races logged yet. Visit the <a href="{{ url_for('multiplayer.index') }}" style="color: var(--accent);">Multiplayer Arena</a>!

  </td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

</div>

</div>

{% endblock %}
```

=== FILE: .\app\templates\dashboard\profile.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width: 960px; margin: 0 auto;">

<!-- Pilot Profile Header -->

<div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.5rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

<div style="display: flex; align-items: center; gap: 1.25rem;">

<div style="width: 64px; height: 64px; border-radius: 50%; background: var(--bg-card); border: 2px solid var(--accent); display: flex; align-items: center; justify-content: center; font-size: 1.8rem; font-weight: 800; color: var(--accent);">

{{ profile_user.username[0]|upper }}

</div>

<div>

<div style="display: flex; align-items: center; gap: 0.5rem;">

<h1 style="font-size: 1.8rem; font-weight: 700; margin: 0;">{{ profile_user.username }}</h1>

{% if profile_user.is_admin %}

ADMIN

{% endif %}

</div>

<p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.25rem;">

Pilot on TypeSphere since {{ profile_user.created_at.strftime('%B %Y') }}

</p>

</div>

</div>

<!-- Ghost Race CTA -->

{% if best_test %}

🏁 Race {{ profile_user.username }}'s Ghost ({{ best_wpm }} WPM)

{% endif %}

</div>

<!-- Primary Stats Grid -->

<div class="stats-hud" style="grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));">

<div class="stat-box">

<div class="stat-label">Personal Best</div>

```
<div class="stat-value" style="color: var(--success);">{{ best_wpm }} <span style="font-size: 0.8rem; color: var(--text-muted);">WPM</span></div>

</div>

<div class="stat-box">

  <div class="stat-label">Average Speed</div>

  <div class="stat-value">{{ avg_wpm }} <span style="font-size: 0.8rem; color: var(--text-muted);">WPM</span></div>

</div>

<div class="stat-box">

  <div class="stat-label">Average Accuracy</div>

  <div class="stat-value">{{ avg_accuracy }}%</div>

</div>

<div class="stat-box">

  <div class="stat-label">Verified Tests</div>

  <div class="stat-value">{{ total_tests }}</div>

</div>

</div>
```

```
<!-- Biometric DNA & Badges Grid -->

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)); gap: 1.5rem; margin-top: 1.5rem;">

  <!-- Biomechanical Hand Balance -->

  <div class="card" style="margin-bottom: 0; background: var(--bg-card);">

    <h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Biomechanical Hand Telemetry</h3>

    <div style="display: flex; flex-direction: column; gap: 0.75rem; font-size: 0.95rem;">

      <div style="display: flex; justify-content: space-between;">

        <span style="color: var(--text-muted);">Left Hand Balance:</span>

        <strong>{{ dna.left_hand_accuracy if dna else 98.0 }}%</strong>

      </div>

      <div style="display: flex; justify-content: space-between;">

        <span style="color: var(--text-muted);">Right Hand Balance:</span>

        <strong>{{ dna.right_hand_accuracy if dna else 98.5 }}%</strong>

      </div>

      <div style="display: flex; justify-content: space-between;">

        <span style="color: var(--text-muted);">Punctuation Precision:</span>
```

```
<strong>{{ dna.punctuation_accuracy if dna else 95.0 }}%</strong>

</div>

<div style="display: flex; justify-content: space-between;">

  <span style="color: var(--text-muted);">Numeric Row Cadence:</span>

  <strong>{{ dna.numbers_accuracy if dna else 94.0 }}%</strong>

</div>

</div>

<!-- Trophy Case -->

<div class="card" style="margin-bottom: 0; background: var(--bg-card);">

  <h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Unlocked Trophy Case ({{ achievements | length }})</h3>

  <div style="display: flex; flex-direction: column; gap: 0.65rem;">

    {% for ua in achievements %}

      <div style="display: flex; align-items: center; justify-content: space-between; background: var(--bg-secondary); padding: 0.6rem 0.85rem; border-radius: var(--radius); border-left: 3px solid var(--accent);">

        <span style="font-weight: 600; font-size: 0.9rem;">🏆 {{ ua.achievement.title }}</span>

        <span style="font-size: 0.75rem; color: var(--text-muted);">{{ ua.unlocked_at.strftime('%b %d, %Y') }}</span>

      </div>

      {% else %}

      <p style="color: var(--text-muted); font-size: 0.85rem;">No trophy badges unlocked yet.</p>

      {% endfor %}

    </div>

  </div>

  {% endblock %}

=== FILE: .\app\templates\feedback\contact.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width: 680px; margin: 2rem auto;">
```

```
<div style="border-bottom: 1px solid var(--border-color); padding-bottom: 1rem; margin-bottom: 1.5rem;">

  <h1 style="font-size: 1.75rem; font-weight: 800; margin: 0;">Contact TypeSphere</h1>

  <p style="color: var(--text-muted); font-size: 0.88rem; margin-top: 0.25rem;">

    Inquiries, partnership proposals, and technical support requests.

  </p>

</div>

<form method="POST" action="{{ url_for('feedback.contact') }}">

  <div style="margin-bottom: 1.25rem;">

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Full Name</label>

    <input type="text" name="name" required style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

  </div>

  <div style="margin-bottom: 1.25rem;">

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Email Address</label>

    <input type="email" name="email" required style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

  </div>

  <div style="margin-bottom: 1.25rem;">

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Subject</label>

    <input type="text" name="subject" required style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

  </div>

  <div style="margin-bottom: 1.5rem;">

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Message</label>

    <textarea name="message" rows="5" required style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary); font-family: inherit; line-height: 1.5;"></textarea>

  </div>

  <button type="submit" class="btn btn-primary" style="padding: 0.75rem 2rem; font-weight: 700;">

    Send Message &rarr;

  </button>

</form>
```

```
</button>

</form>

</div>

{% endblock %}

=== FILE: .\app\templates\feedback\feedback.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width: 860px; margin: 2rem auto;">

  <div style="border-bottom: 1px solid var(--border-color); padding-bottom: 1rem; margin-bottom: 1.5rem;">

    <h1 style="font-size: 1.75rem; font-weight: 800; margin: 0;">Platform Feedback</h1>

    <p style="color: var(--text-muted); font-size: 0.88rem; margin-top: 0.25rem;">

      Help us improve TypeSphere. Submit bug reports, UI suggestions, and feature requests.

    </p>

  </div>

  {% if not current_user.is_authenticated %}

  <div class="alert-box alert-warning" style="margin-bottom: 1.5rem;">

    <div>

      <strong>Sign-In Required:</strong> You must be signed in to submit platform feedback and track engineer responses.

    </div>

    <a href="{{ url_for('auth.login', next=url_for('feedback.submit_feedback')) }}" class="btn btn-primary btn-sm">Sign In</a>

  </div>

  {% else %}

  <form method="POST" action="{{ url_for('feedback.submit_feedback') }}" style="margin-bottom: 2rem;">

    <div style="display: grid; grid-template-columns: 1fr 2fr; gap: 1rem; margin-bottom: 1.25rem;">

      <div>

        <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Category</label>

        <select name="category" style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

          <option value="Bug Report">Bug Report</option>

          <option value="Typing Test">Typing Test & Physics</option>

        </select>

      </div>

    </div>

  </form>

  <div style="margin-bottom: 1.5rem;">

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Feedback</label>

    <input type="text" name="feedback" required style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

  </div>

  <div>

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Comments</label>

    <input type="text" name="comments" required style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

  </div>

  <div>

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Rating</label>

    <input type="radio" name="rating" value="1" style="display: inline-block; width: 10px; height: 10px; border: 1px solid var(--border-color); border-radius: 50%; background: var(--bg-card); margin-right: 5px;"> 1

    <input type="radio" name="rating" value="2" style="display: inline-block; width: 10px; height: 10px; border: 1px solid var(--border-color); border-radius: 50%; background: var(--bg-card); margin-right: 5px;"> 2

    <input type="radio" name="rating" value="3" style="display: inline-block; width: 10px; height: 10px; border: 1px solid var(--border-color); border-radius: 50%; background: var(--bg-card); margin-right: 5px;"> 3

    <input type="radio" name="rating" value="4" style="display: inline-block; width: 10px; height: 10px; border: 1px solid var(--border-color); border-radius: 50%; background: var(--bg-card); margin-right: 5px;"> 4

    <input type="radio" name="rating" value="5" style="display: inline-block; width: 10px; height: 10px; border: 1px solid var(--border-color); border-radius: 50%; background: var(--bg-card); margin-right: 5px;"> 5

  </div>

  <div>

    <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Submit</label>

    <input type="submit" value="Submit" style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

  </div>

</div>

</div>
```

```
<option value="UI/UX">UI & Ergonomics</option>

<option value="Multiplayer">Multiplayer & Races</option>

<option value="Feature Request">Feature Request</option>

</select>

</div>

<div>

  <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Summary Title</label>

  <input type="text" name="title" required placeholder="e.g. Minor delay on punctuation transition..." style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary);">

</div>

</div>

<div style="margin-bottom: 1.5rem;">

  <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Detailed Feedback</label>

  <textarea name="message" rows="4" required placeholder="Describe what occurred or what feature you would love to see..." style="width: 100%; padding: 0.65rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary); font-family: inherit; line-height: 1.5;"></textarea>

</div>

<button type="submit" class="btn btn-primary" style="padding: 0.65rem 1.5rem; font-weight: 700;">

  Submit Feedback

</button>

</form>
```

```
<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Your Past Submissions</h3>
```

```
<div class="table-responsive-container">
```

```
<table>

<thead>

  <tr>

    <th>Category</th>

    <th>Summary</th>

    <th>Status</th>
```

```
<th>Response</th>

</tr>

</thead>

<tbody>

  {% for fb in user_submissions %}

  <tr>

    <td><span class="btn btn-sm" style="padding: 0.15rem 0.4rem; font-size: 0.75rem;">{{ fb.category }}</span></td>

    <td><strong>{{ fb.title }}</strong></td>

    <td><span style="font-weight: 700; font-size: 0.8rem; color: var(--accent);">{{ fb.status }}</span></td>

    <td>{{ fb.admin_response or 'Pending review' }}</td>

  </tr>

  {% else %}

  <tr>

    <td colspan="4" style="text-align: center; color: var(--text-muted); padding: 1.5rem;">No previous submissions recorded.</td>

  </tr>

  {% endfor %}

</tbody>

</table>

</div>

{% endif %}

</div>

{% endblock %}
```

```
=== FILE: .\app\templates\feedback\ratings.html ===
```

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<div class="card" style="max-width: 920px; margin: 2rem auto;">
```

```
<div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1rem; margin-bottom: 1.5rem; flex-wrap: wrap; gap: 1rem;">
```

```
<div>
```

```
<h1 style="font-size: 1.8rem; font-weight: 800; margin: 0;">Community Ratings & Reviews</h1>
```

```
<p style="color: var(--text-muted); font-size: 0.88rem; margin-top: 0.25rem;">
```

```
Overall Community Rating: <strong style="color: var(--warning); font-size: 1.1rem;">★ {{ avg_rating }} / 5.0</strong> {{{ total_reviews }}
verified reviews)

</p>

</div>

</div>

{% if not current_user.is_authenticated %}

<div class="alert-box alert-warning" style="margin-bottom: 2rem;">

<div>

<strong>Authentication Required:</strong> Please sign in to leave a platform rating & review.

</div>

<a href="{{ url_for('auth.login', next=url_for('feedback.ratings')) }}" class="btn btn-primary btn-sm">Sign In</a>

</div>

{% else %}

<!-- Review Submission / Update Box -->

<div class="card" style="background: var(--bg-card); margin-bottom: 2rem;">

<h3 style="font-size: 1.15rem; margin-bottom: 0.85rem;">

{{ 'Update Your Review' if user_review else 'Rate TypeSphere' }}

</h3>

<form method="POST" action="{{ url_for('feedback.ratings') }}">

<div style="display: grid; grid-template-columns: 140px 1fr; gap: 1rem; margin-bottom: 1.25rem;">

<div>

<label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Score</label>

<select name="rating" style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--warning); font-weight: 700;">

<option value="5" {% if user_review and user_review.rating == 5 %}selected{% endif %}>★★★★★ (5/5)</option>

<option value="4" {% if user_review and user_review.rating == 4 %}selected{% endif %}>★★★★☆ (4/5)</option>

<option value="3" {% if user_review and user_review.rating == 3 %}selected{% endif %}>★★★☆☆ (3/5)</option>

<option value="2" {% if user_review and user_review.rating == 2 %}selected{% endif %}>★★☆☆☆ (2/5)</option>

<option value="1" {% if user_review and user_review.rating == 1 %}selected{% endif %}>★☆☆☆☆ (1/5)</option>

</select>

</div>
```

```
<div>

<label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Review Title</label>

<input type="text" name="review_title" required value="{{ user_review.review_title if user_review else '' }}" placeholder="e.g. Best typing
platform for developers..." style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-
radius: var(--radius-sm); color: var(--text-primary);">

</div>

</div>

<div style="margin-bottom: 1.25rem;">

<label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-muted); margin-bottom: 0.35rem;">Your Review
Experience</label>

<textarea name="review_text" rows="3" required placeholder="What do you think of the typing test, multiplayer arena, and biometrics?"
style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm);
color: var(--text-primary); line-height: 1.5;">{{ user_review.review_text if user_review else '' }}</textarea>

</div>

<button type="submit" class="btn btn-primary" style="padding: 0.65rem 1.5rem; font-weight: 700;">

{{ 'Update My Rating' if user_review else 'Publish Rating' }}

</button>

</form>

</div>

{% endif %}

<!-- Reviews Feed -->

<h3 style="font-size: 1.15rem; margin-bottom: 1rem;">Recent Community Reviews</h3>

<div style="display: flex; flex-direction: column; gap: 1rem;">

{% for r in reviews %}

<div style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-sm); padding: 1rem 1.25rem;">

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.35rem;">

<div>

<span style="color: var(--warning); font-size: 1rem; margin-right: 0.5rem;">

{% for _ in range(r.rating) %}★{% endfor %}{% for _ in range(5 - r.rating) %}☆{% endfor %}

</span>

<strong>{{ r.review_title }}</strong>
```



```
</button>

{% endif %}

{% if 'Multiplayer' in game.supports|join(' ') %}

<button onclick="setPlayMode('multiplayer', this)" class="btn" style="justify-content: flex-start;">

    🏁 Multi-Racer Room

</button>

{% endif %}

</div>

</div>

<!-- 2. Difficulty & AI Configuration -->

<div class="card" style="background: var(--bg-card); margin-bottom: 0;">

    <h3 style="font-size: 0.95rem; font-weight: 700; margin-bottom: 0.75rem;">2. Difficulty & Opponents</h3>

    <div style="display: flex; flex-direction: column; gap: 0.85rem;">

        <div>

            <label style="display: block; font-size: 0.75rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.3rem;">Difficulty Tier</label>

            <select id="setup-difficulty" class="form-select" style="width: 100%;">

                <option value="easy">Easy (Slower pacing, simple lexicon)</option>

                <option value="moderate" selected>Moderate (Standard human speed)</option>

                <option value="hard">Hard (Rapid pace, advanced vocabulary)</option>

                <option value="expert">Expert (Esports velocity, complex targets)</option>

            </select>

        </div>

        <div id="setup-objective-wrap">

            <label style="display: block; font-size: 0.75rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.3rem;">Target Objective</label>

            <select id="setup-objective-val" class="form-select" style="width: 100%;">

                {% for opt in game.objective_options %}

                    <option value="{{ opt.val }}" {% if loop.first %}selected{% endif %}>{{ opt.label }}</option>

                {% endfor %}

            </select>

        </div>

    </div>

</div>
```

```
</div>

<div id="setup-room-code-wrap" style="display: none;">

    <label style="display: block; font-size: 0.75rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.3rem;">Room Code</label>

    <input type="text" id="setup-room-code" placeholder="ENTER 4-DIGIT CODE" style="width: 100%; padding: 0.5rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; font-family: var(--font-mono); text-transform: uppercase;">

</div>

</div>

</div>

<!-- 3. Typing Mechanics (Blind Mode & Backspace) -->

<div class="card" style="background: var(--bg-card); margin-bottom: 0;">

    <h3 style="font-size: 0.95rem; font-weight: 700; margin-bottom: 0.75rem;">3. Typing Controls</h3>

    <div style="display: flex; flex-direction: column; gap: 0.75rem;">

        <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.4rem 0;">

            <div>

                <div style="font-weight: 700; font-size: 0.85rem;">Blind Mode</div>

                <div style="font-size: 0.75rem; color: var(--text-muted);">Disables visual error masking</div>

            </div>

            <button onclick="toggleSetupToggle('blind')" id="toggle-blind-btn" class="btn btn-sm">OFF</button>

        </div>

        <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.4rem 0; border-top: 1px solid var(--border-color);">

            <div>

                <div style="font-weight: 700; font-size: 0.85rem;">Backspace Allowance</div>

                <div style="font-size: 0.75rem; color: var(--text-muted);">Allow correcting typos</div>

            </div>

            <button onclick="toggleSetupToggle('backspace')" id="toggle-backspace-btn" class="btn btn-sm active-choice">ON</button>

        </div>

        {% if game.slug == 'bubble_pop' %}

            <div style="border-top: 1px solid var(--border-color); padding-top: 0.6rem;">

            </div>

        {% endif %}

    </div>

</div>
```

```
<label style="display:block; font-size:0.75rem; text-transform:uppercase; color:var(--text-muted); font-weight:700; margin-bottom:0.3rem;">Target Character Category</label>

<select id="setup-bubble-char-mode" class="form-select" style="width:100%;">

  <option value="letters">Lowercase Letters (a-z)</option>

  <option value="uppercase">Uppercase Letters (A-Z)</option>

  <option value="numbers">Numeric Digits (0-9)</option>

  <option value="symbols">Symbols & Punctuation (!@#$%...)</option>

  <option value="letters_numbers">Letters + Numbers</option>

  <option value="mixed_all">Mixed All Characters</option>

  {% if weak_keys %}<option value="weak_keys">Target My Weak Keys ({{ weak_keys |join(' ', ' ') }})</option>{% endif %}

</select>

</div>

{% endif %}

</div>

</div>

<!-- Ready Screen Summary & Launch -->

<div style="text-align:center;">

  <button onclick="startArcadeCountdown()" class="btn btn-primary" style="padding:0.85rem 3.5rem; font-size:1.1rem; font-weight:800; box-shadow:0 0 25px var(--accent-glow);">

    🎮Enter Arena & Start (3..2..1)

  </button>

</div>

</div>

<!-- =====

PHASE 2: LIVE GAMEPLAY CANVAS & OPPONENT TRACKS

===== -->

<div id="game-active-arena" style="display:none;">

<!-- Live Match Telemetry Header -->

<div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:0.75rem; margin-bottom:1rem; flex-wrap:wrap; gap:0.5rem;">

  <div style="display:flex; align-items:center; gap:0.6rem;">
```

```
<span style="font-size:1.6rem;">{{ game.icon }}</span>

<strong id="live-header-title" style="font-size:1.15rem;">{{ game.title }}</strong>

</div>

<!-- Live Opponents Progress -->

<div id="live-opponents-roster" style="display:flex; gap:1.25rem; align-items:center; font-size:0.88rem;">

  <div>You: <strong id="live-my-stat" style="color:var(--accent); font-family:var(--font-mono);">0 pts</strong> (<span id="live-my-wpm">0 WPM</span></div>

  <div id="live-opp-wrapper">Opponent: <strong id="live-opp-stat" style="color:var(--warning); font-family:var(--font-mono);">0 pts</strong> (<span id="live-opp-wpm">0 WPM</span></div>

</div>

<button onclick="abortCurrentGame()" class="btn btn-sm btn-danger">&times; Abort Game</button>

</div>

<!-- Dynamic Game Viewport -->

<div id="arcade-viewport" tabindex="0"></div>

<!-- Feedback & Combo Tracker -->

<div style="display:flex; justify-content:space-between; align-items:center; margin-top:0.75rem; font-size:0.82rem; color:var(--text-muted);">

  <span id="gameplay-objective-hint">WIN: {{ game.win_condition }}</span>

  <span>Streak Combo: <strong id="live-combo-badge" style="color:var(--accent); font-family:var(--font-mono); font-size:0.95rem;">0x</strong></span>

</div>

</div>

<!-- =====

PHASE 3: DEFINITIVE RESULTS SYSTEM

===== -->

<div id="game-results-screen" style="display:none; text-align:center; padding:1.5rem 0;">

  <div id="res-outcome-badge" style="display:inline-block; font-size:1rem; font-weight:900; padding:0.35rem 1rem; border-radius:6px; margin-bottom:0.5rem; text-transform:uppercase;">

    FINISHED

  </div>
```



```
<div id="res-trophy-symbol" style="font-size:4rem; margin-bottom:0.25rem;">🏆</div>

<h2 id="res-headline-title" style="font-size:2rem; font-weight:800; color:var(--accent); margin-bottom:0.25rem;">Match Concluded</h2>

<p id="res-motivational-text" style="color:var(--text-secondary); font-size:0.95rem; max-width:620px; margin:0 auto 1.5rem; line-height:1.5;">

  Game session completed.

</p>


<!-- Essential Result Statistics -->

<div class="stats-hud" style="grid-template-columns:repeat(auto-fit, minmax(120px, 1fr)); max-width:840px; margin:0 auto 1.75rem;">

  <div class="stat-box">

    <div class="stat-label">Final Score</div>

    <div class="stat-value" id="res-score" style="color:var(--accent);">0</div>

  </div>

  <div class="stat-box">

    <div class="stat-label">Net Speed</div>

    <div class="stat-value" id="res-wpm">0 WPM</div>

  </div>

  <div class="stat-box">

    <div class="stat-label">Accuracy</div>

    <div class="stat-value" id="res-acc">100%</div>

  </div>

  <div class="stat-box">

    <div class="stat-label">Errors</div>

    <div class="stat-value" id="res-errors" style="color:var(--danger);">0</div>

  </div>

  <div class="stat-box">

    <div class="stat-label">Max Combo</div>

    <div class="stat-value" id="res-combo" style="color:var(--warning);">0x</div>

  </div>

</div>


<!-- Game-Specific Telemetry Box -->

<div class="card" style="background:var(--bg-card); max-width:680px; margin:0 auto 1.75rem; text-align:left; padding:1rem 1.25rem;">
```

```
<h4 style="font-size:0.85rem; text-transform:uppercase; color:var(--accent); margin-bottom:0.6rem; letter-spacing:0.05em;">Telemetry
Audit</h4>

<div id="res-specific-stats" style="display:flex; flex-direction:column; gap:0.4rem; font-size:0.88rem;"></div>

</div>


<!-- Actions -->

<div style="display:flex; justify-content:center; gap:0.75rem; flex-wrap:wrap;">

  <button onclick="startArcadeCountdown()" class="btn btn-primary" style="padding:0.75rem 1.75rem; font-weight:800;">

    🎮 Play Again (Space)

  </button>

  <button onclick="openPreGameSetup()" class="btn" style="padding:0.75rem 1.25rem;">

    ⏏️ Modify Rules

  </button>

  <a href="{{ url_for('games.index') }}" class="btn" style="padding:0.75rem 1.25rem;">

    Arcade Directory

  </a>

  {% if current_user.is_authenticated %}

  <a href="{{ url_for('typing.practice_page') }}" class="btn" style="border-color:var(--accent); color:var(--accent); padding:0.75rem 1.25rem;">

    Train Weaknesses &rarr;

  </a>

  {% endif %}

</div>

</div>


<div>

  <link rel="stylesheet" href="{{ url_for('static', filename='css/arcade.css') }}">

  {% endblock %}

  {% block scripts %}

  <script src="https://cdn.socket.io/4.7.2/socket.io.min.js"></script>

  <script src="{{ url_for('static', filename='js/arcade_runtime.js') }}"></script>

  <script>
```

```
const GAME_SLUG = "{{ game.slug }}";

let selectedPlayMode = 'solo_ai';

let isBlindMode = false;

let isBackspaceAllowed = true;

function setPlayMode(mode, btn) {

    selectedPlayMode = mode;

    document.querySelectorAll("#setup-play-modes .btn").forEach(b => b.classList.remove('active-choice'));

    btn.classList.add('active-choice');

    const roomWrap = document.getElementById('setup-room-code-wrap');

    const oppWrap = document.getElementById('live-opp-wrapper');

    if (roomWrap) roomWrap.style.display = (mode === '1v1_private' || mode === 'multiplayer') ? 'block' : 'none';

    if (oppWrap) oppWrap.style.display = (mode === 'solo_practice') ? 'none' : 'block';

}

function toggleSetupToggle(type) {

    if (type === 'blind') {

        isBlindMode = !isBlindMode;

        const btn = document.getElementById('toggle-blind-btn');

        btn.textContent = isBlindMode ? 'ON' : 'OFF';

        btn.classList.toggle('active-choice', isBlindMode);

    } else if (type === 'backspace') {

        isBackspaceAllowed = !isBackspaceAllowed;

        const btn = document.getElementById('toggle-backspace-btn');

        btn.textContent = isBackspaceAllowed ? 'ON' : 'OFF';

        btn.classList.toggle('active-choice', isBackspaceAllowed);

    }

}

function openPreGameSetup() {

    document.getElementById('game-setup-screen').style.display = 'block';

    document.getElementById('game-active-arena').style.display = 'none';

}
```

```
document.getElementById('game-results-screen').style.display = 'none';

}

function startArcadeCountdown() {

    const diff = document.getElementById('setup-difficulty').value;

    const objVal = document.getElementById('setup-objective-val') ? document.getElementById('setup-objective-val').value : '60';

    const charMode = document.getElementById('setup-bubble-char-mode') ? document.getElementById('setup-bubble-char-mode').value : 'letters';

    const code = document.getElementById('setup-room-code') ? document.getElementById('setup-room-code').value.trim() : "";

    document.getElementById('game-setup-screen').style.display = 'none';

    document.getElementById('game-results-screen').style.display = 'none';

    document.getElementById('game-active-arena').style.display = 'block';

    window.arcadeEngine.start({

        slug: GAME_SLUG,

        playMode: selectedPlayMode,

        difficulty: diff,

        objectiveVal: objVal,

        blindMode: isBlindMode,

        backspaceAllowed: isBackspaceAllowed,

        charMode: charMode,

        roomCode: code

    });

}

function abortCurrentGame() {

    window.arcadeEngine.stop();

    openPreGameSetup();

}

</script>

{% endblock %}

=== FILE: .\app\templates\games\bubble_pop.html ===
```

```
{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:880px; margin:0 auto; text-align:center;">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.25rem;">

    <h2 style="font-size:1.4rem; font-weight:800; margin:0;">🍁 Bubble Pop Arena</h2>

    <div style="display:flex; gap:1.2rem; align-items:center;">

      <span>Score: <strong id="bubble-score" style="color:var(--accent); font-size:1.25rem;">0</strong></span>

      <span>Lives: <strong id="bubble-lives" style="color:var(--danger); font-size:1.25rem;">♥️❏❏❏❏</strong></span>

    <a href="{{ url_for('games.index') }}" class="btn btn-sm">&times; Exit</a>

  </div>

</div>

<div id="bubble-stage" style="position:relative; width:100%; height:400px; background:var(--bg-secondary); border:2px solid var(--border-color); border-radius:var(--radius); overflow:hidden; outline:none;">

  <div style="position:absolute; top:30px; left:0; right:0; height:2px; background:var(--danger); opacity:0.5;"></div>

</div>

<div style="margin-top:0.75rem; font-size:0.85rem; color:var(--text-muted);">

  Type the word inside any bubble to pop it before it hits the top ceiling.

</div>

</div>

{% endblock %}

{% block scripts %}

<script>

const BUBBLE_WORDS = ["pop", "float", "drift", "breeze", "light", "wave", "glide", "swift", "clear", "spark", "fresh", "shine", "glow", "soar"];

let bScore = 0;

let bLives = 3;

let bubbles = [];

let spawnTimer = null;

let floatTimer = null;
```

```
document.addEventListener('DOMContentLoaded', () => {

  spawnTimer = setInterval(createBubble, 1900);

  floatTimer = setInterval(floatBubbles, 40);

  window.addEventListener('keydown', (e) => {

    if (bLives <= 0) return;

    if (e.key.length !== 1 || e.ctrlKey || e.metaKey || e.altKey) return;

    const char = e.key.toLowerCase();

    const candidate = bubbles.find(b => b.text.startsWith(b.typed + char));

    if (candidate) {

      candidate.typed += char;

      candidate.el.innerHTML = `<span style="color:var(--success); text-decoration:underline;">${candidate.typed}</span>${candidate.text.slice(candidate.typed.length)}</div>

      if (candidate.typed === candidate.text) {

        popBubble(candidate);

      }

      if (window.soundEngine) window.soundEngine.playKey(false);

    } else {

      if (window.soundEngine) window.soundEngine.playKey(true);

    }

  });

});

function createBubble() {

  const stage = document.getElementById('bubble-stage');

  if (!stage || bLives <= 0) return;

  const text = BUBBLE_WORDS[Math.floor(Math.random() * BUBBLE_WORDS.length)];

  const el = document.createElement('div');

  el.textContent = text;
```

```
el.style.position = 'absolute';

el.style.left = `${Math.random() * (stage.clientWidth - 100) + 15}px`;

el.style.bottom = '0px';

el.style.padding = '0.75rem 1rem';

el.style.borderRadius = '50px';

el.style.background = 'rgba(56, 189, 248, 0.2)';

el.style.border = '2px solid var(--accent)';

el.style.color = 'var(--text-primary)';

el.style.fontFamily = 'var(--font-mono)';

el.style.fontWeight = '700';

el.style.boxShadow = '0 0 15px var(--accent-glow)';

stage.appendChild(el);


bubbles.push({ el, text, typed: '', y: 0, speed: 1.2 + Math.random() });

}


function floatBubbles() {

  const stage = document.getElementById('bubble-stage');

  if (!stage || bLives <= 0) return;


  for (let i = bubbles.length - 1; i >= 0; i--) {

    const b = bubbles[i];

    b.y += b.speed;

    b.el.style.bottom = `${b.y}px`;


    // Reached top ceiling

    if (b.y >= stage.clientHeight - 60) {

      b.el.remove();

      bubbles.splice(i, 1);

      bLives--;

      updateBubbleLives();

      if (window.soundEngine) window.soundEngine.playKey(true);
```

```
if (bLives <= 0) {

  clearInterval(spawnTimer);

  clearInterval(floatTimer);

  alert('Bubbles burst! Final Score: ${bScore} points. ');

  window.location.href = "{{ url_for('games.index') }}"

  break;

}

}

}

}


function popBubble(b) {

  b.el.remove();

  const idx = bubbles.indexOf(b);

  if (idx !== -1) bubbles.splice(idx, 1);

  bScore += (b.text.length * 15);

  document.getElementById('bubble-score').textContent = bScore;

}


function updateBubbleLives() {

  let h = '';

  for (let i = 0; i < bLives; i++) h += '♥□';

  document.getElementById('bubble-lives').textContent = h || '💀□';

}

</script>

{% endblock %}


=== FILE: .\app\templates\games\cipher_hacker.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:820px; margin:0 auto; text-align:center;">
```

```
<div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.5rem;">

<h2 style="font-size:1.4rem; font-weight:800; margin:0; color:#10b981;">Ⓜ Cipher Terminal Intrusion</h2>

<div style="display:flex; gap:1.2rem; align-items:center;">

  <span>Decrypted: <strong id="cipher-count" style="color:var(--accent); font-size:1.25rem;">0</strong></span>

  <span>Lockdown: <strong id="cipher-countdown" style="color:var(--danger); font-size:1.25rem;">30s</strong></span>

  <a href="{{ url_for('games.index') }}" class="btn btn-sm">&times; Abort</a>

</div>

</div>

<!-- Terminal Screen -->

<div style="background:#05050d; border:2px solid #10b981; border-radius:var(--radius); padding:2rem; margin-bottom:1.5rem; text-align:left; font-family:var(--font-mono); box-shadow:0 0 20px rgba(16, 185, 129, 0.2);">

  <div style="color:#64748b; font-size:0.85rem; margin-bottom:0.75rem;">// SECURE SYSTEM DECRYPTION ACTIVE:</div>

  <div id="cipher-hash-display" style="font-size:1.8rem; font-weight:800; color:#10b981; word-break:break-all; margin-bottom:0.5rem;">

    0x7FA9B2

  </div>

</div>

<input type="text" id="cipher-input" autocomplete="off" autocorrect="off" autocapitalize="off" spellcheck="false" placeholder="Enter exact decryption sequence..." style="width:100%; max-width:480px; padding:0.85rem; background:var(--bg-card); border:2px solid #10b981; border-radius:var(--radius-sm); color:#10b981; font-family:var(--font-mono); font-size:1.2rem; text-align:center; outline:none;">

</div>

{% endblock %}

{% block scripts %}

<script>

const HASHES = ["0x8FA1", "0xDE4B", "0x91C0", "SHA256", "0xAA12", "PORT:443", "0xFF00", "AES-256", "0x00A1", "KEY:994"];

let cDecrypted = 0;

let cTime = 30;

let currentHash = "0x8FA1";

let cTimer = null;

document.addEventListener('DOMContentLoaded', () => {
```

```
const input = document.getElementById('cipher-input');

input.focus();

loadNewHash();

cTimer = setInterval(() => {

  cTime--;

  document.getElementById('cipher-countdown').textContent = `${cTime}s`;

  if (cTime <= 0) {

    clearInterval(cTimer);

    alert('Access denied! Security firewall triggered. Hashes cracked: ${cDecrypted}.');

    window.location.href = "{{ url_for('games.index') }}";

  }

}, 1000);

input.addEventListener('input', () => {

  if (input.value === currentHash) {

    cDecrypted++;

    cTime = Math.min(45, cTime + 4); // Bonus seconds

    document.getElementById('cipher-count').textContent = cDecrypted;

    input.value = "";

    if (window.soundEngine) window.soundEngine.playKey(false);

    loadNewHash();

  }

});

});

function loadNewHash() {

  currentHash = HASHES[Math.floor(Math.random() * HASHES.length)];

  document.getElementById('cipher-hash-display').textContent = currentHash;

}

</script>

{% endblock %}
```

```
=== FILE: .\app\templates\games\falling_words.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:880px; margin:0 auto; text-align:center;">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.25rem;">

    <h2 style="font-size:1.4rem; font-weight:800; margin:0;">⚡️ 🎮 Falling Words Arcade</h2>

    <div style="display:flex; gap:1.2rem; align-items:center;">

      <span>Score: <strong id="game-score" style="color:var(--accent); font-size:1.25rem;">0</strong></span>

      <span>Lives: <strong id="game-lives" style="color:var(--danger); font-size:1.25rem;">♥️🟩🟩🟩🟩🟩</strong></span>

      <a href="{{ url_for('games.index') }}" class="btn btn-sm">&times; Exit</a>

    </div>

  </div>

  <!-- Falling Arena Canvas (No typing box needed: type directly on your keyboard) -->

  <div id="game-canvas" tabindex="0" style="position:relative; width:100%; height:400px; background:var(--bg-secondary); border:2px solid var(--border-color); border-radius:var(--radius); overflow:hidden; outline:none; cursor:default;">

    <div id="danger-line" style="position:absolute; bottom:35px; left:0; right:0; height:2px; background:var(--danger); opacity:0.6;"></div>

    <div id="active-stroke-indicator" style="position:absolute; bottom:8px; left:50%; transform:translateX(-50%); font-family:var(--font-mono); font-size:0.9rem; color:var(--text-muted);">

      Target word: <span id="current-target-match" style="color:var(--accent); font-weight:700;">[Type any falling word]</span>

    </div>

  </div>

  <div style="margin-top:0.75rem; font-size:0.82rem; color:var(--text-muted);">

    No input box required: simply strike the keys of any descending word on your keyboard.

  </div>

</div>

{% endblock %}

{% block scripts %}

<script>
```

```
const WORD_POOL = [

  "velocity", "cadence", "rhythm", "kinetic", "focus", "flow", "precision",

  "matrix", "stream", "quantum", "tactile", "circuit", "binary", "system",

  "engine", "pilot", "balance", "command", "transit", "standard", "channel"

];

let score = 0;

let lives = 3;

let activeWords = []; // [{ id, text, typed: "", el, x, y, speed }]

let spawnInterval = null;

let physicsInterval = null;

let currentTargetWord = null;

let wordsCleared = 0;

document.addEventListener('DOMContentLoaded', () => {

  const arena = document.getElementById('game-canvas');

  arena.focus();

  spawnInterval = setInterval(spawnWord, 2000);

  physicsInterval = setInterval(updatePhysics, 40);

  // Global Direct Keystroke Interceptor

  window.addEventListener('keydown', (e) => {

    if (lives <= 0) return;

    if (e.key.length !== 1 || e.ctrlKey || e.metaKey || e.altKey) return;

    e.preventDefault();

    const char = e.key.toLowerCase();

    // 1. If currently tracking a word, continue typing it

    if (currentTargetWord) {

      const nextExpected = currentTargetWord.text[currentTargetWord.typed.length];

      if (char === nextExpected) {
```

```

currentTargetWord.typed += char;

updateWordVisual(currentTargetWord);

}

if (currentTargetWord.typed === currentTargetWord.text) {

    destroyWord(currentTargetWord);

    currentTargetWord = null;

    updateTargetLabel();

}

if (window.soundEngine) window.soundEngine.playKey(false);

return;

}

}

// 2. Otherwise search for any active falling word starting with this letter

const candidate = activeWords.find(w => w.text.startsWith(char));

if (candidate) {

    currentTargetWord = candidate;

    currentTargetWord.typed = char;

    updateWordVisual(currentTargetWord);

}

if (currentTargetWord.typed === currentTargetWord.text) {

    destroyWord(currentTargetWord);

    currentTargetWord = null;

}

updateTargetLabel();

if (window.soundEngine) window.soundEngine.playKey(false);

} else {

    if (window.soundEngine) window.soundEngine.playKey(true);

}

});

});

function spawnWord() {

```

```

const container = document.getElementById('game-canvas');

if (!container || lives <= 0) return;

const text = WORD_POOL[Math.floor(Math.random() * WORD_POOL.length)];

const el = document.createElement('div');

el.className = 'falling-word-node';

el.textContent = text;

el.style.position = 'absolute';

el.style.left = `${Math.random() * (container.clientWidth - 130) + 15}px`;

el.style.top = '0px';

el.style.fontFamily = 'var(--font-mono)';

el.style.fontWeight = '700';

el.style.fontSize = '1.05rem';

el.style.color = 'var(--text-primary)';

el.style.background = 'var(--bg-card)';

el.style.padding = '0.3rem 0.65rem';

el.style.borderRadius = '5px';

el.style.border = '1.5px solid var(--border-color)';

container.appendChild(el);

activeWords.push({

    id: Math.random(),

    text: text,

    typed: '',

    el: el,

    y: 0,

    speed: 1.2 + Math.random() * 1.3

});

}

function updatePhysics() {

    const container = document.getElementById('game-canvas');

    if (!container || lives <= 0) return;

```

```

for (let i = activeWords.length - 1; i >= 0; i--) {

  const w = activeWords[i];

  w.y += w.speed;

  w.el.style.top = `${w.y}px`;

  // Word crossed boundary without completion

  if (w.y >= container.clientHeight - 40) {

    w.el.remove();

    if (currentTargetWord && currentTargetWord.id === w.id) {

      currentTargetWord = null;

      updateTargetLabel();

    }

    activeWords.splice(i, 1);

    lives--;

    updateLivesDisplay();

    if (window.soundEngine) window.soundEngine.playKey(true);

    if (lives <= 0) {

      concludeGame();

      break;

    }

  }

}

```

```

function updateWordVisual(w) {

  const typedPart = w.text.substring(0, w.typed.length);

  const remaining = w.text.substring(w.typed.length);

  w.el.innerHTML = `${typedPart}</span><span>${remaining}</span>`;

  w.el.style.borderColor = 'var(--accent)';

  w.el.style.boxShadow = '0 0 10px var(--accent-glow)';

}

```

```

function destroyWord(w) {

  w.el.remove();

  const idx = activeWords.findIndex(item => item.id === w.id);

  if (idx !== -1) activeWords.splice(idx, 1);

  score += (w.text.length * 15);

  wordsCleared++;

  document.getElementById('game-score').textContent = score;

}

```

```

function updateTargetLabel() {

  const lbl = document.getElementById('current-target-match');

  lbl.textContent = currentTargetWord ? `${currentTargetWord.text}` : '[Type any falling word]';

}

```

```

function updateLivesDisplay() {

  let h = "";

  for (let i = 0; i < lives; i++) h += '❤️🟩';

  document.getElementById('game-lives').textContent = h || '💀🟩';

}

```

```

function concludeGame() {

  clearInterval(spawnInterval);

  clearInterval(physicsInterval);

  alert(`Game Over! Final Score: ${score} points (${wordsCleared} words eliminated).`);

}

```

```

fetch('/games/api/record-score', {

  method: 'POST',

  headers: { 'Content-Type': 'application/json' },

  body: JSON.stringify({ game_mode: 'falling_words', score: score, words_typed: wordsCleared })

}).finally(() => {

  window.location.href = `${url_for('games.index')}`;

}

```



```
});

}

</script>

{% endblock %}

=== FILE: \app\templates\games\index.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:1160px; margin:0 auto;">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1.25rem; margin-bottom:1.75rem; flex-wrap:wrap; gap:1rem;">

    <div>

      <div style="display:flex; align-items:center; gap:0.6rem;">

        <h1 style="font-size:1.85rem; font-weight:800; margin:0;">TypeSphere Arcade</h1>

        <span style="background:var(--accent-glow); color:var(--accent); font-weight:800; font-size:0.75rem; padding:0.2rem 0.55rem; border-radius:4px;">

          12 DISCIPLINES

        </span>

      </div>

      <p style="color:var(--text-secondary); font-size:0.88rem; margin-top:0.25rem;">

        Competitive and educational gaming engines engineered to build instinctive tactile recognition, agility under pressure, and velocity.

      </p>

    </div>

    <div style="display:flex; gap:0.6rem;">

      <a href="{% url_for('typing.test_page') %}" class="btn">&larr; Free Test Track</a>

      <a href="{% url_for('learn.index') %}" class="btn">Academy &rarr;</a>

    </div>

  </div>

</div>

<!-- 12-Game Arcade Matrix -->

<div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(320px, 1fr)); gap:1.25rem;">
```

```
{% for g in games %}

  <div class="card" style="background:var(--bg-card); border:1px solid var(--border-color); margin-bottom:0; display:flex; flex-direction:column; justify-content:space-between;">

    <div>

      <div style="display:flex; justify-content:space-between; align-items:flex-start; margin-bottom:0.75rem;">

        <div style="display:flex; align-items:center; gap:0.75rem;">

          <span style="font-size:2.3rem;">{{ g.icon }}</span>

          <div>

            <h3 style="font-size:1.2rem; font-weight:800; margin:0; color:var(--text-primary);">{{ g.title }}</h3>

            <span style="font-size:0.75rem; color:var(--accent); font-weight:700; text-transform:uppercase;">{{ g.category }}</span>

          </div>

        </div>

        <span style="font-size:0.68rem; font-weight:800; background:var(--bg-secondary); border:1px solid var(--border-color); padding:0.15rem 0.45rem; border-radius:4px; color:var(--text-secondary);">

          {{ g.badge }}

        </span>

      </div>

      <p style="color:var(--text-secondary); font-size:0.86rem; line-height:1.55; margin-bottom:1rem;">

        {{ g.summary }}

      </p>

    </div>

    <div>

      <div style="background:var(--bg-secondary); border-radius:var(--radius-sm); padding:0.5rem 0.75rem; margin-bottom:0.85rem; font-size:0.78rem; display:flex; justify-content:space-between;">

        <span style="color:var(--text-muted);">Play Modes:</span>

        <strong style="color:var(--text-primary);">{{ g.supports|join(' &bull; ') }}</strong>

      </div>

      <div style="display:flex; justify-content:space-between; align-items:center; gap:0.5rem;">

        {% set pb = high_scores.get(g.slug, 0) %}

        <div style="font-size:0.8rem; color:var(--text-muted);">

          Record: <strong style="color:{{ 'var(--warning)' if pb > 0 else 'var(--text-secondary)' }}; font-family:var(--font-mono);">{{ pb }} pts</strong>

        </div>

      </div>

    </div>

  </div>

{% endfor %}
```

```
</div>

<a href="{url_for('games.play_arena', game_slug=g.slug)}" class="btn btn-primary" style="padding:0.55rem 1.25rem; font-weight:700;">

    Setup Game &rarr;

</a>

</div>

</div>

</div>

{% endfor %}

</div>

</div>

{% endblock %}
```

=== FILE: .\app\templates\games\speed_racer.html ===

{% extends 'base.html' %}

{% block content %}

```
<div class="card" style="max-width:880px; margin:0 auto; text-align:center;">

    <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.25rem;">

        <h2 style="font-size:1.4rem; font-weight:800; margin:0;">🏎️ Speed Racer Time Trial</h2>

        <div style="display:flex; gap:1.2rem; align-items:center;">

            <span>Speed: <strong id="racer-speed" style="color:var(--accent); font-size:1.2rem;">0 WPM</strong></span>

            <span>Progress: <strong id="racer-pct" style="color:var(--success); font-size:1.2rem;">0%</strong></span>

            <a href="{url_for('games.index')}" class="btn btn-sm">&times; Exit</a>

        </div>

    </div>
```

<!-- Race Track View -->

```
<div style="background:var(--bg-secondary); border:2px solid var(--border-color); border-radius:var(--radius); padding:1.5rem; margin-bottom:1.5rem;">

    <div style="margin-bottom:1rem;">

        <div style="display:flex; justify-content:space-between; font-size:0.85rem; margin-bottom:0.3rem;">
```

```
<span>🚗 Your Car</span>

    <span id="player-track-stat">0m / 500m</span>

</div>

<div style="background:var(--bg-card); height:16px; border-radius:8px; overflow:hidden; border:1px solid var(--border-color);">

    <div id="player-car-bar" style="width:0%; height:100%; background:var(--accent); transition:width 0.15s ease;"></div>

</div>

</div>
```

```
<div>

    <div style="display:flex; justify-content:space-between; font-size:0.85rem; margin-bottom:0.3rem;">
```

🏎️ Pace Car (65 WPM)

```
<span id="bot-track-stat">0m / 500m</span>
```

```
</div>

<div style="background:var(--bg-card); height:16px; border-radius:8px; overflow:hidden; border:1px solid var(--border-color);">

    <div id="bot-car-bar" style="width:0%; height:100%; background:var(--warning); transition:width 0.2s ease;"></div>

</div>

</div>

</div>
```

<!-- Word Display & Input -->

```
<div id="racer-passage-display" class="typing-container" style="min-height:130px; margin-bottom:1.25rem; font-family:var(--font-mono); font-size:1.35rem; line-height:2.2;">
```

<!-- Rendered Words -->

```
</div>
```

```
<input type="text" id="racer-input" autocomplete="off" autocorrect="off" autocapitalize="off" spellcheck="false" placeholder="Start typing the passage to throttle forward..." style="width:100%; max-width:520px; padding:0.85rem 1.15rem; background:var(--bg-card); border:2px solid var(--border-color); border-radius:var(--radius-sm); color:var(--text-primary); font-family:var(--font-mono); font-size:1.1rem; outline:none;">
```

```
</div>
```

{% endblock %}

{% block scripts %}

```
<script>
```

```
const RACE_PASSAGE = "The open highway stretched across the desert floor under a wide expanse of pale morning sky. High velocity demands relaxed control and steady breathing. When the throttle opens every second compounds into pure forward momentum.";
```

```
let currentIdx = 0;
```

```
let startTime = null;
```

```
let botProgress = 0;
```

```
let botInterval = null;
```

```
let raceEnded = false;
```

```
document.addEventListener('DOMContentLoaded', () => {
```

```
  renderPassage();
```

```
  const input = document.getElementById('racer-input');
```

```
  input.focus();
```

```
  // Bot paces at steady 65 WPM
```

```
  botInterval = setInterval(() => {
```

```
    if (!startTime || raceEnded) return;
```

```
    botProgress += 0.8;
```

```
    const pct = Math.min(100, Math.round(botProgress));
```

```
    document.getElementById('bot-car-bar').style.width = `${pct}%`;
```

```
    document.getElementById('bot-track-stat').textContent = `${Math.round(pct * 5)}m / 500m`;
```

```
    if (pct >= 100) {
```

```
      clearInterval(botInterval);
```

```
      if (!raceEnded) {
```

```
        raceEnded = true;
```

```
        alert("Pace car crossed the finish line first! Keep training your velocity.");
```

```
        finishRacerGame();
```

```
      }
```

```
    }
```

```
  }, 200);
```

```
input.addEventListener('input', () => {
```

```
  if (raceEnded) return;
```

```
  if (!startTime) startTime = performance.now();
```

```
const val = input.value;
```

```
const spans = document.querySelectorAll('#racer-passage-display .char');
```

```
currentIdx = Math.min(val.length, RACE_PASSAGE.length);
```

```
let errors = 0;
```

```
for (let i = 0; i < spans.length; i++) {
```

```
  if (i < val.length) {
```

```
    if (val[i] === RACE_PASSAGE[i]) {
```

```
      spans[i].className = 'char correct';
```

```
    } else {
```

```
      spans[i].className = 'char incorrect';
```

```
      errors++;
```

```
    }
```

```
  } else {
```

```
    spans[i].className = 'char';
```

```
  }
```

```
}
```

```
const pct = Math.min(100, Math.round((currentIdx / RACE_PASSAGE.length) * 100));
```

```
const elapsedMinutes = (performance.now() - startTime) / 60000;
```

```
const wpm = elapsedMinutes > 0 ? Math.round(((currentIdx - errors) / 5) / elapsedMinutes) : 0;
```

```
document.getElementById('player-car-bar').style.width = `${pct}%`;
```

```
document.getElementById('player-track-stat').textContent = `${Math.round(pct * 5)}m / 500m`;
```

```
document.getElementById('racer-speed').textContent = `${wpm} WPM`;
```

```
document.getElementById('racer-pct').textContent = `${pct}%`;
```

```
if (val === RACE_PASSAGE) {
```

```
  raceEnded = true;
```

```
  clearInterval(botInterval);
```

```
  alert("🏁 Finish Line Reached! You defeated the pace car with ${wpm} WPM!");
```

```
  finishRacerGame();
```

```
    }

  });

});

function renderPassage() {

  const container = document.getElementById('racer-passage-display');

  container.innerHTML = "";

  for (let i = 0; i < RACE_PASSAGE.length; i++) {

    const s = document.createElement('span');

    s.className = 'char';

    s.textContent = RACE_PASSAGE[i];

    container.appendChild(s);

  }

}

function finishRacerGame() {

  window.location.href = "{{ url_for('games.index') }}";

}

</script>

{% endblock %}

=== FILE: \app\templates\games\whack_a_word.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:820px; margin:0 auto; text-align:center;">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.5rem;">

    <h2 style="font-size:1.4rem; font-weight:800; margin:0;">⚡ Whack-A-Word Grid</h2>

    <div style="display:flex; gap:1.2rem; align-items:center;">

      <span>Whacked: <strong id="mole-score" style="color:var(--accent); font-size:1.25rem;">0</strong></span>

      <span>Time: <strong id="mole-time" style="color:var(--warning); font-size:1.25rem;">45s</strong></span>

    <a href="{{ url_for('games.index') }}" class="btn btn-sm">&times; Exit</a>

  </div>

</div>

</div>
```

```
</div>

</div>

<!-- 3x3 Mole Grid -->

<div style="display:grid; grid-template-columns:repeat(3, 1fr); gap:1rem; max-width:600px; margin:0 auto 1.5rem;">

  {% for i in range(9) %}

    <div id="hole-{{ i }}" style="height:100px; background:var(--bg-secondary); border:2px solid var(--border-color); border-radius:var(--radius); display:flex; align-items:center; justify-content:center; font-family:var(--font-mono); font-weight:800; font-size:1.15rem; color:var(--text-muted);">

      <!-- Word pops here -->

    </div>

    {% endfor %}

  </div>

  <div style="font-size:0.85rem; color:var(--text-muted);">

    Type the active illuminated word immediately to strike the mole.

  </div>

</div>

{% endblock %}

{% block scripts %}

<script>

const MOLE_WORDS = ["swift", "strike", "smash", "rapid", "flash", "clack", "click", "punch", "quick", "whack", "power", "turbo"];

let mScore = 0;

let mTime = 45;

let currentActiveWord = "";

let currentActiveHole = null;

let moleTimer = null;

let clockTimer = null;

let typedBuffer = "";

document.addEventListener('DOMContentLoaded', () => {

  clockTimer = setInterval(() => {

    mTime--;
```

```
document.getElementById('mole-time').textContent = `${mTime}s`;

if (mTime <= 0) {

    clearInterval(clockTimer);

    clearInterval(moleTimer);

    alert('Time up! You struck ${mScore} moles.');
```

 window.location.href = "{{ url_for('games.index') }}";

```

}

}, 1000);
```

```
spawnMole();
```

```
window.addEventListener('keydown', (e) => {

    if (mTime <= 0 || !currentActiveWord) return;

    if (e.key.length !== 1 || e.ctrlKey || e.metaKey || e.altKey) return;
```

```

    const char = e.key.toLowerCase();

    if (currentActiveWord.startsWith(typedBuffer + char)) {

        typedBuffer += char;

        const hole = document.getElementById('hole-${currentActiveHole}');

        hole.innerHTML = `<span style="color:var(--success); text-decoration:underline;">${typedBuffer}</span>${currentActiveWord.slice(typedBuffer.length)}`;
```

```

    if (typedBuffer === currentActiveWord) {

        mScore++;

        document.getElementById('mole-score').textContent = mScore;

        hole.style.background = 'var(--success-bg)';

        hole.style.borderColor = 'var(--success)';

        if (window.soundEngine) window.soundEngine.playKey(false);

        spawnMole();

    }

} else {

    if (window.soundEngine) window.soundEngine.playKey(true);

}
```

```
});
```

```
});
```

```
function spawnMole() {

    clearTimeout(moleTimer);

    typedBuffer = "";
```

```

    // Reset previous hole

    if (currentActiveHole !== null) {

        const prev = document.getElementById('hole-${currentActiveHole}');

        prev.textContent = "";

        prev.style.background = 'var(--bg-secondary)';

        prev.style.borderColor = 'var(--border-color)';

    }
```

```

    currentActiveHole = Math.floor(Math.random() * 9);

    currentActiveWord = MOLE_WORDS[Math.floor(Math.random() * MOLE_WORDS.length)];
```

```

    const hole = document.getElementById('hole-${currentActiveHole}');

    hole.textContent = currentActiveWord;

    hole.style.background = 'var(--bg-card)';

    hole.style.borderColor = 'var(--accent)';

    hole.style.color = 'var(--text-primary)';
```

```

    moleTimer = setTimeout(spawnMole, 2400); // 2.4s window to whack

}
```

```
</script>
```

```
{% endblock %}
```

```
=== FILE: .\app\templates\games\word_bltz.html ===
```

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<div class="card" style="max-width:760px; margin:0 auto; text-align:center;">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.5rem;">

    <h2 style="font-size:1.4rem; font-weight:800; margin:0;"> #60-Second Word Blitz</h2>

    <div style="display:flex; gap:1.2rem; align-items:center;">

      <span>Words: <strong id="blitz-words" style="color:var(--accent); font-size:1.25rem;">0</strong></span>

      <span>Multiplier: <strong id="blitz-mult" style="color:var(--warning); font-size:1.25rem;">1x</strong></span>

      <span>Time: <strong id="blitz-clock" style="color:var(--danger); font-size:1.25rem;">60s</strong></span>

    </div>

  </div>

  <!-- Big Central Word Prompt -->

  <div id="blitz-prompt-card" style="background:var(--bg-secondary); border:2px solid var(--accent); border-radius:var(--radius); padding:2.5rem; margin-bottom:1.5rem;">

    <div id="blitz-word" style="font-family:var(--font-mono); font-size:2.8rem; font-weight:900; color:var(--text-primary); letter-spacing:0.05em;">

      READY

    </div>

  </div>

  <input type="text" id="blitz-input" autocomplete="off" autocorrect="off" autocapitalize="off" spellcheck="false" placeholder="Type the flashing word here..." style="width:100%; max-width:440px; padding:0.85rem; background:var(--bg-card); border:2px solid var(--border-color); border-radius:var(--radius-sm); color:var(--text-primary); font-family:var(--font-mono); font-size:1.3rem; text-align:center; outline:none;">

</div>

{% endblock %}

{% block scripts %}

<script>

const BLITZ_CORPUS = ["energy", "fusion", "voltage", "current", "matrix", "stream", "spark", "motion", "dynamo", "thrust", "ignite", "reactor", "hyper", "velocity", "signal", "photon", "laser"];

let bWordsTyped = 0;

let bMultiplier = 1;

let bStreak = 0;

let bTimeRemaining = 60;

let currentBlitzWord = "blitz";

let bStarted = false;
```

```
let bInterval = null;

document.addEventListener('DOMContentLoaded', () => {

  const input = document.getElementById('blitz-input');

  input.focus();

  setNewBlitzWord();

  input.addEventListener('input', () => {

    if (!bStarted) {

      bStarted = true;

      startBlitzTimer();

    }

    const val = input.value.trim().toLowerCase();

    if (val === currentBlitzWord) {

      bWordsTyped++;

      bStreak++;

      if (bStreak % 5 === 0 && bMultiplier < 4) bMultiplier++;

      document.getElementById('blitz-words').textContent = bWordsTyped;

      document.getElementById('blitz-mult').textContent = `${bMultiplier}x`;

      input.value = '';

      if (window.soundEngine) window.soundEngine.playKey(false);

      setNewBlitzWord();

    }

  });

});

function setNewBlitzWord() {

  currentBlitzWord = BLITZ_CORPUS[Math.floor(Math.random() * BLITZ_CORPUS.length)];

  document.getElementById('blitz-word').textContent = currentBlitzWord.toUpperCase();

}
```

```
function startBlitzTimer() {

  bInterval = setInterval(() => {

    bTimeRemaining--;

    document.getElementById('blitz-clock').textContent = `${bTimeRemaining}s`;

    if (bTimeRemaining <= 0) {

      clearInterval(bInterval);

      alert('Time's up! You completed ${bWordsTyped} blitz words with a peak multiplier of ${bMultiplier}x!');

      window.location.href = "{{ url_for('games.index') }}";

    }

  }, 1000);

}

</script>

{% endblock %}

=== FILE: .\app\templates\games\zombie_outbreak.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:880px; margin:0 auto; text-align:center;">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.25rem;">

    <h2 style="font-size:1.4rem; font-weight:800; margin:0;">☠ Zombie Defense Outbreak</h2>

    <div style="display:flex; gap:1.2rem; align-items:center;">

      <span>Eliminated: <strong id="zombie-score" style="color:var(--accent); font-size:1.25rem;">0</strong></span>

      <span>Base Health: <strong id="zombie-health" style="color:var(--danger); font-size:1.25rem;">100%</strong></span>

      <a href="{{ url_for('games.index') }}" class="btn btn-sm">&times; Exit</a>

    </div>

  </div>

  <div id="zombie-arena" style="position:relative; width:100%; height:380px; background:var(--bg-secondary); border:2px solid var(--border-color); border-radius:var(--radius); overflow:hidden;">

    <div style="position:absolute; right:30px; top:0; bottom:0; width:4px; background:var(--accent); opacity:0.6;"></div>

  </div>

</div>

</div>
```

```
<div style="margin-top:0.75rem; font-size:0.85rem; color:var(--text-muted);">

  Zombies approach your base from the left. Type their word cleanly to fire a laser bolt.

</div>

</div>

{% endblock %}

{% block scripts %}

<script>

const ZOMBIE_WORDS = ["bite", "crawl", "undead", "shadow", "stalker", "ghoul", "decay", "danger", "frenzy", "horde", "curse", "infect"];

let zScore = 0;

let zHealth = 100;

let zombies = [];

let zSpawnTimer = null;

let zMoveTimer = null;

document.addEventListener('DOMContentLoaded', () => {

  zSpawnTimer = setInterval(spawnZombie, 2200);

  zMoveTimer = setInterval(moveZombies, 50);

  window.addEventListener('keydown', (e) => {

    if (zHealth <= 0) return;

    if (e.key.length !== 1 || e.ctrlKey || e.metaKey || e.altKey) return;

    const char = e.key.toLowerCase();

    const candidate = zombies.find(z => z.text.startsWith(z.typed + char));

    if (candidate) {

      candidate.typed += char;

      candidate.el.innerHTML = `☠ <span style="color:var(--success); text-decoration:underline;">${candidate.typed}</span>${candidate.text.slice(candidate.typed.length)}</div>

    if (candidate.typed === candidate.text) {

      candidate.el.remove();

    }

  });

</div>
```

```

    zombies.splice(zombies.indexOf(candidate), 1);

    zScore++;

    document.getElementById('zombie-score').textContent = zScore;

}

if (window.soundEngine) window.soundEngine.playKey(false);

} else {

    if (window.soundEngine) window.soundEngine.playKey(true);

}

});

});
```

```
function spawnZombie() {

    const arena = document.getElementById('zombie-arena');

    if (!arena || zHealth <= 0) return;

    const text = ZOMBIE_WORDS[Math.floor(Math.random() * ZOMBIE_WORDS.length)];

    const el = document.createElement('div');

    el.innerHTML = `<span>${text}</span>`;

    el.style.position = 'absolute';

    el.style.left = '0px';

    el.style.top = `${Math.random() * (arena.clientHeight - 60) + 15}px`;

    el.style.padding = '0.35rem 0.65rem';

    el.style.background = 'var(--bg-card)';

    el.style.border = '1px solid var(--danger)';

    el.style.borderRadius = '4px';

    el.style.fontFamily = 'var(--font-mono)';

    el.style.fontWeight = '700';

    arena.appendChild(el);

    zombies.push({ el, text, typed: '', x: 0, speed: 1.2 + Math.random() });

}
```

```
function moveZombies() {
```

```
const arena = document.getElementById('zombie-arena');
```

```
if (!arena || zHealth <= 0) return;
```

```
for (let i = zombies.length - 1; i >= 0; i--) {
```

```
    const z = zombies[i];
```

```
    z.x += z.speed;
```

```
    z.el.style.left = `${z.x}px`;
```

```
// Breach base barrier
```

```
if (z.x >= arena.clientWidth - 110) {
```

```
    z.el.remove();
```

```
    zombies.splice(i, 1);
```

```
    zHealth -= 25;
```

```
    document.getElementById('zombie-health').textContent = `${zHealth}%`;
```

```
    if (window.soundEngine) window.soundEngine.playKey(true);
```

```
    if (zHealth <= 0) {
```

```
        clearInterval(zSpawnTimer);
```

```
        clearInterval(zMoveTimer);
```

```
        alert('Perimeter breached! You survived and eliminated ${zScore} zombies.');
```

```
        window.location.href = "{{ url_for('games.index') }}";
```

```
        break;
```

```
    }
```

```
}
```

```
}
```

```
}
```

```
</script>
```

```
{% endblock %}
```

```
=== FILE: .\app\templates\leaderboard\index.html ===
```

```
{% extends 'base.html' %}
```

```
{% block content %}
```



```
<div class="card" style="max-width:1080px; margin:0 auto;">

<!-- Header -->

<div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1.25rem; margin-bottom:1.5rem; flex-wrap:wrap; gap:1rem;">

<div>

<h1 style="font-size:1.8rem; font-weight:800; margin:0;">Global Championship Leaderboard</h1>

<p style="color:var(--text-secondary); font-size:0.85rem; margin-top:0.25rem;">

    Official anti-cheat verified rankings. Custom text and unstandardized runs are excluded.

</p>

</div>

{% if current_mode != 'ranked_elo' %}

<div style="display:flex; gap:0.4rem; flex-wrap:wrap;">

    <a href="{{ url_for('leaderboard.index', sort='wpm', mode=current_mode) }}" class="btn {{ 'btn-primary' if current_sort == 'wpm' else '' }}">Speed (WPM)</a>

    <a href="{{ url_for('leaderboard.index', sort='accuracy', mode=current_mode) }}" class="btn {{ 'btn-primary' if current_sort == 'accuracy' else '' }}">Accuracy</a>

    <a href="{{ url_for('leaderboard.index', sort='consistency', mode=current_mode) }}" class="btn {{ 'btn-primary' if current_sort == 'consistency' else '' }}">Consistency</a>

</div>

{% endif %}

</div>

<!-- Filter Pills -->

<div style="display:flex; gap:0.4rem; margin-bottom:1.5rem; flex-wrap:wrap;">

    <a href="{{ url_for('leaderboard.index', sort=current_sort, mode='all') }}" class="btn {{ 'btn-primary' if current_mode == 'all' else '' }}" style="padding:0.35rem 0.75rem; font-size:0.82rem;">All Ranked Runs</a>

    <a href="{{ url_for('leaderboard.index', mode='ranked_elo') }}" class="btn {{ 'btn-primary' if current_mode == 'ranked_elo' else '' }}" style="padding:0.35rem 0.75rem; font-size:0.82rem; border-color:#f59e0b; color:{{ '#04101e' if current_mode == 'ranked_elo' else '#f59e0b' }}; font-weight:700;">🏆Ranked Elo Divisions

</a>

    <a href="{{ url_for('leaderboard.index', sort=current_sort, mode='timed_60') }}" class="btn {{ 'btn-primary' if current_mode == 'timed_60' else '' }}" style="padding:0.35rem 0.75rem; font-size:0.82rem;">1-Min Benchmark</a>

    <a href="{{ url_for('leaderboard.index', sort=current_sort, mode='timed_30') }}" class="btn {{ 'btn-primary' if current_mode == 'timed_30' else '' }}" style="padding:0.35rem 0.75rem; font-size:0.82rem;">30s Sprint</a>

</div>
```

```
<a href="{{ url_for('leaderboard.index', sort=current_sort, mode='timed_15') }}" class="btn {{ 'btn-primary' if current_mode == 'timed_15' else '' }}" style="padding:0.35rem 0.75rem; font-size:0.82rem;">15s Dash</a>

</div>

<!-- User Personal Ranking Summary Card -->

{% if user_rank_info %}

<div class="card" style="background:var(--bg-card); border:1px solid var(--accent); padding:0.85rem 1.25rem; margin-bottom:1.75rem; display:flex; justify-content:space-between; align-items:center; flex-wrap:wrap; gap:1rem;">

<div>

<span style="font-size:0.75rem; font-weight:800; color:var(--accent); text-transform:uppercase;">Your Global Standing</span>

<div style="font-size:1.15rem; font-weight:800; margin-top:0.2rem;">

    Rank #{{ user_rank_info.rank }} &bull; Top {{ user_rank_info.percentile }}% of Verified Typists

</div>

</div>

<div>

    Personal Best: <strong style="color:var(--accent); font-family:var(--font-mono); font-size:1.1rem;">{{ user_rank_info.best_wpm }} WPM</strong> ({{ user_rank_info.best_acc }}%)

</div>

</div>

{% endif %}

<!-- Table with Meaningful Empty State -->

<div class="table-responsive-container">

{% if current_mode == 'ranked_elo' %}

<table>

<thead>

<tr>

<th>Rank</th>

<th>Pilot</th>

<th>Division</th>

<th>MMR Rating</th>

<th>Win / Loss</th>

<th>Win Rate</th>

</tr>
```

```
</thead>

<tbody>

  {% for u in ranked_users %}

  <tr>

    <td style="font-weight:800; color:({{ '#f59e0b' if loop.index == 1 else ('#94a3b8' if loop.index == 2 else ('#b45309' if loop.index == 3 else 'var(--text-muted)')) }});">#{ loop.index }}</td>

    <td><a href="{ { url_for('dashboard.public_profile', username=u.username) }}" style="color:var(--text-primary); text-decoration:none; font-weight:700;">{{ u.username }}</a></td>

    <td><span style="font-weight:700; color:({{ u.rank_badge.color }});">{{ u.rank_badge.icon }} {{ u.rank_division }}</span></td>

    <td style="font-family:var(--font-mono); font-weight:800; color:var(--accent);">{{ u.elo_rating }} MMR</td>

    <td><span style="color:var(--success); font-weight:700;">{{ u.ranked_wins }}W</span> - <span style="color:var(--danger); font-weight:700;">{{ u.ranked_losses }}L</span></td>

    <td>{{ ((u.ranked_wins / (u.ranked_wins + u.ranked_losses)) * 100)|round(1) if (u.ranked_wins + u.ranked_losses) > 0 else 0 }}%</td>

  </tr>

  {% else %}

  <tr>

    <td colspan="6" style="padding:3rem; text-align:center; color:var(--text-muted);">

      No competitive duels recorded yet.<br><br>

      <a href="{ { url_for('multiplayer.index') }}" class="btn btn-primary btn-sm">Find a Ranked Match</a>

    </td>

  </tr>

  {% endfor %}

</tbody>

</table>

{% else %}

<table>

<thead>

<tr>

<th>Rank</th>

<th>Pilot</th>

<th>Duration</th>

<th>Speed</th>

<th>Accuracy</th>

<th>Consistency</th>
```

```
<th>Ghost Race</th>

</tr>

</thead>

<tbody>

  {% for test in tests %}

  <tr>

    <td style="font-weight:800; color:({{ '#f59e0b' if loop.index == 1 else ('#94a3b8' if loop.index == 2 else ('#b45309' if loop.index == 3 else 'var(--text-muted)')) }});">#{ { loop.index }}</td>

    <td><a href="{ { url_for('dashboard.public_profile', username=test.user.username) }}" style="color:var(--accent); text-decoration:none; font-weight:700;">{{ test.user.username }}</a></td>

    <td>{{ test.duration | int }}s</td>

    <td style="font-family:var(--font-mono); font-weight:800; color:var(--accent);">{{ test.wpm }} WPM</td>

    <td>{{ test.accuracy }}%</td>

    <td>{{ test.consistency }}%</td>

    <td>

      <a href="{ { url_for('typing.test_page') }}"?ghost_race={{ test.id }}" class="btn btn-sm" style="border-color:#a371f7; color:#a371f7;">

        🏁 Race Ghost

      </a>

    </td>

  </tr>

  {% else %}

  <tr>

    <td colspan="7" style="padding:3.5rem; text-align:center; color:var(--text-muted);">

      <div style="font-size:2rem; margin-bottom:0.5rem;">🔍</div>

      <div style="font-size:1.05rem; font-weight:700; color:var(--text-primary); margin-bottom:0.25rem;">No verified results in this category yet.</div>

      <p style="margin-bottom:1.25rem;">Be the first verified typist to set the global benchmark score.</p>

      <a href="{ { url_for('typing.test_page') }}" class="btn btn-primary">Start Ranked Test</a>

    </td>

  </tr>

  {% endfor %}

</tbody>

</table>

{% endif %}
```

```
</div>

</div>

{% endblock %}

=== FILE: .\app\templates\learn\index.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:1080px; margin:0 auto;">

<!-- Header with Placement Assessment -->

<div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1.25rem; margin-bottom:1.5rem; flex-wrap:wrap; gap:1rem;">

<div>

<h1 style="font-size:1.8rem; font-weight:800; margin:0;">TypeSphere Academy</h1>

<p style="color:var(--text-secondary); font-size:0.88rem; margin-top:0.25rem;">

    Structured typing curriculum with multi-attempt mastery verification and skill tracks.

</p>

</div>

</div>

<div style="display:flex; gap:0.6rem;">

<a href="{{ url_for('learn.skill_assessment') }}" class="btn" style="border-color:var(--accent); color:var(--accent); font-weight:700;">

    📝 Take Placement Assessment

</a>

<a href="{{ url_for('typing.test_page') }}" class="btn">&larr; Free Test Track</a>

</div>

</div>

<!-- Track Switcher Navigation -->

<div style="display:flex; gap:0.5rem; margin-bottom:2rem; flex-wrap:wrap;">

<a href="{{ url_for('learn.index', track='beginner') }}" class="btn {{ 'btn-primary' if active_track == 'beginner' else '' }}">

    Beginner Track (0–40 WPM)

</a>

<a href="{{ url_for('learn.index', track='intermediate') }}" class="btn {{ 'btn-primary' if active_track == 'intermediate' else '' }}">
```

```
Intermediate Track (40–70 WPM)

</a>

<a href="{{ url_for('learn.index', track='advanced') }}" class="btn {{ 'btn-primary' if active_track == 'advanced' else '' }}">

    Advanced Track (70–100 WPM)

</a>

<a href="{{ url_for('learn.index', track='expert') }}" class="btn {{ 'btn-primary' if active_track == 'expert' else '' }}">

    Expert Track (100+ WPM)

</a>

</div>

<!-- Stage Cards Grid -->

<div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(310px, 1fr)); gap:1.25rem;">

    {% for st in stages %}

        {% set prog = progress_map.get(st.id) %}

        <div class="card" style="background:var(--bg-card); border:1px solid var(--border-color); margin-bottom:0; display:flex; flex-direction:column; justify-content:space-between;">

            <div>

                <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:0.6rem;">

                    <span style="font-size:0.75rem; font-weight:800; color:var(--accent); background:var(--accent-glow); padding:0.15rem 0.5rem; border-radius:4px;">

                        STAGE {{ st.stage_number }}

                    </span>

                    {% if prog and prog.completed %}

                        <span style="font-size:0.75rem; font-weight:800; color:var(--success); background:var(--success-bg); padding:0.15rem 0.5rem; border-radius:4px;">

                            ✓ MASTERED

                        </span>

                    {% else %}

                        <span style="font-size:0.75rem; color:var(--text-muted);">

                            Req: {{ st.min_wpm_to_pass|int }} WPM / {{ st.min_accuracy_to_pass|int }}%

                        </span>

                    {% endif %}

                </div>

            </div>

        </div>

    {% endfor %}

</div>
```

```
<h3 style="font-size:1.15rem; font-weight:700; margin:0 0 0.4rem 0;">{{ st.title }}</h3>

<p style="color:var(--text-secondary); font-size:0.85rem; line-height:1.5; margin-bottom:1rem;">

  {{ st.description }}

</p>

</div>

<div>

  {% if prog %

    <div style="font-size:0.78rem; color:var(--text-muted); margin-bottom:0.75rem;">

      Progress: <strong style="color:var(--accent);">{{ prog.successful_attempts }} / {{ st.required_attempts or 2 }} Verified Passes</strong>
      &bull; PB: {{ prog.best_wpm|int }} WPM

    </div>

    {% endif %

    <a href="{{ url_for('learn.lesson', slug=st.slug) }}" class="btn btn-primary" style="width:100%; justify-content:center; padding:0.65rem; font-weight:700;">

      Practice Stage {{ st.stage_number }} &rarr;

    </a>

  </div>

</div>

{% endfor %}

</div>

{% endblock %}

=== FILE: .\app\templates\learn\lesson.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:880px; margin:0 auto;">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1rem; margin-bottom:1.5rem;">

    <div>

      <span style="font-size:0.75rem; font-weight:800; color:var(--accent);">STAGE {{ stage.stage_number }} EXERCISE</span>

      <h2 style="font-size:1.5rem; font-weight:800; margin:0.2rem 0 0;">{{ stage.title }}</h2>

    </div>

  </div>

</div>

</div>
```

```
</div>

<a href="{{ url_for('learn.index') }}" class="btn btn-sm">&larr; Curriculum</a>

</div>

<p style="color:var(--text-secondary); font-size:0.9rem; line-height:1.5; margin-bottom:1.5rem;">

  {{ stage.description }}

</p>

<div class="alert-box alert-info" style="margin-bottom:1.5rem;">

  Passing Requirements: Minimum <strong>{{ stage.min_wpm_to_pass|int }} WPM</strong> and <strong>{{ stage.min_accuracy_to_pass|int }}% Accuracy</strong>.

</div>

<!-- Lesson Arena -->

<div style="display:grid; grid-template-columns:repeat(3, 1fr); gap:0.75rem; margin-bottom:1.25rem;">

  <div class="stat-box"><div class="stat-label">Live WPM</div><div class="stat-value" id="lesson-wpm">0</div></div>

  <div class="stat-box"><div class="stat-label">Accuracy</div><div class="stat-value" id="lesson-acc">100%</div></div>

  <div class="stat-box"><div class="stat-label">Errors</div><div class="stat-value" id="lesson-err" style="color:var(--danger);">0</div></div>

</div>

<div id="lesson-box" class="typing-container focus-ring" style="min-height:150px; max-height:220px; margin-bottom:1.25rem;">

  <div id="lesson-caret" class="caret caret-smooth"></div>

  <div id="lesson-text" class="typing-text-display"></div>

</div>

<input type="text" id="lesson-input" autocomplete="off" autocorrect="off" autocapitalize="off" spellcheck="false" placeholder="Type the exercise above to begin..." style="width:100%; padding:0.85rem; background:var(--bg-card); border:1px solid var(--border-color); border-radius:var(--radius-sm); color:var(--text-primary); font-family:var(--font-mono); font-size:1.1rem; outline:none;">

</div>

{% endblock %}

{% block scripts %}

<script>

const targetMaterial = {{ stage.practice_material|tojson }};
```

```

const stageId = {{ stage.id }};

const minWpm = {{ stage.min_wpm_to_pass }};

const minAcc = {{ stage.min_accuracy_to_pass }};

let startTime = null;

let currentIdx = 0;

document.addEventListener('DOMContentLoaded', () => {

  renderLessonText();

  const input = document.getElementById('lesson-input');

  input.focus();

  input.addEventListener('input', () => {

    if (!startTime) startTime = performance.now();

    const val = input.value;

    const spans = document.querySelectorAll('#lesson-text .char');

    currentIdx = Math.min(val.length, targetMaterial.length);

    let errors = 0;

    for (let i = 0; i < spans.length; i++) {

      if (i < val.length) {

        if (val[i] === targetMaterial[i]) {

          spans[i].className = 'char correct';

        } else {

          spans[i].className = 'char incorrect';

          errors++;

        }

      } else {

        spans[i].className = 'char';

      }

    }

  });

  // Caret Position

```

```

const caret = document.getElementById('lesson-caret');

if (currentIdx < spans.length) {

  caret.style.left = `${spans[currentIdx].offsetLeft}px`;

  caret.style.top = `${spans[currentIdx].offsetTop + 4}px`;

}

// Telemetry

const elapsedMinutes = (performance.now() - startTime) / 60000;

const netWords = Math.max(0, (currentIdx - errors) / 5.0);

const wpm = elapsedMinutes > 0 ? Math.round(netWords / elapsedMinutes) : 0;

const acc = currentIdx > 0 ? Math.round(((currentIdx - errors) / currentIdx) * 100) : 100;

document.getElementById('lesson-wpm').textContent = wpm;

document.getElementById('lesson-acc').textContent = `${acc}%`;

document.getElementById('lesson-err').textContent = errors;

if (val === targetMaterial) {

  input.disabled = true;

  submitCompletion(wpm, acc);

}

});

});

function renderLessonText() {

  const container = document.getElementById('lesson-text');

  container.innerHTML = "";

  for (let i = 0; i < targetMaterial.length; i++) {

    const s = document.createElement('span');

    s.className = 'char';

    s.textContent = targetMaterial[i];

    container.appendChild(s);

  }

}

```

```
async function submitCompletion(wpm, acc) {

  const passed = (wpm >= minWpm && acc >= minAcc);

  try {

    await fetch('/learn/api/complete-stage', {

      method: 'POST',

      headers: { 'Content-Type': 'application/json' },

      body: JSON.stringify({ stage_id: stageId, wpm: wpm, accuracy: acc })

    });

  } catch {}

  alert(passed

    ? '🎯 Stage Passed! Speed: ${wpm} WPM | Accuracy: ${acc}%. Progression updated!'

    : 'Stage Incomplete. Score: ${wpm} WPM (Target: ${minWpm}) & ${acc}% Acc. Try again!');

  window.location.href = "{{ url_for('learn.index') }}"

}

</script>

{% endblock %}

=== FILE: \app\templates\multiplayer\room.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width: 1060px; margin: 0 auto;">

<!-- =====

1. PERSISTENT THREE-MODE TOP BAR (ALWAYS ACCESSIBLE)

===== -->

<div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 1.5rem; flex-wrap: wrap; gap: 1rem;">

<div>

<h1 style="font-size: 1.75rem; font-weight: 800; margin: 0;">

Multiplayer Arena
```

```
</h1>

<p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.2rem;">

Synchronized competitive typing across Public Rooms, Quick Match, or Private 1v1.

</p>

</div>

<!-- Mode Switcher Navigation Pills (Instant In-Page Switching) -->

<div style="display: flex; gap: 0.4rem; align-items: center; flex-wrap: wrap;">

  <button onclick="switchModeTab('public')" id="tab-pill-public" class="btn btn-primary" style="padding: 0.45rem 0.9rem; font-size: 0.85rem; font-weight: 700;">

    🏠 Public Rooms

  </button>

  <button onclick="switchModeTab('quick')" id="tab-pill-quick" class="btn" style="padding: 0.45rem 0.9rem; font-size: 0.85rem; font-weight: 700;">

    ⚡ Quick Match (1v1)

  </button>

  <button onclick="switchModeTab('private')" id="tab-pill-private" class="btn" style="padding: 0.45rem 0.9rem; font-size: 0.85rem; font-weight: 700;">

    🏠 Private 1v1

  </button>

<!-- Connection Status Badge -->

<div id="socket-conn-badge" style="display: inline-flex; align-items: center; gap: 0.45rem; background: var(--bg-card); border: 1px solid var(--border-color); padding: 0.35rem 0.75rem; border-radius: 20px; font-size: 0.78rem; margin-left: 0.5rem;">

  <span id="socket-conn-dot" style="width: 8px; height: 8px; border-radius: 50%; background: var(--warning);"></span>

  <span id="socket-conn-text" style="color: var(--text-muted);">Connecting...</span>

</div>

</div>

</div>

<!-- =====

VIEW 1: MULTIPLAYER HUB (Mode Selection Cards & Directory)

===== -->

<div id="view-hub">
```

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 1.25rem; margin-bottom: 2rem;">

<!-- 1. Public Rooms Card -->

<div class="card" id="card-mode-public" style="background: var(--bg-card); border: 1.5px solid var(--accent); margin-bottom: 0; display: flex; flex-direction: column; justify-content: space-between;">

<div>

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

<h3 style="margin: 0; color: var(--accent); font-size: 1.2rem;">Public Rooms</h3>

OPEN

</div>

<p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.5; margin-bottom: 1.25rem;">

Browse open rooms or create your own with custom player limits (up to 8), test durations, and custom passages.

</p>

</div>

<button onclick="openCreateModal('public')" class="btn btn-primary" style="width: 100%; justify-content: center; font-weight: 700;">

+ Create Public Room

</button>

</div>

<!-- 2. Quick Match 1v1 Card -->

<div class="card" id="card-mode-quick" style="background: var(--bg-card); border: 1px solid var(--border-color); margin-bottom: 0; display: flex; flex-direction: column; justify-content: space-between;">

<div>

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

<h3 style="margin: 0; font-size: 1.2rem;">Quick Match</h3>

AUTOMATIC

</div>

<p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.5; margin-bottom: 1.25rem;">

Instant pairing. Enter the waiting queue and automatically duel another available pilot without sharing links.

</p>

</div>

<button onclick="startQuickMatchQueue()" class="btn" style="width: 100%; justify-content: center; font-weight: 700; border-color: var(--success); color: var(--success);">

Enter 1v1 Queue

</button>

</div>

<!-- 3. Private 1v1 Card -->

<div class="card" id="card-mode-private" style="background: var(--bg-card); border: 1px solid var(--border-color); margin-bottom: 0; display: flex; flex-direction: column; justify-content: space-between;">

<div>

<h3 style="margin: 0 0 0.5rem 0; font-size: 1.2rem;">Private 1v1</h3>

<p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.5; margin-bottom: 1.25rem;">

Generate an exclusive room code or invite link. Strictly 2 players. Both must be ready before match initiation.

</p>

</div>

<div style="display: flex; flex-direction: column; gap: 0.5rem;">

<button onclick="openCreateModal('private')" class="btn" style="width: 100%; justify-content: center; font-weight: 700;">

Create Private 1v1

</button>

<div style="display: flex; gap: 0.4rem;">

<input type="text" id="join-code-input" placeholder="Room Code (1V1-XXXX)" style="flex: 1; padding: 0.55rem 0.75rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary); font-family: var(--font-mono); font-size: 0.85rem; text-transform: uppercase;">

<button onclick="joinRoomByCodeInput()" class="btn btn-primary" style="padding: 0.55rem 1rem;">Join</button>

</div>

</div>

</div>

</div>

<!-- Public Rooms Live Directory -->

<div id="public-directory-section" style="border-top: 1px solid var(--border-color); padding-top: 1.5rem;">

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1rem; flex-wrap: wrap; gap: 0.5rem;">

<h3 style="font-size: 1.15rem; margin: 0;">Open Public Rooms</h3>

<button onclick="requestPublicRooms()" class="btn btn-sm">

Refresh Directory

</button>

</div>

<div class="table-responsive-container">

<table style="width: 100%; border-collapse: collapse; text-align: left;">

<thead>

<tr>

<th>Code</th>

<th>Room Title</th>

<th>Host</th>

<th>Racers</th>

<th>Duration</th>

<th>Status</th>

<th>Action</th>

</tr>

</thead>

<tbody id="public-rooms-tbody">

<tr>

<td colspan="7" style="padding: 2.5rem; text-align: center; color: var(--text-muted);">

Searching for active public rooms...

</td>

</tr>

</tbody>

</table>

</div>

</div>

</div>

<!-- =====

VIEW 2: QUICK MATCH SEARCHING MODAL

===== -->

<div id="modal-quick-searching" style="display: none; position: fixed; inset: 0; background: rgba(0,0,0,0.85); z-index: 2500; align-items: center; justify-content: center; backdrop-filter: blur(8px); padding: 1rem;">

<div class="card" style="width: 100%; max-width: 440px; text-align: center; border-color: var(--accent); padding: 2.5rem 2rem;">

<div style="font-size: 3rem; margin-bottom: 1rem; animation: pulse 1.2s infinite;"> 🚀</div>

<h2 style="font-size: 1.5rem; font-weight: 800; color: var(--accent); margin-bottom: 0.5rem;">Quick Match Queue</h2>

<p style="color: var(--text-muted); font-size: 0.9rem; line-height: 1.5; margin-bottom: 2rem;">

Searching for an opponent... You will be automatically transferred to the starting grid once paired.

</p>

<button onclick="cancelQuickMatchQueue()" class="btn btn-danger" style="padding: 0.65rem 1.5rem; font-weight: 700;">

Cancel Queue

</button>

</div>

</div>

<!-- =====

VIEW 3: CREATE ROOM CONFIGURATION MODAL

===== -->

<div id="modal-create-room" style="display: none; position: fixed; inset: 0; background: rgba(0,0,0,0.85); z-index: 2500; align-items: center; justify-content: center; backdrop-filter: blur(8px); padding: 1rem;">

<div class="card" style="width: 100%; max-width: 520px; border-color: var(--accent); padding: 2rem; max-height: 90vh; overflow-y: auto;">

<div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 0.75rem; margin-bottom: 1.25rem;">

<h3 id="create-modal-title" style="margin: 0; font-size: 1.3rem; font-weight: 800; color: var(--accent);">Create Room</h3>

<button onclick="closeCreateModal()" class="btn btn-sm">×</button>

</div>

<div style="margin-bottom: 1.25rem;">

<label style="display: block; font-size: 0.8rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.35rem;">Room Title</label>

<input type="text" id="create-room-name" value="Speed Sprint" style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary); font-family: inherit;">

</div>

<div style="margin-bottom: 1.25rem;">

<label style="display: block; font-size: 0.8rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.35rem;">Duration</label>

<div style="display: flex; gap: 0.4rem; flex-wrap: wrap;" id="create-dur-group">

<button type="button" class="btn btn-sm" onclick="selectCreateDur(30, this)">30s</button>


```
<button type="button" class="btn btn-sm active-choice" onclick="selectCreateDur(60, this)">60s</button>

<button type="button" class="btn btn-sm" onclick="selectCreateDur(120, this)">120s</button>

<button type="button" class="btn btn-sm" onclick="selectCreateDur(300, this)">300s</button>

<button type="button" class="btn btn-sm" onclick="selectCreateDur(600, this)">600s</button>

</div>

</div>

<div id="create-players-wrap" style="margin-bottom: 1.25rem;">

  <label style="display: block; font-size: 0.8rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.35rem;">Maximum Racers</label>

  <select id="create-max-players" style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary); font-family: inherit;">

    <option value="2">2 Players (1v1)</option>

    <option value="4" selected>4 Players</option>

    <option value="6">6 Players</option>

    <option value="8">8 Players</option>

  </select>

</div>

<div style="margin-bottom: 1.5rem;">

  <label style="display: block; font-size: 0.8rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.35rem;">Custom Passage (Optional)</label>

  <textarea id="create-custom-text" rows="3" placeholder="Leave blank to use curated passages, or paste custom text..." style="width: 100%; padding: 0.65rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary); font-family: var(--font-mono); font-size: 0.85rem; line-height: 1.5;"></textarea>

</div>

<div style="display: flex; justify-content: flex-end; gap: 0.75rem;">

  <button type="button" onclick="closeCreateModal()" class="btn">Cancel</button>

  <button type="button" onclick="submitCreateRoom()" class="btn btn-primary" style="font-weight: 800; padding: 0.65rem 1.5rem;">

    Launch Room &arr;

  </button>

</div>

</div>

</div>

</div>
```

```
<!-- =====

VIEW 4: WAITING ROOM / LOBBY

===== -->

<div id="view-waiting" style="display: none;">

  <div style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius); padding: 1.5rem; margin-bottom: 1.5rem;">

    <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1rem; margin-bottom: 1.25rem; flex-wrap: wrap; gap: 1rem;">

      <div>

        <span style="font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; color: var(--text-muted);">Active Room Code</span>

        <div style="display: flex; align-items: center; gap: 0.6rem; margin-top: 0.2rem;">

          <span id="waiting-room-code" style="font-size: 1.5rem; font-family: var(--font-mono); font-weight: 800; color: var(--accent);">ROOM-CODE</span>

          <button onclick="copyLobbyInvite()" id="btn-copy-lobby" class="btn btn-sm">

            ⓘ Copy Link

          </button>

        </div>

      </div>

    </div>

    <div style="text-align: right; font-size: 0.85rem; color: var(--text-muted);">

      <div>Type: <strong id="waiting-room-type" style="color: var(--text-primary);">PUBLIC</strong></div>

      <div>Duration: <strong id="waiting-room-dur" style="color: var(--text-primary);">60s</strong></div>

    </div>

  </div>

  <h3 style="font-size: 1.1rem; margin-bottom: 1rem;">Connected Racers</h3>

  <div id="waiting-racers-list" style="display: flex; flex-direction: column; gap: 0.65rem; margin-bottom: 1.5rem;">

    <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 1rem; border-top: 1px solid var(--border-color); padding-top: 1.25rem;">

      <button onclick="safeLeaveRoom()" class="btn btn-danger">

        ⌂ Leave Room

      </button>
```

```
<div style="display: flex; gap: 0.75rem; align-items: center;">

  <button onclick="toggleMyReadyStatus()" id="btn-toggle-ready" class="btn" style="padding: 0.65rem 1.5rem; font-weight: 800;">

    Mark Ready ( ✓ )

  </button>

  <button onclick="hostStartMatch()" id="btn-host-start" class="btn btn-primary" style="background: var(--success); border-color: var(--success); font-weight: 800; padding: 0.65rem 1.75rem;">

    🏁 Start Match

  </button>

</div>

</div>

</div>

</div>

<!-- =====

VIEW 5: ACTIVE RACE ARENA (Individual Finish & Spectate)

===== -->

<div id="view-arena" style="display: none;">

  <!-- Synchronized 3-Second Countdown Overlay -->

  <div id="arena-countdown-banner" style="display: none; text-align: center; margin-bottom: 1.5rem; background: var(--bg-card); border: 2px solid var(--accent); border-radius: var(--radius); padding: 1.5rem;">

    <div style="font-size: 0.85rem; text-transform: uppercase; letter-spacing: 0.1em; color: var(--text-muted);">Synchronized Start</div>

    <div id="arena-countdown-num" style="font-size: 4.5rem; font-family: var(--font-mono); font-weight: 900; color: var(--accent); margin: 0.2rem 0;">3</div>

  </div>

  <!-- Finished Early Banner (Live Spectate Mode) -->

  <div id="finished-spectate-banner" style="display: none; background: var(--success-bg); border: 1.5px solid var(--success); border-radius: var(--radius); padding: 1rem 1.5rem; margin-bottom: 1.25rem; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 1rem;">

    <div>

      <div style="font-size: 1.15rem; font-weight: 800; color: var(--success);" id="spectate-title">

        🏁 You finished! Position: #1

      </div>

      <div style="font-size: 0.85rem; color: var(--text-muted);" id="spectate-subtext">
```

```
Other racers are still on track. You are now spectating live.

</div>

</div>

<div style="display: flex; gap: 0.6rem;">

  <button onclick="leaveEarlyAfterFinish()" class="btn btn-sm">

    🏠 Leave Race (Save Place)

  </button>

</div>

</div>

<!-- Live Progress Tracks -->

<div style="margin-bottom: 1.5rem;">

  <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.75rem;">

    <h3 style="font-size: 1.15rem; margin: 0;">Live Race Tracks</h3>

    <span id="arena-status-badge" style="font-size: 0.85rem; color: var(--text-muted); font-weight: 600;">Race in progress...</span>

  </div>

  <div id="arena-tracks-container" style="display: flex; flex-direction: column; gap: 0.75rem;"></div>

</div>

<!-- Smooth Auto-Centering Passage Box -->

<div style="margin-bottom: 1rem;">

  <div id="arena-passage-box" class="typing-container focus-ring" style="min-height: 140px; max-height: 220px; overflow-y: auto; scroll-behavior: smooth; position: relative;">

    <div id="arena-caret" class="caret caret-block"></div>

    <div id="arena-passage-text" class="typing-text-display"></div>

  </div>

</div>

<!-- Universal Cockpit Input -->

<div>

  <input type="text" id="arena-cockpit-input" placeholder="Type here when race begins (Space advances)..." autocomplete="off" autocorrect="off" autocapitalize="off" spellcheck="false" style="width: 100%; padding: 0.95rem 1.15rem; background: var(--bg-card); border: 2px solid var(--border-color); border-radius: var(--radius-sm); color: var(--text-primary); font-family: var(--font-mono); font-size: 1.1rem; outline: none; transition: border-color var(--transition-fast);">
```

```

<div style="text-align: center; margin-top: 0.5rem; font-size: 0.8rem; color: var(--text-muted);">
  Active line auto-centers smoothly as you type.
</div>

</div>

</div>

<!-- =====

VIEW 6: FINAL RESULTS SCREEN (Celebration & Motivation)

===== -->

<div id="view-results" style="display: none; text-align: center;">

<div class="card" id="results-card-container" style="background: var(--bg-card); border-color: var(--accent); padding: 2.5rem 2rem;">

  <div id="results-trophy-icon" style="font-size: 4rem; margin-bottom: 0.4rem;">🏆</div>

  <h2 id="results-headline-title" style="font-size: 2.1rem; font-weight: 800; color: var(--accent); margin-bottom: 0.4rem;">
    Race Concluded
  </h2>

  <div id="results-motivational-banner" style="background: var(--accent-glow); border: 1px solid var(--accent); border-radius: var(--radius-sm);
padding: 0.85rem 1.25rem; max-width: 680px; margin: 0 auto 1.75rem; font-size: 0.95rem; font-weight: 600; line-height: 1.5;">
    Great race everyone!
  </div>

  <!-- Standings Table -->

  <div class="table-responsive-container" style="margin-bottom: 2rem;">

    <table style="width: 100%; text-align: left;">

      <thead>

        <tr>

          <th>Position</th>

          <th>Pilot</th>

          <th>Net Speed</th>

          <th>Accuracy</th>

          <th>Errors</th>

          <th>Status</th>


```

```

</tr>

</thead>

<tbody id="results-standings-tbody"></tbody>

</table>

</div>

<!-- Actions -->

<div style="display: flex; justify-content: center; gap: 0.85rem; flex-wrap: wrap;">

  <button onclick="requestRematch()" id="btn-results-rematch" class="btn btn-primary" style="background: var(--success); border-color: var(--success); font-weight: 800; padding: 0.75rem 1.75rem; font-size: 1rem;">

    Rematch (Same Racers)

  </button>

  <button onclick="switchModeTab('quick')" class="btn btn-primary" style="font-weight: 800; padding: 0.75rem 1.5rem; font-size: 0.95rem;">

    Quick Match Another

  </button>

  <button onclick="switchModeTab('public')" class="btn" style="padding: 0.75rem 1.25rem;">

    Public Rooms

  </button>

  <button onclick="safeLeaveRoom()" class="btn" style="padding: 0.75rem 1.25rem;">

    Return to Hub

  </button>

</div>

</div>

{% endblock %}

{% block scripts %}

<script src="https://cdn.socket.io/4.7.2/socket.io.min.js"></script>

<script>

// =====

// Master Multiplayer Controller

```

```
// =====

let socket = null;

let mySid = null;

const isMobile = /Mobi|Android|iPhone|iPad/i.test(navigator.userAgent);

const platformTag = isMobile ? "Phone" : "PC";

const authUsername = "{ current_user.username if current_user.is_authenticated else ""}";

const myPlayerName = authUsername ? authUsername : `Pilot_${platformTag}_${Math.floor(100 + Math.random() * 900)}`;

let activeRoomData = null;

let currentPassage = "";

let currentInputIndex = 0;

let raceActive = false;

let isCountingDown = false;

let raceStartTime = null;

let clientRaceTimer = null;

let hasLocallyFinished = false;

let creationSelectedDur = 60;

let creationType = 'public';

const trackElements = {};

document.addEventListener('DOMContentLoaded', () => {

  initSocket();

  const params = new URLSearchParams(window.location.search);

  const joinParam = params.get('join') || params.get('room');

  if (joinParam) {

    setTimeout(() => {

      socket.emit('join_room', { code: joinParam.trim().toUpperCase(), player_name: myPlayerName });

    }, 450);

  }

}
```

```
bindArenaInput();

highlightActiveNavPill('public');

});

function initSocket() {

  socket = io();

  socket.on('connect', () => {

    mySid = socket.id;

    document.getElementById('socket-conn-dot').style.background = 'var(--success)';

    document.getElementById('socket-conn-text').textContent = `Online as ${myPlayerName}`;

    requestPublicRooms();

  });

  socket.on('disconnect', () => {

    document.getElementById('socket-conn-dot').style.background = 'var(--danger)';

    document.getElementById('socket-conn-text').textContent = `Disconnected. Reconnecting...`;

  });

  socket.on('public_rooms_update', (data) => {

    renderPublicRooms(data.rooms || []);

  });

  socket.on('room_joined', (data) => {

    mySid = data.your_sid;

    activeRoomData = data.room;

    currentPassage = data.room.text;

    hasLocallyFinished = false;

    showView('waiting');

    renderWaitingLobby();

  });

  socket.on('room_state_updated', (data) => {
```

```
activeRoomData = data.room;

currentPassage = data.room.text;

if (activeRoomData.status === 'waiting') {

  renderWaitingLobby();

}

});
```

```
socket.on('join_error', (data) => {

  alert(data.message || 'Unable to join room.');
```

```
});

socket.on('action_error', (data) => {

  alert(data.message || 'Action failed.');
```

```
});

socket.on('quick_queue_waiting', () => {

  document.getElementById('modal-quick-searching').style.display = 'flex';

});
```

```
socket.on('quick_queue_cancelled', () => {

  document.getElementById('modal-quick-searching').style.display = 'none';

});
```

```
socket.on('quick_match_paired', (data) => {

  document.getElementById('modal-quick-searching').style.display = 'none';

  mySid = data.your_sid;

  activeRoomData = data.room;

  currentPassage = data.room.text;

  hasLocallyFinished = false;

  showView('waiting');

  renderWaitingLobby();

});
```

```
socket.on('match_countdown_started', (data) => {

  currentPassage = data.text;

  hasLocallyFinished = false;

  showView('arena');

  executeCountdown(data.duration || 60);

});
```

```
socket.on('room_progress_update', (data) => {

  renderArenaTracks(data.players || []);

});
```

```
socket.on('individual_player_finished', (data) => {

  const finishedPlayer = data.player;

  const isMe = (finishedPlayer.sid === mySid);

  if (isMe) {

    hasLocallyFinished = true;

    document.getElementById('finished-spectate-banner').style.display = 'flex';

    const medal = (finishedPlayer.place === 1) ? '🥇 1st' : ((finishedPlayer.place === 2) ? '🥈 2nd' : '🥉 3rd');

    document.getElementById('spectate-title').textContent = `🏁 You finished! Position: ${medal} (${Math.round(finishedPlayer.wpm)} WPM)`;

    document.getElementById('spectate-subtext').textContent = data.remaining_count > 0

      ? ` ${data.remaining_count} racer(s) still on track. You are now spectating live.`

      : 'All racers finished!';

  }

}
```

```
if (data.all_finished) {

  setTimeout(() => {

    showResultsView(data standings);

  }, 900);

}

});
```

```
socket.on('race_concluded', (data) => {
```

```

clearInterval(clientRaceTimer);

setTimeout(() => {

  showResultsView(data.standings);

}, 900);

});

socket.on('rematch_accepted', (data) => {

  activeRoomData = data.room;

  currentPassage = data.room.text;

  hasLocallyFinished = false;

  showView('waiting');

  renderWaitingLobby();

});

socket.on('left_room_confirmed', () => {

  activeRoomData = null;

  hasLocallyFinished = false;

  clearInterval(clientRaceTimer);

  showView('hub');

  requestPublicRooms();

});

}

// =====

// Mode Switching & Safety Guards

// =====

function switchModeTab(targetMode) {

  if (activeRoomData && (activeRoomData.status === 'in_progress' || activeRoomData.status === 'countdown') && !hasLocallyFinished) {

    const confirmLeave = confirm("You are currently inside an active race! Switching modes will record you as DNF. Continue?");

    if (!confirmLeave) return;

  }

}

if (activeRoomData) {

```

```

    socket.emit('leave_room_voluntary');

  }

  cancelQuickMatchQueue();

  highlightActiveNavPill(targetMode);

  showView('hub');

  if (targetMode === 'quick') {

    startQuickMatchQueue();

  } else if (targetMode === 'private') {

    openCreateModal('private');

  } else {

    const dir = document.getElementById('public-directory-section');

    if (dir) dir.scrollToView({ behavior: 'smooth' });

    requestPublicRooms();

  }

}

function highlightActiveNavPill(activeMode) {

  const pPub = document.getElementById('tab-pill-public');

  const pQck = document.getElementById('tab-pill-quick');

  const pPrv = document.getElementById('tab-pill-private');

  pPub.className = (activeMode === 'public') ? 'btn btn-primary' : 'btn';

  pQck.className = (activeMode === 'quick') ? 'btn btn-primary' : 'btn';

  pPrv.className = (activeMode === 'private') ? 'btn btn-primary' : 'btn';

  const cPub = document.getElementById('card-mode-public');

  const cQck = document.getElementById('card-mode-quick');

  const cPrv = document.getElementById('card-mode-private');

  if (cPub) cPub.style.borderColor = (activeMode === 'public') ? 'var(--accent)' : 'var(--border-color)';

  if (cQck) cQck.style.borderColor = (activeMode === 'quick') ? 'var(--accent)' : 'var(--border-color)';

  if (cPrv) cPrv.style.borderColor = (activeMode === 'private') ? 'var(--accent)' : 'var(--border-color)';

```

```
}

function showView(viewName) {

  document.getElementById('view-hub').style.display = (viewName === 'hub') ? 'block' : 'none';

  document.getElementById('view-waiting').style.display = (viewName === 'waiting') ? 'block' : 'none';

  document.getElementById('view-arena').style.display = (viewName === 'arena') ? 'block' : 'none';

  document.getElementById('view-results').style.display = (viewName === 'results') ? 'block' : 'none';

}

function safeLeaveRoom() {

  if (activeRoomData && (activeRoomData.status === 'in_progress' || activeRoomData.status === 'countdown') && !hasLocallyFinished) {

    if (!confirm("Are you sure you want to abandon this race?")) return;

  }

  socket.emit('leave_room_voluntary');

}

function leaveEarlyAfterFinish() {

  socket.emit('leave_race_after_finish');

}

// =====

// Hub Directory

// =====

function requestPublicRooms() {

  if (socket && socket.connected) {

    socket.emit('request_public_rooms');

  }

}

function renderPublicRooms(rooms) {

  const tbody = document.getElementById('public-rooms-tbody');

  if (!tbody) return;

  tbody.innerHTML = '';
```

```
if (rooms.length === 0) {

  tbody.innerHTML = `

    <tr>

      <td colspan="7" style="padding: 2.5rem; text-align: center; color: var(--text-muted);">

        No public rooms waiting. Click <strong>" + Create Public Room"</strong> to host!

      </td>

    </tr>

  `;

  return;

}

rooms.forEach(r => {

  const tr = document.createElement('tr');

  tr.innerHTML = `

    <td style="font-family: var(--font-mono); font-weight: 700; color: var(--accent);">${r.code}</td>

    <td style="font-weight: 600;">${r.name}</td>

    <td>${r.host_name}</td>

    <td>${r.current_players} / ${r.max_players}</td>

    <td>${r.duration}s</td>

    <td>

      <span style="background: var(--success-bg); color: var(--success); padding: 0.2rem 0.5rem; border-radius: 4px; font-size: 0.75rem; font-weight: 700;">WAITING</span>

    </td>

    <td>

      <button onclick="joinRoomByCode('${r.code}')" class="btn btn-primary btn-sm">

        Join Room

      </button>

    </td>

  `;

  tbody.appendChild(tr);

});

}
```

```
function joinRoomByCode(code) {

    socket.emit('join_room', { code: code, player_name: myPlayerName });

}
```

```
function joinRoomByCodeInput() {

    const input = document.getElementById('join-code-input');

    const code = input.value.trim().toUpperCase();

    if (!code) {

        alert('Please enter a room code.');
```

return;

```
    }

    joinRoomByCode(code);

}
```

```
function startQuickMatchQueue() {

    socket.emit('join_quick_queue', { name: myPlayerName });

}
```

```
function cancelQuickMatchQueue() {

    socket.emit('cancel_quick_queue');
```

highlightActiveNavPill('public');

```
}
```

```
function openCreateModal(type) {

    creationType = type;

    document.getElementById('create-modal-title').textContent = (type === 'private') ? "Create Private 1v1 Room" : "Create Public Room";

    document.getElementById('create-players-wrap').style.display = (type === 'private') ? 'none' : 'block';

    document.getElementById('modal-create-room').style.display = 'flex';

}
```

```
function closeCreateModal() {

    document.getElementById('modal-create-room').style.display = 'none';

}
```

```
}
```

```
function selectCreateDur(dur, btn) {

    creationSelectedDur = dur;

    document.querySelectorAll("#create-dur-group .btn").forEach(b => b.classList.remove('active-choice'));

    btn.classList.add('active-choice');
```

```
}
```

```
function submitCreateRoom() {

    const name = document.getElementById('create-room-name').value.trim() || 'Speed Duel';

    const maxP = (creationType === 'private') ? 2 : parseInt(document.getElementById('create-max-players').value, 10);

    const customText = document.getElementById('create-custom-text').value.trim();

}
```

```
socket.emit('create_room', {

    type: creationType,

    name: name,

    player_name: myPlayerName,

    duration: creationSelectedDur,

    max_players: maxP,

    custom_text: customText

});
```

```
closeCreateModal();

}
```

```
// =====

// Waiting Room / Lobby

// =====
```

```
function renderWaitingLobby() {

    if (!activeRoomData) return;
```

```
    document.getElementById('waiting-room-code').textContent = activeRoomData.code;

    document.getElementById('waiting-room-type').textContent = activeRoomData.type.toUpperCase();

}
```



```
document.getElementById('waiting-room-dur').textContent = `${activeRoomData.duration}s`;

const container = document.getElementById('waiting-racers-list');

container.innerHTML = "";

const players = Object.values(activeRoomData.players || {}).filter(p => !p.left_early);

let allReady = true;

players.forEach(p => {

  const isMe = (p.sid === mySid);

  if (!p.ready) allReady = false;

  const div = document.createElement('div');

  div.style.display = 'flex';

  div.style.justifyContent = 'space-between';

  div.style.alignItems = 'center';

  div.style.background = isMe ? 'rgba(56, 189, 248, 0.12)' : 'var(--bg-secondary)';

  div.style.border = isMe ? '1.5px solid var(--accent)' : '1px solid var(--border-color)';

  div.style.padding = '0.75rem 1rem';

  div.style.borderRadius = 'var(--radius-sm)';

  div.innerHTML = `

    <div style="display: flex; align-items: center; gap: 0.6rem;">

      <span style="font-size: 1.15rem;">❏</span>

      <strong style="font-size: 0.95rem;">${isMe ? 'You (${p.name)}' : p.name}</strong>

      ${p.is_host ? '<span style="font-size: 0.7rem; background: var(--accent); color: #04101e; padding: 0.1rem 0.45rem; border-radius: 4px; font-weight: 700;">HOST</span>' : ''}

    </div>

    <div>

      <span style="font-size: 0.8rem; font-weight: 700; padding: 0.25rem 0.65rem; border-radius: 4px; background: ${p.ready ? 'var(--success-bg)' : 'var(--warning-bg)'}; color: ${p.ready ? 'var(--success)' : 'var(--warning)}';">

        ${p.ready ? '✔ Ready' : '❏ Not Ready'}

      </span>

    </div>

  `;

  container.appendChild(div);

});
```

```
</span>

</div>

`;

container.appendChild(div);

});

const amHost = (activeRoomData.host_sid === mySid);

const hostBtn = document.getElementById('btn-host-start');

const readyBtn = document.getElementById('btn-toggle-ready');

if (amHost) {

  hostBtn.style.display = 'inline-flex';

  const canStart = (players.length >= 2 || activeRoomData.type === 'public') && allReady;

  hostBtn.disabled = !canStart;

  hostBtn.style.opacity = canStart ? '1' : '0.5';

} else {

  hostBtn.style.display = 'none';

}

const myRecord = activeRoomData.players[mySid];

if (myRecord) {

  readyBtn.textContent = myRecord.ready ? 'Cancel Ready' : 'Mark Ready (✔)';

  readyBtn.className = myRecord.ready ? 'btn' : 'btn btn-primary';

}

}

function copyLobbyInvite() {

  if (!activeRoomData) return;

  const link = `${window.location.origin}/multiplayer/?join=${encodeURIComponent(activeRoomData.code)}`;

  navigator.clipboard.writeText(link).then(() => {

    const btn = document.getElementById('btn-copy-lobby');

    btn.textContent = 'Link Copied! ✔';

  });

}
```

```
btn.style.color = 'var(--success)';

setTimeout(() => {

    btn.textContent = '📄 Copy Link';

    btn.style.color = "";

}, 2000);

});

}
```

```
function toggleMyReadyStatus() {

    socket.emit('toggle_ready');

}
```

```
function hostStartMatch() {

    socket.emit('start_match');

}
```

```
function requestRematch() {

    socket.emit('rematch_request');

}
```

```
// =====

// Race Arena Execution & Smooth Auto-Centering

// =====
```

```
function executeCountdown(durationSec) {

    isCountingDown = true;

    raceActive = false;

    currentInputIndex = 0;

    raceStartTime = null;

    hasLocallyFinished = false;
```

```
document.getElementById('finished-spectate-banner').style.display = 'none';

const banner = document.getElementById('arena-countdown-banner');

const num = document.getElementById('arena-countdown-num');
```

```
const statusBadge = document.getElementById('arena-status-badge');
```

```
const input = document.getElementById('arena-cockpit-input');
```

```
banner.style.display = 'block';
```

```
statusBadge.textContent = 'Standby for countdown...';
```

```
input.value = "";
```

```
input.disabled = true;
```

```
renderArenaPassage();
```

```
let count = 3;
```

```
num.textContent = count;
```

```
const cdTimer = setInterval(() => {
```

```
    count--;
```

```
    if (count > 0) {
```

```
        num.textContent = count;
```

```
    } else if (count === 0) {
```

```
        num.textContent = 'GO!';
```

```
        num.style.color = 'var(--success)';
```

```
        statusBadge.textContent = 'RACE ACTIVE!';
```

```
        raceActive = true;
```

```
        isCountingDown = false;
```

```
        raceStartTime = performance.now();
```

```
        socket.emit('client_race_active');
```

```
        input.disabled = false;
```

```
        input.focus();
```

```
        startClientRaceTimer(durationSec);
```

```
    } else {
```

```
        clearInterval(cdTimer);
```

```
        banner.style.display = 'none';
```

```
        num.style.color = 'var(--accent)';
```



```

if (!track) {

  track = document.createElement('div');

  track.id = `arena-track-${p.sid}`;

  track.style.background = `var(--bg-card)`;

  track.style.padding = '0.75rem 1rem';

  track.style.borderRadius = `var(--radius-sm)`;

  track.style.border = isMe ? '1.5px solid var(--accent)' : '1px solid var(--border-color)';

  track.innerHTML = `

    <div style="display:flex; justify-content:space-between; font-size:0.85rem; margin-bottom:0.35rem;">

      <span>${label}</span>

      <span class="trk-speed-text" style="font-family:var(--font-mono); font-weight:700; color:${isMe ? `var(--accent)` : `var(--text-primary)`};">0
WPM (0%)</span>

    </div>

    <div style="background:var(--bg-secondary); height:10px; border-radius:5px; overflow:hidden;">

      <div class="trk-speed-bar" style="width:0%; height:100%; background:${isMe ? `var(--accent)` : `#94a3b8`}; transition:width 0.15s
ease;"></div>

    </div>

  `;

  container.appendChild(track);

  trackElements[p.sid] = track;

}

const rankSuffix = p.place ? ` [Finished #${p.place}]` : '';

track.querySelector('.trk-speed-text').textContent = `${Math.round(p.wpm | | 0)} WPM (${Math.round(p.progress | | 0)}%)${rankSuffix}`;

const bar = track.querySelector('.trk-speed-bar');

bar.style.width = `${p.progress | | 0}%`;

if (p.progress >= 100) {

  bar.style.background = `var(--success)`;

}

});

const currentSids = new Set(players.map(p => p.sid));

```

```

Object.keys(trackElements).forEach(sid => {

  if (!currentSids.has(sid)) {

    trackElements[sid].remove();

    delete trackElements[sid];

  }

});

}

```

```

function bindArenaInput() {

  const input = document.getElementById('arena-cockpit-input');

  input.addEventListener('input', () => {

    if (!raceActive || !hasLocallyFinished) {

      input.value = "";

      return;

    }

    const val = input.value;

    const spans = document.querySelectorAll('#arena-passage-text .char');

    currentInputIndex = Math.min(val.length, currentPassage.length);

    let mistakes = 0;

    for (let i = 0; i < spans.length; i++) {

      if (i < val.length) {

        if (val[i] === currentPassage[i]) {

          spans[i].className = 'char correct';

        } else {

          spans[i].className = 'char incorrect';

          mistakes++;

        }

      } else {

        spans[i].className = 'char';

      }

    }

```

```

}

updateArenaCaret();

if (val.length > 0) {

  const lastChar = val[val.length - 1];

  const isCorrect = (val.length <= currentPassage.length) && (lastChar === currentPassage[val.length - 1]);

  if (window.soundEngine) {

    window.soundEngine.playKey(!isCorrect);

  }

}

const progress = Math.min(100, Math.round((currentInputIndex / currentPassage.length) * 100));

const elapsedMin = (performance.now() - raceStartTime) / 60000;

const netWords = Math.max(0, (currentInputIndex - mistakes) / 5.0);

const wpm = elapsedMin > 0 ? Math.round(netWords / elapsedMin) : 0;

const accuracy = currentInputIndex > 0 ? Math.round(((currentInputIndex - mistakes) / currentInputIndex) * 100) : 100;

socket.emit('progress_update', {

  progress: progress,

  wpm: wpm,

  accuracy: accuracy,

  errors: mistakes

});

if (val === currentPassage) {

  hasLocallyFinished = true;

  raceActive = false;

  input.blur();

}

});

input.addEventListener('keydown', (e) => {
```

```

  if (e.key === ' ' && !raceActive) {

    e.preventDefault();

  }

});

}

// =====

// Results Screen & Contextual Feedback

// =====

function showResultsView(standings) {

  showView('results');

  const tbody = document.getElementById('results-standings-tbody');

  tbody.innerHTML = '';

  const winner = standings[0];

  const isWinnerMe = (winner && winner.sid === mySid);

  const trophyIcon = document.getElementById('results-trophy-icon');

  const titleEl = document.getElementById('results-headline-title');

  const bannerEl = document.getElementById('results-motivational-banner');

  if (isWinnerMe) {

    trophyIcon.textContent = "🏆";

    titleEl.textContent = "VICTORY! Outstanding Speed!";

    titleEl.style.color = "var(--warning)";

    bannerEl.style.borderColor = "var(--warning)";

    bannerEl.style.background = "var(--warning-bg)";

    bannerEl.innerHTML = `🏆 <strong>First Place Champion!</strong> You dominated the grid with unmatched precision!`;

  } else {

    trophyIcon.textContent = "🏆";

    titleEl.textContent = "Race Concluded";

    titleEl.style.color = "var(--accent)";

    bannerEl.style.borderColor = "var(--accent)";
```

```
bannerEl.style.background = "var(--accent-glow)";

const myIndex = standings.findIndex(p => p.sid === mySid);

const myRank = myIndex + 1;

let motivation = "Great effort! Every race sharpens your tactile muscle memory.";

if (myRank === 2) {

    motivation = "🥈 Silver finish! You were so close to 1st place — keep this momentum!";

} else if (myRank === 3) {

    motivation = "🥉 Podium finish! Solid precision, keep pushing to close the gap!";

} else {

    motivation = "Don't give up! Challenge the winner to a rematch and take back the lead.";

}

bannerEl.innerHTML = `You placed <strong>#${myRank > 0 ? myRank : 'DNF'}</strong> &bull; ${motivation}`;

}

standings.forEach((p, idx) => {

    const isMe = (p.sid === mySid);

    const tr = document.createElement('tr');

    if (isMe) {

        tr.style.background = 'rgba(56, 189, 248, 0.1)';

    }

    const medal = (idx === 0) ? '🥇 1st' : ((idx === 1) ? '🥈 2nd' : ((idx === 2) ? '🥉 3rd' : ` ${idx + 1}th`));

    const statusTag = p.left_early

        ? '<span style="color:var(--danger); font-weight:700;">DNF / Left</span>'

        : (p.finished ? '<span style="color:var(--success); font-weight:700;">Finished</span>' : '<span style="color:var(--warning); font-weight:700;">Racing</span>');

    tr.innerHTML = `

        <td style="font-weight: 800; color: ${idx === 0 ? 'var(--warning)' : 'var(--text-primary)'};">${medal}</td>

        <td style="font-weight: 700;">${(isMe ? `You (${p.name})` : p.name)}</td>

        <td style="font-family: var(--font-mono); font-weight: 800; color: var(--accent);">${Math.round(p.wpm || 0)} WPM</td>
    `
}
```

```
<td>${Math.round(p.accuracy || 100)}%</td>

<td style="color: var(--danger); font-weight: 700;">${p.errors || 0}</td>

<td>${statusTag}</td>

`;

tbody.appendChild(tr);

});

}

</script>

{% endblock %}

=== FILE: .\app\templates\multiplayer\texts.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card">

    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1.5rem;">

        <h1 style="font-size: 1.8rem; font-weight: 700;">Typing Prompts Database</h1>

        <a href="{{ url_for('admin.index') }}" class="btn">&larr; Back to Admin</a>

    </div>

    <form method="POST" style="background: var(--bg-card); padding: 1.5rem; border-radius: var(--radius); margin-bottom: 2rem;">

        <h3 style="margin-bottom: 1rem;">Add New Prompt Text</h3>

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; margin-bottom: 1rem;">

            <div>

                <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem;">Category</label>

                <input type="text" name="category" placeholder="Literature, Technology, Science, etc." required style="width: 100%; padding: 0.6rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary); font-family: inherit;">

            </div>

            <div>

                <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem;">Code Language (Optional)</label>

                <input type="text" name="code_lang" placeholder="python, javascript, sql" style="width: 100%; padding: 0.6rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary); font-family: inherit;">

            </div>

        </div>
    </form>
</div>
```

```
</div>

<div style="margin-bottom: 1rem;">

  <label style="display: block; font-size: 0.85rem; margin-bottom: 0.4rem;">Content Body</label>

  <textarea name="content" rows="4" required style="width: 100%; padding: 0.75rem; background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: 4px; color: var(--text-primary); font-family: var(--font-mono);"></textarea>

</div>

<div style="margin-bottom: 1.5rem; display: flex; align-items: center; gap: 0.5rem;">

  <input type="checkbox" name="is_code" id="is_code">

  <label for="is_code" style="font-size: 0.85rem;">This prompt is programming source code</label>

</div>

<button type="submit" class="btn btn-primary">Save Prompt</button>

</form>

<h3 style="margin-bottom: 1rem;">Existing Prompts {{{ texts|length }}}</h3>

<table style="width: 100%; border-collapse: collapse;">

  <thead>

    <tr style="border-bottom: 1px solid var(--border-color); color: var(--text-muted); text-align: left; font-size: 0.85rem;">

      <th style="padding: 0.5rem;">ID</th>

      <th style="padding: 0.5rem;">Category</th>

      <th style="padding: 0.5rem;">Preview</th>

      <th style="padding: 0.5rem;">Is Code</th>

    </tr>

  </thead>

  <tbody>

    {% for t in texts %}

      <tr style="border-bottom: 1px solid var(--border-color); font-size: 0.85rem;">

        <td style="padding: 0.5rem;">#{{ t.id }}</td>

        <td style="padding: 0.5rem;">{{ t.category }}</td>

        <td style="padding: 0.5rem; max-width: 450px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap;">{{ t.content }}</td>

        <td style="padding: 0.5rem;">{{ 'Yes (' + t.code_lang + ')' if t.is_code else 'No' }}</td>

      </tr>

    </tbody>

  </table>
```

```
</tr>

{% endfor %}

</tbody>

</table>

</div>

{% endblock %}

=== FILE: .\app\templates\settings\index.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width: 900px; margin: 0 auto;">

  <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 1.75rem; flex-wrap: wrap; gap: 1rem;">

    <div>

      <h1 style="font-size: 1.8rem; font-weight: 700;">Preferences & Ergonomics</h1>

      <p style="color: var(--text-muted); font-size: 0.85rem;">

        {% if current_user.is_authenticated %}

          Logged in as <strong>{{ current_user.username }}</strong>. Changes are saved to your permanent profile.

        {% else %}

          Anonymous Session. Changes apply for this session; refreshing restores default settings.

        {% endif %}

      </p>

    </div>

  </div>

  <!-- Prominent Reset to Default Settings Button -->

  <button onclick="window.settingsManager.resetToDefaults()" class="btn btn-danger" style="font-weight: 700;">

    ⚙️ Reset to Default Settings

  </button>

</div>

<!-- Live Interactive Sandbox Preview -->

<div style="margin-bottom: 2rem;">
```

```
<div style="font-size: 0.8rem; text-transform: uppercase; color: var(--text-muted); font-weight: 700; margin-bottom: 0.5rem;">

  Live Preview Sandbox

</div>

<div class="typing-container focus-ring" style="min-height: 110px; padding: 1.25rem;" onclick="document.getElementById('sandbox-input').focus()">

  <div id="caret" class="caret caret-smooth" style="left: 20px; top: 20px;"></div>

  <div id="sandbox-display" class="typing-text-display">

    The quick brown fox jumps over the lazy dog. Type here to test theme, font, and caret rendering!

  </div>

  <input type="text" id="sandbox-input" style="opacity: 0; position: absolute; left: 0; top: 0;" onkeydown="handleSandboxKey(event)">

</div>

</div>

<div style="display: flex; flex-direction: column; gap: 2rem;">

  <!-- 1. Theme Selection -->

  <div>

    <h3 style="font-size: 1.1rem; margin-bottom: 0.75rem;">Theme</h3>

    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 0.75rem;">

      <button onclick="selectTheme('dark')" class="btn" style="justify-content: center;">Dark Matter</button>

      <button onclick="selectTheme('light')" class="btn" style="justify-content: center;">Clean White</button>

      <button onclick="selectTheme('cyber')" class="btn" style="justify-content: center;">Neon Cyber</button>

      <button onclick="selectTheme('sepia')" class="btn" style="justify-content: center;">Warm Sepia</button>

    </div>

  </div>

  <!-- 2. Typography -->

  <div>

    <h3 style="font-size: 1.1rem; margin-bottom: 0.75rem;">Monospace Typography</h3>

    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 0.75rem;">

      <button onclick="selectFont('JetBrains Mono')" class="btn" style="font-family: 'JetBrains Mono', monospace; justify-content: center;">JetBrains Mono</button>

      <button onclick="selectFont('Fira Code')" class="btn" style="font-family: 'Fira Code', monospace; justify-content: center;">Fira Code</button>

    </div>

  </div>

</div>
```

```
</div>

<!-- 3. Font Size & Caret Variant -->

<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 2rem;">

  <div>

    <div style="display: flex; justify-content: space-between; margin-bottom: 0.75rem;">

      <h3 style="font-size: 1.1rem;">Font Size</h3>

      <span id="font-size-label" style="font-weight: 700; color: var(--accent);">22px</span>

    </div>

    <input type="range" min="16" max="32" value="22" id="font-size-slider" oninput="selectFontSize(this.value)" style="width: 100%; cursor: pointer;">

  </div>

  <div>

    <h3 style="font-size: 1.1rem; margin-bottom: 0.75rem;">Caret Style</h3>

    <div style="display: flex; gap: 0.5rem;">

      <button onclick="selectCaret('smooth')" class="btn" style="flex: 1; justify-content: center;">Smooth Line</button>

      <button onclick="selectCaret('block')" class="btn" style="flex: 1; justify-content: center;">Solid Block</button>

      <button onclick="selectCaret('underline')" class="btn" style="flex: 1; justify-content: center;">Underline</button>

    </div>

  </div>

</div>

<!-- 4. Audio Synthesis -->

<div>

  <h3 style="font-size: 1.1rem; margin-bottom: 0.75rem;">Acoustic Audio Theme</h3>

  <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 0.75rem;">

    <button onclick="selectSound('mechanical')" class="btn" style="justify-content: center;">Cherry MX</button>

    <button onclick="selectSound('bubble')" class="btn" style="justify-content: center;">Bubble Pop</button>

    <button onclick="selectSound('beep')" class="btn" style="justify-content: center;">Digital Synth</button>

  </div>

  </div>

</div>
```



```
</div>

{% endblock %}

{% block scripts %}

<script>

function selectTheme(t) { window.settingsManager.update('theme', t); }

function selectFont(f) { window.settingsManager.update('font_family', f); }

function selectFontSize(s) {

    document.getElementById('font-size-label').textContent = s + 'px';

    window.settingsManager.update('font_size', parseInt(s, 10));

}

function selectCaret(c) { window.settingsManager.update('caret_style', c); }

function selectSound(s) {

    window.settingsManager.update('sound_theme', s);

    if (window.soundEngine) window.soundEngine.playKey(false);

}

function handleSandboxKey(e) {

    if (window.soundEngine) window.soundEngine.playKey(e.key === 'Backspace');

}

</script>

{% endblock %}
```

```
=== FILE: .\app\templates\typing\certificate.html ===

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content=" width=device-width, initial-scale=1.0">

    <title>Official Certificate of Typing Proficiency - {{ candidate_name }}</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/main.css') }}">

<style>

    * {

        box-sizing: border-box;
```

```
margin: 0;

padding: 0;

}

body {

    background: #0b0f17;

    color: #1a202c;

    font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;

    min-height: 100vh;

    display: flex;

    flex-direction: column;

    align-items: center;

    justify-content: center;

    padding: 1.5rem 1rem;

}

/* Mobile orientation recommendation */

.mobile-rotate-hint {

    display: none;

    background: rgba(88, 166, 255, 0.15);

    border: 1px solid var(--accent);

    color: var(--accent);

    padding: 0.5rem 1rem;

    border-radius: 20px;

    font-size: 0.8rem;

    font-weight: 600;

    margin-bottom: 1rem;

    text-align: center;

}

@media (max-width: 768px) {

    .mobile-rotate-hint {

        display: block;
```

```
}

}

/* Fluid Responsive Certificate Container */

.cert-container {

  width: 100%;

  max-width: 860px;

  min-height: 560px;

  background: #ffffff;

  border: 10px solid #1e293b;

  outline: 3px solid {{ tier_color }};

  outline-offset: -7px;

  padding: clamp(1.25rem, 4vw, 3rem);

  position: relative;

  box-shadow: 0 20px 45px rgba(0, 0, 0, 0.7);

  background-image: radial-gradient(#e2e8f0 1px, transparent 1px);

  background-size: 20px 20px;

  display: flex;

  flex-direction: column;

  justify-content: space-between;

}

.cert-header {

  text-align: center;

  border-bottom: 2px solid #e2e8f0;

  padding-bottom: 1rem;

  margin-bottom: 1.25rem;

}

.cert-title {

  font-size: clamp(1.2rem, 3.5vw, 2.1rem);

  font-weight: 800;

  letter-spacing: 0.05em;
```

```
text-transform: uppercase;

color: #0f172a;

}

.cert-subtitle {

  font-size: clamp(0.7rem, 2vw, 0.95rem);

  color: #64748b;

  margin-top: 0.25rem;

  text-transform: uppercase;

  letter-spacing: 0.12em;

}

.cert-body {

  text-align: center;

  flex: 1;

  display: flex;

  flex-direction: column;

  justify-content: center;

}

.cert-awarded-to {

  font-size: clamp(0.75rem, 2vw, 0.85rem);

  color: #64748b;

  text-transform: uppercase;

  letter-spacing: 0.1em;

  margin-bottom: 0.35rem;

}

.cert-name {

  font-size: clamp(1.5rem, 5vw, 2.4rem);

  font-weight: 800;

  color: #0f172a;

  border-bottom: 2px solid #cbd5e1;
```

```
display: inline-block;

padding: 0 1.5rem 0.25rem;

margin: 0 auto 1.25rem;

font-family: Georgia, serif;

word-break: break-word;

}

.cert-text {

font-size: clamp(0.85rem, 2.2vw, 1rem);

line-height: 1.6;

color: #334155;

max-width: 620px;

margin: 0 auto 1.5rem;

}

.cert-stats {

display: flex;

justify-content: center;

gap: clamp(1rem, 4vw, 3rem);

margin-bottom: 1.5rem;

flex-wrap: wrap;

}

.cert-stat-box {

text-align: center;

min-width: 80px;

}

.cert-stat-val {

font-size: clamp(1.4rem, 4vw, 2.1rem);

font-weight: 800;

color: #0f172a;

font-family: 'JetBrains Mono', monospace;
```

```
}

.cert-stat-lbl {

font-size: clamp(0.65rem, 1.8vw, 0.78rem);

color: #64748b;

text-transform: uppercase;

letter-spacing: 0.05em;

}

/* Footer & Seal Section */

.cert-bottom-row {

display: flex;

justify-content: space-between;

align-items: flex-end;

margin-top: 1rem;

flex-wrap: wrap;

gap: 1rem;

}

.cert-footer-meta {

font-size: clamp(0.7rem, 2vw, 0.8rem);

color: #64748b;

line-height: 1.5;

text-align: left;

}

.cert-seal {

width: clamp(75px, 15vw, 105px);

height: clamp(75px, 15vw, 105px);

border-radius: 50%;

border: 3px dashed {{ tier_color }};

display: flex;

flex-direction: column;
```

```
align-items: center;

justify-content: center;

background: #f8fafc;

box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);

transform: rotate(-10deg);

flex-shrink: 0;

}

.cert-seal-text {

font-size: clamp(0.7rem, 2.2vw, 0.85rem);

font-weight: 800;

color: {{ tier_color }};

text-transform: uppercase;

letter-spacing: 0.1em;

}

.action-bar {

margin-top: 1.5rem;

display: flex;

gap: 0.75rem;

flex-wrap: wrap;

justify-content: center;

width: 100%;

max-width: 860px;

}

/* Print & PDF formatting */

@media print {

body {

background: none;

padding: 0;

min-height: auto;

}

}
```

```
.action-bar, .mobile-rotate-hint, .navbar {

display: none !important;

}

.cert-container {

width: 100%;

max-width: 100%;

height: 600px;

box-shadow: none;

page-break-inside: avoid;

}

}

</style>

</head>

<body>

<div class="mobile-rotate-hint">

 Tip: Rotate your phone to landscape for full certificate view

</div>

<div class="cert-container">

<!-- Header -->

<div class="cert-header">

<h1 class="cert-title">TypeSphere Certificate of Competence</h1>

<div class="cert-subtitle">Standardized Benchmark & Biometric Verification</div>

</div>

<!-- Body -->

<div class="cert-body">

<div class="cert-awarded-to">This certifies that</div>

<div class="cert-name">{{ candidate_name }}</div>

<div class="cert-text">

has successfully completed an authorized typing benchmark under standard test parameters, exhibiting verified speed and accuracy.

</div>

</div>

</div>
```

```
<!-- Telemetry Stats -->

<div class="cert-stats">

  <div class="cert-stat-box">

    <div class="cert-stat-val">{{ test.wpm }}</div>

    <div class="cert-stat-lbl">Net Speed (WPM)</div>

  </div>

  <div class="cert-stat-box">

    <div class="cert-stat-val">{{ test.accuracy }}%</div>

    <div class="cert-stat-lbl">Accuracy</div>

  </div>

  <div class="cert-stat-box">

    <div class="cert-stat-val">{{ test.duration }}s</div>

    <div class="cert-stat-lbl">Duration</div>

  </div>

</div>

<!-- Bottom Row: Meta & Seal -->

<div class="cert-bottom-row">

  <div class="cert-footer-meta">

    <div><strong>Verification ID:</strong> {{ cert_code }}</div>

    <div><strong>Issued On:</strong> {{ test.completed_at.strftime("%B %d, %Y") }}</div>

    <div><strong>Classification:</strong> {{ cert_tier }}</div>

  </div>

  <div class="cert-seal">

    <span class="cert-seal-text">{{ tier_badge }}</span>

    <span style="font-size: 0.6rem; color: #64748b; font-weight: 700;">VERIFIED</span>

  </div>

</div>

</div>
```

```
<!-- Actions -->

<div class="action-bar">

  <button onclick="window.print()" class="btn btn-primary" style="padding: 0.75rem 1.5rem; font-size: 0.95rem;">

    🖨️ Print / Download PDF

  </button>

  <a href="{{ url_for('typing.result_page', test_id=test.id) }}" class="btn" style="padding: 0.75rem 1.5rem; font-size: 0.95rem;">

    ⏪ Back to Results

  </a>

</div>

</body>

</html>

=== FILE: .\app\templates\typing\custom.html ===

{% extends 'base.html' %}

{% block content %}

  <div class="card" style="max-width: 840px; margin: 0 auto;">

    <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

      <div>

        <h1 style="font-size: 1.8rem; font-weight: 700; margin: 0;">Custom Text Practice Studio</h1>

        <p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.35rem;">

          Paste your text, choose your timer countdown, and launch your personalized benchmark.

        </p>

      </div>

      <a href="{{ url_for('typing.test_page') }}" class="btn">&larr; Return to Test Track</a>

    </div>

    <!-- Real-time Text Telemetry -->

    <div class="stats-hud" style="grid-template-columns: repeat(3, 1fr); margin-bottom: 1.5rem;">

      <div class="stat-box">

        <div class="stat-label">Word Count</div>
```

```
<div class="stat-value" id="stat-word-count">0</div>

</div>

<div class="stat-box">

  <div class="stat-label">Character Count</div>

  <div class="stat-value" id="stat-char-count">0</div>

</div>

<div class="stat-box">

  <div class="stat-label">Est. Time (@ 60 WPM)</div>

  <div class="stat-value" id="stat-est-time">0s</div>

</div>

</div>

<!-- Duration Mode Selection -->

<div style="margin-bottom: 1.5rem; background: var(--bg-card); padding: 1rem 1.25rem; border-radius: var(--radius); border: 1px solid var(--border-color);">

  <label style="display: block; font-size: 0.85rem; font-weight: 700; color: var(--text-primary); margin-bottom: 0.5rem; text-transform: uppercase; letter-spacing: 0.05em;">

    Select Test Timer (Countdown Limit)

  </label>

  <div style="display: flex; gap: 0.75rem; flex-wrap: wrap; id="duration-selector">

    <button type="button" class="btn active-choice" onclick="selectDuration(60, this)" data-tooltip="Timer will start at 60s and count down to 0, automatically finishing test">1 Minute (60s Countdown)</button>

    <button type="button" class="btn" onclick="selectDuration(120, this)" data-tooltip="Timer will start at 120s and count down to 0">2 Minutes (120s Countdown)</button>

    <button type="button" class="btn" onclick="selectDuration(180, this)" data-tooltip="Timer will start at 180s and count down to 0">3 Minutes (180s Countdown)</button>

    <button type="button" class="btn" onclick="selectDuration(0, this)" data-tooltip="No countdown limit. Test runs until you type the entire custom paragraph">Full Passage (No Time Limit)</button>

  </div>

</div>

<!-- Main Text Input -->

<div style="margin-bottom: 1.5rem;">

  <label style="display: block; font-size: 0.85rem; margin-bottom: 0.5rem; color: var(--text-muted); text-transform: uppercase; letter-spacing: 0.05em;">

    Paste Content Below

  </div>

</div>
```

```
</label>

  <textarea id="custom-studio-input" rows="8" placeholder="Paste your custom study text, lyrics, or programming code here..."
oninput="updateStats()" style="width: 100%; padding: 1rem; background: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius); color: var(--text-primary); font-family: var(--font-mono); font-size: 1rem; line-height: 1.6; outline: none;"></textarea>

</div>

<!-- Quick Presets -->

<div style="margin-bottom: 2rem;">

  <div style="font-size: 0.8rem; color: var(--text-muted); margin-bottom: 0.5rem;">Or load a curated preset:</div>

  <div style="display: flex; gap: 0.5rem; flex-wrap: wrap;">

    <button onclick="loadPreset('tech')" class="btn" style="padding: 0.3rem 0.65rem; font-size: 0.8rem;">Distributed Systems</button>

    <button onclick="loadPreset('lit')" class="btn" style="padding: 0.3rem 0.65rem; font-size: 0.8rem;">Classic Literature</button>

    <button onclick="loadPreset('code')" class="btn" style="padding: 0.3rem 0.65rem; font-size: 0.8rem;">Python Quicksort</button>

  </div>

</div>

<!-- Action Button -->

<div style="display: flex; justify-content: flex-end; gap: 0.75rem;">

  <button onclick="clearText()" class="btn">Clear</button>

  <button onclick="launchCustomTest()" class="btn btn-primary" style="padding: 0.75rem 2rem; font-size: 1rem; font-weight: 700;">

    Launch Custom Practice Track &#x2192;

  </button>

</div>

</div>

{% endblock %}

{% block scripts %}

<script>

let selectedCustomDuration = 60; // default 1 min

const PRESETS = {

  tech: "Distributed event streaming architectures enable microservices to communicate asynchronously with high throughput and horizontal scalability. By decoupling producers from consumers through immutable partitioned logs, systems achieve resilience against node failures.",

  lit: "The Great Gatsby is a 1925 novel by American writer F. Scott Fitzgerald. It is a critique of the American Dream, showing the moral decay of the 1920s. The story is set in West Egg, Long Island, and follows the lives of Jay Gatsby, a mysterious millionaire, and Nick Carraway, a stock bond trader who becomes Gatsby's friend and confidant. The novel is a classic of American literature, known for its lyrical prose and exploration of themes such as wealth, power, and the pursuit of the American Dream.",

  code: "A quicksort algorithm is a divide-and-conquer sorting algorithm. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then recursively sorted. Quicksort has an average-case time complexity of O(n log n) and a worst-case time complexity of O(n^2). It is an in-place algorithm, meaning it does not require additional memory for sorting."

}
```

lit: "It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of light, it was the season of darkness."

```
code: "def binary_search(array, target):\n    low, high = 0, len(array) - 1\n    while low <= high:\n        mid = (low + high) // 2\n        if array[mid] == target:\n            return mid\n        elif array[mid] < target:\n            low = mid + 1\n        else:\n            high = mid - 1\n    return -1"
```

```
function selectDuration(seconds, btn) {\n\n    selectedCustomDuration = seconds;\n\n    document.querySelectorAll('#duration-selector .btn').forEach(b => b.classList.remove('active-choice'));\n\n    btn.classList.add('active-choice');\n\n}
```

```
function updateStats() {\n\n    const text = document.getElementById('custom-studio-input').value.trim();\n\n    const words = text ? text.split(/\\s+/).length : 0;\n\n    const chars = text.length;\n\n    const estSeconds = Math.round((words / 60) * 60);\n\n\n    document.getElementById('stat-word-count').textContent = words;\n\n    document.getElementById('stat-char-count').textContent = chars;\n\n    document.getElementById('stat-est-time').textContent = `${estSeconds}s`;\n\n}
```

```
function loadPreset(key) {\n\n    document.getElementById('custom-studio-input').value = PRESETS[key];\n\n    updateStats();\n\n}
```

```
function clearText() {\n\n    document.getElementById('custom-studio-input').value = "";\n\n    updateStats();\n\n}
```

```
function launchCustomTest() {\n\n    const text = document.getElementById('custom-studio-input').value.trim();\n\n    if (!text || text.length < 5) {\n\n        alert('Please enter at least 5 characters to start a custom typing test.');
```

```
document.addEventListener('DOMContentLoaded', () => {\n\n    updateStats();\n\n});\n</script>\n{% endblock %}
```

=== FILE: .\\app\\templates\\typing\\practice.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width: 1060px; margin: 0 auto;">

<!-- Academy Header -->

<div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 1.25rem; margin-bottom: 2rem; flex-wrap: wrap; gap: 1rem;">

<div>

<h1 style="font-size: 1.8rem; font-weight: 800; margin: 0; display: flex; align-items: center; gap: 0.6rem;">

📖 TypeSphere Academy & Practice Studio

</h1>

<p style="color: var(--text-muted); font-size: 0.85rem; margin-top: 0.35rem;">

AI-driven biometric weakness drills and structured touch-typing curriculum from Home Row to Advanced Code.

</p>

</div>

<div style="display: flex; gap: 0.6rem;">

← Return to Test Track

</div>

</div>

<!-- ===== -->

<!-- 1. AI BIOMETRIC WEAKNESS COACH -->

<!-- ===== -->

<div class="card" style="background: var(--bg-card); border: 1.5px solid var(--accent); margin-bottom: 2rem;">

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1rem; flex-wrap: wrap; gap: 0.75rem;">

<div>

AI Telemetry Coach

<h3 style="font-size: 1.25rem; margin: 0.35rem 0 0;">Personalized Weakness Prescription</h3>

</div>

<button onclick="startAdaptiveDrill()" class="btn btn-primary" style="font-weight: 800; padding: 0.65rem 1.5rem;">

⚡Train My Weaknesses Now

</button>

</div>

<p style="color: var(--text-secondary); font-size: 0.9rem; line-height: 1.6; margin-bottom: 1.25rem;">

{% if weak_keys %}

Our analytical engine inspected your typing DNA and identified sub-second latency and error spikes around keys:

{% for k, err, cnt in weak_keys %}'{ k }' {{ err }}{% err %}{% if not loop.last %}, {% endif %}{% endfor %}.

Starting an adaptive session isolates these specific n-grams to retrain finger muscle memory.

{% else %}

Complete a few benchmark typing tests while signed in to unlock deep biometric diagnosis of your error keys and finger imbalances.

{% endif %}

</p>

<!-- Weakness Metrics Grid -->

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 1rem;">

<div style="background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); padding: 0.85rem 1rem;">

<div style="font-size: 0.75rem; color: var(--text-muted); text-transform: uppercase; font-weight: 700;">Left vs Right Balance</div>

<div style="font-size: 1.25rem; font-weight: 800; font-family: var(--font-mono); color: var(--accent); margin-top: 0.25rem;">

{ { dna.left_hand_accuracy | round(1) if dna else 98.2 } }% / { { dna.right_hand_accuracy | round(1) if dna else 99.1 } }%

</div>

</div>

<div style="background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); padding: 0.85rem 1rem;">

<div style="font-size: 0.75rem; color: var(--text-muted); text-transform: uppercase; font-weight: 700;">Primary Confusion Pair</div>

<div style="font-size: 1.15rem; font-weight: 800; color: var(--warning); margin-top: 0.25rem;">

{% if top_confusions %}

Expected '{ { top_confusions[0][0] } }' → Typed '{ { top_confusions[0][1] } }'

{% else %}

None detected yet

{% endif %}

</div>

</div>

<div style="background: var(--bg-secondary); border: 1px solid var(--border-color); border-radius: var(--radius-sm); padding: 0.85rem 1rem;">

<div style="font-size: 0.75rem; color: var(--text-muted); text-transform: uppercase; font-weight: 700;">Punctuation Stability</div>

<div style="font-size: 1.25rem; font-weight: 800; font-family: var(--font-mono); color: var(--success); margin-top: 0.25rem;">

{ { dna.punctuation_accuracy | round(1) if dna else 96.0 } }%

</div>

</div>

</div>

</div>

<!-- ===== -->

<!-- 2. STRUCTURED TOUCH-TYPING CURRICULUM -->

<!-- ===== -->


```

<div style="border-top: 1px solid var(--border-color); padding-top: 2rem;">

<h2 style="font-size: 1.4rem; font-weight: 800; margin-bottom: 0.4rem;">6-Stage Touch Typing Curriculum</h2>

<p style="color: var(--text-muted); font-size: 0.85rem; margin-bottom: 1.5rem;">

  Progressive mechanical mastery designed for touch typists, developers, and speed competitors.

</p>

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(310px, 1fr)); gap: 1.25rem;">

  {% for key, lesson in curriculum.items() %}

    <div class="card" style="background: var(--bg-card); border: 1px solid var(--border-color); margin-bottom: 0; display: flex; flex-direction:
column; justify-content: space-between;">

      <div>

        <div style="display: flex; justify-content: space-between; align-items: flex-start; margin-bottom: 0.5rem;">

          <h3 style="font-size: 1.1rem; font-weight: 700; margin: 0; color: var(--text-primary);">

            {{ lesson.title }}

          </h3>

          <span style="font-size: 0.7rem; font-weight: 800; background: var(--bg-secondary); border: 1px solid var(--border-color); color: var(--accent);
padding: 0.15rem 0.45rem; border-radius: 4px;">

            STAGE {{ loop.index }}

          </span>

        </div>

        <div style="font-family: var(--font-mono); font-size: 0.85rem; color: var(--accent); font-weight: 700; margin-bottom: 0.6rem;">

          Keys: {{ lesson.keys }}

        </div>

        <p style="color: var(--text-muted); font-size: 0.85rem; line-height: 1.5; margin-bottom: 1rem;">

          {{ lesson.description }}

        </p>

        </div>

        <button onclick="launchLesson('{{ key }})" class="btn btn-primary" style="width: 100%; justify-content: center; font-weight: 700; padding:
0.65rem;">

          Practice Lesson &rrr;

        </button>

```

```

</div>
{% endfor %}

</div>

</div>

{% endblock %}

{% block scripts %}

<script>

function startAdaptiveDrill() {

    window.location.href = "{{ url_for('typing.test_page') }}"?mode=adaptive";

}

function launchLesson(lessonKey) {

    window.location.href = "{{ url_for('typing.test_page') }}"?lesson=${encodeURIComponent(lessonKey)}";

}

</script>

{% endblock %}

=== FILE: .\app\templates\typing\result.html ===

{% extends 'base.html' %}

{% block content %}

<div class="card" style="max-width:980px; margin:0 auto;">

<!-- Result Header -->

<div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:1.25rem; margin-bottom:1.5rem; flex-wrap:wrap; gap:1rem;">

<div>

<div style="display:flex; align-items:center; gap:0.6rem;">

<h1 style="font-size:1.8rem; font-weight:800; margin:0;">Test Results</h1>

<span style="background:{{ tier_color }}; color:#04101e; font-weight:800; font-size:0.75rem; padding:0.2rem 0.55rem; border-radius:4px; text-transform:uppercase;">

    {{ skill_tier }}


```

```
</span>

{% if test.is_ranked %}

    <span style="background:var(--success-bg); color:var(--success); font-weight:800; font-size:0.75rem; padding:0.2rem 0.55rem; border-radius:4px;">

        VERIFIED RANKED

    </span>

{% else %}

    <span style="background:var(--bg-card); color:var(--text-muted); font-size:0.75rem; padding:0.2rem 0.55rem; border-radius:4px;">

        PRACTICE RUN

    </span>

{% endif %}

</div>

<p style="color:var(--text-secondary); font-size:0.85rem; margin-top:0.25rem;">

    Completed on {{ test.completed_at.strftime('%b %d, %Y at %H:%M') }} &bull; Mode: {{ test.mode }}

</p>

</div>

<!-- Essential Actions -->

<div style="display:flex; gap:0.5rem; flex-wrap:wrap;">

    <a href="{{ url_for('typing.test_page') }}" class="btn btn-primary">&larr; Next Test (Tab)</a>

    <a href="{{ url_for('typing.test_page') }}"?retry_test_id={{ test.id }}" class="btn">↺ Retry Text</a>

    <a href="{{ url_for('typing.certificate_page', test_id=test.id) }}" class="btn" style="border-color:var(--warning); color:var(--warning);">↻ Certificate</a>

</div>

</div>

<!-- Primary Results Card: High-Impact Essential Data First -->

<div class="stats-hud" style="grid-template-columns:repeat(auto-fit, minmax(140px, 1fr)); margin-bottom:1.75rem;">

    <div class="stat-box">

        <div class="stat-label">Net Speed</div>

        <div class="stat-value" style="font-size:2.2rem; color:var(--accent);">{{ test.wpm }} <span style="font-size:0.9rem; color:var(--text-muted);">WPM</span></div>

    </div>

    <div class="stat-box">
```

```
<div class="stat-label">Accuracy</div>

<div class="stat-value" style="font-size:2.2rem; color:{{ 'var(--success)' if test.accuracy >= 95 else 'var(--warning)' }};">{{ test.accuracy }}%</div>

</div>

<div class="stat-box">

    <div class="stat-label">Consistency</div>

    <div class="stat-value" style="font-size:2.2rem;">{{ test.consistency }}%</div>

</div>

<div class="stat-box">

    <div class="stat-label">Errors Logged</div>

    <div class="stat-value" style="font-size:2.2rem; color:var(--danger);">{{ test.errors }}</div>

</div>

</div>

<!-- =====

    INTELLIGENT ECOSYSTEM RECOMMENDATION (CONNECTING ALL FEATURES)

    ===== -->

{% if recommendation %}

<div class="card" style="background:var(--bg-card); border:1.5px solid var(--accent); margin-bottom:1.75rem; padding:1.25rem 1.5rem;">

    <div style="display:flex; justify-content:space-between; align-items:flex-start; flex-wrap:wrap; gap:1rem;">

        <div style="max-width:640px;">

            <span style="font-size:0.75rem; font-weight:800; background:var(--accent-glow); color:var(--accent); padding:0.15rem 0.5rem; border-radius:4px; text-transform:uppercase;">

                Intelligent Next Step

            </span>

            <h3 style="font-size:1.15rem; font-weight:800; margin:0.4rem 0 0.25rem 0;">{{ recommendation.title }}</h3>

            <p style="color:var(--text-secondary); font-size:0.88rem; line-height:1.5; margin:0;">

                {{ recommendation.reason }}

            </p>

        </div>

    </div>

    <div style="display:flex; gap:0.5rem; flex-wrap:wrap;">

        <a href="{{ recommendation.action_url }}" class="btn btn-primary" style="font-weight:700;">

            {{ recommendation.action_label }} &rarr;

        </a>

    </div>

</div>
```

```
</a>

</div>

</div>

<!-- Suggested Curriculum & Game Pathways -->

<div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(220px, 1fr)); gap:0.75rem; margin-top:1.25rem; border-top:1px solid
var(--border-color); padding-top:0.85rem; font-size:0.82rem;">

  <div>

    <span style="color:var(--text-muted);">Recommended Academy Lesson:</span><br>

    <a href="{{ url_for('learn.index') }}" style="color:var(--accent); font-weight:700; text-decoration:none;">

      {{ recommendation.stage_recommendation }} &rarr;

    </a>

  </div>

  <div>

    <span style="color:var(--text-muted);">Recommended Skill-Building Game:</span><br>

    <a href="{{ url_for('games.index') }}" style="color:var(--warning); font-weight:700; text-decoration:none;">

      ☞ {{ recommendation.game_recommendation }} &rarr;

    </a>

  </div>

</div>

</div>

{% endif %}
```

```
<!-- Speed Velocity Timeline -->

<div style="margin-bottom:1.75rem;">

  <h3 style="font-size:1.1rem; font-weight:700; margin-bottom:0.75rem;">Speed Progression</h3>

  <div id="chart-timeline" style="background:var(--bg-card); border:1px solid var(--border-color); border-radius:var(--radius-sm); padding:1rem;
min-height:220px;"></div>

</div>

<!-- Detailed Diagnostics Dropdown -->

<div style="border-top:1px solid var(--border-color); padding-top:1.25rem;">

  <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:1rem;">
```

```
<h3 style="font-size:1.05rem; font-weight:700; margin:0;">Detailed Biometric Metrics</h3>

<button onclick="toggleDetails()" id="btn-toggle-details" class="btn btn-sm">

  View Detailed Diagnostics &darr;

</button>

</div>

<div id="detailed-diagnostics-panel" style="display:none; grid-template-columns:repeat(auto-fit, minmax(280px, 1fr)); gap:1rem;">

  <div class="card" style="background:var(--bg-card); margin-bottom:0;">

    <h4 style="font-size:0.9rem; margin-bottom:0.6rem;">Keystroke Breakdown</h4>

    <div style="font-size:0.85rem; display:flex; flex-direction:column; gap:0.4rem;">

      <div style="display:flex; justify-content:space-between;"><span style="color:var(--text-muted);">Correct Characters:</span><strong>{{
test.correct_chars }}</strong></div>

      <div style="display:flex; justify-content:space-between;"><span style="color:var(--text-muted);">Incorrect Characters:</span><strong>{{
style="color:var(--danger);">{{ test.incorrect_chars }}</strong></div>

      <div style="display:flex; justify-content:space-between;"><span style="color:var(--text-muted);">Gross Uncorrected Speed:</span><strong>{{
test.raw_wpm }} WPM</strong></div>

    </div>

  </div>

  <div class="card" style="background:var(--bg-card); margin-bottom:0;">

    <h4 style="font-size:0.9rem; margin-bottom:0.6rem;">Mistake Locations in this Run</h4>

    <div style="font-size:0.85rem; display:flex; flex-direction:column; gap:0.35rem;">

      {% for k, cnt in difficult_keys %}

        <div style="display:flex; justify-content:space-between; background:var(--bg-secondary); padding:0.3rem 0.6rem; border-radius:4px;">

          <span>Key '<strong>{{ k }}</strong>'</span>

          <span style="color:var(--danger); font-weight:700;">{{ cnt }} typo{{ 's' if cnt > 1 else '' }}</span>

        </div>

      {% else %}

        <span style="color:var(--success);">Zero character errors detected!</span>

      {% endfor %}

    </div>

  </div>

</div>

</div>

</div>

</div>
```

```
</div>

{% endblock %}

{% block scripts %}

<script>

document.addEventListener('DOMContentLoaded', () => {

  try {

    const rawData = {{ test.timeline_data|tojson|safe if test.timeline_data else '[]' }};

    const timelineData = typeof rawData === 'string' ? JSON.parse(rawData) : rawData;

    if (Array.isArray(timelineData) && timelineData.length > 1) {

      SvgLineChart.render('chart-timeline', timelineData, '#38bdf8');

    }

  } catch {}

});

function toggleDetails() {

  const panel = document.getElementById('detailed-diagnostics-panel');

  const btn = document.getElementById('btn-toggle-details');

  if (panel.style.display === 'none') {

    panel.style.display = 'grid';

    btn.textContent = 'Hide Detailed Diagnostics ↑';

  } else {

    panel.style.display = 'none';

    btn.textContent = 'View Detailed Diagnostics ↓';

  }

}

</script>

{% endblock %}

=== FILE: .\app\templates\typing\test.html ===

{% extends 'base.html' %}

{% block content %}
```

```
<!-- Pause Modal -->

<div id="pause-modal" class="modal-backdrop" onclick="iff(event.target===this)window.typingEngine.closePauseModal()" style="display:none;">

<div class="card modal-dialog" style="text-align:center;">

  <button onclick="window.typingEngine.closePauseModal()" class="modal-close-btn" aria-label="Close">&times;</button>

  <div style="font-size:2rem; margin-bottom:0.4rem;">⏸</div>

  <h2 style="font-size:1.4rem; font-weight:800; margin-bottom:0.3rem;">Benchmark Paused</h2>

  <p style="color:var(--text-muted); font-size:0.85rem; margin-bottom:1.5rem;">

    Timer is frozen. Closing this window will not cancel your progress.

  </p>

  <div style="display:flex; flex-direction:column; gap:0.5rem;">

    <button onclick="window.typingEngine.resumeTest()" class="btn btn-primary" style="padding:0.75rem; font-weight:700;">

      ▶ Resume Benchmark (Space / Esc)

    </button>

    <button onclick="window.typingEngine.loadPrompt(false)" class="btn" style="padding:0.65rem;">

      ⌂ Restart Benchmark (Tab)

    </button>

  </div>

</div>

</div>

</div>

<!-- Paused Floating HUD Bar -->

<div id="paused-floating-bar" style="display:none; justify-content:center; align-items:center; gap:1rem; background:var(--bg-card); border:1.5px solid var(--warning); padding:0.65rem 1.25rem; border-radius:var(--radius); margin-bottom:1.25rem; max-width:620px; margin-left:auto; margin-right:auto;">

  <span style="color:var(--warning); font-weight:800; font-size:0.9rem;">⏸ TEST IS PAUSED</span>

  <div style="display:flex; gap:0.5rem;">

    <button onclick="window.typingEngine.resumeTest()" class="btn btn-primary btn-sm">▶ Resume</button>

    <button onclick="window.typingEngine.loadPrompt(false)" class="btn btn-sm">⌂ Restart</button>

  </div>

</div>

<!-- =====

TIER 1: CLEAN TEST CONFIGURATION & STATUS
```

```
===== -->

<div class="card" style="padding:0.85rem 1.25rem; margin-bottom:1.25rem;">

  <div style="display:flex; justify-content:space-between; align-items:center; flex-wrap:wrap; gap:0.85rem;">

    <div style="display:flex; align-items:center; gap:0.6rem; flex-wrap:wrap;">

      <!-- Time Selector -->

      <div class="dropdown-control">

        <label for="cfg-time-select" class="dropdown-label">Time</label>

        <select id="cfg-time-select" onchange="onTimeChange(this.value)" class="form-select">

          <option value="15">15 sec</option>

          <option value="30">30 sec</option>

          <option value="60" selected>1 min</option>

          <option value="120">2 min</option>

          <option value="300">5 min</option>

          <option value="600">10 min</option>

          <option value="1200">20 min</option>

          <option value="0">No Time Limit</option>

        </select>

      </div>

      <!-- Content Type -->

      <div class="dropdown-control">

        <label for="cfg-type-select" class="dropdown-label">Content</label>

        <select id="cfg-type-select" onchange="onTypeChange(this.value)" class="form-select">

          <option value="words" selected>Words & Sentences</option>

          <option value="paragraphs">Full Passages</option>

          <option value="quotes">Quotes</option>

          <option value="numbers">Numbers Only</option>

          <option value="punctuation">Punctuation Focus</option>

          <option value="professional">Office / Work Memos</option>

          <option value="data_entry">Data Entry</option>

          <option value="code">Programming Code</option>

          <option value="custom">Custom Text</option>

        </select>

      </div>

    </div>

  </div>

</div>
```

```
</select>

</div>

<!-- Difficulty Level -->

<div class="dropdown-control">

  <label for="cfg-diff-select" class="dropdown-label">Difficulty</label>

  <select id="cfg-diff-select" onchange="onDifficultyChange(this.value)" class="form-select">

    <option value="easy">Easy (Common, short words)</option>

    <option value="moderate" selected>Moderate (Standard English)</option>

    <option value="hard">Hard (Advanced syllables)</option>

    <option value="expert">Expert (Complex vocabulary)</option>

  </select>

</div>

<!-- Ranked Match Eligibility Badge -->

<div id="rank-eligibility-badge" style="font-size:0.8rem; font-weight:700; margin-left:0.25rem;"></div>

</div>

<!-- Quick Action Controls -->

<div style="display:flex; align-items:center; gap:0.4rem; flex-wrap:wrap;">

  <button onclick="toggleBlindQuick()" id="btn-blind" class="btn btn-sm active-choice" title="Disables Backspacing">

    Blind: ON

  </button>

  <button onclick="toggleGhostQuick()" id="btn-ghost" class="btn btn-sm" title="Ghost Pacer">

    Ghost: OFF

  </button>

  <button onclick="window.typingEngine.togglePause()" class="btn btn-sm" style="color:var(--accent);">

    ⏸ Pause

  </button>

  <button onclick="window.typingEngine.loadPrompt(false)" class="btn btn-sm" style="color:var(--warning);">

    ⏮ Restart

  </button>

</div>
```

```
</div>

<!-- Dynamic Content & Level Hint -->

<div id="config-explainer-hint" style="font-size:0.8rem; color:var(--text-muted); margin-top:0.6rem; border-top:1px solid var(--border-color); padding-top:0.5rem;">

    Moderate Level: Standard vocabulary, common punctuation, and flowing sentence structures.

</div>

</div>

<!-- =====

LIVE TELEMETRY HUD (Essential Live Information Only)

===== -->

<div class="stats-hud">

<div class="stat-box">

<svg class="speedometer-svg" viewBox="0 0 36 36">

<path class="speedo-bg" d="M18 2.0845 a 15.9155 15.9155 0 0 1 0 31.831 a 15.9155 15.9155 0 0 1 0 -31.831" />

<path id="speedo-arc" class="speedo-fill" stroke-dasharray="100, 100" stroke-dashoffset="100" d="M18 2.0845 a 15.9155 15.9155 0 0 1 0 31.831 a 15.9155 15.9155 0 0 1 0 -31.831" />

</svg>

<div class="stat-label">Net WPM</div>

<div class="stat-value" id="hud-wpm">0</div>

</div>

<div class="stat-box">

<div class="stat-label">Accuracy</div>

<div class="stat-value" id="hud-acc">100%</div>

</div>

<div class="stat-box">

<div class="stat-label">Errors</div>

<div class="stat-value" id="hud-errors" style="color:var(--danger);">0</div>

</div>
```

```
<div class="stat-box">

<div class="stat-label" id="hud-label-time">Time Remaining</div>

<div class="stat-value" id="hud-time">1m</div>

</div>

</div>

<!-- Main Typing Display Area with Infinite Streaming -->

<div id="typing-box" class="typing-container focus-ring" tabindex="0">

<input type="text" id="mobile-text-proxy" autocomplete="off" autocorrect="off" autocapitalize="off" spellcheck="false" style="position:absolute; opacity:0; left:0; top:0; width:100%; height:100%; cursor:default; z-index:1;">

<div id="caret" class="caret caret-smooth"></div>

<div id="ghost-caret" class="ghost-caret" style="display:none;"></div>

<div id="text-display" class="typing-text-display"></div>

</div>

<!-- Bottom Utility Strip -->

<div style="display:flex; justify-content:space-between; align-items:center; margin-top:1rem; flex-wrap:wrap; gap:0.5rem; font-size:0.82rem; color:var(--text-muted);">

<div>

<button onclick="toggleAdvancedAnalyticsDrawer()" id="btn-toggle-advanced" class="btn btn-sm" style="border-color:var(--border-color);">

    View Advanced Telemetry & Heatmap &darr;

</button>

</div>

<div>

    Passage streams infinitely &bull; <kbd style="background:var(--bg-card); padding:0.15rem 0.35rem; border-radius:4px;">Tab</kbd> to restart &bull; <kbd style="background:var(--bg-card); padding:0.15rem 0.35rem; border-radius:4px;">Esc</kbd> for focus mode

</div>

</div>
```

```
<!-- =====

TIER 2: ADVANCED TELEMETRY & HEATMAP DRAWER (EXPANDABLE)

===== -->
```

```
<div id="advanced-analytics-drawer" class="card" style="display:none; margin-top:1.5rem; background:var(--bg-card); border:1px solid var(--border-color);">

  <div style="display:flex; justify-content:space-between; align-items:center; border-bottom:1px solid var(--border-color); padding-bottom:0.75rem; margin-bottom:1rem;">

    <h3 style="font-size:1.05rem; font-weight:800; margin:0; color:var(--accent);">

      Advanced Biometric Telemetry & Key Latency

    </h3>

    <span style="font-size:0.8rem; color:var(--text-muted);">Real-time cognitive metrics</span>

  </div>

  <div style="display:grid; grid-template-columns:repeat(auto-fit, minmax(220px, 1fr)); gap:1rem; margin-bottom:1.5rem;">

    <div style="background:var(--bg-secondary); padding:0.75rem 1rem; border-radius:var(--radius-sm);">

      <div style="font-size:0.72rem; color:var(--text-muted); text-transform:uppercase; font-weight:700;">Finger Distribution</div>

      <div style="font-size:1.1rem; font-weight:800; margin-top:0.2rem; color:var(--text-primary);">Home-Row Anchored</div>

    </div>

    <div style="background:var(--bg-secondary); padding:0.75rem 1rem; border-radius:var(--radius-sm);">

      <div style="font-size:0.72rem; color:var(--text-muted); text-transform:uppercase; font-weight:700;">Cadence Stability</div>

      <div style="font-size:1.1rem; font-weight:800; margin-top:0.2rem; color:var(--success);">High Regularity</div>

    </div>

    <div style="background:var(--bg-secondary); padding:0.75rem 1rem; border-radius:var(--radius-sm);">

      <div style="font-size:0.72rem; color:var(--text-muted); text-transform:uppercase; font-weight:700;">Mean Inter-Key Latency</div>

      <div style="font-size:1.1rem; font-weight:800; margin-top:0.2rem; color:var(--accent);">142 ms</div>

    </div>

  </div>

  <!-- Heatmap Keyboard -->

  <div class="keyboard-wrapper" id="keyboard-container" style="margin-top:0; border:none; padding:0; background:transparent;">

    <div style="display:flex; justify-content:space-between; width:100%; max-width:820px; align-items:center; margin-bottom:0.5rem; font-size:0.8rem; color:var(--text-muted);">

      <span>Biometric Key Heatmap</span>

      <span style="display:flex; gap:0.5rem; align-items:center;">

        <span style="display:inline-block; width:9px; height:9px; border-radius:2px; background:var(--success);"></span> Accurate

        <span style="display:inline-block; width:9px; height:9px; border-radius:2px; background:var(--danger); margin-left:0.4rem;"></span> Weak

      </span>

    </div>

  </div>

</div>
```

```
</div>

<div id="on-screen-keyboard"></div>

</div>

</div>

{% endblock %}

{% block scripts %}

<script src="{{ url_for('static', filename='js/keyboard.js') }}"></script>

<script src="{{ url_for('static', filename='js/engine.js') }}"></script>

<script>

const HINT_DESCRIPTIONS = {

  'easy': 'Easy Level: Short, high-frequency words, simple sentence structures, and minimal punctuation.',

  'moderate': 'Moderate Level: Standard vocabulary, common punctuation, and natural sentence structures.',

  'hard': 'Hard Level: Advanced vocabulary, longer words, complex sentence structures, and increased punctuation.',

  'expert': 'Expert Level: Sophisticated vocabulary, technical terminology, complex punctuation, and rapid reaches.'

};

function onTypeChange(typeVal) {

  if (typeVal === 'custom') {

    window.location.href = "{{ url_for('typing.custom_text_page') }}"

    return;

  }

  window.typingEngine.contentType = typeVal;

  window.typingEngine.loadPrompt(false);

}

function onTimeChange(timeVal) {

  const dur = parseInt(timeVal, 10);

  window.typingEngine.durationLimit = dur;

  window.typingEngine.loadPrompt(false);

}

function onDifficultyChange(diffVal) {
```

```

window.typingEngine.level = diffVal;

const hint = document.getElementById('config-explainer-hint');

if (hint && HINT_DESCRIPTIONS[diffVal]) {

    hint.textContent = HINT_DESCRIPTIONS[diffVal];

}

window.typingEngine.loadPrompt(false);

}
```

```

function toggleBlindQuick() {

    const next = !window.typingEngine.blindModeActive;

    window.typingEngine.blindModeActive = next;

    const btn = document.getElementById('btn-blind');

    btn.textContent = `Blind: ${next ? 'ON' : 'OFF'} `;

    btn.classList.toggle('active-choice', next);

}
```

```

function toggleGhostQuick() {

    const next = !window.typingEngine.ghostEnabled;

    window.typingEngine.ghostEnabled = next;

    const btn = document.getElementById('btn-ghost');

    btn.textContent = `Ghost: ${next ? 'ON' : 'OFF'} `;

    btn.classList.toggle('active-choice', next);

}
```

```

function toggleAdvancedAnalyticsDrawer() {

    const drawer = document.getElementById('advanced-analytics-drawer');

    const btn = document.getElementById('btn-toggle-advanced');

    if (drawer.style.display === 'none') {

        drawer.style.display = 'block';

        btn.textContent = 'Hide Advanced Telemetry ↑';

        drawer.scrollIntoView({ behavior: 'smooth' });

    } else {

        drawer.style.display = 'none';

    }

}
```

```

        btn.textContent = `📊 View Advanced Telemetry & Heatmap ↓`;

    }

}

</script>

{% endblock %}
```